# SQL & NoSQL Interview Mastery

Senior Software Engineer Edition

11 modules · PostgreSQL-first · MySQL / MongoDB / Redis / Cassandra / DynamoDB comparisons

Core concepts · Internals · Production scenarios · FAANG-style questions

165 practice problems (5 easy / 5 medium / 5 hard per module)

Interview patterns: Google · Meta · Amazon · Microsoft · Uber · Netflix · Stripe · LinkedIn

## SQL & NoSQL Interview Mastery — Senior Software Engineer Edition

> Written from the perspective of a Principal Database Engineer & Senior Interviewer (Google, Meta, Amazon, Microsoft, Uber, Stripe, Netflix, LinkedIn).
>
> **Audience:** Backend engineers with 5+ years of experience. **Primary database:** PostgreSQL. Comparisons include MySQL, MongoDB, Redis, Cassandra, DynamoDB. **Scope:** Only what is actually asked in senior SWE interviews. No beginner filler, no rarely used features.

---

### How to Use This Guide

Every chapter follows the same 12-part format:

1. **Why Interviewers Ask This** — what signal the interviewer is trying to extract
2. **Core Concept** — the deep mental model
3. **Internal Working** — what the engine actually does
4. **Visualization (ASCII)** — diagram of the mechanism
5. **Real Production Example** — how this shows up at FAANG scale
6. **Common Interview Questions** — the exact questions you will hear
7. **Common Mistakes** — the traps that fail candidates
8. **Best Practices** — what a senior answer sounds like
9. **Coding Questions** — hands-on exercises
10. **SQL Examples** — runnable PostgreSQL

11. **Optimization Techniques** — how to make it fast
12. **Follow-up Questions** — the interviewer's next move after your answer

Each module ends with **5 easy, 5 medium, and 5 hard practice problems**.

Work through modules in order. Module 4 (Indexes) and Module 6 (Transactions & Concurrency) are the highest-signal modules in senior interviews — do not skim them.

---

## Modules

| # | Module | File | Interview Weight |
|---|---|---|---|
| 1 | Database Fundamentals | modules/module-01-fundamentals.md | ★★★★★ |
| 2 | SQL Core | modules/module-02-sql-core.md | ★★★★☆ |
| 3 | Joins | modules/module-03-joins.md | ★★★★☆ |
| 4 | Indexes (Highest Priority) | modules/module-04-indexes.md | ★★★★★ |
| 5 | Query Optimization | modules/module-05-query-optimization.md | ★★★★★ |
| 6 | Transactions & Concurrency | modules/module-06-transactions-concurrency.md | ★★★★★ |
| 7 | Database Scaling | modules/module-07-scaling.md | ★★★★★ |
| 8 | NoSQL | modules/module-08-nosql.md | ★★★★☆ |
| 9 | Database Design | modules/module-09-database-design.md | ★★★★☆ |
| 10 | Interview SQL (Top Problems) | modules/module-10-interview-sql.md | ★★★★★ |
| 11 | Production Debugging | modules/module-11-production-debugging.md | ★★★★☆ |

---

## The Senior-Level Answer Framework

When answering *any* database interview question, structure your answer as:

```
1. Definition        (one crisp sentence — shows precision)
2. Mechanism         (how the engine implements it — shows depth)
3. Trade-off         (what you give up — shows judgment)
4. Production story  (where you used/broke it — shows experience)
5. Scale limit       (when it stops working — shows senior thinking)
```

Interviewers at Google/Meta/Amazon are not testing whether you know the word "index." They are testing whether you can **predict system behavior under load** and **defend design decisions with trade-offs**. Every chapter in this guide is written to arm you with exactly that.

---

## Quick Self-Assessment (Can you answer these cold?)

- Why does `SELECT COUNT(*)` on a 500M-row Postgres table take minutes even with indexes?
- Why can a composite index on `(a, b)` serve `WHERE a = 1` but not `WHERE b = 1`?
- Why does `READ COMMITTED` still allow lost updates, and how do you fix it without `SERIALIZABLE`?
- Why does `OFFSET 1000000 LIMIT 10` get slower as offset grows, and what replaces it?
- Why does Cassandra write faster than Postgres, and what does it sacrifice?
- How do you shard a payments table, and what breaks when you do?

If any of these makes you pause, the corresponding module will fix it.

# MODULE 1 — Database Fundamentals

> The module interviewers use to calibrate your level in the first 10 minutes. A senior engineer is expected to answer these with mechanisms and trade-offs, not definitions.

Chapters: 1.1 Why Databases Exist & Database Architecture 1.2 RDBMS vs NoSQL (SQL vs NoSQL) 1.3 ACID vs BASE 1.4 CAP Theorem & Strong vs Eventual Consistency 1.5 OLTP vs OLAP 1.6 Normalization vs Denormalization 1.7 Primary Key vs Unique Key vs Foreign Key 1.8 Transactions (Introduction — deep dive in Module 6)

---

## Chapter 1.1 — Why Databases Exist & Database Architecture

### 1. Why Interviewers Ask This

It sounds trivial, but it separates engineers who treat the DB as a magic box from those who understand it as a **concurrent, crash-safe, indexed file manager**. If you can explain what a database gives you *over writing to files*, you can reason about every other topic: WAL explains replication, buffer pool explains why indexes matter, the planner explains slow queries.

### 2. Core Concept

A database solves five problems that files don't:

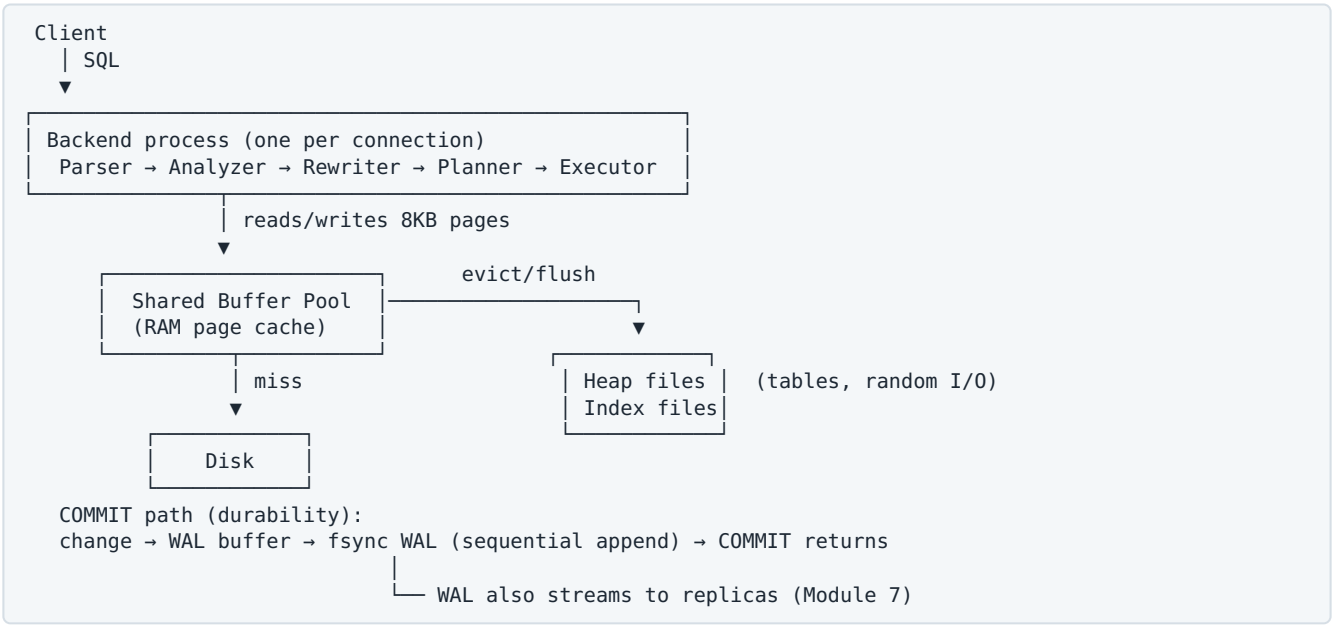
| Problem | File approach fails because | Database solution |
|---|---|---|
| Concurrent access | Two writers corrupt each other | Locking + MVCC |
| Crash safety | Partial write = corrupted file | Write-Ahead Log (WAL) + recovery |
| Fast lookup | Full scan of the file | Indexes (B+Tree, hash) |
| Data integrity | Nothing stops bad data | Constraints, types, FKs |
| Declarative access | You hand-code every scan | SQL + cost-based optimizer |

### 3. Internal Working (PostgreSQL architecture)

Life of a query:

1. **Client → connection**: Postgres forks one *backend process* per connection (this is why connection pooling matters — see Module 7/11).
2. **Parser** → parse tree. **Analyzer** resolves tables/columns. **Rewriter** expands views.
3. **Planner/Optimizer** enumerates plans (seq scan vs index scan, join orders, join algorithms), estimates cost using **statistics** (`pg_statistic`), picks the cheapest.
4. **Executor** pulls tuples through the plan tree (Volcano/iterator model: each node's `next()` calls its children).
5. **Storage**: tables are heap files split into **8KB pages**. Reads go through the **shared buffer pool** (RAM cache of pages). Dirty pages are flushed lazily by the background writer / checkpointer.
6. **Durability**: before a commit returns, the change record is appended to the **WAL** and fsynced. The heap page itself can be written later — if the server crashes, WAL replay reconstructs it. This is why sequential WAL append makes commits fast even though data pages are random-access.

## 4. Visualization (ASCII)




## 5. Real Production Example

**Stripe**: every payment mutation must survive a machine dying mid-write. WAL-before-data (write-ahead logging) is the exact mechanism that guarantees a charge marked "captured" is never lost, and the same WAL stream feeds read replicas and change-data-capture into Kafka. When an interviewer asks "how does a DB not lose data on power failure," WAL + fsync + recovery replay is the expected answer.

## 6. Common Interview Questions

- "Why not just store data in files / S3?"
- "Walk me through what happens when I run a query." (classic Google warm-up)
- "How does a database survive a crash mid-transaction?"
- "Why is one-connection-per-request a problem in Postgres?"
- "What is the buffer pool and why does it matter for performance?"

## 7. Common Mistakes

- Answering "databases are for storing data" — zero signal.
- Claiming COMMIT writes the data pages to disk. It writes the **WAL**; data pages flush later.
- Forgetting that Postgres connections are processes (~5–10MB each), so 5,000 app connections will take the DB down without a pooler.
- Confusing the query *cache* (MySQL, removed in 8.0) with the buffer pool.

## 8. Best Practices

- Frame every performance question in terms of **pages touched**: a query is fast when it reads few 8KB pages, and those pages are in the buffer pool.
- Know your DB's memory knobs: `shared_buffers` (~25% RAM), `work_mem` (per-sort/hash!), `wal_buffers`, `effective_cache_size` (planner hint, not allocation).
- Always put PgBouncer (transaction mode) or RDS Proxy in front of Postgres at scale.

## 9. Coding Questions

1. Given a 100GB table and 16GB RAM, estimate why a full scan is disk-bound and what fraction of the buffer pool a hot 2GB index consumes.
2. Write pseudocode for crash recovery given a WAL of `[BEGIN T1, T1: x=5, COMMIT T1, BEGIN T2, T2: y=7, CRASH]` (redo committed, ignore/undo uncommitted).

### 10. SQL Examples

```
-- See the plan and real execution (Module 5 covers reading it)
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM orders WHERE user_id = 42;

-- Buffer cache hit ratio (want > 0.99 for OLTP)
SELECT sum(blks_hit)::float / nullif(sum(blks_hit) + sum(blks_read), 0) AS cache_hit_ratio
FROM pg_stat_database;

-- Table size vs index size (pages on disk)
SELECT relname,
       pg_size_pretty(pg_table_size(oid))  AS table_size,
       pg_size_pretty(pg_indexes_size(oid)) AS index_size
FROM pg_class WHERE relname = 'orders';
```

### 11. Optimization Techniques

- Keep the **working set** (hot pages) in RAM; monitor cache hit ratio.
- Batch writes: 1 transaction with 1,000 inserts ≫ 1,000 transactions (one fsync vs a thousand).
- `synchronous_commit = off` for tolerable-loss workloads (metrics, logs): commit returns before WAL fsync — huge throughput win, risk of losing last few ms on crash (not corruption).

### 12. Follow-up Questions

- "What happens if the WAL disk is slow but data disk is fast?" (commits stall — WAL fsync is the commit critical path)
- "How does the WAL enable replication?" (ship/stream the same log — Module 7)
- "What's a checkpoint and why does it cause I/O spikes?" (flush all dirty pages; spread with `checkpoint_completion_target`)

---

## Chapter 1.2 — RDBMS vs NoSQL (SQL vs NoSQL)

### 1. Why Interviewers Ask This

It's the #1 system-design gate question. They're testing whether you choose databases based on **access patterns, consistency needs, and scale**, or based on hype. A senior answer never says "NoSQL is faster" — it names the specific trade-off.

### 2. Core Concept

| Dimension | RDBMS (Postgres/MySQL) | NoSQL (Mongo/Cassandra/DynamoDB/Redis) |
|---|---|---|
| Data model | Relations, rigid schema | Document / wide-column / KV / graph |
| Query model | Declarative SQL, ad-hoc joins | API by primary key; limited/no joins |
| Consistency | ACID transactions, strong by default | Mostly tunable / eventual (Dynamo, Cassandra); Mongo has multi-doc txns since 4.0 |
| Scaling | Vertical + read replicas; sharding is manual/painful | Horizontal sharding is native |
| Schema evolution | Migrations (DDL) | Flexible, app enforces shape |
| Best at | Transactions, integrity, ad-hoc queries | Massive scale on *known* access patterns |

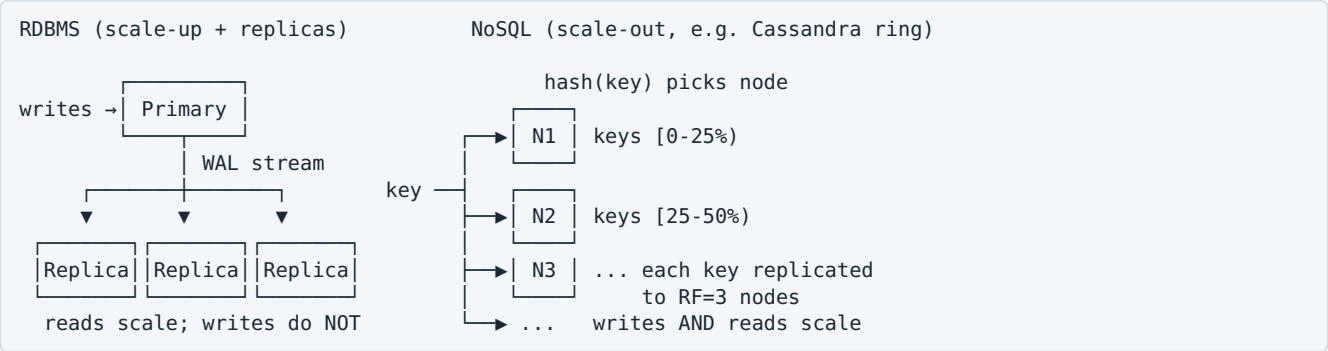
The real dividing line: **RDBMS optimizes for flexible queries over one node's worth of data with strong guarantees. NoSQL optimizes for one or two access patterns over unbounded data.**

### 3. Internal Working

- Postgres/MySQL: B+Tree storage, WAL, MVCC, cost-based planner — random-access read-optimized.

- Cassandra/DynamoDB-style: **LSM trees** (Module 8) — sequential-write-optimized, reads may check several SSTables; consistent hashing spreads partitions across nodes with no master.
- MongoDB: B+Tree (WiredTiger) documents, replica sets with a single primary, range/hash sharding.
- Redis: everything in RAM, single-threaded event loop per shard — microsecond latency.

## 4. Visualization (ASCII)




## 5. Real Production Example

- **Uber**: trips/payments in MySQL-based Schemaless + Postgres early on (integrity), but the driver-location firehose (millions of writes/sec, only queried by driver_id) went to Cassandra-style stores and Redis.
- **Amazon**: the shopping cart famously moved to Dynamo because a cart must accept writes even during a partition (availability > consistency for carts); orders/payments stayed strongly consistent.
- **Netflix**: viewing history in Cassandra (append-heavy, per-user key), billing in MySQL.

## 6. Common Interview Questions

- "You're designing X — SQL or NoSQL? Why?"
- "What do you lose when you move from Postgres to DynamoDB?" (ad-hoc queries, joins, multi-row ACID by default, constraints)
- "When would you use both?" (almost always: system of record in RDBMS + NoSQL for hot paths)
- "Is MongoDB ACID?" (single-document always; multi-document transactions exist but are expensive and limited — using them heavily means you probably wanted an RDBMS)

## 7. Common Mistakes

- "NoSQL because we need scale" without stating the write volume or access pattern.
- Ignoring that **Postgres handles 10k+ writes/sec and multi-TB tables** fine — most startups never outgrow it. Premature NoSQL adoption costs you joins, transactions, and migrations agility.
- Modeling relational data (many-to-many, ad-hoc reporting) in a document store, then doing joins in application code.
- Saying "schemaless means no schema" — the schema moves into your application code, unversioned.

## 8. Best Practices

- Default to PostgreSQL. Move a *specific workload* to NoSQL when you can name the access pattern and the scale number that breaks the RDBMS.
- Choose by access pattern: key-value at huge scale → DynamoDB/Cassandra; caching/ephemeral → Redis; flexible nested documents, mid-scale → MongoDB; ad-hoc queries + transactions → RDBMS.
- In interviews, always state what you sacrifice — that's the senior signal.

## 9. Coding Questions

1. Model "user has many orders, order has many items" in Postgres (3 tables) and in DynamoDB (single table, `PK=USER#id`, `SK=ORDER#id#ITEM#n`) and list which queries each supports cheaply.
2. Given 1M writes/sec of IoT sensor readings queried only by `(device_id, time range)`, pick a store and justify (Cassandra/LSM wins; B+Tree random inserts + index maintenance lose).

### 10. SQL Examples

```
-- Relational strength: an ad-hoc question you never planned for
SELECT u.country, count(*) AS orders, avg(o.total_cents) AS avg_order
FROM orders o JOIN users u ON u.id = o.user_id
WHERE o.created_at >= now() - interval '30 days'
GROUP BY u.country ORDER BY orders DESC;
-- In DynamoDB this is a full-table export + offline job unless you pre-modeled it.

-- Postgres as a document store (often the right middle ground)
CREATE TABLE events (
  id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  payload jsonb NOT NULL
);
CREATE INDEX ON events USING gin (payload jsonb_path_ops);
SELECT * FROM events WHERE payload @> '{"type": "checkout", "status": "failed"}';
```

### 11. Optimization Techniques

- Hybrid architecture: RDBMS as source of truth, Redis cache in front, stream changes (CDC) to Elasticsearch/ analytics. This answers most "scale the reads" questions.
- In RDBMS, use `jsonb` for genuinely schemaless fragments instead of adopting a second database.
- In NoSQL, design the table *from the queries backwards* (Module 8).

### 12. Follow-up Questions

- "Your startup on Postgres just hit 50k writes/sec on one table — options?" (partition, shard, queue+batch, or move that table to Cassandra/DynamoDB; discuss trade-offs)
- "How do you keep Redis/Elasticsearch in sync with Postgres?" (CDC via WAL → Debezium/Kafka; discuss ordering + at-least-once)
- "What does DynamoDB single-table design cost you?" (rigid access patterns, painful evolution)

---

## Chapter 1.3 — ACID vs BASE

### 1. Why Interviewers Ask This

ACID is the most-asked database question, at every level. At senior level they push past the acronym: *how* is each letter implemented, and what does BASE actually buy you.

### 2. Core Concept

**ACID** (per transaction): - **Atomicity** — all statements apply or none do. No partial effects, even on crash. - **Consistency** — the transaction moves the DB from one valid state to another; constraints (PK/FK/CHECK/triggers) hold. (Note: the "C" is partly the *application's* job.) - **Isolation** — concurrent transactions behave as if serialized (to a degree set by the isolation level — Module 6). - **Durability** — once COMMIT returns, the data survives crash/power loss.

**BASE** (distributed, availability-first systems): - **Basically Available** — the system answers even during failures/ partitions. - **Soft state** — state may change without input (replicas converging). - **Eventual consistency** — stop writing, and all replicas converge to the same value.

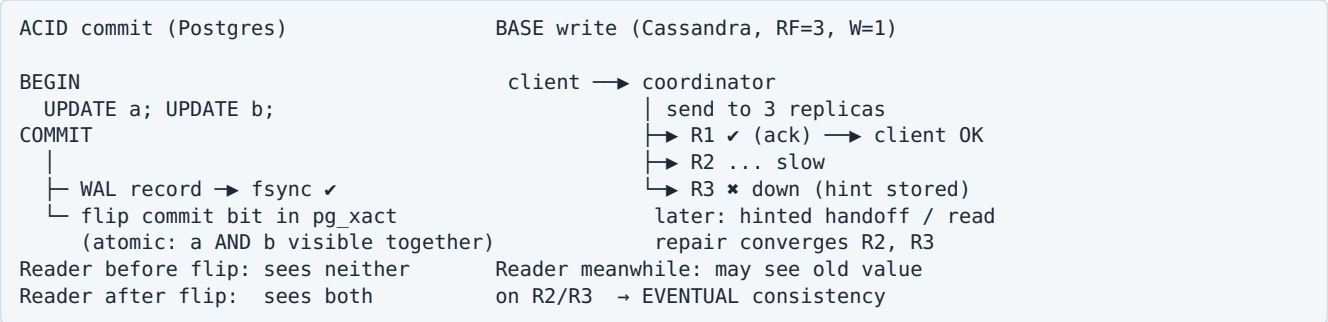
BASE is not "no guarantees" — it's a deliberate trade: accept stale reads and conflict resolution in exchange for availability and horizontal write scale.

## 3. Internal Working

| Property | PostgreSQL mechanism |
|---|---|
| Atomicity | MVCC: new row versions carry the writing transaction's XID; a single atomic flip of the transaction's status in `pg_xact` (commit log) makes *all* its versions visible or dead at once. Rollback = mark aborted; dead versions cleaned by VACUUM. |
| Consistency | Constraint checks at statement/commit time; deferred constraints checked at COMMIT |
| Isolation | MVCC snapshots + row locks; SSI for SERIALIZABLE (Module 6) |
| Durability | WAL fsync before COMMIT returns; `synchronous_standby` extends this to replicas |

BASE example (DynamoDB/Cassandra): a write goes to N replicas; with `W=1` the coordinator acks after one replica accepts; anti-entropy (hinted handoff, read repair, Merkle-tree sync) converges the rest. Conflicts resolved by last-write-wins timestamps (Cassandra) or vector clocks (Dynamo).

## 4. Visualization (ASCII)




## 5. Real Production Example

**Stripe / any payments system**: moving money between two balances *must* be atomic — debit and credit in one transaction, or you invent/destroy money. Meanwhile the "recent transactions" feed shown in the dashboard can be BASE — a few seconds stale is fine. Senior answer: *split the system by invariant strength*, don't pick one model globally.

## 6. Common Interview Questions

- "Explain ACID with how each property is implemented." (the senior version)
- "Which ACID property does a crash test? A concurrent update test?" (D/A; I)
- "Is eventual consistency acceptable for a bank?" (balances: no; notification feed: yes — and real banks use *reconciliation*, which is eventual consistency with auditing)
- "What anomalies can users see under eventual consistency?" (stale reads, non-monotonic reads, read-your-own-write failures)

## 7. Common Mistakes

- Reciting the acronym with no mechanism — automatic mid-level cap.
- Saying Consistency (ACID) = Consistency (CAP). They're different words: ACID-C is about integrity constraints; CAP-C is linearizability across replicas.
- Believing durability is absolute: with `synchronous_commit=off` or async replication you can lose acknowledged writes; know your configuration.
- Thinking BASE systems can't do transactions at all (DynamoDB has `TransactWriteItems`, Mongo has multi-doc txns — limited and costly, but exist).

## 8. Best Practices

- Identify each business invariant and assign it a consistency budget: money movement = ACID + synchronous replication; feeds/counters/caches = eventual.

- For cross-service "transactions," don't distribute ACID — use sagas/outbox with idempotency keys (mention this; it's a favorite Stripe/Uber follow-up).
- Make writes idempotent everywhere eventual consistency or retries exist.

### 9. Coding Questions

1. Write the transfer transaction (below) and explain what happens if the process crashes between the two UPDATEs.
2. Design an idempotent "charge customer" API: unique idempotency key + `INSERT ... ON CONFLICT DO NOTHING` returning the prior result.

### 10. SQL Examples

```
-- Atomic money transfer
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
-- crash here? WAL has no COMMIT record → recovery discards both updates
COMMIT;

-- Consistency via constraint: overdrafts impossible even under bugs
ALTER TABLE accounts ADD CONSTRAINT non_negative CHECK (balance >= 0);

-- Idempotency for BASE-world retries
CREATE TABLE payments (
  idempotency_key text PRIMARY KEY,
  amount_cents int NOT NULL,
  status text NOT NULL DEFAULT 'pending'
);
INSERT INTO payments (idempotency_key, amount_cents)
VALUES ('req-abc-123', 4999)
ON CONFLICT (idempotency_key) DO NOTHING;
```

### 11. Optimization Techniques

- Group related writes into one transaction: atomicity for free *and* one WAL fsync.
- Keep transactions short — long transactions hold back VACUUM and increase lock/serialization conflicts (Module 6/11).
- Relax durability only where loss is acceptable and say so explicitly (`synchronous_commit=off` per-session for bulk loads).

### 12. Follow-up Questions

- "Two services must both commit or neither — how, without 2PC?" (outbox + saga with compensating actions; discuss why 2PC is avoided: blocking on coordinator failure)
- "How does Postgres make a multi-row transaction visible atomically?" (single commit-bit flip; snapshots decide visibility)
- "What's read-your-own-writes and how do you guarantee it with replicas?" (session pinning to primary, or wait-for-LSN on the replica)

---

## Chapter 1.4 — CAP Theorem & Strong vs Eventual Consistency

### 1. Why Interviewers Ask This

Every distributed-database decision routes through CAP. Interviewers use it to check you can reason under failure, and to catch the common misstatement ("pick 2 of 3").

### 2. Core Concept

In a distributed system, when a **network partition** (P) happens, you must choose: - **CP**: refuse some requests (lose availability) to stay consistent — e.g., a minority-side node rejects writes. - **AP**: keep answering (stay available) and accept divergence — reconcile later.

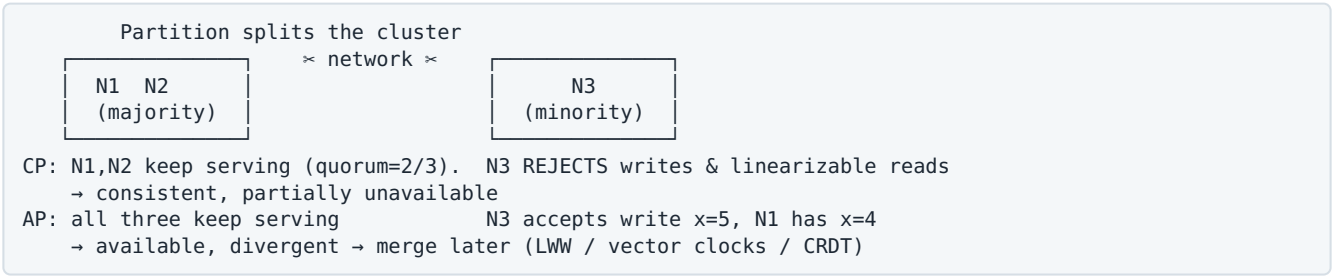
Key corrections seniors are expected to make: - **P is not optional.** Networks partition; the only choice is C vs A *during* the partition. - CAP-Consistency = **linearizability** (every read sees the latest acknowledged write, as if one copy). Not ACID-C. - **PACELC** extension: *if Partition, trade A vs C; Else (normal operation), trade Latency vs Consistency.* Even with no partition, synchronous replication costs latency. DynamoDB ≈ PA/EL. Postgres+sync replica ≈ PC/EC. Cassandra tunable.

Consistency spectrum (strong → weak): `Linearizable → Sequential → Causal → Read-your-writes / Monotonic reads → Eventual`

## 3. Internal Working

- **CP systems** (Spanner, etcd, Zookeeper, Postgres with sync replication + failover): quorum/consensus (Paxos/Raft). A write commits when a majority accepts it; minority partitions can't serve linearizable reads or accept writes.
- **AP systems** (Dynamo, Cassandra with `W=1,R=1`): any replica accepts writes; sloppy quorums and hinted handoff keep availability; convergence via read-repair/anti-entropy; conflicts via LWW or vector clocks.
- **Tunable**: Cassandra per-query consistency. If `R + W > RF` (e.g. QUORUM reads + QUORUM writes with RF=3 → 2+2>3), read and write sets overlap → you read the latest write (strongly consistent read, at latency/availability cost).

## 4. Visualization (ASCII)




## 5. Real Production Example

- **Amazon cart (AP)**: never refuse "add to cart." Divergent carts merge by union — worst case a deleted item reappears. Business chose availability.
- **Google Spanner (CP)**: AdWords billing needs global consistency; Spanner uses Paxos + TrueTime. During partition, minority regions stall — Google accepts that.
- **Netflix (AP)**: viewing state in Cassandra; a stale "continue watching" row is harmless.

## 6. Common Interview Questions

- "Explain CAP. Is CA possible?" (only in a single-node/no-partition world — i.e., not a distributed system guarantee)
- "Where does DynamoDB / Cassandra / Postgres / Spanner sit?"
- "How do you get a strongly consistent read from Cassandra?" (`R+W>RF`)
- "Design X: which consistency level per operation?" (the real question behind CAP)

## 7. Common Mistakes

- "Pick two of three" framing — P isn't a choice.
- Applying CAP to a single-node Postgres ("Postgres is CA") — CAP is about replicated systems.
- Treating eventual consistency as unbounded chaos: in practice convergence is ms–seconds, and you can layer session guarantees (read-your-writes) on top.
- Forgetting PACELC: consistency costs latency even without failures.

## 8. Best Practices

- Answer per-operation, not per-system: "checkout is CP, product views are AP."
- Name the anomaly the user would see under AP and whether the business tolerates it.
- Mention quorum arithmetic (`R+W>RF`) — it's the concrete tool interviewers want to hear.

### 9. Coding Questions

1. RF=3. Give (W,R) pairs for: fastest writes ever (W=1,R=1), strong reads with balanced cost (W=2,R=2), write-heavy strong (W=1,R=3). State availability impact of each.
2. Sketch read-your-own-writes for a Postgres primary + replicas: after write, capture `pg_current_wal_lsn()`; on replica, wait until `pg_last_wal_replay_lsn() >= lsn` or route that session to the primary.

### 10. SQL Examples

```
-- Replication lag on a Postgres replica (staleness you'd serve under async replication)
SELECT now() - pg_last_xact_replay_timestamp() AS replica_lag;

-- Synchronous replication = choosing C (and paying latency / availability)
-- postgresql.conf: synchronous_standby_names = 'ANY 1 (r1, r2)'
-- Per-transaction downgrade for non-critical writes:
SET LOCAL synchronous_commit = off;
```

### 11. Optimization Techniques

- Use causal/session consistency instead of full linearizability when possible — cheaper, fixes the visible anomalies (user sees own writes).
- Local quorums (`LOCAL_QUORUM`) in multi-region Cassandra: strong-ish within region without cross-region latency.
- CRDTs for AP counters/sets (mention: Riak, Redis CRDT modules) to make merges automatic.

### 12. Follow-up Questions

- "What does TrueTime buy Spanner?" (bounded clock uncertainty → external consistency without cross-region locks on reads)
- "Your AP system shows a user their comment disappearing and reappearing — fix?" (monotonic reads: sticky routing to the same replica, or client-tracked versions)
- "Why is 2PC not a CAP escape hatch?" (coordinator failure blocks everyone — it sacrifices A *and* liveness)

---

## Chapter 1.5 — OLTP vs OLAP

### 1. Why Interviewers Ask This

"Why is this analytics query killing the production DB?" is a real incident at every company. This topic tests whether you know workloads shape storage engines — row vs column layout.

### 2. Core Concept

| | OLTP | OLAP |
|---|---|---|
| Work unit | Many tiny transactions (point reads/writes) | Few huge scans/aggregations |
| Pattern | `WHERE id = ?`, touch few rows, all columns | Touch millions of rows, few columns |
| Storage | **Row-oriented** (whole row contiguous) | **Column-oriented** (each column contiguous) |
| Index | B+Tree point/range lookups | Zone maps, min/max pruning, partitions |
| Latency target | ms | seconds–minutes acceptable |
| Systems | Postgres, MySQL, DynamoDB | Snowflake, BigQuery, Redshift, ClickHouse |

### 3. Internal Working

- **Row store**: one 8KB page holds complete rows → `SELECT * WHERE id=?` reads ~1–4 pages. Perfect for OLTP; terrible for "sum one column over 500M rows" (reads *every* column's bytes).
- **Column store**: `amount` values stored contiguously → scan reads only that column; compression is extreme (similar values together: RLE, dictionary, delta) → 10–50x less I/O; vectorized execution (SIMD over column batches). But updating one "row" touches many files → bad at OLTP.

- Bridge: **CDC/ETL** replicates OLTP data into the warehouse (minutes of lag); HTAP systems (TiDB, AlloyDB, SingleStore) keep both layouts internally.

## 4. Visualization (ASCII)

```
Row store page:                    Column store files:
[id1|name1|city1|amt1]             ids:   [1,2,3,4,...]  ←compressed
[id2|name2|city2|amt2]             names: [n1,n2,n3,...]
[id3|name3|city3|amt3]             citys: [c1,c1,c1,...] ←RLE crushes this
                                   amts:  [a1,a2,a3,...] ←SUM reads ONLY this
SELECT * WHERE id=2  → 1 page ✔    SELECT * WHERE id=2 → touch every file ✖
SUM(amt) 500M rows  → all pages ✖  SUM(amt) → one compressed column ✔
```

## 5. Real Production Example

**Netflix**: playback start/stop events land in OLTP-ish stores and Kafka, then flow into an S3/Iceberg warehouse queried by Trino/Spark for engagement analytics. Nobody runs `GROUP BY title` over the serving database. The standard incident: an analyst's dashboard query on the production replica saturates I/O and lags replication → move analytics off OLTP.

## 6. Common Interview Questions

- "Why are column stores faster for analytics?" (I/O reduction + compression + vectorization)
- "Why not run reports on the primary / replica?" (I/O + cache pollution + replication lag + long transactions blocking vacuum)
- "How does data get from OLTP to the warehouse?" (CDC → Kafka → warehouse; batch ETL)
- "What's HTAP?"

## 7. Common Mistakes

- Adding indexes to make an OLAP query fast on Postgres — indexes don't help scan-90%-of-table queries; the layout is wrong.
- Running month-long aggregates in the request path instead of precomputing (materialized views, rollup tables).
- Letting BI tools hit production with `SELECT *` over years of data.

## 8. Best Practices

- Separate serving from analytics from day one: replica → then CDC to a real warehouse.
- Precompute aggregates consumed by product surfaces (counts, totals) — don't aggregate at read time.
- In Postgres, `BRIN` indexes + partitioning give "poor-man's OLAP" for append-only time-series.

## 9. Coding Questions

1. Estimate: 500M rows × 200B/row row-store vs a 4-byte column with 8x compression — how many bytes does `SUM(amount)` read in each? (~100GB vs ~250MB)
2. Design the pipeline: orders table → hourly revenue dashboard with <5 min lag (CDC → Kafka → ClickHouse/ warehouse → dashboard; or Postgres materialized view if small).

### 10. SQL Examples

```
-- OLTP query: point lookup, index, ms
SELECT * FROM orders WHERE id = 91823;

-- OLAP query: don't run this on the OLTP primary
SELECT date_trunc('month', created_at) AS month,
       country, sum(total_cents) / 100.0 AS revenue
FROM orders
WHERE created_at >= '2025-01-01'
GROUP BY 1, 2 ORDER BY 1, 2;

-- Poor-man's OLAP in Postgres: precomputed rollup
CREATE MATERIALIZED VIEW monthly_revenue AS
SELECT date_trunc('month', created_at) AS month, sum(total_cents) AS revenue_cents
FROM orders GROUP BY 1;
REFRESH MATERIALIZED VIEW CONCURRENTLY monthly_revenue;
```

### 11. Optimization Techniques

- Partition big fact tables by time → partition pruning replaces index for range scans.
- BRIN index on `created_at` for append-only tables: tiny index, prunes page ranges.
- Aggregate incrementally (upsert into rollup on write, or streaming) instead of re-scanning.

### 12. Follow-up Questions

- "Your replica lags whenever the ETL runs — why?" (big reads evict cache + long snapshot holds; fix: dedicated replica or CDC)
- "Why is `COUNT(*)` slow in Postgres but instant in a column store?" (MVCC forces visibility checks per row vs column-store metadata; Module 5)
- "When is HTAP worth it vs a pipeline?" (freshness requirements vs operational complexity)

---

## Chapter 1.6 — Normalization vs Denormalization

### 1. Why Interviewers Ask This

Schema-design round staple. Tests whether you understand *why* redundancy is dangerous (anomalies) and *when* it's the right choice anyway (read-path performance). Senior candidates are expected to defend deliberate denormalization.

### 2. Core Concept

**Normalization** removes redundancy so every fact lives in exactly one place. - **1NF**: atomic values, no repeating groups (no CSV-in-a-column). - **2NF**: no partial dependency — non-key columns depend on the *whole* composite key. - **3NF**: no transitive dependency — non-key columns depend *only* on the key. ("The key, the whole key, and nothing but the key.") - BCNF: every determinant is a candidate key (rarely probed beyond name-dropping).

Update/insert/delete **anomalies** are what normalization prevents: change a product's name stored in 10M order rows and miss some → contradictory data.

**Denormalization** deliberately re-introduces redundancy to kill joins/aggregations on hot read paths — accepting that the application must now keep copies in sync.

### 3. Internal Working

- Normalized reads = joins; each join is a B+Tree probe or hash build (Module 3). 5-way joins at p99 under load add up.
- Denormalized writes = multi-row/multi-table updates (fan-out). Consistency maintained by transactions, triggers, or async jobs — each with failure modes.
- The classic middle ground: **snapshot columns** (order stores `product_name`, `unit_price` at purchase time — this is *correct*, not just fast: historical facts shouldn't change) and **counter caches** (`users.post_count`).

## 4. Visualization (ASCII)

```
Normalized                          Denormalized read model
users(id, name)                     order_view(order_id, user_name,
orders(id, user_id, product_id)                product_name, price, ...)
products(id, name, price)                       ▲
                                                │ kept in sync by txn /
read = 3-way JOIN                               │ trigger / async consumer
write = 1 row, 1 place ✔            read = 1 row, 0 joins ✔
                                    write = N copies to update ✖
Anomaly demo (denormalized): product renamed → update 10M rows or serve two names
```

## 5. Real Production Example

**Meta / Twitter feeds**: fully normalized "read the feed" = join follows × posts × users at request time — impossible at scale. Solution: denormalized per-user feed lists (fan-out on write into Redis/Cassandra) — the canonical denormalization story. Meanwhile the *source of truth* (users, posts) stays normalized in MySQL. **Amazon orders**: item name & price copied into `order_items` forever — a price change must not rewrite history.

## 6. Common Interview Questions

- "Explain 1NF/2NF/3NF with an example." (be able to do it in 60 seconds)
- "When would you denormalize?" (measured read bottleneck on a join/aggregate hot path; read≫write ratio; or snapshot semantics)
- "How do you keep denormalized copies consistent?" (same-transaction, triggers, CDC/async + reconciliation)
- "Design an orders schema — why did you copy price into order_items?" (snapshot correctness)

## 7. Common Mistakes

- Denormalizing preemptively "for performance" with no measurement — you inherit sync bugs for nothing.
- Normalizing history away: joining `orders → products.price` shows *today's* price on old orders. Classic trap question.
- Storing lists as CSV/JSON arrays then needing to query by element (violates 1NF; forces full scans — in Postgres at least use `jsonb` + GIN if you must).
- Saying "3NF is always right" or "joins are slow" — both junior signals; indexed joins on OLTP point queries are fast.

## 8. Best Practices

- Normalize the write model (source of truth) to 3NF by default.
- Denormalize *read models* (CQRS-lite): rollups, counter caches, feed tables — rebuildable from the source of truth (that's your escape hatch when they drift).
- Snapshot immutable business facts (prices, addresses, tax rates) at event time.
- If a counter/copy can drift, schedule a reconciliation job. Say the word "reconciliation" — it lands well.

## 9. Coding Questions

1. Normalize `orders(id, user_email, user_name, product_names_csv, total)` to 3NF — identify which normal form each fix addresses.
2. Add a consistent `posts_count` to `users`: same-transaction `UPDATE users SET posts_count = posts_count + 1` on insert, and explain the lock contention it creates on hot users (and the async alternative).

### 10. SQL Examples

```
-- 3NF write model with snapshot columns on the read-critical child
CREATE TABLE products (
  id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name text NOT NULL,
  price_cents int NOT NULL
);
CREATE TABLE orders (
  id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  user_id bigint NOT NULL REFERENCES users(id),
  created_at timestamptz NOT NULL DEFAULT now()
);
CREATE TABLE order_items (
  order_id bigint REFERENCES orders(id),
  product_id bigint REFERENCES products(id),
  product_name text NOT NULL,     -- snapshot: history must not change
  unit_price_cents int NOT NULL,  -- snapshot
  quantity int NOT NULL CHECK (quantity > 0),
  PRIMARY KEY (order_id, product_id)
);

-- Counter cache maintained transactionally
BEGIN;
INSERT INTO posts (user_id, body) VALUES (42, '...');
UPDATE users SET posts_count = posts_count + 1 WHERE id = 42;
COMMIT;
```

### 11. Optimization Techniques

- Prefer a **covering index** (Module 4) or **materialized view** before physically denormalizing base tables — cheaper to maintain, easy to drop.
- For hot counters, batch increments (accumulate in Redis, flush periodically) to avoid single-row lock contention.
- Keep every denormalized artifact rebuildable with one SQL statement; document it next to the DDL.

### 12. Follow-up Questions

- "Your counter cache drifted — how do you detect and fix without downtime?" (periodic diff against `COUNT(*)`, repair in batches)
- "Trigger vs application code vs CDC for sync — trade-offs?" (triggers: consistent but hidden and add write latency; CDC: decoupled but eventual)
- "How does this change in DynamoDB?" (denormalization is the *default*; you duplicate into item collections/GSIs by design)

---

## Chapter 1.7 — Primary Key vs Unique Key vs Foreign Key

### 1. Why Interviewers Ask This

Screener question with senior-level depth hiding inside: UUID vs sequence, FK locking costs, and "should we drop FKs at scale" are real design fights at Uber/Stripe-scale companies.

## 2. Core Concept

| | Primary Key | Unique Constraint/Key | Foreign Key |
|---|---|---|---|
| Purpose | Row identity | Enforce alternate uniqueness | Referential integrity (child → parent) |
| NULLs | Never | Allowed (Postgres: NULLs don't collide by default; 15+ has `NULLS NOT DISTINCT`) | NULL = "no parent", allowed |
| Per table | Exactly one | Many | Many |
| Implementation | Unique B+Tree index (+ NOT NULL) | Unique B+Tree index | Constraint checked via parent's PK/unique index |
| Extra role | Target of FKs; **MySQL/InnoDB: the clustered index** | Natural-key guard | Cascades (`ON DELETE ...`) |

Surrogate keys (`bigint identity` / UUID) vs natural keys (email, SSN): natural keys change and leak semantics — use surrogates for identity, add unique constraints on natural keys.

## 3. Internal Working

- PK/unique = B+Tree where insert descends to the leaf; if the key exists (and is committed or from a concurrent txn — insert *waits* on the conflicting txn) → unique violation.
- **FK enforcement**: inserting a child takes a `FOR KEY SHARE` lock on the parent row (so the parent can't be deleted mid-insert); deleting a parent must check the child table — **if there's no index on the child FK column, that check is a sequential scan per delete** and cascades take row locks on every child. This is the #1 FK performance trap.
- MySQL/InnoDB: the table *is* the PK B+Tree (clustered). Random PKs (UUIDv4) cause page splits and buffer-pool churn; sequential PKs append. Postgres heaps don't cluster, but UUIDv4 still bloats indexes and wrecks cache locality → prefer `bigint identity` or time-ordered UUIDv7/ULID.

## 4. Visualization (ASCII)

```
FK check on INSERT child(order_id=7):
 child row ──lookup──▶ parent PK index ──▶ row 7 exists?
                                           │ yes: lock FOR KEY SHARE, proceed
                                           │ no : ERROR fk violation
DELETE parent(7):
 parent ──must check──▶ child table WHERE order_id=7
     with index on child.order_id : B+Tree probe  ✔ fast
     without index                : FULL SCAN per delete ✖ + long row locks

UUIDv4 inserts into B+Tree:            bigint/UUIDv7 inserts:
 scattered ▼   ▼    ▼   (page splits)   append ▶▶▶ (rightmost leaf, hot cache)
[..][..][..][..][..]                    [..][..][..][▶]
```

## 5. Real Production Example

**Uber-style shard-out**: FKs can't span shards, so at extreme scale companies drop DB-level FKs and enforce integrity in services + async auditors — a *deliberate* trade of integrity for scalability/ownership. Conversely, **Stripe-style ledgers** keep FKs and constraints exactly because money rows must never orphan. Know both positions and when each is right.

## 6. Common Interview Questions

- "PK vs unique key — all differences?" (NULLs, count, FK target, clustering in InnoDB)
- "UUID or auto-increment for the PK?" (discuss: guessability, sharding/merge-ability, B+Tree locality; answer: bigint internally or UUIDv7 if globally unique needed)
- "Why index foreign key columns?" (delete/cascade scans + join performance)
- "Would you use FKs at massive scale?" (trade-off discussion, not yes/no)

## 7. Common Mistakes

- Forgetting Postgres does **not** auto-index the child side of an FK (it only requires the *parent* side to be unique). Missing child indexes → slow deletes, lock pileups.
- Choosing natural PKs (email) that later change → cascading updates everywhere.
- UUIDv4 PKs on write-heavy InnoDB tables → page-split storm.
- `ON DELETE CASCADE` on huge child tables → one parent delete silently deletes millions of rows in one transaction (lock + WAL explosion).

## 8. Best Practices

- `bigint GENERATED ALWAYS AS IDENTITY` internal PK; expose a separate public ID (UUIDv7/ULID) if IDs leave the system.
- Index every FK column; add composite indexes that *start* with the FK when you filter further.
- Prefer `ON DELETE RESTRICT` + explicit application deletes for big graphs; use soft deletes where audit matters.
- Enforce natural uniqueness with unique constraints, not application checks (race-proof).

## 9. Coding Questions

1. Find FK columns missing indexes (join `pg_constraint` contype='f' against `pg_index` — write the catalog query).
2. Enforce "a user has at most one active subscription" — `CREATE UNIQUE INDEX ON subscriptions(user_id) WHERE status = 'active';` (partial unique).

## 10. SQL Examples

```
CREATE TABLE users (
  id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  public_id uuid NOT NULL DEFAULT gen_random_uuid() UNIQUE,  -- exposed ID
  email citext NOT NULL UNIQUE                               -- natural key guarded
);

CREATE TABLE orders (
  id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  user_id bigint NOT NULL REFERENCES users(id) ON DELETE RESTRICT
);
CREATE INDEX ON orders (user_id);          -- FK index: NOT automatic in Postgres

-- Race-proof "insert if new" on a natural key
INSERT INTO users (email) VALUES ('a@b.com')
ON CONFLICT (email) DO UPDATE SET email = EXCLUDED.email
RETURNING id;
```

## 11. Optimization Techniques

- Time-ordered keys (identity, UUIDv7) keep B+Tree inserts append-only → fewer splits, hot rightmost pages cached.
- Add FKs to existing big tables with `NOT VALID` then `VALIDATE CONSTRAINT` (validation scans without blocking writes).
- Batch parent deletes; delete children first in chunks to bound transaction size.

## 12. Follow-up Questions

- "How do you generate unique IDs across shards?" (Snowflake IDs: timestamp+worker+sequence; or UUIDv7; or per-shard ranges)
- "Deferrable constraints — when?" (swap-type updates where uniqueness is violated mid-txn: `DEFERRABLE INITIALLY DEFERRED`)
- "What lock does an FK insert take on the parent, and what does it block?" (`FOR KEY SHARE`; blocks parent key updates/deletes, not normal column updates)

---

# Chapter 1.8 — Transactions (Introduction)

*(Full concurrency treatment — isolation levels, MVCC, locks, deadlocks — is Module 6. This chapter covers what belongs in "fundamentals" rounds.)*

## 1. Why Interviewers Ask This

Transactions are where correctness meets performance. Fundamentals rounds check you know the lifecycle and the *shape* of correct transactional code: short, idempotent, retry-aware.

## 2. Core Concept

A transaction is the unit of atomicity, isolation and durability: `BEGIN` → statements → `COMMIT` (all effects visible atomically, durable) or `ROLLBACK` (no effects). Postgres runs *every* statement in a transaction (autocommit wraps single statements). `SAVEPOINT` gives partial rollback inside a transaction.

Three rules of production transactions: 1. **Short** — hold locks & snapshots for milliseconds, never across network calls or user think-time. 2. **Idempotent/retryable** — serialization failures (40001) and deadlocks (40P01) are *normal*; the app must retry. 3. **Right-sized** — exactly the rows that must change together; no more (lock contention), no less (broken invariants).

## 3. Internal Working

- `BEGIN` assigns a virtual txn ID; the first write gets a real **XID**.
- Every written row version is stamped `xmin=XID` (creator); deletes/updates stamp `xmax` (deleter) on the old version — updates are delete+insert in MVCC (Module 6).
- `COMMIT`: WAL commit record fsynced → commit bit set in `pg_xact` → all versions atomically visible to later snapshots.
- `ROLLBACK`: cheap — just mark the XID aborted; dead versions swept by VACUUM later.
- Statement failure inside a txn aborts the whole txn in Postgres (must ROLLBACK or use SAVEPOINTs) — unlike MySQL which by default aborts only the statement.

## 4. Visualization (ASCII)


## 5. Real Production Example

A checkout at **Amazon/Stripe** scale: reserve inventory, create order, record payment intent — one transaction against one DB, *but* the external card-network call happens OUTSIDE the transaction (never hold row locks across an HTTP call — the classic incident is a payment provider slowdown turning into a database lock pileup). Pattern: txn1 = write `pending` + outbox row → call provider → txn2 = finalize.

## 6. Common Interview Questions

- "What happens if a transaction fails halfway?" (nothing persists; how: WAL/MVCC)
- "Why are long transactions bad?" (hold locks; block VACUUM → bloat; snapshot forces keeping dead versions; replication conflicts)
- "How do you call an external API 'inside' a transaction?" (you don't — outbox pattern)
- "Difference between ROLLBACK and crash recovery?" (same end state; rollback is explicit mark, recovery replays WAL and discards uncommitted)

## 7. Common Mistakes

- Transactions spanning user interaction ("BEGIN, show form, COMMIT on submit") — lock disaster.

- No retry logic: treating 40001/deadlock as a bug instead of a signal to retry.
- Doing reads that don't need transactional consistency inside the write transaction, inflating its duration.
- Assuming autocommit batches are atomic — 1,000 separate INSERTs without BEGIN are 1,000 transactions.

## 8. Best Practices

- Wrap multi-statement invariants in explicit transactions; keep them under ~100ms.
- Acquire locks in a **consistent order** across code paths (deadlock prevention — Module 6).
- Retry on `40001`/`40P01` with jittered backoff, bounded attempts, idempotent statements.
- Use the outbox pattern for DB-write + message-publish atomicity.

## 9. Coding Questions

1. Write the inventory-safe checkout below and explain why the `CHECK`/`WHERE` guard prevents overselling even under concurrency.
2. Implement retry-on-serialization-failure pseudocode around a transaction block.

## 10. SQL Examples

```
-- Oversell-proof decrement (atomic check-and-set in one statement)
BEGIN;
UPDATE inventory
SET quantity = quantity - 1
WHERE product_id = 77 AND quantity >= 1;
-- rowcount = 0 → out of stock → ROLLBACK and tell the user
INSERT INTO orders (user_id, product_id) VALUES (42, 77);
COMMIT;

-- Savepoints: tolerate a failing optional step
BEGIN;
INSERT INTO orders ...;
SAVEPOINT coupon;
INSERT INTO coupon_redemptions ...;   -- may violate "already used" unique
-- on error: ROLLBACK TO SAVEPOINT coupon;  (order survives)
COMMIT;
```

## 11. Optimization Techniques

- Combine per-row round trips into set-based statements (one UPDATE with a join beats N updates).
- Move derivable work (counters, projections) out of the critical transaction to async consumers when the invariant allows.
- Monitor long transactions: `SELECT * FROM pg_stat_activity WHERE xact_start < now() - interval '1 min';`

## 12. Follow-up Questions

- "Two replicas of your service run this transaction concurrently — walk me through the race." (leads into isolation levels → Module 6)
- "How does this transaction behave at READ COMMITTED vs SERIALIZABLE?"
- "What's in the WAL for this transaction, and when is it safe to tell the user 'saved'?"

---

# Module 1 — Practice Problems

## Easy (5)

1. List the ACID properties and, for each, the single Postgres mechanism that implements it.
2. A table stores `tags` as a comma-separated string. Which normal form is violated, and what two problems will you hit? Rewrite the schema.
3. State the difference between a primary key and a unique constraint regarding NULLs, count per table, and FK targeting.
4. Your read replica shows an order 800ms after the primary commits it. Name the consistency model the user experiences and one fix for "user doesn't see their own order."
5. Classify each as OLTP or OLAP and pick row-store or column-store: (a) fetch cart by user_id, (b) revenue by country by month for 3 years, (c) update shipment status, (d) funnel analysis over 2B events.

## Medium (5)

1. Design the consistency policy for a food-delivery app: order placement, driver GPS location, restaurant menu, payment capture. For each: ACID or BASE, and the user-visible anomaly you accept.
2. Cassandra RF=3. For (W=1,R=1), (W=2,R=2), (W=3,R=1): state whether reads are strongly consistent, and what happens to writes when one replica is down.
3. A parent table `merchants` (10k rows) and child `payments` (2B rows, FK `merchant_id`, no index on it). Deleting one merchant takes 40 minutes and blocks writes. Explain exactly why and give two fixes.
4. You must add `orders.user_email` (denormalized from `users`) to kill a hot join. Specify the sync mechanism, the failure mode, and the reconciliation query.
5. Your API does: BEGIN → INSERT payment → call card network (p99 = 8s) → UPDATE status → COMMIT. Production shows lock waits and connection exhaustion. Redesign it.

## Hard (5)

1. Design read-your-own-writes for a Postgres primary with 5 async replicas behind a load balancer, without pinning all traffic to the primary. (LSN token flow: capture on write, compare on replica, fallback route.)
2. Prove with a two-transaction interleaving that "check balance, then insert withdrawal in a second statement" oversells at READ COMMITTED, then give three distinct fixes (atomic UPDATE guard, `SELECT ... FOR UPDATE`, SERIALIZABLE + retry) with their costs.
3. You're sharding `orders` by `user_id` across 8 Postgres nodes. Enumerate everything that breaks: FKs to shared tables, unique constraints on `order_number`, multi-user reports, transactions spanning users — and give the standard mitigation for each.
4. A migration from auto-increment bigint to UUIDv4 PKs cut insert throughput 5x on MySQL but only ~20% on Postgres. Explain the asymmetry (clustered PK page splits vs heap+index bloat) and the fix that keeps global uniqueness (UUIDv7/ULID).
5. Sketch a transactional outbox end-to-end: DDL for the outbox table, the write transaction, the relay (poll vs logical decoding), delivery guarantees, and how consumers deduplicate.

---

*Next: Module 2 — SQL Core*

# MODULE 2 — SQL Core

> The screening-round module. Senior candidates fail SQL screens not on syntax but on **logical execution order, NULL semantics, and window functions**. Those three carry this module.

Chapters: 2.1 Logical Query Execution Order (SELECT, WHERE, GROUP BY, HAVING, ORDER BY, LIMIT, DISTINCT, CASE) 2.2 Subqueries, Correlated Subqueries, EXISTS / IN / ANY / ALL 2.3 CTEs & Recursive CTEs 2.4 Views & Materialized Views 2.5 UNION vs UNION ALL (Set Operations) 2.6 Window Functions (the interview kingmaker) 2.7 Aggregate, Date & String Functions

---

## Chapter 2.1 — Logical Query Execution Order

### 1. Why Interviewers Ask This

80% of "why is this query wrong?" screen questions reduce to execution order: why you can't use a SELECT alias in WHERE, why HAVING exists, how DISTINCT interacts with ORDER BY. It's also the mental model you need to reason about what the optimizer may rearrange.

### 2. Core Concept

SQL is written `SELECT ... FROM ... WHERE ...` but **evaluated logically** as:

```
FROM / JOIN  →  WHERE  →  GROUP BY  →  HAVING  →  SELECT (expressions, aliases)
→  DISTINCT  →  ORDER BY  →  LIMIT / OFFSET
```

Consequences you must be able to recite: - **WHERE** runs before grouping → cannot reference aggregates (`WHERE count(*) > 5` is invalid) → that's **HAVING**'s job. - **SELECT aliases** don't exist yet in WHERE/GROUP BY/HAVING (Postgres allows them in GROUP BY/ORDER BY as an extension — don't rely on it in interviews). - **ORDER BY** runs after SELECT → it *can* use aliases and, with DISTINCT, may only order by selected expressions. - **LIMIT without ORDER BY** returns *arbitrary* rows — a classic trap. - **WHERE filters rows; HAVING filters groups.** Put every non-aggregate condition in WHERE (it prunes before the expensive grouping).

**CASE** is an expression (not control flow), usable anywhere an expression goes — including inside aggregates (`SUM(CASE WHEN ... THEN 1 ELSE 0 END)` = conditional counting, better written in Postgres as `COUNT(*) FILTER (WHERE ...)`).

### 3. Internal Working

The planner turns logical order into a physical plan and may reorder aggressively as long as results are equivalent: predicates are pushed down below joins, `LIMIT` can stop an index scan early (top-N), `DISTINCT`/`GROUP BY` become HashAggregate or Sort+Unique, `ORDER BY` is satisfied either by an explicit Sort node (spills to disk beyond `work_mem`) or **for free by an index** that already returns rows in order. `ORDER BY x LIMIT 10` with an index on `x` = read 10 index entries and stop — this powers all fast pagination (Module 5).

### 4. Visualization (ASCII)

```
written order            logical evaluation          physical plan (example)
SELECT c, count(*)    ┌─▶ FROM orders                  Limit(10)
FROM orders           │   WHERE status='paid'           └─ Sort DESC / or IndexScan
WHERE status='paid'   │   GROUP BY c                       └─ HashAggregate
GROUP BY c            │   HAVING count(*)>5                   └─ SeqScan
HAVING count(*)>5     │   SELECT c, count(*)                     Filter: status='paid'
ORDER BY 2 DESC       │   ORDER BY count DESC
LIMIT 10              └─  LIMIT 10
alias visibility: WHERE ✖ | GROUP BY ✖(std) | HAVING ✖ | ORDER BY ✔
```

## 5. Real Production Example

A Netflix-style dashboard query: "top 10 titles by completed plays yesterday." The difference between `WHERE event_date = yesterday` (prunes billions of rows before aggregation, hits the partition) and putting the date check in HAVING-adjacent logic (aggregating everything, then filtering) is minutes vs milliseconds. Filter-early is not style — it's the whole game.

## 6. Common Interview Questions

- "Why does `WHERE total_spend > 100` fail when `total_spend` is a SELECT alias?"
- "Difference between WHERE and HAVING? Which is faster for a non-aggregate condition and why?"
- "What does LIMIT 1 without ORDER BY return?"
- "Count paid vs unpaid orders in one query." (CASE/FILTER inside aggregates)
- "What's the logical order of clause evaluation?" (recite it)

## 7. Common Mistakes

- `HAVING status = 'paid'` — works if `status` is grouped, but forces the condition after aggregation; belongs in WHERE.
- `SELECT DISTINCT user_id ORDER BY created_at` — invalid/ambiguous: which `created_at` for a deduped user? (Postgres errors; fix with GROUP BY + aggregate, or `DISTINCT ON`.)
- Relying on implicit ordering ("it always came back sorted") — plans change, order changes.
- `WHERE x <> 'a'` silently dropping NULL rows (NULL comparisons are UNKNOWN → filtered). NULL handling: use `IS DISTINCT FROM` for null-safe inequality.

## 8. Best Practices

- Push every sargable condition into WHERE; reserve HAVING strictly for aggregate conditions.
- Always pair LIMIT with a **deterministic** ORDER BY (add a unique tiebreaker column: `ORDER BY created_at DESC, id DESC`).
- Prefer `COUNT(*) FILTER (WHERE ...)` over `SUM(CASE ...)` in Postgres — clearer, same plan.
- Know `DISTINCT ON (col)` (Postgres): "one row per key, pick which by ORDER BY" — the idiomatic greatest-n-per-group tool.

## 9. Coding Questions

1. One scan, one row per country: total orders, paid orders, paid ratio, only countries with

   > 1000 orders, top 10 by paid ratio.

2. "Latest order per user" three ways: `DISTINCT ON`, window function `ROW_NUMBER`, and a correlated subquery — and rank them by expected performance with an index on `(user_id, created_at DESC)`.

### 10. SQL Examples

```
-- Conditional aggregation + correct clause placement
SELECT u.country,
       count(*)                                    AS orders,
       count(*) FILTER (WHERE o.status = 'paid')  AS paid_orders,
       round(count(*) FILTER (WHERE o.status='paid')::numeric / count(*), 3) AS paid_ratio
FROM orders o JOIN users u ON u.id = o.user_id
WHERE o.created_at >= now() - interval '30 days'     -- row filter: BEFORE grouping
GROUP BY u.country
HAVING count(*) > 1000                                -- group filter: aggregates only
ORDER BY paid_ratio DESC
LIMIT 10;

-- DISTINCT ON: latest order per user (Postgres idiom)
SELECT DISTINCT ON (user_id) user_id, id, created_at
FROM orders
ORDER BY user_id, created_at DESC;

-- CASE as expression: bucketing
SELECT CASE WHEN total_cents < 1000  THEN 'small'
            WHEN total_cents < 10000 THEN 'medium'
            ELSE 'large' END AS bucket,
       count(*)
FROM orders GROUP BY 1;
```

### 11. Optimization Techniques

- Make ORDER BY free: index matching the sort (`(created_at DESC, id DESC)`) lets `ORDER BY ... LIMIT k` read k entries and stop.
- Filter before aggregating; aggregate before joining when the join is only for labels (aggregate the fact table, then join dimension tables).
- Watch `Sort Method: external merge Disk:` in EXPLAIN ANALYZE — raise `work_mem` for that session or add the index.

### 12. Follow-up Questions

- "Your GROUP BY query spills to disk — what are your options?" (work_mem, pre-filter, index-ordered GroupAggregate, pre-aggregated rollup)
- "Why can `LIMIT` change which join algorithm wins?" (startup cost vs total cost — nested loop streams first rows early; hash join must build first)
- "How would you make `paid_ratio` per country available at 10ms p99?" (rollup table / materialized view — Chapter 2.4)

---

## Chapter 2.2 — Subqueries, Correlated Subqueries, EXISTS / IN / ANY / ALL

### 1. Why Interviewers Ask This

They test two things: can you express "rows that have / don't have related rows" correctly, and do you know the **NOT IN + NULL** landmine and the performance difference between correlated and uncorrelated forms.

### 2. Core Concept

- **Scalar subquery**: returns one value — usable anywhere an expression goes.
- **Uncorrelated subquery**: independent of the outer row; executed once.
- **Correlated subquery**: references outer-row columns; *logically* re-executed per outer row.
- **EXISTS (SELECT 1 ...)**: true if the subquery returns ≥1 row; stops at the first match (semi-join). `NOT EXISTS` = anti-join.
- **IN (subquery)**: membership test. **`NOT IN` with any NULL in the subquery returns zero rows** — because `x <> NULL` is UNKNOWN, so no row can ever pass. The most famous SQL trap.

- **ANY/SOME**: `x = ANY(sub)` ≡ `x IN (sub)`; useful with other operators (`> ANY` = greater than the minimum). **ALL**: must hold vs every row (`> ALL` = greater than the maximum; vacuously true on empty set — second trap).

## 3. Internal Working

Postgres rewrites aggressively: - `IN`/`EXISTS` subqueries → **semi-joins** (hash or nested-loop) — usually identical plans, so the "EXISTS is faster than IN" folklore is mostly false *in Postgres* (was true in old MySQL). - `NOT EXISTS` → **anti-join** (efficient). `NOT IN` often *cannot* become an anti-join unless the planner proves non-nullability → materializes the subquery and probes with three-valued logic → slower **and** semantically dangerous. - Correlated subqueries the planner can't decorrelate become a **SubPlan**: executed per outer row (O(N×cost)). EXPLAIN shows `SubPlan` — a red flag on big outer sets unless the inner probe is a cheap index hit or it's a *hashed* SubPlan.

## 4. Visualization (ASCII)

```
Semi-join (EXISTS/IN):                    Correlated SubPlan:
outer row ──► probe inner hash/index      for each of 1,000,000 outer rows:
  match found? emit once, stop              run inner query (index probe: OK,
  (never duplicates outer rows)              seq scan: catastrophe)

NOT IN with NULL:
ids in subquery: {1, 2, NULL}
x NOT IN (1,2,NULL) ⇒ x<>1 AND x<>2 AND x<>NULL
                                  └── UNKNOWN ⇒ whole predicate UNKNOWN ⇒ row dropped
RESULT: empty set, silently.  Use NOT EXISTS.
```

## 5. Real Production Example

Churn query at a Stripe-like company: "customers with no successful payment in 90 days." Written with `NOT IN (SELECT customer_id FROM payments ...)`, it returned zero rows for a week because one legacy payment row had `customer_id NULL` — a silent correctness incident, not a performance one. Rewritten with `NOT EXISTS`, correct and anti-join fast.

## 6. Common Interview Questions

- "Find users with no orders" — expected: `NOT EXISTS` (or LEFT JOIN ... IS NULL), and *why not* `NOT IN`.
- "EXISTS vs IN — semantics and performance?"
- "When does a correlated subquery hurt, and how do you rewrite it?" (as a join or window function)
- "What does `salary > ALL (SELECT salary FROM emp WHERE dept='X')` return when dept X is empty?" (all rows — vacuous truth)

## 7. Common Mistakes

- `NOT IN` against a nullable column — the #1 trap; know it cold.
- Correlated aggregate per row (`(SELECT count(*) FROM orders o WHERE o.user_id = u.id)`) on a large outer table without an index on `orders(user_id)`.
- Using `IN` to deduplicate a join — `EXISTS` never multiplies rows; a JOIN does.
- Scalar subquery returning >1 row → runtime error that only appears with production data.

## 8. Best Practices

- Default to `EXISTS`/`NOT EXISTS` for presence/absence — null-safe, dup-safe, plans well.
- Rewrite per-row correlated aggregates as a grouped join or window function when reading many rows.
- `= ANY(array)` is the idiomatic Postgres way to bind a list parameter (`WHERE id = ANY($1::bigint[])`).
- Keep subqueries sargable: correlate on indexed columns.

## 9. Coding Questions

1. "Users whose *every* order is over $100" — two ways: `NOT EXISTS` (an order ≤ 100) and `100 < ALL (subquery)`; explain the empty-set semantics difference (users with no orders).

2. Rewrite this per-row subquery as a join: `SELECT u.*, (SELECT max(created_at) FROM orders o WHERE o.user_id=u.id) FROM users u;`

### 10. SQL Examples

```
-- Presence: users with at least one refunded order (semi-join, no duplicates)
SELECT u.* FROM users u
WHERE EXISTS (SELECT 1 FROM orders o
              WHERE o.user_id = u.id AND o.status = 'refunded');

-- Absence done safely (anti-join)
SELECT u.* FROM users u
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.user_id = u.id);

-- The trap, for contrast: returns 0 rows if ANY orders.user_id IS NULL
SELECT u.* FROM users u
WHERE u.id NOT IN (SELECT o.user_id FROM orders o);

-- Correlated → join rewrite
SELECT u.id, u.name, m.last_order_at
FROM users u
LEFT JOIN (SELECT user_id, max(created_at) AS last_order_at
           FROM orders GROUP BY user_id) m ON m.user_id = u.id;

-- ANY with an array parameter
SELECT * FROM orders WHERE status = ANY(ARRAY['paid','shipped']);
```

### 11. Optimization Techniques

- Check EXPLAIN for `SubPlan` (per-row execution) vs `Hash Semi Join`/`Hash Anti Join` (set-based) — rewrite until you get the latter on large inputs.
- Ensure the correlated column is the leading column of an index on the inner table.
- For "top/latest per group" prefer `DISTINCT ON` or window functions over correlated `= (SELECT max(...))` — one scan instead of N probes.

### 12. Follow-up Questions

- "Why can't the planner turn NOT IN into an anti-join here?" (nullable column; add NOT NULL or `WHERE sub.col IS NOT NULL` and re-plan)
- "The EXISTS probe is fast but runs 10M times — what now?" (decorrelate: aggregate inner once, hash join)
- "Semi-join vs inner join + DISTINCT — result and cost differences?" (same rows here, but DISTINCT pays for dedup after multiplication)

---

## Chapter 2.3 — CTEs & Recursive CTEs

### 1. Why Interviewers Ask This

CTEs test query decomposition; **recursive CTEs are the only standard SQL for hierarchies/graphs** (org charts, category trees, dependency chains) — a fixture of Amazon/Google SQL rounds. Bonus signal: knowing the Postgres 12 materialization change.

### 2. Core Concept

- `WITH name AS (...)` names a subquery for reuse and readability; CTEs can chain (later CTEs reference earlier ones).
- **Postgres ≥12**: CTEs referenced once are **inlined** (predicates push into them, indexes usable). Referenced multiple times, containing writes (`INSERT ... RETURNING`), or marked `AS MATERIALIZED` → computed once into a tuplestore. `AS NOT MATERIALIZED` forces inlining. Pre-12 (and lore you'll hear): CTEs were *always* optimization fences.
- **Recursive CTE** = iteration until fixpoint:

```
WITH RECURSIVE r AS (
  <base case / anchor>
  UNION ALL
  <recursive step referencing r>   -- runs on the PREVIOUS iteration's rows only
)
SELECT * FROM r;
```

`UNION` (vs `UNION ALL`) additionally dedups each iteration — that's how you survive cycles.

## 3. Internal Working

Executor keeps a **working table** (last iteration's rows). Each step joins the recursive term against the working table only (not all accumulated rows), appends results to the output and makes them the next working table; stops when an iteration yields zero rows. Total cost ≈ depth × (per-level join cost) — so an index on the join key (`parent_id`) is mandatory. Cycle without protection = infinite loop; Postgres 14+ adds `CYCLE col SET is_cycle USING path`.

## 4. Visualization (ASCII)

```
employees(id, manager_id): reports chain of CEO(1)
iter 0 (anchor): {1}
iter 1: children of {1}        → {2,3}
iter 2: children of {2,3}      → {4,5,6}
iter 3: children of {4,5,6}    → {}  → STOP
output = 1,2,3,4,5,6 with depth/path accumulated per row

working table (small, last level only) ──join on parent_id──▶ employees (indexed)
```

## 5. Real Production Example

Amazon-style product taxonomy: "all products under 'Electronics'" where categories nest arbitrarily deep. Recursive CTE walks category → descendants, then joins products. At very high read volume, teams precompute a **closure table** or use `ltree`/materialized paths — a follow-up interviewers love ("what if this tree is read a million times a minute?").

## 6. Common Interview Questions

- "Employees under manager X, with depth and path." (the canonical one)
- "Detect a cycle in the reporting chain."
- "Generate a row per day for the last 30 days and left-join sales onto it" (recursive CTE or `generate_series` — know both; the series version is the Postgres answer).
- "Are CTEs optimization fences?" (version-aware answer = senior signal)

## 7. Common Mistakes

- `UNION ALL` on cyclic data → infinite recursion (killed only by memory/`statement_timeout`).
- Filtering *outside* the recursion what could be filtered *inside* (walk the whole tree, then discard — push predicates into the recursive term when semantics allow).
- Assuming the recursive term can reference `r` twice or use aggregation over it — not allowed.
- Using a chain of CTEs where each one scans a huge table separately instead of reusing one scan.

## 8. Best Practices

- Always carry `depth` and a `path` array — costs little, answers the follow-ups (ordering, cycle check via `id = ANY(path)`).
- Cap depth defensively: `WHERE depth < 20` in the recursive term.
- Name CTEs like functions (`active_users`, `daily_totals`) — interviewers grade readability.
- Know the alternatives for hot hierarchies: closure table, materialized path, `ltree`.

## 9. Coding Questions

1. Org chart: all reports of employee 42 with depth and full path, cycle-safe.

2. Running dependency resolution: given `tasks(id, depends_on)`, output a valid execution order (topological sort via recursive CTE with depth, order by max depth).

### 10. SQL Examples

```
-- Subordinates with depth & path, cycle-safe
WITH RECURSIVE reports AS (
  SELECT id, name, manager_id, 1 AS depth, ARRAY[id] AS path
  FROM employees WHERE id = 42
  UNION ALL
  SELECT e.id, e.name, e.manager_id, r.depth + 1, r.path || e.id
  FROM employees e
  JOIN reports r ON e.manager_id = r.id
  WHERE NOT e.id = ANY(r.path)          -- cycle guard
    AND r.depth < 20                    -- depth cap
)
SELECT * FROM reports ORDER BY path;

-- Calendar spine + left join (report with zero-filled days)
SELECT d::date AS day, coalesce(sum(o.total_cents), 0) AS revenue
FROM generate_series(current_date - 29, current_date, interval '1 day') d
LEFT JOIN orders o ON o.created_at::date = d::date
GROUP BY 1 ORDER BY 1;

-- Materialization control (Postgres 12+)
WITH heavy AS MATERIALIZED (SELECT ... expensive, reused 3x ...)
SELECT ... FROM heavy h1 JOIN heavy h2 ON ...;
```

### 11. Optimization Techniques

- Index the recursion join key (`employees(manager_id)`) — turns each level into index probes.
- For read-heavy trees: closure table (`ancestor, descendant, depth` rows) trades O(depth) recursion for O(1) lookup at write-time cost.
- Use `NOT MATERIALIZED` when a filter from the outer query should push into the CTE.

### 12. Follow-up Questions

- "This tree has 50M nodes and the query runs 10k times/sec — now what?" (closure table / cached subtree sets / denormalized path column)
- "How do you paginate a recursive result stably?" (order by path, keyset on path)
- "Same query in MySQL?" (MySQL 8.0 `WITH RECURSIVE` — nearly identical; know it exists)

---

## Chapter 2.4 — Views & Materialized Views

### 1. Why Interviewers Ask This

Views test whether you know the difference between an **abstraction** (view = stored query) and a **precomputation** (matview = stored result) — and the staleness/refresh trade-off is a mini CAP question inside SQL.

### 2. Core Concept

| | View | Materialized View |
|---|---|---|
| Storage | None — macro expanded at plan time | Real table of results on disk |
| Freshness | Always current | Stale until `REFRESH` |
| Read cost | Cost of the underlying query | Cost of reading a (indexable!) table |
| Write path | N/A (simple views auto-updatable) | Refresh: full recompute in PG |
| Use | API stability, security (column/row hiding), DRY | Expensive aggregates read often, written rarely |

## 3. Internal Working

- View: rewriter splices the view definition into the query tree, then the planner optimizes the *whole* thing — predicates push into the view; performance identical to writing the full query (footgun: stacking views 5 deep hides a monster join).
- Matview: `REFRESH MATERIALIZED VIEW` re-runs the query and swaps the heap (locks readers); `REFRESH ... CONCURRENTLY` diffs against the old contents using a **unique index** (required) and applies deltas without blocking reads — slower to run, non-blocking to readers. Postgres has no built-in incremental matview maintenance (unlike Oracle) — that's what rollup-table + trigger/CDC patterns are for.

## 4. Visualization (ASCII)

```
View:    query ──rewrite──► [query ⋈ view definition] ──plan──► execute (fresh, full cost)

Matview:                     write path            read path
 base tables ──REFRESH──► [stored result table] ◄──index scan, ms
                 ▲ cron/trigger                    (stale by up to refresh interval)
CONCURRENTLY: new snapshot ──diff via unique idx──► apply deltas (readers unblocked)
```

## 5. Real Production Example

A LinkedIn-style "profile view stats" page: counting 90 days of view events per user at request time = OLAP query on the OLTP path. Matview (or rollup table) refreshed every 10 minutes serves it in single-digit ms; product accepts the staleness and the UI says "updated recently." That sentence — *the product accepted staleness* — is what interviewers want to hear.

## 6. Common Interview Questions

- "View vs materialized view — when each?"
- "Does a view improve performance?" (no — it's the same query; matviews do)
- "How do you refresh a matview without downtime?" (`CONCURRENTLY` + unique index)
- "How would you keep a matview near-real-time?" (you wouldn't — incremental rollup via triggers/CDC, or streaming aggregation)

## 7. Common Mistakes

- Believing views cache results.
- Matview without a unique index → `REFRESH CONCURRENTLY` fails; plain refresh blocks reads.
- View-on-view-on-view stacks where nobody can find the 12-way join anymore.
- Using matviews for data that must be fresh (inventory counts at checkout).

## 8. Best Practices

- Views for security boundaries (expose `users_public` without PII) and query DRY.
- Matviews for expensive, read-heavy, staleness-tolerant aggregates; always index them like tables.
- Schedule refreshes off-peak; monitor refresh duration growth (it's a full recompute).
- When freshness matters, prefer an incrementally-maintained rollup table.

## 9. Coding Questions

1. Create a security view exposing users without email/phone and grant a read-only role access to the view but not the table.
2. Build `daily_revenue` as a matview with the unique index required for concurrent refresh.

### 10. SQL Examples

```
-- Security view
CREATE VIEW users_public AS
SELECT id, display_name, country, created_at FROM users;
GRANT SELECT ON users_public TO analyst_role;

-- Materialized aggregate + concurrent refresh
CREATE MATERIALIZED VIEW daily_revenue AS
SELECT created_at::date AS day,
       count(*) AS orders,
       sum(total_cents) AS revenue_cents
FROM orders GROUP BY 1;

CREATE UNIQUE INDEX ON daily_revenue (day);      -- required for CONCURRENTLY
REFRESH MATERIALIZED VIEW CONCURRENTLY daily_revenue;

-- Incremental rollup alternative (fresh, no full recompute)
CREATE TABLE daily_revenue_rt (day date PRIMARY KEY, orders bigint, revenue_cents bigint);
INSERT INTO daily_revenue_rt VALUES (current_date, 1, 4999)
ON CONFLICT (day) DO UPDATE
SET orders = daily_revenue_rt.orders + 1,
    revenue_cents = daily_revenue_rt.revenue_cents + EXCLUDED.revenue_cents;
```

### 11. Optimization Techniques

- Index matviews on their query patterns — they're tables.
- Partition-wise refresh: split the matview per time range so only recent partitions recompute.
- If the view is hot and simple, check the plan — sometimes a covering index on the base table beats maintaining a matview at all.

### 12. Follow-up Questions

- "Refresh takes 30 min and runs hourly — trajectory and fix?" (incremental rollup / only recompute recent window)
- "How does MySQL handle this?" (no matviews — summary tables by hand)
- "Row-level security vs views for multi-tenant isolation?" (RLS policies are enforced on the table for all paths; views can be bypassed by direct table access)

---

## Chapter 2.5 — UNION vs UNION ALL (Set Operations)

### 1. Why Interviewers Ask This

Small topic, reliable trap: the hidden cost of deduplication, and NULL/type alignment rules. Also a favorite in "combine data from two sources" questions.

### 2. Core Concept

- `UNION ALL`: concatenate results. No dedup. Cheap — essentially free append.
- `UNION`: concatenate **then deduplicate whole rows** (implicit DISTINCT across the combined set).
- `INTERSECT` / `EXCEPT`: rows in both / in first-not-second (also dedup by default; `ALL` variants exist).
- Rules: same column count, compatible types; column names come from the **first** branch; `ORDER BY` applies to the final combined result only.
- Set ops treat `NULL = NULL` as equal for dedup purposes (unlike `=` in WHERE) — a subtle interview point.

### 3. Internal Working

`UNION ALL` = Append node streaming children (and enables partition-style pruning per branch: branches whose WHERE contradicts the outer predicate can be skipped). `UNION` = Append + HashAggregate (or Sort+Unique) over *all* columns of the combined set — memory/spill cost proportional to total rows. On 100M+2M rows, that dedup is the whole query cost.

## 4. Visualization (ASCII)

```
UNION ALL                          UNION
  Append ──► out                    HashAggregate(all cols)  ← dedup cost lives here
    ├─ scan A (stream)                └─ Append
    └─ scan B (stream)                    ├─ scan A
rows: |A|+|B|, streaming                  └─ scan B
                                   rows: distinct(|A|+|B|), blocking, can spill
```

## 5. Real Production Example

Uber-style "activity feed" merging `trips`, `eats_orders`, `payments` into one timeline: `UNION ALL` with a `source` tag column, ordered by time, keyset-paginated. Using `UNION` here would burn CPU deduplicating rows that *cannot* collide (different sources) — a real code-review catch that interviewers simulate.

## 6. Common Interview Questions

- "UNION vs UNION ALL — difference and performance?"
- "When is UNION actually required?" (true duplicates across branches must collapse)
- "JOIN vs UNION — when each?" (JOIN widens columns; UNION stacks rows)
- "Simulate FULL OUTER JOIN with UNION" (left join UNION ALL right-anti part — MySQL <8 lacks FULL JOIN)

## 7. Common Mistakes

- Defaulting to `UNION` "to be safe" — silently deduplicates legitimate duplicate business rows *and* pays the sort/hash.
- Mismatched column order across branches (types align positionally — `id, name` vs `name, id` may still run if types coerce, returning garbage).
- Putting `ORDER BY`/`LIMIT` intended for one branch without parentheses — it binds to the whole set operation.

## 8. Best Practices

- Default `UNION ALL`; use `UNION` only when dedup is a stated requirement.
- Add a discriminator column (`'trip' AS source`) when merging heterogeneous rows.
- Parenthesize branches when applying per-branch ORDER BY/LIMIT.
- For "top 10 across N sources": take top 10 per branch (indexed), UNION ALL, then top 10 overall — bounds the work.

## 9. Coding Questions

1. Merge `web_events` and `mobile_events` into one ordered, keyset-paginated feed with a source tag.
2. Given `employees_2024` and `employees_2025`, produce: joined both years (INTERSECT), left in 2025 (EXCEPT), and explain the NULL-equality behavior if `team` is nullable.

## 10. SQL Examples

```sql
-- Heterogeneous feed
SELECT 'trip'  AS source, id, created_at, driver_id::text AS ref FROM trips   WHERE user_id = 42
UNION ALL
SELECT 'order' AS source, id, created_at, restaurant::text      FROM eats     WHERE user_id = 42
ORDER BY created_at DESC
LIMIT 20;

-- Per-branch limits done right
(SELECT * FROM trips ORDER BY created_at DESC LIMIT 10)
UNION ALL
(SELECT * FROM eats  ORDER BY created_at DESC LIMIT 10)
ORDER BY created_at DESC LIMIT 10;

-- Set difference: users active last month but not this month (churn)
SELECT user_id FROM activity WHERE month = '2026-05-01'
EXCEPT
SELECT user_id FROM activity WHERE month = '2026-06-01';
```

### 11. Optimization Techniques

- Push filters/limits into each branch (planner often does, but write it explicitly).
- Matching indexes per branch + `Merge Append` keeps the merged stream ordered without a sort.
- Replace `UNION` with `UNION ALL` + a smarter key when you can prove disjointness.

### 12. Follow-up Questions

- "The combined ORDER BY ... LIMIT is slow — how do you avoid sorting |A|+|B| rows?" (per-branch indexed top-k, then merge)
- "EXCEPT vs NOT EXISTS?" (EXCEPT dedups and compares whole rows with NULL-safe equality; NOT EXISTS is per-key and keeps duplicates)

---

## Chapter 2.6 — Window Functions

### 1. Why Interviewers Ask This

**The single highest-yield SQL topic.** Ranking, dedup, running totals, moving averages, gaps/islands, sessionization — the entire Module 10 problem set is window functions. Screens at Meta/Amazon/Stripe assume fluency.

### 2. Core Concept

A window function computes over a set of rows **related to the current row, without collapsing rows** (unlike GROUP BY).

```
fn(...) OVER (PARTITION BY ...   -- restart per group
              ORDER BY ...       -- ordering within partition
              frame)             -- which neighbors: ROWS/RANGE BETWEEN ...
```

The functions that matter: - **Ranking**: `ROW_NUMBER()` (unique 1,2,3), `RANK()` (ties share, gaps: 1,1,3), `DENSE_RANK()` (ties share, no gaps: 1,1,2). Interviewers *always* probe the difference. - **Offsets**: `LAG(col, n)`, `LEAD(col, n)` — previous/next row's value. - **Edges**: `FIRST_VALUE`, `LAST_VALUE` (⚠ default frame makes LAST_VALUE = current row — the classic trap; fix the frame), `NTH_VALUE`. - **Aggregates as windows**: `SUM/AVG/COUNT/MIN/MAX ... OVER (...)` — running totals, moving averages. - **Distribution**: `NTILE(n)`, `PERCENT_RANK()`, `CUME_DIST()`.

**Frames**: default with ORDER BY is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` (peers-inclusive!). For "previous 6 rows" moving windows use `ROWS BETWEEN 6 PRECEDING AND CURRENT ROW`. ROWS = physical rows; RANGE = value distance; GROUPS = peer groups.

Evaluation position: windows are computed **after WHERE/GROUP BY/HAVING, before ORDER BY/LIMIT** → you cannot filter on a window function directly; wrap it in a subquery/CTE (this is `QUALIFY` in Snowflake/BigQuery — Postgres needs the wrap).

### 3. Internal Working

WindowAgg node: sort rows by `PARTITION BY, ORDER BY` (or consume an index providing that order), then stream, maintaining per-partition state. Running aggregates are O(N); frame-based ones may buffer the frame. Multiple window clauses with the *same* window spec share one sort; different specs = multiple sorts — a real cost on big data. Big sorts spill to disk (`work_mem`).

## 4. Visualization (ASCII)

```
PARTITION BY user ORDER BY day    SUM(amt) OVER (... ROWS UNBOUNDED PRECEDING)
user day amt | running
u1   d1  10  | 10
u1   d2  20  | 30    ◄ state carried within partition
u1   d3   5  | 35
u2   d1   7  |  7    ◄ state RESET at partition boundary
u2   d2   3  | 10

RANK vs DENSE_RANK vs ROW_NUMBER on scores 90,90,80:
score  ROW_NUMBER  RANK  DENSE_RANK
 90        1        1        1
 90        2        1        1
 80        3        3        2
```

## 5. Real Production Example

Stripe-style "flag the second-and-later charge attempts on the same card within a day" (`ROW_NUMBER() OVER (PARTITION BY card_id, day ORDER BY created_at)` > 1); Netflix-style 7-day rolling watch-hours per profile (`AVG ... ROWS 6 PRECEDING`); dedup pipelines everywhere (`ROW_NUMBER() = 1` per business key keeps the newest record).

## 6. Common Interview Questions

- "ROW_NUMBER vs RANK vs DENSE_RANK?" (near-guaranteed)
- "Top 3 earners per department." (partition + rank + wrap)
- "Running total / 7-day moving average."
- "Compare each row to the previous one" (LAG — month-over-month growth)
- "Delete duplicates, keep newest." (ROW_NUMBER in a CTE, delete rn>1)
- "Why can't I put ROW_NUMBER() in WHERE?"

## 7. Common Mistakes

- `LAST_VALUE` with the default frame (returns current row) — fix with `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`.
- Filtering on the window function in the same SELECT's WHERE.
- Wrong tool for "Nth highest": `RANK()` skips values after ties; "3rd highest *salary value*" needs `DENSE_RANK`.
- Moving average with `RANGE` when you mean `ROWS` (peers collapse) — or vice versa for time-based windows with gaps.
- Forgetting `PARTITION BY` → one global window; or partitioning by an unindexed expression on huge data → giant sort.

## 8. Best Practices

- Name windows once: `WINDOW w AS (PARTITION BY user_id ORDER BY created_at)` then `OVER w` — readable and guarantees shared sorts.
- Always pick the tiebreaker consciously (`ORDER BY created_at DESC, id DESC`) — nondeterministic ROW_NUMBER is a real prod bug.
- For time-based moving windows with gaps, either densify with a calendar spine or use `RANGE BETWEEN interval '7 days' PRECEDING AND CURRENT ROW` (Postgres 11+).
- Index `(partition_cols, order_cols)` to feed WindowAgg without a sort.

## 9. Coding Questions

1. Top 3 products by revenue per category, ties included (RANK ≤ 3).
2. Month-over-month revenue growth % with LAG, handling the first month (NULLIF/COALESCE).
3. Deduplicate `users` on email keeping the most recently active row — as a DELETE.

### 10. SQL Examples

```
-- Top-N per group
SELECT * FROM (
  SELECT e.*, DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk
  FROM employees e
) t WHERE rnk <= 3;

-- Running total + 7-row moving average
SELECT day, revenue,
       SUM(revenue) OVER (ORDER BY day)                                    AS running_total,
       AVG(revenue) OVER (ORDER BY day ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS ma7
FROM daily_revenue;

-- MoM growth
SELECT month, revenue,
       round(100.0 * (revenue - LAG(revenue) OVER (ORDER BY month))
             / NULLIF(LAG(revenue) OVER (ORDER BY month), 0), 1) AS growth_pct
FROM monthly_revenue;

-- Dedup delete (keep newest per email)
DELETE FROM users u
USING (
  SELECT id, ROW_NUMBER() OVER (PARTITION BY email ORDER BY last_seen DESC, id DESC) rn
  FROM users
) d
WHERE u.id = d.id AND d.rn > 1;

-- Named window (shared sort)
SELECT user_id, created_at,
       ROW_NUMBER() OVER w AS attempt_no,
       LAG(created_at) OVER w AS prev_attempt
FROM logins
WINDOW w AS (PARTITION BY user_id ORDER BY created_at);
```

### 11. Optimization Techniques

- One window spec per query where possible (shared sort); check EXPLAIN for stacked WindowAgg+Sort pairs.
- Pre-filter partitions before windowing (window runs after WHERE — use it).
- For top-N per group over huge tables, compare with `DISTINCT ON` / lateral top-N per key (`JOIN LATERAL (... ORDER BY ... LIMIT n)`) which can use the index per key instead of sorting everything.

### 12. Follow-up Questions

- "The window sort spills 200GB — options?" (partition pruning first, lateral per-key top-N, pre-aggregation, more work_mem, index-fed order)
- "How is `SUM OVER` different from a self-join running total?" (O(N) vs O(N²); pre-window-function interviews used the self-join — know it historically)
- "Sessionize these events with a 30-minute gap rule." (LAG + conditional flag + running SUM — Module 10 covers it fully)

---

## Chapter 2.7 — Aggregate, Date & String Functions

### 1. Why Interviewers Ask This

Not to test memorization — to catch three recurring bugs: **NULLs in aggregates, timezone handling, and non-sargable function calls on indexed columns**.

### 2. Core Concept

**Aggregates**: `COUNT(*)` counts rows; `COUNT(col)` counts non-NULL; `COUNT(DISTINCT col)` non-NULL distinct. `SUM/AVG/MIN/MAX` **ignore NULLs**; `SUM` of no rows = NULL (not 0 — wrap in COALESCE). `AVG` of int is numeric in PG but

int division bites elsewhere: `sum(a)::numeric/count(*)`. High-value extras: `FILTER (WHERE ...)`, `string_agg`, `array_agg`, `bool_or/bool_and`, `percentile_cont` (Module 10).

**Dates**: `timestamptz` stores UTC instant, renders in session TZ — use it, not `timestamp`. `date_trunc('month', ts)` for bucketing; `age()/extract(epoch from ...)` for durations; intervals do calendar math (`+ interval '1 month'` handles month lengths). Half-open ranges (`>= start AND < end`) avoid boundary bugs and stay index-friendly.

**Strings**: `lower/upper`, `substring`, `position`, `split_part`, `concat_ws`, `trim`, regex (`~`, `regexp_replace`), `LIKE 'abc%'` (can use btree index with `text_pattern_ops`) vs `LIKE '%abc%'` (cannot — needs `pg_trgm` GIN). `citext` or `lower()` expression index for case-insensitive uniqueness.

**Sargability rule**: wrapping the **column** in a function defeats the index (`WHERE date(created_at) = '2026-01-01'` → seq scan) — move the function to the constant side or create an expression index.

## 3. Internal Working

Aggregates run as HashAggregate (hash per group; memory-bound; can spill PG13+) or GroupAggregate (requires sorted input — pairs beautifully with an index). `COUNT(DISTINCT)` forces per-group dedup state — expensive; approximate alternative is HyperLogLog (`postgresql-hll`, or `approx_count_distinct` in warehouses). Expression indexes store the computed value (`lower(email)`, `date(created_at)`) so the function call becomes indexable.

## 4. Visualization (ASCII)

```
COUNT semantics on col = [1, NULL, 2, 2]:
COUNT(*)=4   COUNT(col)=3   COUNT(DISTINCT col)=2   SUM(col)=5   AVG(col)=5/3 (NULL ignored)

Sargability:
WHERE date(created_at) = '2026-01-01'      WHERE created_at >= '2026-01-01'
      └── f(column): index UNUSABLE          AND created_at <  '2026-01-02'
          → Seq Scan                               └── bare column: Index Scan ✔
```

## 5. Real Production Example

Timezone incident every company has: daily revenue computed with `created_at::date` (session TZ) disagreeing with finance's UTC-based numbers around midnight boundaries → all rollups re-specified as `date_trunc('day', created_at AT TIME ZONE 'UTC')`. And the perf twin: `WHERE lower(email) = lower($1)` scanning 50M rows until someone adds `CREATE INDEX ON users (lower(email))`.

## 6. Common Interview Questions

- "COUNT(*) vs COUNT(col) vs COUNT(1)?" (COUNT(1)=COUNT(*); COUNT(col) skips NULLs)
- "Why does AVG differ from SUM/COUNT here?" (NULLs in col)
- "Query all of yesterday's orders — write the WHERE." (half-open range, index-safe, TZ-explicit)
- "Case-insensitive email uniqueness — how?" (unique expression index on lower(email) or citext)
- "Why did the index stop being used when we added date()/lower()?"

## 7. Common Mistakes

- `sum(...)` returning NULL on empty sets breaking downstream math (COALESCE it).
- `BETWEEN '2026-01-01' AND '2026-01-31'` on timestamps — silently drops Jan 31 after midnight; use `< '2026-02-01'`.
- `timestamp` (no TZ) columns in multi-region systems.
- `COUNT(DISTINCT user_id)` over billions of rows in a dashboard (use HLL sketches / rollups).
- String concatenation with `||` where a NULL nukes the whole string (use `concat_ws`).

## 8. Best Practices

- Store instants as `timestamptz`, money as integer cents (or `numeric` — never float), case-insensitive text as `citext`/lower-indexed.
- Bucket with `date_trunc`, filter with half-open ranges, always state the timezone.
- `FILTER (WHERE ...)` for multi-metric single-scan queries.

- Keep predicates sargable; when you can't, create the expression index and remember it must match the query's expression *exactly*.

## 9. Coding Questions

1. One scan over `orders`: per month — orders, revenue, distinct buyers, refund rate, avg order value excluding refunds.
2. Parse `full_name` into first/last with `split_part`, flag rows that don't fit the pattern.

## 10. SQL Examples

```
-- Multi-metric single scan
SELECT date_trunc('month', created_at AT TIME ZONE 'UTC') AS month,
       count(*)                                            AS orders,
       coalesce(sum(total_cents), 0)                       AS revenue_cents,
       count(DISTINCT user_id)                             AS buyers,
       round(count(*) FILTER (WHERE status='refunded')::numeric / count(*), 4) AS refund_rate,
       avg(total_cents) FILTER (WHERE status <> 'refunded') AS aov_cents
FROM orders
WHERE created_at >= '2026-01-01' AND created_at < '2026-07-01'
GROUP BY 1 ORDER BY 1;

-- Sargable "yesterday" + expression index pattern
SELECT * FROM orders
WHERE created_at >= (current_date - 1)::timestamptz
  AND created_at <  current_date::timestamptz;

CREATE UNIQUE INDEX ON users (lower(email));   -- makes lower(email)=... indexable AND unique

-- String utilities
SELECT split_part(email, '@', 2)                   AS domain,
       string_agg(DISTINCT country, ', ' ORDER BY country) AS countries
FROM users GROUP BY 1;
```

## 11. Optimization Techniques

- Replace `COUNT(DISTINCT)` at scale with HLL or pre-aggregated distinct-count rollups.
- Feed GroupAggregate with an index on the GROUP BY column to skip hashing entirely for ordered output.
- `%text%` search → `pg_trgm` GIN index; prefix search → `text_pattern_ops` btree; full-text → `tsvector` GIN.

## 12. Follow-up Questions

- "Your expression index on `date(created_at)` isn't used by `date_trunc('day', created_at)` — why?" (expression must match exactly)
- "AVG over money in float drifted by cents — why and fix?" (binary float representation; numeric/integer cents)
- "How would you compute distinct daily users over a year, fast and approximately?" (daily HLL sketches, union them)

---

# Module 2 — Practice Problems

## Easy (5)

1. Per status, count orders in the last 7 days; only statuses with >100 orders; order by count. State which clause each condition belongs in and why.
2. `COUNT(*)`=1000, `COUNT(discount)`=700, `AVG(discount)`=5. What is `SUM(discount)`, and what happens to AVG if the NULLs were really "no discount = 0"?
3. Latest login per user: write it with `DISTINCT ON` and with `ROW_NUMBER`.
4. Explain the output difference of RANK, DENSE_RANK, ROW_NUMBER over salaries 100, 100, 90, 80.
5. Rewrite non-sargable to sargable: `WHERE year(created_at)=2026 AND lower(status)='paid'`.

## Medium (5)

1. Users who ordered in May 2026 but not June 2026 — solve twice: EXCEPT and NOT EXISTS; explain the duplicate/NULL semantics difference.
2. 7-day *time-based* moving average of daily revenue where some days have no rows — produce correct values (calendar spine or RANGE frame; explain why plain `ROWS 6 PRECEDING` is wrong).
3. Category tree: all descendant categories of id=5 with depth and path, cycle-safe; then count products per *subtree*.
4. A report uses `UNION` across 6 monthly tables and takes minutes. Diagnose (dedup sort over everything) and fix (UNION ALL + why it's safe, or partitioned table).
5. Write a DELETE that removes duplicate `payments` rows sharing `idempotency_key`, keeping the earliest, and explain why you partition by the key and order by `created_at, id`.

## Hard (5)

1. Percent of each department's payroll earned by its top earner, one scan, no self-join (window SUM per dept + window MAX or RANK; handle ties).
2. Given `events(user_id, ts)`, compute per user the longest streak of consecutive *days* with activity (dedup to days, LAG/day-diff, gap flag, running group id, count — write it fully).
3. A matview refresh (45 min, hourly) powers a customer-facing dashboard needing <2 min staleness. Design the replacement: incremental rollup DDL, the upsert on write path, the backfill, and the reconciliation query proving correctness.
4. `WHERE id NOT IN (subquery)` returns 0 rows in prod, correct rows in staging. Explain the exact three-valued-logic evaluation, find the data difference, and give two fixes.
5. You need "top 20 posts per user for 10M users" and the window-sort approach spills terabytes. Design the lateral-join-per-key alternative with its supporting index, and explain when it beats WindowAgg and when it loses.

---

*Next: Module 3 — Joins*

# MODULE 3 — Joins

> Two layers, both examined: **logical** (which rows come back — INNER/LEFT/RIGHT/FULL/CROSS/SELF) and **physical** (how the engine computes them — Nested Loop / Hash / Merge). Senior interviews live in the second layer and in the traps between the two.

Chapters: 3.1 Logical Joins: INNER, LEFT, RIGHT, FULL, CROSS, SELF 3.2 Physical Joins: Nested Loop, Hash Join, Merge Join 3.3 Join Traps & Optimization Playbook

---

## Chapter 3.1 — Logical Joins

### 1. Why Interviewers Ask This

Everyone claims to know joins; interviewers filter with three probes: **row multiplication**, **WHERE vs ON placement on outer joins**, and **NULL behavior in join keys**. Getting these right is table stakes for the SQL screen.

### 2. Core Concept

| Join | Returns | Typical use |
|---|---|---|
| INNER | Only matching pairs | Facts that must have both sides |
| LEFT (OUTER) | All left rows + matches (right side NULL-padded when absent) | Optional relationships, "with or without" |
| RIGHT | Mirror of LEFT (rewrite as LEFT — everyone does) | Rarely written |
| FULL | All rows both sides, NULL-padded where unmatched | Reconciliation/diff of two sets |
| CROSS | Cartesian product (m×n) | Generating combinations (calendar × products) |
| SELF | Table joined to itself (any type) | Hierarchies, pairs, sequential comparison |

Three rules that decide interview questions: 1. **Multiplication**: joins are pair-matching. If the key isn't unique on one side, rows duplicate; if not unique on *both* sides, rows multiply (m×n per key). Every "why did my SUM double?" bug is this. 2. **ON vs WHERE on outer joins**: `ON` decides *matching*; `WHERE` filters the *result*. A WHERE condition on the right table's column (other than `IS NULL`) discards the NULL-padded rows → **silently converts LEFT JOIN to INNER JOIN**. The single most-tested join trap. 3. **NULL keys never match**: `NULL = NULL` is UNKNOWN. Rows with NULL join keys vanish from INNER joins.

Anti-join idioms: "left rows with no match" = `LEFT JOIN ... WHERE right.id IS NULL` or `NOT EXISTS` (prefer the latter — intent-revealing and plans as Anti Join directly).

### 3. Internal Working

The planner treats logical join type as a *constraint* on reordering: inner joins commute and associate freely (join reordering — Module 5); outer joins restrict reorder legality. It also tries **outer-join simplification**: if a later predicate rejects NULLs from the right side, the LEFT JOIN is internally converted to INNER (which is exactly the WHERE-vs-ON trap, done on purpose by the optimizer). FULL JOIN limits algorithm choice (hash full join supported in recent PG; merge join handles it naturally).

## 4. Visualization (ASCII)

```
users u LEFT JOIN orders o ON o.user_id = u.id
u: (1,ann) (2,bob) (3,cat)      o: (10,u1) (11,u1) (12,u3)

result:               1|ann|10          -- ann duplicated: 2 matches
                      1|ann|11
                      2|bob|NULL        -- padded: bob kept
                      3|cat|12

...WHERE o.status='paid'    → bob's row has o.status = NULL → dropped
                              LEFT JOIN just became INNER JOIN (trap!)
...ON  o.status='paid'      → bob kept (padded); ann keeps only paid orders ✔

NULL key: u4 with id NULL never equals anything → absent from INNER result
```

## 5. Real Production Example

A Meta-style metrics bug: "signups with their first payment" written as `LEFT JOIN payments p ... WHERE p.status='completed'` silently dropped all non-paying signups from a conversion funnel — the funnel showed 100% conversion for a quarter. The fix (move the condition into ON) is a one-line diff; the interview question "what changed?" is asked verbatim. FULL JOIN in the wild: reconciling internal `payments` vs the processor's settlement file — mismatched rows on either side are exactly the NULL-padded rows.

## 6. Common Interview Questions

- "Difference between putting the filter in ON vs WHERE on a LEFT JOIN?" (near-guaranteed)
- "Users with zero orders — write it twice (anti-join both idioms)."
- "Why did the revenue SUM double after adding a join?" (one-to-many multiplication)
- "When is CROSS JOIN legitimate?" (spines: dates × products for zero-filled reports)
- "Employees earning more than their manager." (classic self-join)

## 7. Common Mistakes

- The WHERE-kills-LEFT-JOIN trap (both directions: causing it, and not spotting it in review).
- Joining one-to-many then aggregating without dedup — `SUM` counts parents multiple times (fix: pre-aggregate the many side in a subquery/CTE *before* joining).
- `COUNT(*)` vs `COUNT(right.id)` after LEFT JOIN: `COUNT(*)` counts padded rows as 1; users with zero orders show 1 unless you count the right column.
- Accidental cross join via missing ON in comma-syntax (`FROM a, b WHERE ...` and the WHERE gets edited away) — always use explicit JOIN ... ON.
- Self-join without aliasing both sides distinctly.

## 8. Best Practices

- Left-table = the set you must not lose; conditions on the optional side go in ON.
- Pre-aggregate the many side to one row per key before joining when you'll aggregate the one side.
- Prefer `NOT EXISTS` over LEFT-JOIN-IS-NULL for anti-joins (clearer, and immune to being broken by later WHERE edits).
- Every FK you join on should be indexed (Module 4) — logical joins are only as good as the physical access path.

## 9. Coding Questions

1. Per user: name, order count, lifetime revenue — including users with zero orders showing 0 (LEFT JOIN + COUNT(o.id) + COALESCE(SUM)) — then rewrite with a pre-aggregated subquery and explain when the second wins.
2. Pairs of employees in the same department with the same salary, each pair once (self-join with `e1.id < e2.id`).
3. Full reconciliation: rows only in `ledger`, only in `settlement`, and amount mismatches — one FULL JOIN query with a CASE classification column.

### 10. SQL Examples

```
-- Correct optional-side filtering
SELECT u.id, u.name, count(o.id) AS paid_orders, coalesce(sum(o.total_cents),0) AS revenue
FROM users u
LEFT JOIN orders o ON o.user_id = u.id AND o.status = 'paid'   -- condition in ON
GROUP BY u.id, u.name;

-- Anti-join, both idioms
SELECT u.* FROM users u
LEFT JOIN orders o ON o.user_id = u.id
WHERE o.id IS NULL;                        -- idiom 1

SELECT u.* FROM users u
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.user_id = u.id);  -- idiom 2 (preferred)

-- Self join: employee vs manager
SELECT e.name AS employee, m.name AS manager
FROM employees e JOIN employees m ON m.id = e.manager_id
WHERE e.salary > m.salary;

-- CROSS JOIN spine: zero-filled daily sales per product
SELECT d::date AS day, p.id AS product_id, coalesce(sum(s.qty),0) AS qty
FROM generate_series(current_date-6, current_date, '1 day') d
CROSS JOIN products p
LEFT JOIN sales s ON s.product_id = p.id AND s.sold_on = d::date
GROUP BY 1, 2;

-- FULL JOIN reconciliation
SELECT coalesce(l.txn_id, s.txn_id) AS txn_id,
       CASE WHEN l.txn_id IS NULL THEN 'missing_internally'
            WHEN s.txn_id IS NULL THEN 'missing_at_processor'
            WHEN l.amount <> s.amount THEN 'amount_mismatch'
            ELSE 'ok' END AS status
FROM ledger l FULL JOIN settlement s ON s.txn_id = l.txn_id
WHERE l.txn_id IS NULL OR s.txn_id IS NULL OR l.amount <> s.amount;
```

### 11. Optimization Techniques

- Reduce before you join: filter and pre-aggregate each side to the minimal row set.
- Join on compact, indexed, same-typed keys (type mismatch `int = varchar` blocks index use).
- For "one row from the many side" (latest order per user), `JOIN LATERAL (...ORDER BY...LIMIT 1)` probes the index per user instead of joining everything.

### 12. Follow-up Questions

- "Rewrite this RIGHT JOIN chain as LEFT JOINs — why do teams ban RIGHT JOIN?" (reading order)
- "The anti-join is slow on 100M rows — what plan do you want to see?" (Hash Anti Join, not a per-row SubPlan)
- "What happens to join results when the key is nullable on both sides, and how does that differ in a FULL JOIN's output?"

---

## Chapter 3.2 — Physical Joins: Nested Loop, Hash Join, Merge Join

### 1. Why Interviewers Ask This

This is the difference between "knows SQL" and "can fix a slow query." Reading EXPLAIN output means recognizing which algorithm fired and whether it was the right one for the cardinalities. Google/Uber onsite favorite: "here's a plan, tell me why it's slow."

## 2. Core Concept

| Algorithm | How | Best when | Cost shape |
|---|---|---|---|
| **Nested Loop** | For each outer row, find matches in inner | Outer is SMALL and inner probe is an **index hit**; also the only option for non-equi joins (`<`, `BETWEEN`, `LIKE`) | O(outer × inner-probe). Index probe: great. Inner seq scan: O(m×n) disaster |
| **Hash Join** | Build hash table on smaller side, probe with larger | Large unsorted inputs, **equality** keys, enough memory | O(m + n), memory for build side; spills to disk in batches if > work_mem |
| **Merge Join** | Sort both sides (or use index order), zipper through | Both sides **already sorted** (indexes!), huge inputs, or FULL joins | O(m + n) after order; sorts cost n·log n if needed |

Rules of thumb the interviewer expects: - Few outer rows + indexed inner → Nested Loop wins (best startup time too — pairs with LIMIT). - Two big tables, equi-join → Hash Join wins. - Both sides indexed on the join key / pre-sorted → Merge Join wins, output comes out sorted (can eliminate a later ORDER BY). - Only Nested Loop handles arbitrary (non-equality) join conditions.

## 3. Internal Working

- **Nested Loop**: `for o in outer: for i in inner_lookup(o.key): emit`. Planner picks inner path per outer row — parameterized index scan (`Index Cond: user_id = o.id`). Chosen based on *estimated* outer rows: if stats say 10 but reality is 100k, you get 100k index probes — the classic misestimate blowup.
- **Hash Join**: build phase hashes the (estimated) smaller input into `work_mem`; probe phase streams the big side. Overflow → **batching**: both inputs partitioned to disk by hash and joined batch-by-batch (`Batches: 16` in EXPLAIN = spilling). Wrong side chosen for build (misestimate) → giant hash table, memory pressure.
- **Merge Join**: both inputs in key order — from B+Tree index scans (free order) or explicit Sorts. Two cursors advance; duplicate keys on both sides cause mini cross-products with rescans ("mark/restore"). Handles FULL OUTER naturally.

## 4. Visualization (ASCII)

```
NESTED LOOP                    HASH JOIN                     MERGE JOIN
outer (5 rows)                 build: small side → hash tbl  A sorted: 1 3 5 7 9
  │ per row                      {k1:[r],k2:[r],...} (RAM)   B sorted: 1 2 3 3 7
  ▼                            probe: big side streams       zipper:  advance the
inner INDEX probe ──► emit       hash(k) → bucket → emit       smaller cursor, emit
cost ≈ 5 index descents        cost ≈ read A + read B          on equality
✔ tiny outer, LIMIT-friendly   ✖ spills if build > work_mem  ✔ order for free from
✖ 1M outer rows = 1M probes      (EXPLAIN: Batches > 1)        indexes; output sorted
```

## 5. Real Production Example

Uber-scale incident pattern: a query joining `trips` to `cities` runs fine (nested loop, 20 outer rows) until a code change removes a filter — outer side becomes 40M rows, planner stats are stale so it *still* picks nested loop → 40M index probes → p99 explodes and connections pile up. Fix: `ANALYZE` (fresh stats flip it to hash join). The interview version: "same query, fast yesterday, slow today, no code change — go" (stats drift / plan flip).

## 6. Common Interview Questions

- "Explain the three join algorithms and when the planner picks each."
- "What does `Hash Join ... Batches: 32` mean?" (work_mem overflow, disk partitioning)
- "Why is nested loop sometimes the fastest and sometimes the worst?" (outer cardinality × inner path)
- "Which algorithm for `a.ts BETWEEN b.start AND b.end`?" (nested loop — no equality key; mention range-type GiST index as the real fix)
- "Why did adding LIMIT 10 change the join algorithm?" (startup cost: NL streams immediately)

## 7. Common Mistakes

- Reading EXPLAIN top-down only and missing that the *inner* node runs `loops=N` times — multiply `actual time × loops`.
- Blaming the algorithm when the real issue is a **misestimate** (compare `rows=` estimated vs actual in EXPLAIN ANALYZE — Module 5).
- Disabling nested loops globally (`enable_nestloop=off`) as a "fix" — punishes every small-outer query.
- Forgetting Merge Join may include hidden Sort nodes — the sort, not the join, is the cost.
- Missing index on the join FK → hash join forced for even tiny lookups, or NL with inner seq scan (worst case).

## 8. Best Practices

- Index every join key (FKs first); it enables NL-with-index and merge join, and gives the planner options.
- Keep stats fresh (`ANALYZE`, autovacuum analyze thresholds tuned down for big tables).
- Size `work_mem` so common hash joins run `Batches: 1` — but remember it's per-node, per-query, per-connection (multiply!).
- When diagnosing: check estimated-vs-actual rows first, algorithm second, indexes third.

## 9. Coding Questions

1. Given `EXPLAIN ANALYZE` output showing `Nested Loop (actual rows=2,400,000 loops=1)` with inner `Index Scan (loops=2400000)` — state the problem, the two possible fixes (stats / rewrite), and the plan you expect after.
2. For `orders (500M) JOIN users (50M) ON user_id` filtered to one day of orders (~1M): predict the chosen algorithm and build side; then predict again with no date filter.

## 10. SQL Examples

```
-- See the algorithm and the estimates vs reality
EXPLAIN (ANALYZE, BUFFERS)
SELECT u.country, count(*)
FROM orders o JOIN users u ON u.id = o.user_id
WHERE o.created_at >= now() - interval '1 day'
GROUP BY u.country;
-- Look for: Hash Join / Nested Loop / Merge Join, rows= vs actual rows=, Batches:, loops=

-- Force-compare algorithms while investigating (session-local, never in prod code)
SET enable_hashjoin = off;   -- re-run EXPLAIN ANALYZE, compare
RESET enable_hashjoin;

-- Give the planner what it needs
CREATE INDEX ON orders (user_id);            -- join key
CREATE INDEX ON orders (created_at);         -- filter
ANALYZE orders;                              -- fresh stats
```

## 11. Optimization Techniques

- Shrink the build side: filter/aggregate before joining so the hash table is small.
- Feed merge joins from indexes to avoid explicit sorts; exploit its sorted output to erase a later `ORDER BY`.
- Batch key lookups from the app (`= ANY($ids)`) so one indexed NL/hash query replaces N round-trip queries (N+1 problem — Module 5).
- Partition-wise joins: joining two tables partitioned identically on the join key joins partition-pairs (parallelizable, smaller hashes).

## 12. Follow-up Questions

- "work_mem is 4MB and the build side is 4GB — walk through exactly what the executor does." (multi-batch grace hash join, disk partitions, recursive re-partitioning if skewed)
- "One join key value holds 30% of rows (skew) — which algorithm suffers and why?" (hash: one huge bucket / batch; merge: mini cross-product rescans; discuss salting)
- "How do distributed engines join across shards?" (broadcast vs shuffle joins — nice bridge to Module 7/8)

---

# Chapter 3.3 — Join Traps & Optimization Playbook

## 1. Why Interviewers Ask This

Senior loops end joins with a debugging scenario. This chapter is the checklist you run.

## 2. Core Concept — The Playbook

When a join query is slow or wrong, check in order:

1. **Wrong results first**: row multiplication? LEFT-turned-INNER via WHERE? NULL keys dropped?
2. **Cardinality**: how many rows does each side contribute *after* filters? (EXPLAIN ANALYZE actual rows)
3. **Estimates vs actuals**: off by >10x → stale stats (`ANALYZE`), correlated columns (extended statistics), or non-representable predicates.
4. **Access paths**: is every join key indexed? Types identical? Expression mismatch?
5. **Algorithm sanity**: NL with big outer? Hash with `Batches > 1`? Merge with giant Sorts?
6. **Shape rewrites**: pre-aggregate, LATERAL for top-1-per-key, `EXISTS` instead of JOIN+DISTINCT, split OR-joins into UNION ALL.

## 3. Internal Working

Why joins go quadratic: the planner's row estimates multiply through the join tree, so one bad estimate at the bottom corrupts every decision above it (join order, algorithms, memory grants). That's why "fix the estimate" (stats, extended stats, rewrite) usually beats "hint the plan."

## 4. Visualization (ASCII)

```
Estimate error propagates upward:
            Hash Join (est 100 rows, actual 8M)  ← everything above is misplanned
           /        \
   IndexScan A     Hash
   est 10 act 4000  └─ SeqScan B  ← root cause: stale stats on A's filter column
FIX ORDER: stats → indexes → rewrite → (last resort) planner knobs
```

## 5. Real Production Example

Stripe-style reporting join across `charges`, `refunds`, `disputes` where each is one-to-many: joining all three multiplies rows (charge × refunds × disputes) and inflates SUMs. Production fix: aggregate each child to one row per charge in CTEs, then join three tiny aggregates — also what turns three hash joins of 100M rows into three of 1M.

## 6. Common Interview Questions

- "SUM tripled after adding the disputes join — why, and fix it live."
- "This 5-table join is slow; the plan shows all seq scans — walk your checklist."
- "When do you denormalize instead of joining?" (measured hot path, after covering indexes fail — bridges Module 1.6)

## 7. Common Mistakes

- Fixing symptoms with `DISTINCT` on a multiplied result instead of pre-aggregating (hides the bug, keeps the cost).
- Adding indexes to fix a join that spills — the spill is `work_mem`/build-side size, not access path.
- OR in join conditions (`ON a.x=b.x OR a.y=b.y`) — kills hash/merge; split into UNION ALL of two equi-joins.

## 8. Best Practices

- One-row-per-key discipline: know the grain of every table in the join and assert it (unique index) — most wrong-join bugs are grain bugs.
- Keep join keys `bigint`-typed and identical across tables; never join `text` to `int`.
- Standard rewrite kit memorized: pre-aggregate / LATERAL top-N / EXISTS semi-join / UNION-ALL-split OR / calendar-spine cross join.

## 9. Coding Questions

1. Fix live: `SELECT c.id, sum(r.amount), sum(d.amount) FROM charges c LEFT JOIN refunds r ON r.charge_id=c.id LEFT JOIN disputes d ON d.charge_id=c.id GROUP BY c.id;` (multiplied sums — pre-aggregate each child).
2. Rewrite `ON o.user_id = u.id OR o.guest_email = u.email` as UNION ALL of two indexable joins, handling the overlap (dedup on a business key).

## 10. SQL Examples

```
-- Grain-safe multi-child aggregation
WITH r AS (SELECT charge_id, sum(amount) AS refunded FROM refunds  GROUP BY 1),
     d AS (SELECT charge_id, sum(amount) AS disputed FROM disputes GROUP BY 1)
SELECT c.id, coalesce(r.refunded,0) AS refunded, coalesce(d.disputed,0) AS disputed
FROM charges c
LEFT JOIN r ON r.charge_id = c.id
LEFT JOIN d ON d.charge_id = c.id;

-- Top-1-per-key via LATERAL (index-driven, no giant sort)
SELECT u.id, o.*
FROM users u
LEFT JOIN LATERAL (
  SELECT * FROM orders o WHERE o.user_id = u.id
  ORDER BY o.created_at DESC LIMIT 1
) o ON true;
-- supporting index: orders(user_id, created_at DESC)
```

## 11. Optimization Techniques

- Extended statistics for correlated filter columns feeding joins: `CREATE STATISTICS s (dependencies) ON city, country FROM users; ANALYZE users;`
- Parallel hash joins (PG 11+) — check `Workers Planned/Launched` for big analytic joins.
- `join_collapse_limit` awareness: beyond ~8 tables the planner stops exhaustive reordering — write the critical join order explicitly or raise the limit.

## 12. Follow-up Questions

- "The plan is optimal but still too slow — what's left?" (data volume: pre-aggregate, denormalize, cache, move to OLAP store)
- "How would you catch grain bugs before prod?" (unique indexes as executable documentation, row-count assertions in tests)

---

# Module 3 — Practice Problems

## Easy (5)

1. Products never ordered — both anti-join idioms; which plan node do you want to see?
2. Predict the output row count: `A(5 rows, key k unique)` INNER JOIN `B(8 rows, 3 rows with k=1, others unmatched)` — then for LEFT JOIN both directions.
3. Spot the bug: `FROM users u LEFT JOIN orders o ON o.user_id=u.id WHERE o.created_at > now()-interval '30 days'` — what does it return and how do you fix it two ways?
4. Employees and their managers' names, including employees with no manager.
5. Which join algorithm and why: 12-row `countries` joined to 300M-row `events` on `country_code` with an index on `events(country_code)`?

## Medium (5)

1. Per user: signup date, first order date, days-to-convert, including never-converted users — without a correlated subquery.
2. `EXPLAIN ANALYZE` shows `Hash Join ... Batches: 64, actual rows 90M` under a `Sort` feeding `Limit 50`. Give three independent fixes and the plan you'd expect from each.
3. Sessions table has `(user_id, started_at, ended_at)`. Find overlapping session *pairs* per user (self non-equi join), then explain why this can't hash-join and how a GiST index on `tstzrange(started_at, ended_at)` changes the game.
4. Reconcile `inventory_counts` (system) vs `warehouse_scan` (physical): produce missing-in-system, missing-in-warehouse, and quantity mismatches in one FULL JOIN pass.
5. A 6-table reporting join is correct but 40s. Only 2 tables are large. Restructure it (aggregate-then-join) and state the expected algorithm for each remaining join.

## Hard (5)

1. Skewed join: 25% of `events` share `user_id = 0` (anonymous). The hash join stalls on one batch. Show the split: handle `user_id=0` branch separately (pre-aggregated) UNION ALL the normal join, and explain why this fixes both memory and parallelism.
2. Design the indexes so `orders JOIN users` filtered by `orders.created_at` range and `users.country='DE'`, ordered by `orders.created_at DESC LIMIT 50`, runs as NL-with-index without a sort. Explain each index column's role.
3. Interval join at scale: match each `payment(ts)` to the `fx_rate` valid at that instant (`valid_from <= ts < valid_to`, 200M payments, 100k rates). Compare NL+btree, LATERAL top-1, and range-type GiST approaches with expected costs.
4. Prove (with a 3-row example) that `LEFT JOIN` then `WHERE right.col = X` differs from `ON right.col = X`, and that the WHERE form equals INNER JOIN — then find the one condition for which WHERE placement on the right side is legitimate. (`IS NULL` anti-join.)
5. You must join two 1B-row tables on `session_id` nightly. Both are partitioned by day. Design a partition-wise join strategy (co-partitioning, per-partition hash joins, parallelism), and what breaks if one table is partitioned by week instead.

---

*Next: Module 4 — Indexes*

# MODULE 4 — Indexes (Highest Priority)

> If you master one module, master this one. "How do indexes work / why is my index ignored / design the index for this query" appears in virtually every senior backend loop.

Chapters: 4.1 B+Tree Internals 4.2 Clustered vs Non-Clustered Indexes 4.3 Composite Indexes & the Leftmost Prefix Rule 4.4 Covering, Partial & Unique Indexes 4.5 Scan Types: Index Scan, Index-Only Scan, Bitmap Scan, Sequential Scan 4.6 When Indexes Fail: Cardinality, Selectivity & Why Queries Ignore Indexes 4.7 How to Choose Indexes (Design Method)

---

## Chapter 4.1 — B+Tree Internals

### 1. Why Interviewers Ask This

"How does an index work?" is the canonical depth probe. The B+Tree answer separates candidates who can predict costs (log N descent, range scans via leaf chain, write amplification) from those who say "it makes queries fast."

### 2. Core Concept

A **B+Tree** is a balanced, high-fanout ordered tree: - **Internal nodes**: only separator keys + child pointers (no data) → huge fanout. - **Leaf nodes**: all keys in sorted order + pointers to table rows (TIDs in Postgres), linked left↔right into a chain → range scans are sequential leaf walks. - **Balanced**: every leaf at the same depth; lookup = fixed log_fanout(N) page reads.

Fanout math you should say out loud: 8KB page ÷ ~40B per entry ≈ 200–400 entries/page → **height 3–4 covers billions of rows**. Root and internal levels are almost always cached, so a point lookup ≈ 1–2 actual disk reads worst case.

Why B+Tree and not a hash or binary tree: - vs hash: B+Tree supports **range, prefix, ORDER BY**; hash only equality. - vs binary tree: fanout 300 vs 2 → height 4 vs 30 → 4 page reads, not 30 (disk pages are the unit of cost).

### 3. Internal Working

- **Search**: binary-search the root for the child range, descend, repeat; scan the leaf.
- **Range scan**: descend to the first match, then follow the leaf chain — no re-descent.
- **Insert**: descend to the leaf; if full → **page split** (half the entries move to a new page, separator key inserted into the parent; splits can cascade upward, growing the tree by raising the root). Sequential keys always hit the rightmost leaf → cheap, cache-hot appends; random keys (UUIDv4) split everywhere → write amplification + cold cache.
- **Delete**: mark/remove entries; Postgres cleans dead index entries via VACUUM (index bloat when deletes/updates churn).
- **Every secondary index adds write cost**: each INSERT touches every index; each UPDATE of an indexed column touches that index (Postgres non-HOT updates touch *all* indexes — see 4.2).

### 4. Visualization (ASCII)

```
                      root [ 40 | 80 ]
                     /       |        \
       [10|25]            [55|70]           [90|95]        internal: keys only
      /   |   \          /   |   \         /   |   \
  leaf  leaf  leaf   leaf  leaf  leaf   leaf  leaf  leaf   leaves: keys + row ptrs
  [1..9]→[10..24]→[25..39]→[40..54]→[55..69]→[70..79]→[80..89]→[90..94]→[95..99]
     └──────────────── doubly-linked leaf chain: range scan = walk right ──────────┘

Point lookup 57:  root→[55|70]→leaf[55..69] = 3 pages
Range 25..70:     descend once to 25, walk chain to 70

Page split on insert into full leaf:
  [10,12,14,16] + insert 13 → [10,12,13] [14,16] + push "14" up to parent
```

## 5. Real Production Example

Payments table at Stripe scale: `charges(id bigint)` grows by billions of rows; because ids are monotonically increasing, all B+Tree inserts append to the rightmost leaf — index stays fast and hot. A team switches to UUIDv4 for "security" → inserts scatter across the whole tree, page splits everywhere, buffer pool churns, p99 insert latency triples. Fix: UUIDv7 (time-ordered) or keep bigint internally. This exact story is a Meta/Uber interview favorite.

## 6. Common Interview Questions

- "Explain how a B+Tree index finds a row." (with page-count arithmetic)
- "B+Tree vs B-Tree vs hash index?" (B+Tree: data only in leaves + leaf chain; hash: equality only)
- "Why are databases' trees wide and shallow?" (page = I/O unit; minimize page reads)
- "What happens inside the index during INSERT? During a range query?"
- "Why do random UUIDs hurt insert performance?"

## 7. Common Mistakes

- Saying data is stored in internal nodes (that's B-Tree, not B+Tree; and matters for fanout).
- Ignoring write cost: "just add an index" on a table with 10 indexes and heavy writes.
- Missing that ORDER BY on the indexed column is free (leaf chain is sorted) — huge for pagination.
- Thinking index lookup is O(1) or "instant" — it's log with a small base, plus heap access (see 4.5).

## 8. Best Practices

- Time-ordered keys for high-insert tables (bigint identity / UUIDv7 / ULID).
- Watch index bloat on churny tables (`pgstattuple`, `REINDEX CONCURRENTLY` to rebuild).
- Count your indexes: each one taxes every write; delete unused ones (`pg_stat_user_indexes.idx_scan = 0`).
- Know the specialized types exist and when: GIN (jsonb, arrays, full-text, trigram), GiST (ranges, geometry), BRIN (huge append-only, correlated columns), Hash (rarely worth it).

## 9. Coding Questions

1. Compute tree height: 2B rows, 8KB pages, 40B index entries. (fanout ≈ 200 → $200^3$ = 8M, $200^4$ = 1.6B < 2B → height ≈ 4–5; with realistic fanout 300+: 4.)
2. Estimate pages read for `WHERE id BETWEEN 1000 AND 2000` (height descents = 3, plus 1000 entries ÷ ~200/leaf ≈ 5 leaf pages, plus heap pages per 4.5).

## 10. SQL Examples

```
CREATE INDEX CONCURRENTLY idx_orders_created ON orders (created_at);
-- CONCURRENTLY: builds without blocking writes (mandatory on live tables; can't run in a txn)

-- Inspect the tree
CREATE EXTENSION IF NOT EXISTS pageinspect;
SELECT * FROM bt_metap('idx_orders_created');           -- root, level (height)
SELECT avg_leaf_density, leaf_fragmentation
FROM pgstatindex('idx_orders_created');                 -- bloat check (pgstattuple ext)

-- Find dead-weight indexes
SELECT indexrelname, idx_scan, pg_size_pretty(pg_relation_size(indexrelid))
FROM pg_stat_user_indexes ORDER BY idx_scan ASC LIMIT 10;
```

## 11. Optimization Techniques

- `REINDEX CONCURRENTLY` (or `pg_repack`) for bloated indexes on churn-heavy tables.
- Postgres 13+ **B+Tree deduplication** compresses duplicate keys automatically — low-cardinality btrees got much smaller (worth mentioning).
- `fillfactor < 100` on indexes for update-heavy tables leaves split headroom.

### 12. Follow-up Questions

- "Index is 5x the size it should be — why and what do you do?" (bloat from churn; REINDEX CONCURRENTLY; autovacuum tuning)
- "Why might an index speed reads but be net-negative for the system?" (write amplification, cache pressure, maintenance)
- "How does an LSM tree differ for the same workload?" (Module 8 bridge: write-optimized vs read-optimized)

---

## Chapter 4.2 — Clustered vs Non-Clustered Indexes

### 1. Why Interviewers Ask This

Classic MySQL-vs-Postgres discriminator question, and it explains real phenomena: why MySQL secondary lookups are double lookups, why hot ranges are fast in InnoDB, why Postgres has HOT updates and visibility maps.

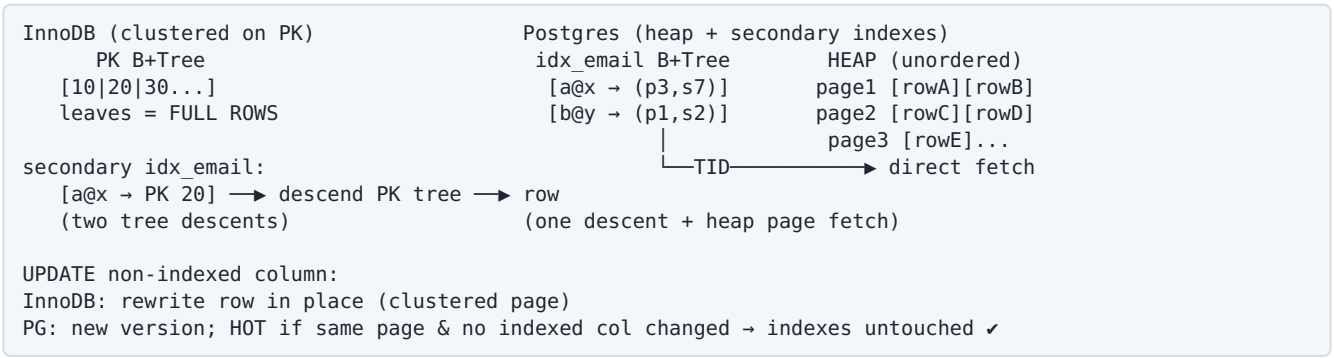
### 2. Core Concept

- **Clustered index**: the table rows are physically stored *in* the index's leaf pages, in key order. One per table (data can only be in one order). **InnoDB: the PK is always the clustered index** — the table *is* a B+Tree on the PK.
- **Non-clustered (secondary) index**: separate structure whose leaves point to the row — in InnoDB, secondary leaves store **the PK value** (→ lookup = secondary tree + PK tree = two descents); in Postgres, **all** indexes are secondary and leaves store the **TID** (physical page, slot) into the heap.
- **Postgres has no clustered index.** `CLUSTER` physically reorders the heap once (not maintained). Correlation between index order and heap order is tracked in stats and matters for range-scan cost.

### 3. Internal Working

- InnoDB point lookup by PK: single tree descent, row is right there. By secondary key: descend secondary → get PK → descend PK tree ("back to the clustered index"). Wide PKs inflate *every* secondary index.
- Postgres lookup: descend index → TID → fetch heap page. Updates write a **new row version** at a new TID; normally all indexes need new entries — unless the update touches no indexed columns and the new version fits on the same page (**HOT update**: heap-only tuple, indexes untouched — a big deal for write performance).
- Range scans: clustered = sequential disk reads of the data itself; Postgres range scan on a poorly-correlated column = random heap hops (planner may switch to bitmap scan — 4.5).

### 4. Visualization (ASCII)




### 5. Real Production Example

A multi-tenant SaaS on MySQL keys `events` with PK `(tenant_id, created_at, id)` — all a tenant's recent events are physically contiguous → tenant dashboards read a handful of pages. The same schema ported to Postgres lost that locality (heap is insert-ordered); the fix was BRIN/btree on `(tenant_id, created_at)` + periodic `pg_repack` ordering, or partitioning by tenant bucket. Interviewers use this to test whether you know the storage difference has *design* consequences.

## 6. Common Interview Questions

- "Clustered vs non-clustered index?" (and "how does Postgres differ from MySQL here?")
- "Why does InnoDB secondary lookup do two tree traversals?"
- "Why should InnoDB PKs be small and sequential?" (secondary index size; page splits)
- "What's a HOT update?" (Postgres senior signal)
- "What does CLUSTER do in Postgres and what's its catch?" (one-time, exclusive lock, not maintained)

## 7. Common Mistakes

- Saying "the PK is the clustered index" about Postgres.
- Choosing UUID PKs in InnoDB → clustered page-split storm *and* fat secondary indexes.
- Overlooking that in InnoDB a covering secondary index avoids the second descent (index contains the PK implicitly).
- Assuming physical order persists in Postgres after `CLUSTER`.

## 8. Best Practices

- InnoDB: small monotonic PK always; put frequently-range-scanned dimension first in the PK only when the access pattern justifies it.
- Postgres: rely on indexes + correlation; consider partitioning for locality; design for HOT (avoid indexing hot-updated columns; `fillfactor` 85–90 on churn tables).
- Know which engine your interview question is about — ask if ambiguous; it changes the answer.

## 9. Coding Questions

1. For InnoDB table `orders(PK id bigint, INDEX(user_id))`: count tree descents for (a) `WHERE id=5`, (b) `WHERE user_id=7` selecting `*`, (c) `WHERE user_id=7` selecting `user_id, id` only.
2. Postgres table with indexes on (a) `status` (updated constantly) — explain why removing that index doubled write throughput (non-HOT → HOT updates).

## 10. SQL Examples

```
-- Postgres: check index/heap correlation (drives range-scan cost)
SELECT attname, correlation FROM pg_stats
WHERE tablename = 'orders' AND attname IN ('id','created_at','user_id');

-- One-time physical reorder (locks table exclusively!)
CLUSTER orders USING idx_orders_created;

-- HOT update ratio (are updates dodging index maintenance?)
SELECT n_tup_upd, n_tup_hot_upd,
       round(n_tup_hot_upd::numeric / nullif(n_tup_upd,0), 2) AS hot_ratio
FROM pg_stat_user_tables WHERE relname = 'orders';
```

## 11. Optimization Techniques

- InnoDB: exploit clustering — design the PK so your hottest range scan is a physical scan.
- Postgres: `fillfactor=90` + drop indexes on frequently-updated columns to maximize HOT.
- `pg_repack` for online physical reordering / bloat removal without CLUSTER's lock.

## 12. Follow-up Questions

- "Why do Postgres index-only scans need a visibility map?" (heap holds MVCC visibility; see 4.5)
- "You migrate MySQL→Postgres and tenant queries slow down 5x — hypothesis?" (lost clustering locality)
- "InnoDB: why can a covering index be even more valuable than in Postgres?" (skips the second descent entirely)

---

# Chapter 4.3 — Composite Indexes & the Leftmost Prefix Rule

## 1. Why Interviewers Ask This

The most practical index question there is: "given this query, design the index" — and it's almost always a composite. Column *order* is where candidates fail.

## 2. Core Concept

A composite index on `(a, b, c)` sorts entries by `a`, then `b` within `a`, then `c` within `(a,b)` — like a phone book (last name, first name, middle).

**Leftmost prefix rule**: the index can efficiently serve predicates that constrain a *contiguous prefix* of the columns: - `a = ?` ✔ `a = ? AND b = ?` ✔ `a = ? AND b = ? AND c = ?` ✔ - `b = ?` alone ✖ (b values are scattered across all a's) - `a = ? AND c = ?` → index navigates on `a`, then **filters** c inside the a-range (c isn't contiguous without b).

**Range stops the key**: after a range/inequality on a column, later columns can't be used for navigation — `a = ? AND b > ? AND c = ?` navigates on (a, b-range), filters c. Hence the design rule: **equality columns first, then the (one) range column, then ORDER BY/covering columns**. ("ESR": Equality, Sort, Range — MongoDB's mnemonic, works for SQL with the nuance that sort-then-range vs range-then-sort depends on which preserves order.)

**ORDER BY**: an index on `(a, b)` serves `WHERE a = ? ORDER BY b` with zero sorting. Mixed directions need matching index directions (`(a ASC, b DESC)`).

Column order also decides **reusability**: `(a,b)` serves a-only queries → an extra `(a)` index is redundant.

## 3. Internal Working

The B+Tree key is the concatenation `(a,b,c)`; comparison is lexicographic. `a=5 AND b=7` descends directly to the `(5,7,*)` leaf range and walks it. `b=7` alone would require probing inside every distinct `a` — Postgres *can* still choose the index and filter (or in some engines "skip scan"), but it reads the whole index → usually not worth it → planner picks seq scan → "why is my index ignored" (4.6).

## 4. Visualization (ASCII)

```
Index (city, age):  entries sorted lexicographically
(Austin,22) (Austin,25) (Austin,31) | (Boston,19) (Boston,25) | (Cairo,40) ...
└──────── city=Austin contiguous ──────┘

WHERE city='Boston' AND age=25   → descend to (Boston,25): direct ✔
WHERE city='Boston' AND age>20   → descend to (Boston,20), walk while city=Boston ✔
WHERE age=25                     → 25s scattered: (Austin,25),(Boston,25)... ✖ full index
WHERE city>'B' AND age=25        → range on city first → age unusable for navigation
                                   (equality first, THEN range!)

ORDER BY: WHERE city='Austin' ORDER BY age  → rows come out pre-sorted, no Sort node ✔
```

## 5. Real Production Example

Uber-style trips query: `WHERE driver_id = ? AND status = 'completed' AND started_at >= ? ORDER BY started_at DESC LIMIT 20`. Correct index: `(driver_id, status, started_at DESC)` — two equalities, then the range/sort column (here range and sort are the same column: ideal). The team originally had `(started_at, driver_id)` — planner scanned weeks of index for one driver. Flipping column order took p99 from 900ms to 3ms. This *exact* shape is the most common index-design interview exercise.

## 6. Common Interview Questions

- "Index on (a,b,c): which of these five WHERE clauses can use it, and how fully?"
- "Design the index for this query." (state the ESR reasoning aloud)
- "Does `(a,b)` make a separate `(a)` index unnecessary? What about `(b)`?"
- "Why is `(range_col, eq_col)` worse than `(eq_col, range_col)`?"
- "How do you index `WHERE a=? ORDER BY b DESC LIMIT 10`?"

## 7. Common Mistakes

- Putting the range/timestamp column first "because we always filter by date."
- One index per column (`(a)`, `(b)`, `(c)`) expecting them to combine like a composite — bitmap-AND exists but is far weaker than the right composite.
- Ignoring ORDER BY when designing — the sort is often the expensive part LIMIT-queries need the index for.
- Redundant indexes (`(a)` alongside `(a,b)`) taxing every write.
- Believing "most selective column first" universally — for a *single* query the prefix-usability and ESR rules dominate; selectivity-first matters when comparing equality columns that are all always present.

## 8. Best Practices

- Design from the query set, not the table: list the WHERE/ORDER BY shapes, then cover them with the fewest composites exploiting prefixes.
- Equality columns (always-present ones first) → then the sort column if it lets LIMIT short-circuit → then range → then INCLUDE covering columns (4.4).
- Verify with `EXPLAIN`: you want `Index Cond:` to contain all intended columns; anything under `Filter:` is being checked row-by-row after navigation.

## 9. Coding Questions

1. Given queries: `(tenant_id=?, created_at range, ORDER BY created_at DESC)`, `(tenant_id=?, status=?)`, `(tenant_id=? , status=?, created_at range)` — design the minimal index set. (One index `(tenant_id, status, created_at DESC)` + one `(tenant_id, created_at DESC)`.)
2. For `WHERE a=1 AND c=3` on index `(a,b,c)`: describe exactly which part navigates and which filters, and estimate rows read if a=1 has 100k rows and c=3 matches 1%.

## 10. SQL Examples

```
CREATE INDEX CONCURRENTLY idx_trips_driver
ON trips (driver_id, status, started_at DESC);

EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM trips
WHERE driver_id = 42 AND status = 'completed'
  AND started_at >= now() - interval '30 days'
ORDER BY started_at DESC
LIMIT 20;
-- Want: Index Scan using idx_trips_driver
--       Index Cond: (driver_id=42 AND status='completed' AND started_at >= ...)
--       (no Sort node, Limit stops after 20 entries)

-- Anti-pattern check: is a column being filtered instead of navigated?
-- "Filter: (status = 'completed')" under an index scan = wrong column order.
```

## 11. Optimization Techniques

- Merge overlapping indexes into one composite serving multiple query shapes via prefixes.
- Match index direction to ORDER BY for mixed-direction sorts.
- If you truly need `b`-only queries too, that's a second index — measure whether the write tax is worth it.

## 12. Follow-up Questions

- "MySQL 8 / Oracle have skip scans — what do they do and when do they help?" (iterate distinct a values, probe b within each; helps only when a has few distinct values)
- "Index is `(a,b)` and the query is `WHERE a IN (1,2,3) AND b = 5` — how does the scan work?" (three sub-descents, one per a value — IN behaves like multiple equalities)
- "Why might `(a,b)` beat `(b,a)` even if b is more selective?" (a is the always-present equality / prefix reuse for a-only queries / ORDER BY b within a)

---

# Chapter 4.4 — Covering, Partial & Unique Indexes

## 1. Why Interviewers Ask This

These three are the "senior toolkit": covering indexes remove heap access entirely, partial indexes exploit workload skew, unique indexes are correctness tools. Interviewers use them to see if you optimize with intent rather than blanket indexing.

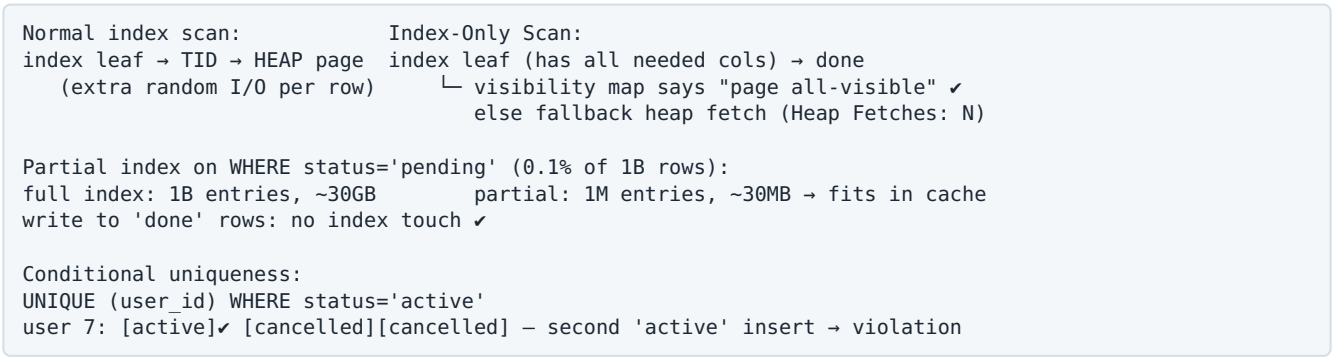
## 2. Core Concept

- **Covering index**: contains every column the query needs → **Index-Only Scan**, heap never touched. Postgres: either put columns in the key or use `INCLUDE (col, ...)` (payload-only: not sortable/searchable, but covering — keeps the key compact and works with UNIQUE).
- **Partial index**: `CREATE INDEX ... WHERE predicate` — indexes only matching rows. Tiny, cheap to maintain, and *only* usable when the query's WHERE provably implies the predicate. Killer feature for skewed data: index the 0.1% `status='pending'` rows, ignore the billions of `'done'`.
- **Unique index**: enforces uniqueness (it *is* how PK/UNIQUE constraints are implemented); combines with partial → **conditional uniqueness** ("one active subscription per user") and with expressions → **normalized uniqueness** (`lower(email)`).

## 3. Internal Working

- Index-only scans in Postgres must still answer MVCC visibility: the **visibility map** marks heap pages where all tuples are visible to everyone; for those, no heap check. Pages dirtied since last VACUUM force heap fetches (`Heap Fetches: N` in EXPLAIN — high N = vacuum needed).
- Partial index maintenance: writes touch it only when the row matches the predicate — this is why it's nearly free on the write path for rare states.
- Unique enforcement: insert descends; on existing equal key, if the owning transaction is still in flight, the inserter **waits** (this serializes concurrent same-key inserts — the mechanism behind `ON CONFLICT`).

## 4. Visualization (ASCII)




## 5. Real Production Example

Job-queue pattern (used at every company): workers poll `SELECT id FROM jobs WHERE status='pending' ORDER BY created_at LIMIT 100`. Table is 500M rows, 99.9% terminal states. Partial index `ON jobs (created_at) WHERE status='pending'` is a few MB, always cached, and the poll is a sub-ms index-only-ish scan. Without it: full index (or seq scan) over half a billion rows every few seconds. Follow-up they'll ask: pair it with `FOR UPDATE SKIP LOCKED` (Module 6) for concurrent workers.

## 6. Common Interview Questions

- "What's a covering index / index-only scan, and what can prevent the 'only' part in Postgres?" (visibility map / vacuum)
- "INCLUDE vs putting the column in the key?" (payload not searchable/sortable; smaller internal pages; works with UNIQUE)

- "How do you index a status column where 99% of rows are 'done'?" (partial — NOT a plain index on the low-cardinality column)
- "Enforce one active session per user at the DB level."
- "Why did `Heap Fetches` spike and the index-only scan get slow?" (bloat/dirty pages → vacuum)

## 7. Common Mistakes

- Covering "just in case" with many INCLUDEs → fat index, write amplification, less of it cached.
- Partial index whose predicate the query doesn't literally imply (`WHERE status = ANY($1)` can't use `WHERE status='pending'` index) — parameterized predicates break partial-index matching.
- Expecting index-only scans without healthy autovacuum.
- Application-level uniqueness checks (SELECT then INSERT) instead of unique indexes — race conditions; the DB check is the only atomic one.

## 8. Best Practices

- Design covering indexes for the handful of hottest, narrowest queries (id-list fetches, existence checks, pagination keys).
- Partial indexes for: rare states (pending/failed), soft-delete filters (`WHERE deleted_at IS NULL`), tenant-specific hot subsets.
- Express business uniqueness in DDL: partial unique + expression unique indexes are executable specifications.
- Monitor `Heap Fetches` and `n_dead_tup`; tune autovacuum per hot table (`autovacuum_vacuum_scale_factor` down).

## 9. Coding Questions

1. Make this an index-only scan: `SELECT user_id, created_at FROM orders WHERE user_id = ? AND created_at >= ?` — write the index (key `(user_id, created_at)`; nothing else needed — explain why INCLUDE is unnecessary here).
2. Enforce: at most one default payment method per user; emails unique case-insensitively; SKU unique among non-deleted products. (Three indexes.)

## 10. SQL Examples

```
-- Covering with INCLUDE (key stays compact & unique-compatible)
CREATE UNIQUE INDEX idx_users_email ON users (lower(email)) INCLUDE (id, display_name);

-- Partial: hot subset only
CREATE INDEX idx_jobs_pending ON jobs (created_at) WHERE status = 'pending';

-- Conditional uniqueness
CREATE UNIQUE INDEX one_default_pm ON payment_methods (user_id) WHERE is_default;

-- Soft-delete-aware uniqueness
CREATE UNIQUE INDEX sku_live ON products (sku) WHERE deleted_at IS NULL;

-- Verify index-only behavior
EXPLAIN (ANALYZE, BUFFERS)
SELECT user_id, created_at FROM orders WHERE user_id = 42 AND created_at >= '2026-01-01';
-- Want: Index Only Scan ... Heap Fetches: 0
```

## 11. Optimization Techniques

- Aggressive autovacuum on tables backing index-only scans (visibility map freshness is the whole trick).
- Replace `(status)` full index with partial per hot status; drop the full one.
- Covering index as a "vertical partition": for a wide table with one hot narrow query, the covering index acts as a narrow copy of the table.

### 12. Follow-up Questions

- "The partial index stopped being used after switching to a prepared statement with $1 for status — why?" (planner can't prove predicate implication for parameters; generic plan)
- "How would you do 'one active per user' in MySQL, which lacks partial indexes?" (generated column trick: nullable column = user_id when active else NULL, unique on it)
- "When does a covering index become the wrong call?" (write-heavy column in payload; index approaching table size)

---

## Chapter 4.5 — Scan Types: Sequential, Index, Index-Only, Bitmap

### 1. Why Interviewers Ask This

Reading EXPLAIN requires knowing the four access paths and — the senior twist — **why seq scan is often correct**. "Why is Postgres not using my index?" is the most common real-world question this knowledge answers.

### 2. Core Concept

| Scan | What it does | When optimal |
|---|---|---|
| **Seq Scan** | Read the whole heap, sequentially | Large fraction of rows needed (rule of thumb: >5–10%), tiny tables, no usable index |
| **Index Scan** | Descend B+Tree, fetch heap row per match (random I/O) | Few rows, high selectivity, or ORDER BY exploitation |
| **Index-Only Scan** | Index Scan without heap fetches (covering + visibility map) | Covering queries on well-vacuumed tables |
| **Bitmap Index/ Heap Scan** | Collect all matching TIDs into a bitmap, sort by page, then fetch heap pages sequentially | Medium selectivity (thousands–millions of rows); combining multiple indexes (BitmapAnd/Or) |

The core economics: an index scan costs a **random heap page read per row** (worst case); a seq scan costs **every page once, sequentially**. There's a crossover — around a few percent of the table — where seq scan wins. Bitmap scan is the middle ground: index precision + sequential heap access, at the cost of losing index order (needs a Sort if you wanted ORDER BY).

### 3. Internal Working

- Planner computes: seq = pages × seq_page_cost; index = descents + matched_rows × (random_page_cost × correlation-adjusted heap cost); bitmap = index read + bitmap build + sorted page fetches. `random_page_cost` default 4 assumes spinning disk — on SSD/cloud set 1.1–1.5 or the planner over-penalizes index scans (a real, common misconfiguration).
- Correlation matters: index on a column physically correlated with heap order (like insert-time timestamps) makes index range scans nearly sequential → cheap.
- Bitmap can degrade to **lossy** mode when `work_mem` is exceeded (stores pages, not tuples → rechecks rows: `Recheck Cond`).

### 4. Visualization (ASCII)

```
selectivity →  0.001%          0.1%            5%              80%
best path   →  Index Scan   Index/Bitmap    Bitmap/Seq        Seq Scan

Index Scan heap access:        Bitmap Scan heap access:
idx→p9, idx→p2, idx→p9,        idx→{p2,p2,p5,p9,p9,p9}
idx→p5 (random, p9 twice!)       └─ sort TIDs → read p2,p5,p9 once each, in order

BitmapAnd (two single-col indexes standing in for a missing composite):
idx_a(a=1) → bitmap A ┐
                      ├─ AND → fetch only pages in both
idx_b(b=2) → bitmap B ┘
```

## 5. Real Production Example

Analytics-ish query on the OLTP DB: `WHERE created_at >= now() - interval '90 days'` on a table where 90 days = 40% of rows. Engineer adds an index, planner ignores it, engineer force-hints… and it gets *slower* (40% of a TB via random I/O). Correct outcomes: accept the seq scan, partition by month (prune to 3 partitions), or BRIN. The interview form: "the planner ignores my index — is it wrong?" — the senior answer starts with "maybe it's right."

## 6. Common Interview Questions

- "When is a full table scan faster than an index scan?" (guaranteed question)
- "What's a bitmap heap scan and why does Postgres use it?"
- "Explain `Recheck Cond` / lossy bitmaps."
- "Why does `random_page_cost` matter and what should it be on SSDs?"
- "Query uses index for `LIMIT 10` but seq scan for `LIMIT 10000` — why?" (cost crossover)

## 7. Common Mistakes

- Treating Seq Scan in EXPLAIN as automatically bad.
- Comparing timings with cold vs warm cache and concluding the plan changed.
- Not knowing bitmap scans discard index ordering (surprise Sort node appears).
- Assuming `count(*)` uses "the index" trivially — it can index-only scan but still must check visibility (Module 5 covers COUNT).

## 8. Best Practices

- Judge plans by **pages touched** (`BUFFERS`) not by scan-type prejudice.
- Tune `random_page_cost` to storage reality; `effective_cache_size` to actual RAM.
- For medium-selectivity recurring filters, prefer a better composite/partial index over relying on BitmapAnd of single-column indexes.

## 9. Coding Questions

1. Table 100GB, 12.8M pages. Query matches 2M rows spread uniformly. Compare page reads: seq scan (12.8M sequential) vs index scan (≈2M random) vs bitmap (≈min(2M pages, clustered subset) sequentialized) — which wins on SSD vs HDD?
2. From `EXPLAIN (ANALYZE, BUFFERS)` output showing `Bitmap Heap Scan ... Heap Blocks: exact= 41235 lossy=182000`, diagnose and fix (work_mem too small for the bitmap).

## 10. SQL Examples

```
-- Watch the crossover live
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM orders WHERE created_at >= now() - interval '1 hour';
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM orders WHERE created_at >= now() - interval '1 year';
-- hour → Index Scan; year → Seq Scan (both correct)

-- SSD-appropriate costing
ALTER SYSTEM SET random_page_cost = 1.1;
SELECT pg_reload_conf();

-- Investigate "index ignored" honestly (session-local experiment)
SET enable_seqscan = off;      -- if index plan is now CHOSEN but SLOWER, planner was right
EXPLAIN (ANALYZE, BUFFERS) SELECT ...;
RESET enable_seqscan;
```

## 11. Optimization Techniques

- Increase correlation for hot range columns: partitioning, or BRIN on naturally-ordered data.
- Cluster occasionally (pg_repack) if one range access pattern dominates.
- Parallel seq scans (PG9.6+) make honest big scans much faster — check `Workers Launched`.

### 12. Follow-up Questions

- "Same query, index scan on replica but seq scan on primary — how?" (different stats/settings/ cache: e.g., ANALYZE timing or cost params differ)
- "How does this decision change in MySQL?" (optimizer cost model differs; no bitmap scans in InnoDB — index_merge instead; clustered PK changes heap-fetch economics)
- "What plan do you expect for `WHERE a=1 OR b=2` with separate indexes on a and b?" (BitmapOr)

---

## Chapter 4.6 — When Indexes Fail: Cardinality, Selectivity & Ignored Indexes

### 1. Why Interviewers Ask This

"Why is my index not being used?" is the highest-frequency database question in real work. The answer catalog below *is* the interview answer.

### 2. Core Concept

**Cardinality** = number of distinct values in a column (`n_distinct`). **Selectivity** = fraction of rows a predicate matches (matched/total; lower = more selective). Indexes pay off when predicates are selective. The planner estimates selectivity from statistics (histograms + most-common-values lists) and compares plan costs — the index loses when estimated selectivity is poor **or** when the estimate is wrong.

**The "index ignored" catalog** (memorize — this answers the question in any interview): 1. **Low selectivity**: predicate matches too much (status='done' on 95% of rows) — seq scan is genuinely cheaper. 2. **Function/expression on the column**: `WHERE date(created_at)=...`, `lower(email)=...` — needs a matching expression index. 3. **Type mismatch / implicit cast**: `WHERE bigint_col = '42'::text`-ish situations, joining varchar to int, numeric literal against text column — the cast wraps the column. 4. **Leading wildcard / pattern**: `LIKE '%foo'` (no prefix to navigate). Also plain `LIKE` under non-C collation needs `text_pattern_ops`. 5. **Leftmost prefix violated**: composite `(a,b)` with only `b` predicated. 6. **OR across columns**: may need BitmapOr or a UNION rewrite; single index can't serve it. 7. **Inequality/NOT**: `<>`, `NOT IN` — matches almost everything. 8. **Stale/insufficient statistics**: planner thinks the predicate matches 40% when it matches 0.01% (or vice versa) — `ANALYZE`, raise the stats target, extended statistics for correlated columns. 9. **Cost misconfiguration**: `random_page_cost=4` on SSD biases against index scans. 10. **Tiny table**: everything fits in one page — seq scan is correct, stop worrying. 11. **Parameterized/generic plans**: prepared statement generic plan can't use a partial index or assumes average selectivity (`plan_cache_mode`). 12. **NULL semantics**: `col = NULL` never true; `IS NULL` *is* indexable (btree stores NULLs) — but expression/partial coverage may be needed.

### 3. Internal Working

Stats pipeline: `ANALYZE` samples rows → per-column `n_distinct`, MCV list (most common values + frequencies), equi-depth histogram → selectivity for `col = x`: if x in MCV use its frequency, else (1 − MCV mass)/(n_distinct − MCV count); ranges use histogram bucket interpolation. Multi-predicate selectivities multiply assuming **independence** — correlated columns (city→country) get catastrophically underestimated → `CREATE STATISTICS ... (dependencies)`. Wrong stats → wrong cost → wrong plan, regardless of what indexes exist.

## 4. Visualization (ASCII)

```
Histogram (created_at):  [Jan|Feb|Mar|...|Dec]  each bucket = 1/12 of rows
WHERE created_at >= Dec-01 → est. ~8% → index viable
WHERE created_at >= Feb-01 → est. ~92% → seq scan

MCV (status): done:0.94  cancelled:0.04  pending:0.02
WHERE status='pending' → 2% → maybe bitmap/partial idx
WHERE status='done'    → 94% → seq scan (index correctly ignored)

Correlation trap: WHERE city='SF' AND state='CA'
independent est: P(city)×P(state) = 0.01×0.03 = 0.0003 (way low if all SF is in CA)
→ planner picks NL for "3 rows", gets 300k → disaster.  Fix: extended statistics.
```

## 5. Real Production Example

Netflix-scale table, query `WHERE country='US' AND platform='tv' AND event_date = yesterday`. Planner multiplies three independent selectivities, estimates 900 rows, picks nested loop into a join — actual 40M rows (US+tv are correlated), query melts a replica. Fix: `CREATE STATISTICS (dependencies, ndistinct) ON country, platform` + `ANALYZE`. Second act interviewers love: after a bulk backfill, everything is slow until `ANALYZE` runs — stale stats after mass writes.

## 6. Common Interview Questions

- "List reasons an index gets ignored." (rapid-fire the catalog)
- "Define cardinality vs selectivity; how does the planner estimate selectivity?"
- "Index on `email` exists; `WHERE lower(email)=?` seq-scans — fix it two ways." (expression index; citext)
- "Query fast with literal, slow as prepared statement — why?" (generic plan/parameter sniffing)
- "When is indexing a boolean column worthwhile?" (almost never plain; partial index on the rare value)

## 7. Common Mistakes

- Force-hinting (`enable_seqscan=off` in prod code) instead of fixing stats/predicates.
- Indexing low-cardinality columns whole instead of partial indexes on the rare states.
- Missing that ORMs generate casts/functions (e.g., timezone conversions) that silently break sargability.
- Never running `ANALYZE` after bulk loads/migrations.

## 8. Best Practices

- Sargable predicates by habit: bare column on the left, constants/expressions on the right.
- After bulk changes: `ANALYZE` explicitly; for skewed big tables raise per-column stats target (`ALTER TABLE ... ALTER COLUMN ... SET STATISTICS 1000`).
- Extended statistics for known correlated pairs.
- Test with production-shaped data — 1k-row dev tables seq-scan everything and hide all of this.

## 9. Coding Questions

1. For each, say if the index on the mentioned column is used and why: `WHERE upper(code)='X'`; `WHERE price*1.1 > 100`; `WHERE created_at::date = current_date`; `WHERE id IN (1,2,3)`; `WHERE name LIKE 'ab%'`; `WHERE name LIKE '%ab'`.
2. Write the fix for each broken one (expression index / rewrite to sargable / trigram index).

### 10. SQL Examples

```
-- Inspect what the planner believes
SELECT attname, n_distinct, most_common_vals, most_common_freqs
FROM pg_stats WHERE tablename = 'orders' AND attname = 'status';

-- Correlated-column fix
CREATE STATISTICS orders_geo (dependencies, ndistinct) ON country, platform FROM events;
ANALYZE events;

-- Expression index to restore sargability
CREATE INDEX ON users (lower(email));
-- and/or rewrite: WHERE created_at >= d AND created_at < d + 1 instead of ::date

-- Trigram index for %substring% search
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE INDEX ON products USING gin (name gin_trgm_ops);
SELECT * FROM products WHERE name ILIKE '%widget%';   -- now indexable
```

### 11. Optimization Techniques

- `auto_explain` with `log_min_duration` to capture real bad plans with real parameters.
- `plan_cache_mode = force_custom_plan` for statements whose parameter skew breaks generic plans.
- Periodic index audit: unused indexes (drop), missing indexes (from `pg_stat_statements` top offenders — Module 5/11).

### 12. Follow-up Questions

- "n_distinct is wildly wrong on a 2B-row table — why and what do you do?" (sampling limits; set n_distinct manually via ALTER TABLE ... SET (n_distinct=...))
- "How do hints work in MySQL/Oracle and why does Postgres refuse them?" (philosophy: fix inputs, not outputs; pg_hint_plan exists if forced)
- "Explain exactly why the generic plan can't use a partial index." (predicate proof requires the parameter value)

---

## Chapter 4.7 — How to Choose Indexes (Design Method)

### 1. Why Interviewers Ask This

This is the synthesis question: "here's the workload — design the indexes." It tests everything above at once, plus the discipline of *not* over-indexing.

### 2. Core Concept — The Method

1. **Inventory the queries** (from code / pg_stat_statements), not the table. Rank by frequency × latency.
2. For each hot query, write the ideal index: **equality columns → sort column → range column → INCLUDE covering columns**; partial WHERE for skewed constant predicates.
3. **Merge**: collapse indexes that are prefixes of others; one composite often serves several queries.
4. **Price the writes**: each index taxes every insert and non-HOT update; on write-heavy tables demand stronger justification.
5. **Verify** with EXPLAIN (ANALYZE, BUFFERS) against production-scale data; confirm Index Cond vs Filter, no unexpected Sort, Heap Fetches ≈ 0 where intended.
6. **Monitor and prune**: unused-index report monthly; every index must earn its keep.

### 3. Internal Working

Why merging works: a B+Tree on `(a,b,c)` *contains* perfect indexes on `(a)` and `(a,b)` as prefixes. Why pricing writes matters: an insert into a table with k indexes does 1 heap write + k index descents/inserts (plus WAL for each); index count linearly scales write cost and vacuum work.

## 4. Visualization (ASCII)

```
Query set                                    Index plan
Q1: WHERE user_id=? ORDER BY created DESC   ┐
Q2: WHERE user_id=? AND created>=?          ├─► (user_id, created_at DESC)   [serves Q1,Q2]
Q3: WHERE user_id=? AND status=? ORDER BY created DESC
                                            └─► (user_id, status, created_at DESC) [Q3; also Q1? no—
                                                status gap breaks prefix for Q1's sort]
Q4: WHERE status='pending' (0.1%)           ──► partial (created_at) WHERE status='pending'
write cost: 3 indexes ≈ acceptable; audit quarterly
```

## 5. Real Production Example

Stripe-like `payments` table review: 14 indexes accreted over years; write p99 degrading and autovacuum永 behind. Audit finds 5 unused, 3 redundant prefixes, 2 replaceable by one composite + partial. Down to 6 indexes: insert latency −40%, index storage −60%, no read regressions. The interview lesson: index design is a *portfolio* problem, not per-query.

## 6. Common Interview Questions

- "Design indexes for this schema + these five queries." (the standard exercise)
- "How do you decide an index is worth its write cost?"
- "How do you find missing and unused indexes in production?"
- "What changes about indexing strategy for a write-heavy vs read-heavy table?"

## 7. Common Mistakes

- Indexing every column / every FK reflexively without workload evidence (FK child columns are usually justified — but by the delete/join pattern, cite it).
- Designing for one query in isolation and accumulating near-duplicates.
- Never revisiting: indexes for deleted features running forever.
- Skipping the verify step — "should use the index" is not "does."

## 8. Best Practices

- Keep an "index ledger": every index annotated with the queries it serves.
- Standard per-table starting kit: PK; FK columns used in joins/deletes; the main list-view composite `(tenant/user, created_at DESC)`; partial for hot rare states. Everything else must argue its way in.
- Create with `CONCURRENTLY`, drop with `CONCURRENTLY` (PG 14+ for drop), always.

## 9. Coding Questions

1. Given `messages(id, chat_id, sender_id, created_at, is_deleted, body)` and queries: recent messages per chat (paginated), unread count per chat, user's sent messages by date, full-text search in a chat — produce the full index DDL with justifications.
2. Write the two monitoring queries: top time-consuming statements missing indexes (pg_stat_statements join) and unused indexes (pg_stat_user_indexes).

## 10. SQL Examples

```
-- The chat example answered:
CREATE INDEX ON messages (chat_id, created_at DESC) WHERE NOT is_deleted;  -- feed + pagination
CREATE INDEX ON messages (sender_id, created_at DESC);                      -- user's messages
CREATE INDEX ON messages USING gin (to_tsvector('english', body));          -- search
-- unread: usually a per-member counter table, not an index (design > index)

-- Unused index report
SELECT s.indexrelname, s.idx_scan,
       pg_size_pretty(pg_relation_size(s.indexrelid)) AS size
FROM pg_stat_user_indexes s
JOIN pg_index i ON i.indexrelid = s.indexrelid
WHERE s.idx_scan = 0 AND NOT i.indisunique
ORDER BY pg_relation_size(s.indexrelid) DESC;
```

## 11. Optimization Techniques

- HypoPG extension: hypothetical indexes — test whether the planner *would* use an index before paying to build it.
- Stage big index builds off-peak; on partitioned tables build per-partition then attach.
- For multi-tenant systems, lead almost every index with `tenant_id` — locality + prefix reuse.

## 12. Follow-up Questions

- "An index build with CONCURRENTLY failed halfway — what state is the system in?" (INVALID index: taxes writes, serves no reads — must drop/rebuild)
- "How would your strategy differ on the read replica vs primary?" (same physical indexes — streaming replicas can't diverge; that's a partitioning-of-workload argument, or logical replication if you truly need different indexes)
- "Index or partition — how do you decide?" (selectivity/pruning granularity vs maintenance; Module 7)

---

# Module 4 — Practice Problems

## Easy (5)

1. Index `(customer_id, order_date, status)` exists. For each query say used/partially used/unused: `WHERE customer_id=1`; `WHERE order_date>'2026-01-01'`; `WHERE customer_id=1 AND status='paid'`; `WHERE customer_id=1 AND order_date>'2026-01-01' ORDER BY order_date`.
2. Why can `LIKE 'john%'` use a btree but `LIKE '%john'` cannot? What index fixes the latter?
3. Compute approximate B+Tree height for 50M rows with fanout 250, and the max page reads for a point lookup with a cold cache.
4. `WHERE status='active'` matches 60% of rows. Index or not? What if it matches 0.5%?
5. Explain `Heap Fetches: 48210` on an Index Only Scan and the fix.

## Medium (5)

1. Design the minimal index set for: `WHERE seller_id=? AND state=? ORDER BY listed_at DESC LIMIT 24`; `WHERE seller_id=? ORDER BY listed_at DESC`; `WHERE state='pending_review'` (0.2% of rows) — justify each column position.
2. A prepared statement runs 200x slower than the same query with literals. Explain generic vs custom plans, how partial-index matching fails, and two fixes.
3. InnoDB table, PK is UUIDv4 char(36), five secondary indexes, heavy inserts. Enumerate every cost this design incurs and the migration path.
4. `EXPLAIN (ANALYZE, BUFFERS)` shows: `Bitmap Heap Scan (Recheck Cond..., Heap Blocks: exact=1200 lossy=890000)` under `BitmapAnd` of two indexes. Diagnose both problems and fix (work_mem; replace BitmapAnd with a composite).
5. Soft-deleted rows are 70% of a table; all queries filter `deleted_at IS NULL`. Redesign the indexing (partial everything) and estimate the size/perf impact.

## Hard (5)

1. Time-series table, 4B rows, insert-only, queries are `device_id + time range`. Compare btree `(device_id, ts)`, BRIN on `ts`, and monthly partitions + `(device_id, ts)` per partition — for insert cost, query cost, and storage. Recommend and defend.
2. The planner estimates 12 rows, actual is 2.1M, for `WHERE city='SF' AND category='food'`. Walk the full diagnosis (MCV/histogram inspection, independence assumption) and fix chain (ANALYZE → stats target → extended statistics → rewrite), explaining what each step changes.
3. Design uniqueness for: "an email may be reused after account deletion, but only one live account per email; deleted accounts keep their email for audit." Write the DDL and prove the race-safety of concurrent signups.
4. A covering index made reads perfect but write p99 regressed 30% and autovacuum can't keep up. Explain the mechanics (non-HOT updates, index churn, visibility map invalidation) and design the compromise (drop payload columns, fillfactor, split hot/cold columns to a separate table).
5. You may add exactly ONE index to `events(tenant_id, user_id, type, ts, payload)` serving: (a) tenant dashboard: `tenant_id + ts range ORDER BY ts DESC`; (b) user timeline: `tenant_id + user_id ORDER BY ts DESC`; (c) type analytics: `tenant_id + type + ts range`. Choose, quantify what each query pays with your choice, and argue why no single index serves all three optimally.

---

*Next: Module 5 — Query Optimization*

# MODULE 5 — Query Optimization

> The "can you actually fix a slow query?" module. Interviewers hand you a plan or a scenario and watch your process. Have a *method*, not vibes.

Chapters: 5.1 EXPLAIN & EXPLAIN ANALYZE — Reading Execution Plans 5.2 The Cost-Based Optimizer & Statistics 5.3 Plan Transformations: Predicate Pushdown & Join Reordering 5.4 Sorting & Filtering at Scale 5.5 Pagination (OFFSET vs Keyset) 5.6 COUNT Optimization 5.7 The N+1 Query Problem

---

## Chapter 5.1 — EXPLAIN & EXPLAIN ANALYZE

### 1. Why Interviewers Ask This

"Here's a slow query — what do you do first?" The expected first move is always EXPLAIN (ANALYZE, BUFFERS). Then they hand you output and grade how you read it.

### 2. Core Concept

- `EXPLAIN` — the plan and **estimates** only (doesn't run the query).
- `EXPLAIN ANALYZE` — runs it, adds **actual** times, row counts, loops. (Careful: it executes — wrap DML in `BEGIN; ... ROLLBACK;`.)
- `BUFFERS` — pages touched (shared hit = cache, read = disk, written, temp = spills). The honest cost currency.
- Plans are trees; execution flows leaves → root. Read inner/deepest nodes first.

The five things to extract, in order: 1. **Estimate vs actual rows** per node — the #1 diagnostic. Off by >10x = stats problem, and every decision above that node is suspect. 2. **loops=** — a node's true cost is `actual time × loops` (nested loop inners run N times). 3. **Where the time actually goes** — find the node(s) owning the elapsed time. 4. **Spills** — `Sort Method: external merge Disk: ...`, `Batches: >1`, lossy bitmaps → work_mem. 5. **Scan/join choices vs your expectation** — Filter vs Index Cond, unexpected Seq Scan or Sort.

### 3. Internal Working

Costs are in abstract units: `cost=startup..total`; `rows` = estimated output rows; `width` = avg row bytes. ANALYZE adds `actual time=first..last rows=N loops=M`. `Rows Removed by Filter` shows wasted reads (fetched then discarded — an index-shape smell). BUFFERS distinguishes "slow because disk" from "slow because CPU/rows". `auto_explain` logs plans of slow production queries with real parameters — the way you catch what you can't reproduce.

### 4. Visualization (ASCII)

```
Limit (actual time=0.1..812.4 rows=50)
  └─ Sort (actual time=812.3.. rows=50)          ← time lives HERE
       Sort Method: external merge  Disk: 480MB  ← spill! work_mem
       └─ Hash Join (rows est=1200 actual=9,400,000)   ← 7,800x misestimate!
            ├─ Seq Scan on orders (Filter: status='paid'
            │        Rows Removed by Filter: 88,000,000)  ← index-shape smell
            └─ Hash └─ Seq Scan on users

Reading order: bottom-up. Root cause chain: misestimate → wrong join input sizes →
oversized sort → disk spill. Fix stats & index BEFORE touching work_mem.
```

### 5. Real Production Example

The universal incident: ORM-generated query is fine for months, then a customer with 4M rows (vs median 200) hits it. `auto_explain` captures the plan: nested loop chosen on a 12-row estimate, actual 4M → 4M index probes. Fix: extended statistics + a composite index; the deeper fix is testing with skewed tenants. Interviewers replay exactly this with printed plans.

## 6. Common Interview Questions

- "Walk me through this EXPLAIN output." (practice verbalizing: bottom-up, estimates vs actuals, time attribution)
- "EXPLAIN vs EXPLAIN ANALYZE — and when is ANALYZE dangerous?" (DML executes; also its timing overhead can distort very hot nodes)
- "What does `Rows Removed by Filter: 88M` tell you?"
- "Query is slow in prod, fast in staging — what do you compare?" (plans, stats, data volume, parameters, cache, settings)

## 7. Common Mistakes

- Reading only the top line / total cost and stopping.
- Ignoring `loops` and blaming the wrong node.
- Fixing the symptom node (raise work_mem) when a misestimate below caused it.
- Comparing one warm-cache run vs one cold run and declaring victory (run twice; check BUFFERS hit vs read).

## 8. Best Practices

- Always `EXPLAIN (ANALYZE, BUFFERS)`; format JSON + a visualizer (explain.dalibo / pev2) for big plans.
- Enable `pg_stat_statements` (aggregate top offenders) + `auto_explain` (individual bad runs) in every production system.
- Keep a before/after plan in every optimization PR — plans are the proof.

## 9. Coding Questions

1. Given a plan with `Nested Loop (rows=3 actual rows=1)` wrapping `Index Scan (loops=1,200,000, actual time=0.04)`, compute where the time went and name the fix.
2. Write the pg_stat_statements query returning the top 10 statements by total time with mean time and call counts.

## 10. SQL Examples

```
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT ...;

-- Safe ANALYZE for DML
BEGIN; EXPLAIN ANALYZE DELETE FROM jobs WHERE ...; ROLLBACK;

-- Top offenders
SELECT calls, round(total_exec_time)::bigint AS total_ms,
       round(mean_exec_time,1) AS mean_ms, rows, query
FROM pg_stat_statements ORDER BY total_exec_time DESC LIMIT 10;

-- Log every plan slower than 500ms
-- postgresql.conf: shared_preload_libraries='pg_stat_statements,auto_explain'
--                  auto_explain.log_min_duration='500ms'  auto_explain.log_analyze=on
```

## 11. Optimization Techniques

- Diagnosis order: estimates → access paths → join algorithms → memory/spills → rewrite → (last) config. Say this order in interviews.
- `track_io_timing = on` for real read/write ms in BUFFERS output.

## 12. Follow-up Questions

- "Estimates are perfect but the query is still slow — next?" (it's doing exactly what you asked: reduce work — rewrite, pre-aggregate, cache, better index shape)
- "How do you EXPLAIN a query you can't reproduce?" (auto_explain with real params; pg_stat_statements to find it)

---

# Chapter 5.2 — The Cost-Based Optimizer & Statistics

## 1. Why Interviewers Ask This

"Why did the planner do that?" questions test whether you model the optimizer as a deterministic cost machine driven by statistics — because then every weird plan has a findable cause.

## 2. Core Concept

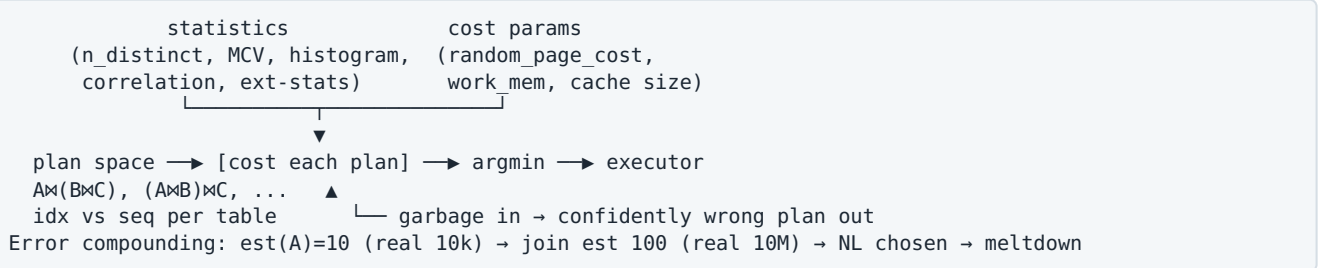
The optimizer: enumerate equivalent plans (access paths per table × join orders × join algorithms) → estimate each plan's cost from **statistics** and **cost parameters** → execute the cheapest. It is only as good as: - **Statistics**: per-column n_distinct, MCV lists, histograms, correlation (from ANALYZE / autovacuum's analyze); extended statistics for column correlations; expression statistics. - **Cost parameters**: `seq_page_cost=1`, `random_page_cost` (lower on SSD!), `cpu_tuple_cost`, `effective_cache_size` (how much of the index/table it may assume cached), `work_mem`. - **Cardinality propagation**: output-row estimates flow up the tree; errors compound multiplicatively through joins.

Postgres deliberately has **no hints** (core) — the philosophy: fix the inputs (stats, indexes, sargability), not the output. Know that MySQL/Oracle/SQL Server do have hints, and pg_hint_plan exists for emergencies.

## 3. Internal Working

- ANALYZE samples 300 × stats_target rows (default target 100 → 30k rows) — huge tables get sampling error, especially for n_distinct (systematically underestimated).
- Join size estimate: |A ⋈ B| ≈ |A|×|B| × selectivity(join pred), assuming independence and containment — skewed keys break it.
- Search space: exhaustive (dynamic programming) up to `geqo_threshold` (12 rels), then genetic algorithm (plans get nondeterministic!); `join_collapse_limit` (8) bounds reorder freedom — beyond it, your written join order partially binds.
- Plan caching: prepared statements switch to a **generic plan** after ~5 executions if it looks competitive — parameter-skew regressions appear exactly then (`plan_cache_mode` overrides).

## 4. Visualization (ASCII)




## 5. Real Production Example

After a Black-Friday-scale backfill inserts 400M rows, dashboards die. Nothing changed but the data: autovacuum's ANALYZE hadn't caught up, the planner still believed the table had 2M rows, picked nested loops everywhere. On-call runs `ANALYZE big_table;` — recovery in seconds. This "stats after bulk load" story is a standard senior screen scenario, verbatim.

## 6. Common Interview Questions

- "How does a cost-based optimizer decide between plans?"
- "What statistics does Postgres keep and how do they become stale?"
- "Why no hints in Postgres, and what do you do instead?"
- "What's the risk of prepared statements at plan level?" (generic plan vs skewed params)
- "Same query, different plan on two identical replicas — how?" (stats timing, config drift, GEQO nondeterminism)

## 7. Common Mistakes

- Assuming autovacuum keeps stats fresh through bulk operations (thresholds are proportional — 10% of a 2B-row table is 200M changes).
- Cranking work_mem globally (it's per sort/hash node per connection — OOM factory).
- "The optimizer is broken" — it's deterministic; find the wrong input.
- Forgetting expression stats: `WHERE lower(email)=...` has no column stats unless an expression index (or PG14 extended expression statistics) exists.

## 8. Best Practices

- Bulk load runbook ends with `ANALYZE` (and often `VACUUM (ANALYZE)`).
- Raise `default_statistics_target` (or per-column) for large skewed tables; add extended statistics for known correlations.
- Set `random_page_cost≈1.1`, `effective_cache_size≈75% RAM` on SSD/cloud boxes.
- Treat plan flips as incidents with a cause: diff stats & settings, check autoanalyze timestamps (`pg_stat_user_tables.last_autoanalyze`).

## 9. Coding Questions

1. Estimate like the planner: table 10M rows, `WHERE status='pending'` (MCV freq 0.02) AND `region='EU'` (freq 0.25) → independent estimate 50k rows. Real: pending is EU-only → 200k. What plan damage results and which feature fixes the estimate?
2. Write the query to find tables whose stats are stale relative to churn (`pg_stat_user_tables`: n_mod_since_analyze vs reltuples).

## 10. SQL Examples

```
-- What the planner believes about a table
SELECT reltuples::bigint AS est_rows, relpages FROM pg_class WHERE relname='orders';
SELECT last_analyze, last_autoanalyze, n_mod_since_analyze
FROM pg_stat_user_tables WHERE relname='orders';

-- Deeper stats for a skewed column + correlated pair
ALTER TABLE orders ALTER COLUMN status SET STATISTICS 1000;
CREATE STATISTICS orders_dep (dependencies) ON status, region FROM orders;
ANALYZE orders;

-- Force per-execution planning for a skew-sensitive prepared statement
SET plan_cache_mode = force_custom_plan;
```

## 11. Optimization Techniques

- For truly untamable queries: pg_hint_plan, or shape the SQL (materialized CTE as an optimization fence, `OFFSET 0` trick historically) — declare these last resorts.
- Partition big tables partly *for the planner*: per-partition stats are finer-grained.

## 12. Follow-up Questions

- "How would you detect a plan regression automatically?" (pg_stat_statements mean-time deltas, plan hash tracking, canary EXPLAIN in CI against a prod-stats snapshot)
- "What's the equivalent story in MySQL?" (persistent stats tables, ANALYZE TABLE, optimizer hints + histograms since 8.0)

---

# Chapter 5.3 — Plan Transformations: Predicate Pushdown & Join Reordering

## 1. Why Interviewers Ask This

These two transformations explain most "why is this equivalent query 100x faster" puzzles, and they're the vocabulary of data-engineering interviews too (Spark/warehouse pushdown).

## 2. Core Concept

- **Predicate pushdown**: apply filters as close to the data as possible — below joins, into subqueries/views/CTEs (PG12+), into partition pruning, into the index itself (Index Cond), down to remote sources (FDW) or storage formats (Parquet row groups). Less data flows up.
- **Join reordering**: join order changes intermediate result sizes by orders of magnitude; the planner searches orders to minimize intermediate rows (start with the most filtered/smallest effective inputs). Constrained by outer-join semantics and `join_collapse_limit`.

What *blocks* pushdown (know this list): - Volatile functions, `LIMIT`/`OFFSET` inside the subquery, window functions/ DISTINCT/GROUP BY boundaries (can't push a filter below an aggregate over different grain), `MATERIALIZED` CTEs, security barriers (RLS views), type-mismatched predicates.

## 3. Internal Working

The rewriter/planner hoists subqueries into the main query ("subquery pullup") when legal, then distributes quals to the lowest legal level. Equivalence classes propagate predicates across join keys: `a.id = b.a_id AND a.id = 5` ⇒ `b.a_id = 5` derived automatically. Outer joins restrict: a filter on the nullable side can't move below the join without changing semantics (the Module 3 ON/WHERE distinction is the user-visible face of the same rule).

## 4. Visualization (ASCII)

```
Naive:                          Pushed down:
      σ country='DE'                    ⋈ user_id
          |                            /          \
      ⋈ user_id               σ country='DE'   σ created>=…
      /        \                   |                |
users(50M)  orders(500M)      users→80k        orders→2M (partition pruned)
join input: 50M × 500M        join input: 80k × 2M   (≈ 4 orders of magnitude less)

Join order: ((F ⋈ small_dim) ⋈ big_dim) vs ((F ⋈ big_dim) ⋈ small_dim)
intermediate sizes differ 1000x — same result, different universe of cost
```

## 5. Real Production Example

A BI view `v_orders_enriched` (5-way join) queried as `SELECT * FROM v_orders_enriched WHERE tenant_id=? AND day=?`. Because views inline, the predicates push through to every base table and prune partitions — milliseconds. Someone "optimizes" by materializing the view's subquery per-day with a MATERIALIZED CTE → fence → five full scans. Undo → fast. Interviews test this as "why did adding MATERIALIZED slow it down?"

## 6. Common Interview Questions

- "What is predicate pushdown? Where does it apply in Postgres / Spark / Parquet?"
- "Why does join order matter if results are identical?"
- "What prevents a filter from being pushed into this subquery?"
- "Explain how `a.id=b.id AND a.id=5` filters b too." (equivalence classes)

## 7. Common Mistakes

- Filtering after aggregation in hand-written SQL when the filter is on the group key (put it in WHERE, not HAVING — pushdown by hand).
- MATERIALIZED CTEs / OFFSET-0 fences left in code as cargo cult.

- 10-way joins written in arbitrary order assuming infinite planner freedom (join_collapse_limit).
- Expecting pushdown through window functions or DISTINCT.

### 8. Best Practices

- Write filters at the lowest level you semantically can; don't rely on the planner for clarity-critical cases.
- Keep views thin and inlinable; avoid stacking aggregates in views that consumers will filter.
- For >8-table joins, write the join order you mean or raise `join_collapse_limit`.

### 9. Coding Questions

1. Rewrite so the date filter prunes partitions: `SELECT * FROM (SELECT *, row_number() OVER (...) rn FROM events) t WHERE t.day='2026-07-01'` (move the day predicate inside — legality argument included).
2. Given tables F(1B), D1(1k), D2(100M) and filters making D2 contribute 500 rows, argue the optimal join order and what estimate error would flip it.

### 10. SQL Examples

```
-- Pushdown-friendly view usage: predicates inline through the view
CREATE VIEW v_paid AS
SELECT o.*, u.country FROM orders o JOIN users u ON u.id=o.user_id
WHERE o.status='paid';
EXPLAIN SELECT * FROM v_paid WHERE country='DE' AND created_at >= '2026-06-01';
-- both predicates appear as Index/Seq conditions on base tables ✔

-- Fence demonstration
WITH c AS MATERIALIZED (SELECT * FROM orders)         -- fence: full scan of orders
SELECT * FROM c WHERE id = 5;
WITH c AS NOT MATERIALIZED (SELECT * FROM orders)     -- inlined: index scan on id
SELECT * FROM c WHERE id = 5;
```

### 11. Optimization Techniques

- Partition pruning is pushdown's biggest win — verify with EXPLAIN (`Subplans Removed`, or only some partitions listed).
- For foreign tables (FDW), check the remote SQL in EXPLAIN VERBOSE — unpushed quals mean shipping whole tables over the network.
- Pre-filter fact tables into CTE-per-branch before UNION ALL merges.

### 12. Follow-up Questions

- "Why can't the filter on the LEFT JOIN's right side be pushed below the join?" (padding rows semantics — ties back to Module 3)
- "How does this concept map to microservice APIs?" (filter at the source — same principle, wider audience answer interviewers enjoy)

---

## Chapter 5.4 — Sorting & Filtering at Scale

### 1. Why Interviewers Ask This

Sorts are the silent killer in real plans (top-N, GROUP BY, window functions, DISTINCT, merge joins all sort). "How do you make ORDER BY cheap?" has one senior answer: don't sort — read in order.

### 2. Core Concept

- In-memory sort: quicksort within `work_mem`. Bigger → **external merge sort** on temp files (`Sort Method: external merge Disk: ...`).
- **Top-N heapsort**: `ORDER BY ... LIMIT k` keeps only k rows in a heap — tiny memory, huge win; but only if the sort node knows about the limit.

- **Best sort is no sort**: a B+Tree index on the ORDER BY prefix (matching directions, compatible collation) streams rows pre-sorted; with LIMIT it reads k entries and stops.
- Incremental sort (PG13+): input sorted by (a) and you need (a,b) — sorts small per-a batches.
- Filtering at scale: cheapest predicate order is the planner's job, but *shape* is yours — sargable predicates (Module 4.6), pre-filtering before expensive functions (`WHERE cheap_cond AND expensive_fn(...)`), avoiding row-by-row regex over billions (trigram index, generated columns).

## 3. Internal Working

External sort: produce sorted runs of work_mem size → merge runs (multi-way); cost ≈ read+write data × passes. Parallel sorts split by workers then merge. `DISTINCT`/`GROUP BY` choose HashAggregate (no order, memory-bound, spillable PG13+) vs Sort+Group (ordered, index-friendly). Collation matters: `ORDER BY text_col` under ICU/libc locales is expensive comparison-wise and must match the index's collation to reuse it (`COLLATE "C"` indexes for byte order).

## 4. Visualization (ASCII)

```
ORDER BY created DESC LIMIT 20:
without index: scan 300M rows → top-N heap (keep 20) → 20      [reads everything]
with (created DESC) index:  read 20 leaf entries → done        [reads 20 + descent]

External merge sort (work_mem=64MB, data=6GB):
pass1: 96 sorted runs of 64MB → disk
pass2: merge 96 runs → output          Disk traffic ≈ 2×6GB (plus reread)
EXPLAIN: "Sort Method: external merge  Disk: 6291456kB" ← the smoking gun
```

## 5. Real Production Example

"Recent activity" endpoint sorting a user's merged events: fine at 1k events, times out for power users with 2M. Plan shows external sort. Fix: index `(user_id, created_at DESC)` on each source, per-branch LIMIT before merge, keyset pagination — sort nodes vanish. The pattern (indexed order + limit pushdown + merge) is the standard senior answer for any feed.

## 6. Common Interview Questions

- "How does the database sort data that doesn't fit in memory?"
- "How do you make `ORDER BY created_at DESC LIMIT 20` O(20)?"
- "GROUP BY: hash vs sort aggregation — trade-offs and how to influence it?"
- "Why did DISTINCT add 30s to this query?" (dedup = sort/hash over everything)

## 7. Common Mistakes

- Raising work_mem globally to silence spills (per-node × per-connection multiplication → OOM).
- Index exists but unusable for the sort: direction mismatch, expression mismatch, collation mismatch, or a bitmap scan discarded the order.
- Sorting wide rows (`SELECT *`) when only keys are needed — sort payload matters; sort ids, join details after.
- DISTINCT slapped on to fix join multiplication (Module 3) — pay dedup forever.

## 8. Best Practices

- Feed sorts from indexes for anything user-facing; reserve real sorts for batch/analytics.
- Sort narrow: order by keys, fetch wide rows afterwards (deferred join / late row lookup).
- Set work_mem per-session for known heavy jobs (`SET LOCAL work_mem='512MB'` in the report transaction), not globally.

## 9. Coding Questions

1. Rewrite wide sort as deferred join: `SELECT * FROM posts ORDER BY score DESC LIMIT 20` on a 300-column-ish table →

`SELECT p.* FROM posts p JOIN (SELECT id FROM posts ORDER BY score DESC LIMIT 20) t USING (id) ORDER BY p.score DESC;` Explain when this wins (payload width × spill).

2. Given `GROUP BY user_id` over 2B rows with 50M groups: estimate hash-agg memory, decide hash vs sort strategy, and propose the pre-aggregated alternative.

### 10. SQL Examples

```
-- Diagnose spills
EXPLAIN (ANALYZE, BUFFERS) SELECT ... ORDER BY ...;
-- "Sort Method: quicksort Memory: 25kB"        good
-- "Sort Method: top-N heapsort Memory: 26kB"   good (LIMIT)
-- "Sort Method: external merge Disk: 480MB"    fix me

-- Session-scoped memory for a legit big sort
BEGIN; SET LOCAL work_mem = '512MB';
COPY (SELECT ... ORDER BY ...) TO STDOUT;
COMMIT;

-- Index-fed order, mixed directions
CREATE INDEX ON posts (user_id, score DESC, id DESC);
SELECT * FROM posts WHERE user_id=42 ORDER BY score DESC, id DESC LIMIT 20;
```

### 11. Optimization Techniques

- Incremental sort exploitation: index on (a), ORDER BY a,b — near-free.
- `COLLATE "C"` (or ICU deterministic) indexes for machine-sorted text (ids, codes) — faster compares, index/sort reuse.
- Parallel query for honest big sorts/aggregations: check workers actually launched.

### 12. Follow-up Questions

- "Why does the same ORDER BY use the index with LIMIT 10 but sort with LIMIT 100000?" (cost crossover: random-ish index reads × N vs one scan+sort)
- "Top 20 posts per user for all users — why is this harder than global top 20, and what are the two plans?" (window sort vs per-key lateral probes — Module 2/4 tie-in)

---

## Chapter 5.5 — Pagination (OFFSET vs Keyset)

### 1. Why Interviewers Ask This

Every list endpoint paginates, and OFFSET pagination is the most common self-inflicted performance wound in production. This is a guaranteed API-design + DB question at Stripe/Meta.

### 2. Core Concept

- **OFFSET/LIMIT**: `... ORDER BY x LIMIT 20 OFFSET 100000` — the executor must **produce and discard** 100,000 rows first. Cost grows linearly with page depth; deep pages = O(offset). Also *unstable*: concurrent inserts/deletes shift rows between pages (skipped/duplicated items).
- **Keyset (cursor/seek) pagination**: remember the last row's sort key; next page = `WHERE (x, id) < (last_x, last_id) ORDER BY x DESC, id DESC LIMIT 20` — a pure index descent: O(page size) at any depth, and stable under concurrent writes.
- Requirements: deterministic total order (unique tiebreaker column!), an index matching it, and an opaque cursor in the API (encode the keyset, don't expose raw columns).
- Trade-off: no "jump to page 47" (offer filters/date-jumps instead), and bidirectional paging needs the reversed condition.

## 3. Internal Working

OFFSET plan: Index Scan → Limit node that *counts off* offset rows (they're fully fetched — heap and all) then emits 20. Keyset plan: the composite predicate `(x,id) < (a,b)` is a **row-value comparison** that maps directly to a B+Tree position — descend once, read 20 leaf entries. This is why Postgres row-value syntax matters (MySQL 8 optimizes it too; otherwise expand to `x < a OR (x = a AND id < b)`).

## 4. Visualization (ASCII)

```
OFFSET 100000 LIMIT 20:
index ▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶[discard 100,000 rows]▶[emit 20]
page 1: 3ms   page 500: 90ms   page 5000: 1.4s  (and heap-fetches all discards)

Keyset  WHERE (created,id) < ('2026-06-01T10:31', 88123):
index ──descend to key──▶[emit 20]      any page: 3ms

Stability under concurrent insert:
OFFSET: new row shifts everything → item seen twice on next page
keyset: position anchored to a value → unaffected
```

## 5. Real Production Example

Public API at a Stripe-like company: `GET /v1/charges?page=4000` from a crawler = worst query in the fleet. Migration to cursor pagination (`starting_after=<opaque id>`) — exactly what Stripe's real API does — made page depth irrelevant and killed the incident class. Also the standard infinite-scroll answer for feeds at Meta.

## 6. Common Interview Questions

- "Why does OFFSET get slower with depth? Fix it." (guaranteed)
- "Design pagination for an API — what's in the cursor?"
- "What breaks with OFFSET pagination under concurrent writes?"
- "How do you paginate a non-unique sort key?" (tiebreaker in key and index)
- "When is OFFSET acceptable?" (small bounded depth, admin UIs, jump-to-page requirement)

## 7. Common Mistakes

- Keyset without a unique tiebreaker → skipped/duplicated rows on ties.
- Cursor as raw `WHERE id < ?` while sorting by a different column (key must be the full sort key).
- Expanded OR form without row-value comparison in engines that then can't use the index cleanly.
- Returning total page count (requires COUNT of everything — see 5.6; return `has_more` instead).

## 8. Best Practices

- Keyset by default for anything user-scrollable; index = exactly the sort key including tiebreaker.
- Opaque, signed cursors (base64 of keyset) — API stability + no client-forged positions.
- Deep-export use cases: don't paginate — stream (server-side cursor, COPY) or snapshot to object storage.

## 9. Coding Questions

1. Write page-1 and page-N queries for a feed ordered by `(score DESC, created_at DESC, id DESC)` with its index, plus the previous-page query.
2. Design the cursor payload for a filtered list (`status='paid'` + sort key) and explain why filters must be inside the cursor or re-validated.

### 10. SQL Examples

```
CREATE INDEX ON orders (user_id, created_at DESC, id DESC);

-- page 1
SELECT * FROM orders WHERE user_id = 42
ORDER BY created_at DESC, id DESC LIMIT 20;

-- next page (cursor = last row's (created_at, id))
SELECT * FROM orders
WHERE user_id = 42
  AND (created_at, id) < ('2026-06-01 10:31:00+00', 88123)
ORDER BY created_at DESC, id DESC
LIMIT 20;

-- previous page (reverse, then re-sort in app or wrap)
SELECT * FROM (
  SELECT * FROM orders
  WHERE user_id = 42 AND (created_at, id) > ('2026-06-01 10:31:00+00', 88123)
  ORDER BY created_at ASC, id ASC LIMIT 20
) t ORDER BY created_at DESC, id DESC;
```

### 11. Optimization Techniques

- Row-value comparisons `(a,b) < (?,?)` — index-perfect in Postgres; verify the plan.
- `has_more` via LIMIT k+1 (fetch one extra) instead of COUNT.
- For hybrid needs (page numbers + speed), precompute page anchors periodically (every 1000th key) and keyset from the nearest anchor.

### 12. Follow-up Questions

- "How does this work across shards?" (query k from each shard by keyset, merge-heap, return k; cursor stores per-shard keys — Module 7 tie-in)
- "User re-sorts by a different column — what happens to your cursor design?" (cursor is sort-specific; new sort = new cursor space + supporting index decision)

---

## Chapter 5.6 — COUNT Optimization

### 1. Why Interviewers Ask This

"Why is COUNT(*) slow in Postgres?" is a top-5 senior screen question because the correct answer requires MVCC understanding, and the fix requires product thinking (do you even need the exact number?).

### 2. Core Concept

`SELECT count(*) FROM big_table` is slow in Postgres because **MVCC visibility lives in the rows**: no central "row count" is maintained per snapshot, so the executor must visit every row (heap, or index+visibility-map) and test visibility. It's O(N) by design — not a bug.

The decision tree (memorize): - **Exact count of everything** → estimated count from stats (`pg_class.reltuples`) is almost always what the product needs. - **Exact filtered count, small result** → normal indexed count is fine. - **Big filtered count, repeated** → **counter table/rollup** maintained transactionally or async (with reconciliation). - **Approximate distinct** → HyperLogLog. - **Pagination totals** → don't. `has_more`, or "about 1,200 results" (estimate from EXPLAIN). - `count(*)` vs `count(col)` vs `count(distinct col)`: rows vs non-null vs distinct-non-null; `count(1)` = `count(*)` (same plan — kill that myth in the interview).

### 3. Internal Working

Index-only-scan counting still checks the visibility map per page (all-visible pages counted from the index alone; others fetch heap). After heavy churn, VM coverage drops → counts slow until VACUUM. MySQL InnoDB has the same O(N) property (MyISAM's free count is the historical confusion). Column stores / Redshift-style engines keep per-block

metadata → near-free counts. Estimated counts: planner's `rows` for arbitrary predicates via `EXPLAIN (FORMAT JSON)` — good for "~how many."

## 4. Visualization (ASCII)

```
count(*) on 800M rows:
heap: [page][page][page]... visit ALL, test visibility per row  → minutes
index-only: [leaf][leaf]... + VM lookup per page                → better, still O(N)

counter table:
writes ─┬─► orders                          read: SELECT n FROM counters
        └─► UPDATE counters SET n=n+1              WHERE key='orders:paid'   → 0.1ms
            (same txn, or async+reconcile)
tradeoff: hot-row contention on the counter ← shard the counter into K rows if hot
```

## 5. Real Production Example

Dashboard tile "Total orders: 84,213,991" recomputed per page view = a minute-long seq scan per load. Fixes shipped in order: cached count (5-min TTL) → counter table with async increments + nightly reconciliation → product change to "84.2M". The interview wants you to walk exactly that escalation, including asking "does anyone need the last digit?"

## 6. Common Interview Questions

- "Why is COUNT(*) O(N) in Postgres? Isn't there an index?" (MVCC visibility answer)
- "Fast approximate row count — how?" (reltuples; EXPLAIN rows)
- "Design a correct-enough live counter for likes at Instagram scale." (sharded counters, batched flush, reconciliation)
- "COUNT(*) vs COUNT(1) vs COUNT(col)?"

## 7. Common Mistakes

- Adding an index "to speed up count(*)" with no predicate — barely helps without VM hygiene, never changes O(N).
- Transactional counter on one hot row at high write rates → lock convoy (shard it).
- Exact totals in paginated APIs (double the cost of every list call).
- Using reltuples right after bulk load without ANALYZE (stale).

## 8. Best Practices

- Ask "what precision does the product need?" before optimizing — the senior move.
- Counter tables: increment in-transaction for correctness-critical, async batched for high-volume; either way schedule reconciliation.
- Keep autovacuum healthy for index-only counting paths.

## 9. Coding Questions

1. Implement a sharded counter: table `counters(key, shard, n)`, increment = `ON CONFLICT ... DO UPDATE` on random shard, read = `SUM(n)`; explain the contention math (K shards ≈ K× less conflict).
2. Return an estimated count for an arbitrary filtered query from `EXPLAIN (FORMAT JSON)` in application code (parse Plan Rows).

### 10. SQL Examples

```
-- Instant estimate (whole table)
SELECT reltuples::bigint FROM pg_class WHERE relname = 'orders';

-- Counter table
CREATE TABLE counters (key text, shard int, n bigint NOT NULL DEFAULT 0,
                       PRIMARY KEY (key, shard));
-- increment (random shard 0..7)
INSERT INTO counters VALUES ('orders:paid', floor(random()*8)::int, 1)
ON CONFLICT (key, shard) DO UPDATE SET n = counters.n + 1;
-- read
SELECT sum(n) FROM counters WHERE key = 'orders:paid';

-- has_more instead of total
SELECT * FROM orders WHERE user_id=42 ORDER BY id DESC LIMIT 21;  -- 21st row ⇒ has_more
```

### 11. Optimization Techniques

- `count(*) FILTER (WHERE ...)` to get many counts in one scan instead of N queries.
- HLL for distinct counts across time windows (mergeable sketches).
- For batch analytics, run counts on the replica/warehouse, never the primary.

### 12. Follow-up Questions

- "Your async counter drifted 2% — detect, fix, prevent." (reconciliation diff job, idempotent consumers, versioned events)
- "Why is count fast in ClickHouse/BigQuery?" (columnar metadata, no per-row MVCC visibility)

---

## Chapter 5.7 — The N+1 Query Problem

### 1. Why Interviewers Ask This

The most common real performance bug in application code, and a favorite because it spans ORM, SQL, and system thinking. "Page is slow, DB is idle-ish, APM shows 400 queries per request" — they want the instant diagnosis.

### 2. Core Concept

N+1: fetch N parent rows (1 query), then loop fetching children per parent (N queries). Each query is fast; the *request* dies of round trips (N × (network RTT + parse/plan/execute)).

Fixes, in order of preference: 1. **Batch load**: one query with `WHERE parent_id = ANY($ids)` (or `IN`), group in app — what ORM "eager loading" does (`includes`/`selectinload`/DataLoader). 2. **JOIN** parent+child in one query (beware row multiplication with multiple child collections — Module 3; often better as separate batched queries per collection). 3. **Lateral top-k per parent** when you need only a few children each. 4. **Denormalize/cache** for read-hot aggregates (child counts on the parent).

GraphQL note: resolvers are N+1 factories → **DataLoader pattern** (batch + per-request cache) is the expected vocabulary.

### 3. Internal Working

Why 400 tiny queries ≫ 1 medium query: per-query cost = RTT (0.5–2ms in-VPC) + parse/plan (unless prepared) + executor startup + row fetch. 400 × ~2ms ≈ 800ms of pure overhead before any real work — while one `ANY($ids)` query is a single round trip resolving to one index bitmap/loop over 400 keys. Also: connection-pool occupancy scales with query count → N+1 under load exhausts pools (Module 11 tie-in).

## 4. Visualization (ASCII)

```
N+1:                                    batched:
app ──q──► db   (users: 1 query)        app ──q──► db   users
app ──q──► db   orders u1               app ──q──► db   orders WHERE user_id=ANY([u1..u100])
app ──q──► db   orders u2                   2 round trips total ✔
 ...×100                                app-side: group rows by user_id
102 round trips ✖ (~200ms network alone)

timeline per request:  |q|q|q|q|q|q|q|...      |query|query|
                        └─ death by a thousand RTTs
```

## 5. Real Production Example

Uber-style trip-history screen: ORM lazily loads `trip.driver`, `trip.payment`, `trip.city` → 1 + 3N queries; at N=50 that's 151 queries per screen. APM flamegraph shows the staircase. Fix: eager-load the three associations (3 batched queries) + a covering index each; p95 from 1.8s → 90ms. Every company has this incident; interviewers just change the nouns.

## 6. Common Interview Questions

- "What's the N+1 problem and how do you detect it in production?" (APM query counts per request; `pg_stat_statements` calls column exploding)
- "Eager loading vs JOIN — when is each right?" (multiple hasMany collections multiply in a join; batched selects keep grains separate)
- "How does DataLoader solve N+1 in GraphQL?" (per-tick batch + cache by key)
- "Fix N+1 without touching the ORM query code?" (harder: view/denormalized read model/cache; usually the answer is 'touch the code')

## 7. Common Mistakes

- "Fixing" with one giant JOIN across three one-to-many collections → cartesian blowup (rows = orders × items × shipments per user).
- Batch queries with 100k-id IN lists (plan/parse bloat, parameter limits) — chunk the batches.
- Caching each child individually (N cache round trips — same problem, new backend; use MGET).
- Not noticing write-path N+1: per-row UPDATE loops instead of one set-based UPDATE.

## 8. Best Practices

- Turn on ORM lazy-load warnings/strict mode in dev; assert max-queries-per-request in tests.
- Standard access patterns: batch by key list, chunked `ANY($1)`, prepared statements.
- For aggregates-of-children shown in lists (counts, latest item), maintain them on the parent (denormalized) instead of loading children at all.

## 9. Coding Questions

1. Given `users` page (100 rows) needing latest order + order count each: write the two batched queries (lateral top-1 with `= ANY`, grouped count with `= ANY`) and the app-side merge.
2. Rewrite a per-row update loop (`for id in ids: UPDATE ... WHERE id=?`) as one statement with `unnest($ids, $values)`.

## 10. SQL Examples

```
-- Batch children for a page of parents
SELECT * FROM orders WHERE user_id = ANY($1::bigint[]) ORDER BY user_id, created_at DESC;

-- Batch latest-order-per-parent (lateral over the id list)
SELECT o.* FROM unnest($1::bigint[]) AS u(id)
JOIN LATERAL (
  SELECT * FROM orders o WHERE o.user_id = u.id
  ORDER BY created_at DESC LIMIT 1
) o ON true;

-- Set-based write instead of update loop
UPDATE products p SET price_cents = v.price
FROM (SELECT unnest($1::bigint[]) AS id, unnest($2::int[]) AS price) v
WHERE p.id = v.id;
```

## 11. Optimization Techniques

- Cap batch sizes (~1–5k keys) and parallelize chunks if needed.
- Prepared statements + pooler to amortize parse/plan for the remaining queries.
- Request-scoped memoization (DataLoader cache) to dedupe repeated key loads within one request.

## 12. Follow-up Questions

- "Your fix made one query slow instead of 400 fast ones — how do you know you won?" (total latency + pool occupancy + DB CPU; measure request-level, not query-level)
- "Same problem calling a microservice per item — carry the lesson over." (batch APIs, the N+1 lesson generalizes; interviewers love this bridge)

---

# Module 5 — Practice Problems

## Easy (5)

1. In an EXPLAIN ANALYZE node: `(cost=0.43..8.45 rows=1 width=8) (actual time=0.031..0.032 rows=412 loops=850)` — extract every fact and name the concern.
2. Why is `SELECT count(*)` slow on an 800M-row Postgres table, in two sentences including the word "visibility"?
3. Convert to keyset pagination: `SELECT * FROM posts ORDER BY created_at DESC LIMIT 20 OFFSET 4000` (include the tiebreaker and the index).
4. `Sort Method: external merge Disk: 2.1GB` — give three distinct remedies in preference order.
5. A request issues 1 query for 50 carts + 50 queries for items. Write the single batched item query and describe the app-side regrouping.

## Medium (5)

1. A prepared statement flipped to a generic plan on its 6th execution and latency went 10x for skewed parameters. Explain the mechanism and give three fixes with trade-offs.
2. Query joins events (1B, partitioned by day) to users, filters `day BETWEEN ...` inside a subquery wrapped by a window function. Partitions aren't pruned. Explain why and restructure.
3. Design "search results: about 12,400 items, pages of 20, infinite scroll" — cursor format, the has_more trick, the estimate source, and the index.
4. `pg_stat_statements` shows a statement with calls=48M/day, mean=0.4ms, total=5.3h/day. Nothing is "slow." Argue whether and how to optimize (N+1 batch consolidation, caching, prepared statements — total-time thinking).
5. A nightly report sets work_mem='2GB' globally via ALTER SYSTEM "so sorts don't spill." Enumerate the failure modes and rewrite the change correctly (SET LOCAL in the job).

## Hard (5)

1. You get this plan shape: NestedLoop(est 40 rows) → over HashJoin(est 39, actual 2.2M) → over two SeqScans with accurate estimates. The misestimate is in the join selectivity itself. What causes join-selectivity misestimates (FK skew, cross-column correlation, non-uniform key distribution), how do you confirm, and what are your options when extended statistics can't express it? (rewrite: pre-aggregate/temp table with ANALYZE; pg_hint_plan last resort.)
2. Build the plan-regression safety net for a deploy pipeline: pg_stat_statements baselines, query fingerprinting, canary EXPLAIN against production-stats snapshot, auto_explain sampling in prod, alert thresholds. Specify what each layer catches that the others miss.
3. An API must serve "page 5000" (regulatory export). OFFSET is banned. Design anchored pagination: precomputed keyset anchors every N rows (materialized, refreshed), anchor lookup
    - keyset from anchor, staleness math, and the fallback when anchors are stale.
4. COUNT problem at Instagram-likes scale: 500k increments/sec on hot posts. Design the full pipeline: client dedup, sharded in-memory accumulation, batched DB flush, exact reconciliation from the event log, read path with staleness bound. State what each layer tolerates and what invariant survives crashes at every point.
5. A 14-way reporting join is planner-nondeterministic (GEQO) and occasionally 100x slow. Options: raise geqo_threshold/join_collapse_limit (planning-time explosion?), decompose into materialized stages with ANALYZE between, or precompute star-schema style. Design the decomposition, argue where the stage boundaries go and prove the plan is now stable.

---

*Next: Module 6 — Transactions & Concurrency*

# MODULE 6 — Transactions & Concurrency

> The deepest technical module in senior loops. Isolation levels + MVCC + locking scenarios are where FAANG interviewers separate senior from staff. Every anomaly below comes with the two-transaction interleaving you must be able to write on a whiteboard.

Chapters: 6.1 Read Anomalies & Isolation Levels (Dirty Reads, Non-Repeatable Reads, Phantoms, Lost Updates) 6.2 MVCC — How Postgres Implements Isolation 6.3 Locks — Row, Table, Advisory; FOR UPDATE / SKIP LOCKED 6.4 Deadlocks 6.5 Optimistic vs Pessimistic Locking 6.6 SERIALIZABLE & Write Skew

---

## Chapter 6.1 — Read Anomalies & Isolation Levels

### 1. Why Interviewers Ask This

The single most reliable senior-depth probe. Definitions are entry stakes; the real test is (a) producing the interleaving that causes each anomaly, (b) knowing which level stops it, (c) knowing the Postgres-specific deviations from the SQL standard.

### 2. Core Concept

The anomalies: - **Dirty read**: read another transaction's *uncommitted* write. (Postgres never allows this — even READ UNCOMMITTED behaves as READ COMMITTED.) - **Non-repeatable read**: read the same row twice in one txn, get different values (someone committed between reads). - **Phantom read**: re-run the same *predicate* query, get different *row sets* (someone inserted/deleted matching rows). - **Lost update**: two txns read-modify-write the same row; the second write silently overwrites the first (`balance = balance_read + x` racing). - **Write skew**: two txns read overlapping data, write *disjoint* rows, and jointly violate an invariant neither violates alone (see 6.6 — the staff-level anomaly).

The levels (SQL standard × what Postgres actually does):

| Level | Dirty | Non-repeat | Phantom | Lost update | Write skew |
|---|---|---|---|---|---|
| READ UNCOMMITTED | std: possible / **PG: never** | possible | possible | possible | possible |
| READ COMMITTED (PG default) | no | possible | possible | **possible** | possible |
| REPEATABLE READ | no | no | std: possible / **PG: no** (snapshot isolation) | **prevented in PG** (first-updater-wins error) | possible |
| SERIALIZABLE | no | no | no | no | no (SSI) |

Postgres specifics worth saying unprompted: - READ COMMITTED = **new snapshot per statement**; REPEATABLE READ = one snapshot per txn. - PG's REPEATABLE READ is **snapshot isolation** — stronger than the standard (no phantoms) but still allows write skew. - Under RR/SERIALIZABLE, write conflicts raise `40001 serialization_failure` → **retry loops are mandatory**.

### 3. Internal Working

Snapshots decide reads (MVCC — 6.2); write conflicts decide the rest. At READ COMMITTED, an UPDATE that hits a row changed by a concurrent committed txn **re-reads the latest version, re-checks the WHERE, and proceeds** (this "requery" behavior is why READ COMMITTED avoids some naive races but still permits lost updates in read-then-write application patterns). At REPEATABLE READ, the same situation errors (`could not serialize access due to concurrent update`) — first-updater-wins.

## 4. Visualization (ASCII)

```
Lost update at READ COMMITTED (app-level read-modify-write):
T1: SELECT balance → 100
T2: SELECT balance → 100
T1: UPDATE balance = 100 - 30  → commits (70)
T2: UPDATE balance = 100 - 50  → commits (50)   ← T1's -30 LOST (should be 20)

Same at REPEATABLE READ:
T2's UPDATE sees the row changed since its snapshot → ERROR 40001 → retry → correct

Non-repeatable read (READ COMMITTED):
T1: SELECT price → 10        T2: UPDATE price=12; COMMIT
T1: SELECT price → 12   (new statement = new snapshot)

Phantom (READ COMMITTED):
T1: SELECT count(*) WHERE dept='eng' → 5     T2: INSERT eng row; COMMIT
T1: same query → 6
```

## 5. Real Production Example

Classic incident: coupon codes with `max_uses=1000` implemented as `SELECT uses FROM coupons; if uses < 1000: UPDATE coupons SET uses = uses + 1` at READ COMMITTED. Black Friday concurrency → 1,180 redemptions. Three real fixes (grade yourself on producing all three): atomic single-statement `UPDATE coupons SET uses=uses+1 WHERE id=? AND uses<1000` (rowcount tells you if you won), `SELECT ... FOR UPDATE` around the read, or SERIALIZABLE + retry.

## 6. Common Interview Questions

- "Define all four anomalies with interleavings." (whiteboard-ready)
- "What's Postgres's default level and what anomalies does it allow?" (READ COMMITTED; non-repeatable, phantom, lost update, write skew)
- "How does PG's REPEATABLE READ differ from the SQL standard? From MySQL's?" (PG: snapshot isolation, no phantoms, first-updater-wins; InnoDB RR: next-key locks block phantom *writes*, reads are consistent snapshots, but current-reads (UPDATE/FOR UPDATE) see latest → different lost-update behavior)
- "Why not run everything SERIALIZABLE?" (retry rate, throughput, predicate-lock memory — though for many workloads it's fine; measured answer wins)

## 7. Common Mistakes

- Reciting the standard table without PG's deviations (that's the differentiating knowledge).
- Believing READ COMMITTED prevents lost updates because "the UPDATE re-checks" — the re-check saves single-statement arithmetic (`SET x = x - 1`), not app-level read-then-write.
- No retry logic under RR/SERIALIZABLE.
- Claiming MySQL and Postgres REPEATABLE READ are the same thing.

## 8. Best Practices

- Default READ COMMITTED + **make each critical mutation atomic in one statement** (guarded UPDATE with WHERE + rowcount check) — this removes most anomaly exposure without level changes.
- Escalate per-transaction, not globally: `BEGIN ISOLATION LEVEL SERIALIZABLE` for the few invariant-critical flows, wrapped in retry.
- Never trust "we tested it" for concurrency — reason from interleavings.

## 9. Coding Questions

1. Write the coupon fix all three ways and state the failure mode of each under 10k concurrent redemptions (guarded UPDATE: hot-row lock queue; FOR UPDATE: same + longer hold; SSI: retry storm).
2. Demonstrate PG's READ COMMITTED "requery" semantics: two concurrent `UPDATE accounts SET balance = balance - 10 WHERE balance >= 10` on balance=15 — final state and why (one succeeds, second re-evaluates WHERE on the new version → matches? 5 >= 10 no → 0 rows).

### 10. SQL Examples

```
-- Atomic guarded mutation (the READ COMMITTED-safe idiom)
UPDATE coupons SET uses = uses + 1
WHERE id = $1 AND uses < max_uses;          -- app checks rowcount = 1

-- Per-transaction escalation with retry (app pseudocode around it)
BEGIN ISOLATION LEVEL SERIALIZABLE;
  SELECT ...; UPDATE ...;
COMMIT;   -- on SQLSTATE 40001: rollback, jittered backoff, retry (bounded)

-- Observe your current level
SHOW transaction_isolation;
```

### 11. Optimization Techniques

- Keep hot-row transactions minimal (acquire the contended lock as LATE as possible in the txn).
- Prefer single-statement atomic ops over FOR UPDATE where possible (shorter lock hold).
- Monitor serialization failure rates; a rising 40001 rate = contention hotspot to redesign (shard the row, queue the mutations).

### 12. Follow-up Questions

- "Interviewer: 'increment a view counter' vs 'transfer money' — same isolation needs?" (no: counter tolerates approximate/async; transfer needs atomic guarded logic)
- "How would you *test* for lost updates in CI?" (concurrent harness, invariant assertions, jepsen-style property checks)
- "What does MySQL's `SELECT ... LOCK IN SHARE MODE` / `FOR SHARE` change in these stories?"

---

## Chapter 6.2 — MVCC: Multi-Version Concurrency Control

### 1. Why Interviewers Ask This

"How can readers not block writers?" is the mechanism question behind every isolation answer. MVCC also explains VACUUM, bloat, txn-ID wraparound — Postgres operational maturity in one topic.

### 2. Core Concept

MVCC = keep **multiple versions of each row**; give each transaction a **snapshot** deciding which versions it sees. Readers never block writers, writers never block readers (writers still block writers on the same row).

Postgres implementation: - Every tuple has hidden columns: `xmin` (creating txn), `xmax` (deleting/locking txn), ctid. - **UPDATE = insert new version + set xmax on old**; DELETE = set xmax. Old versions stay in the heap until VACUUM removes them. - **Snapshot** = (xmin horizon, xmax horizon, in-progress txn list). Tuple visible if its creator committed before the snapshot and it isn't deleted by a committed-before-snapshot txn. - **VACUUM** reclaims dead versions (autovacuum runs it); also maintains the visibility map (index-only scans!) and **freezes** old tuples to prevent **transaction ID wraparound** (32-bit XIDs — the famous operational cliff).

Contrast: MySQL/InnoDB keeps only the newest version in the clustered index and reconstructs old versions from **undo logs** (rollback segments) — old versions live in undo space, purged by the purge thread; long-running transactions bloat undo instead of the heap. Oracle likewise (undo).

### 3. Internal Working

- Commit status lives in `pg_xact`; hint bits on tuples cache "known committed/aborted" to skip lookups.
- **HOT updates** (Module 4.2): new version on the same page with no indexed-column change → no index churn.
- **Long-running transactions are toxic**: they pin the "oldest visible xmin" → VACUUM cannot remove any version newer than that horizon → bloat grows table- and cluster-wide, replicas conflict, wraparound clock keeps ticking.
- Bloat = dead-version space; monitored via `pg_stat_user_tables.n_dead_tup`, fixed by (auto) vacuum; only `VACUUM FULL` / `pg_repack` returns space to the OS.

## 4. Visualization (ASCII)

```
UPDATE row (id=7) by txn 200:
heap:  v1 [xmin=120, xmax=200] "amount=50"   ← old version, now "deleted by 200"
       v2 [xmin=200, xmax=∅  ] "amount=80"   ← new version

snapshot taken before 200 committed → sees v1 (50)
snapshot taken after               → sees v2 (80)
VACUUM (once no snapshot can see v1) → v1 space reclaimed

Long transaction pins the horizon:
oldest snapshot xmin=120 ──────────┐
dead versions from txns 121..900   │ ALL unvacuumable while it lives
                                   ▼ table bloats, scans slow, autovacuum spins
```

## 5. Real Production Example

The canonical Postgres incident: an analyst's psql session left `BEGIN;` open over a weekend → autovacuum couldn't reclaim anything → hot tables tripled in size, index-only scans died (visibility map stale), p99 crept up cluster-wide. Detection: `pg_stat_activity` ordered by `xact_start`. Prevention: `idle_in_transaction_session_timeout`. Every senior Postgres interview touches some version of this.

## 6. Common Interview Questions

- "How do readers avoid blocking writers in Postgres?" (versions + snapshots)
- "What exactly does UPDATE do at the storage level?" (insert new version + xmax old)
- "What is VACUUM for — all three jobs?" (dead tuples, visibility map, freeze/wraparound)
- "Why are long transactions harmful even if they only read?" (pin xmin horizon)
- "How does InnoDB's MVCC differ?" (undo-log reconstruction vs heap versions; where bloat accumulates)

## 7. Common Mistakes

- "MVCC means no locks" — writers still take row locks; DDL takes table locks.
- Thinking DELETE frees space immediately.
- Ignoring wraparound until the `database is not accepting commands` emergency.
- Disabling autovacuum on busy tables "because it causes I/O" (it prevents the larger disaster; tune, don't disable).

## 8. Best Practices

- `idle_in_transaction_session_timeout` set cluster-wide (e.g. 60s); alert on old `xact_start`.
- Tune autovacuum per hot table: lower `autovacuum_vacuum_scale_factor` (e.g. 0.01), more workers/cost limit on big clusters.
- Watch `n_dead_tup`, bloat estimates, and `datfrozenxid` age dashboards.
- Design for HOT: don't index churn-columns; fillfactor headroom.

## 9. Coding Questions

1. Write the monitoring query: sessions idle in transaction > 5 min, with age and query.
2. Explain the visibility decision: tuple (xmin=500 committed, xmax=520 in-progress) under a snapshot where 520 is in the in-progress list → visible or not? (Visible — deleter hasn't committed for this snapshot.)

### 10. SQL Examples

```
-- See version churn and vacuum health
SELECT relname, n_live_tup, n_dead_tup, last_autovacuum
FROM pg_stat_user_tables ORDER BY n_dead_tup DESC LIMIT 10;

-- Long transaction hunt
SELECT pid, now() - xact_start AS duration, state, query
FROM pg_stat_activity
WHERE xact_start IS NOT NULL ORDER BY xact_start LIMIT 10;

-- Wraparound safety margin
SELECT datname, age(datfrozenxid) FROM pg_database ORDER BY 2 DESC;

-- Inspect tuple versions (educational)
SELECT xmin, xmax, ctid, * FROM accounts WHERE id = 7;
```

### 11. Optimization Techniques

- pg_repack for online bloat removal (VACUUM FULL takes an exclusive lock).
- Batch large DELETEs/UPDATEs in chunks with pauses so autovacuum keeps pace; for "delete most of the table" prefer partition drops or CTAS-swap.
- On replicas, `hot_standby_feedback` trade-off: stops query-vs-vacuum conflicts but exports replica xmin to the primary (bloat) — know both directions.

### 12. Follow-up Questions

- "Replication conflict: replica query cancelled by vacuum cleanup — explain and fix options." (max_standby_streaming_delay vs hot_standby_feedback vs dedicated replica)
- "Why does Postgres need freeze but MySQL doesn't?" (32-bit XID comparisons vs InnoDB's different versioning scheme)
- "What's the cost of MVCC for update-heavy tables vs an in-place-update engine?" (write amplification + vacuum debt vs undo/redo complexity)

---

## Chapter 6.3 — Locks: Row, Table, Advisory; FOR UPDATE / SKIP LOCKED

### 1. Why Interviewers Ask This

Lock questions are incident questions: "the deploy added a column and the site went down," "workers grab the same job." You're expected to know lock granularities, what blocks what, and the queue-worker idiom.

### 2. Core Concept

- **Row locks** (stored on tuples, unlimited count): exclusive (UPDATE/DELETE/`FOR UPDATE`), plus weaker shares (`FOR NO KEY UPDATE`, `FOR SHARE`, `FOR KEY SHARE` — FK machinery). Writers queue per row FIFO.
- **Table locks** (8 modes): every statement takes one — SELECT takes ACCESS SHARE; most DDL takes ACCESS EXCLUSIVE (blocks *everything*, including SELECTs). The killer detail: a waiting ACCESS EXCLUSIVE **blocks all newcomers behind it** → one ALTER TABLE behind a long SELECT stalls the whole table.
- **`SELECT ... FOR UPDATE`**: lock rows you're about to modify (pessimistic). Modifiers: `NOWAIT` (error instead of wait), **`SKIP LOCKED`** (skip locked rows — the concurrent job-queue primitive).
- **Advisory locks**: application-defined lock IDs unrelated to rows (`pg_advisory_lock(key)`) — leader election, migration mutexes, dedup of cron jobs.
- Lock waits are invisible until you look: `pg_locks` + `pg_blocking_pids()`.

### 3. Internal Working

Row lock = xmax field + lock bits on the tuple (no lock-table blowup; contrast SQL Server escalation). Waiters sleep on the transaction holding the row until it ends. Table locks live in shared memory with a wait **queue**: conflicts are

checked against holders *and* queued requests (that's the DDL-pileup mechanism). `lock_timeout` bounds how long a statement waits; `deadlock_timeout` (1s) is when the deadlock detector wakes.

## 4. Visualization (ASCII)

```
Row-lock queue on id=7:      DDL pileup:
T1 UPDATE (holds)            long SELECT (ACCESS SHARE, running)
T2 UPDATE (waits)               ▲
T3 UPDATE (waits)            ALTER TABLE (ACCESS EXCLUSIVE) — WAITS
  FIFO per row                  ▲
                             every new SELECT/INSERT — WAITS BEHIND THE ALTER
                             table effectively down ✖
                             fix: lock_timeout on DDL + retry

Job queue with SKIP LOCKED:
jobs: [J1][J2][J3][J4]...
W1: SELECT ... FOR UPDATE SKIP LOCKED LIMIT 1 → locks J1
W2: same query → skips J1 (locked) → locks J2   ← no contention, no double-work
```

## 5. Real Production Example

Two canonical ones: 1. **Migration outage**: `ALTER TABLE orders ADD COLUMN ...` queued behind a 10-min analytics query on the primary → all order traffic queued behind the ALTER → checkout down. Fix pattern: `SET lock_timeout='2s'` + retry loop for DDL; run analytics elsewhere. 2. **Job queue**: workers doing `SELECT ... WHERE status='pending' LIMIT 1 FOR UPDATE` all blocked on the same first row (or worse, without FOR UPDATE, processed it twice). The `FOR UPDATE SKIP LOCKED` idiom (below) is the industry-standard answer and a very common interview design question.

## 6. Common Interview Questions

- "What blocks what: two UPDATEs same row? UPDATE + SELECT? SELECT + ALTER TABLE?"
- "Design a Postgres-backed job queue for N concurrent workers." (SKIP LOCKED expected)
- "Why did adding a nullable column with a default lock the table?" (pre-PG11 rewrite; PG11+ fast default — version nuance = points)
- "How do you find who's blocking whom right now?"
- "What are advisory locks for?"

## 7. Common Mistakes

- Believing plain SELECT can be blocked by row locks (it can't — MVCC) or that it takes no lock at all (ACCESS SHARE, conflicts with ACCESS EXCLUSIVE DDL).
- Queue workers with `LIMIT 1 FOR UPDATE` and no SKIP LOCKED → convoy.
- Long-held FOR UPDATE across external calls (Module 1.8's cardinal sin).
- Running DDL without lock_timeout on hot tables.
- Forgetting `NOWAIT`/`SKIP LOCKED` exist and building polling/retry loops around blocking waits.

## 8. Best Practices

- All migrations: `lock_timeout` + retries; know your non-blocking DDL menu (CREATE INDEX CONCURRENTLY, ADD COLUMN without volatile default, ADD CONSTRAINT NOT VALID + VALIDATE, DETACH PARTITION CONCURRENTLY).
- Lock ordering discipline + acquire late, release fast (commit promptly).
- Job queues: `FOR UPDATE SKIP LOCKED` + status transitions + visibility timeout for crashes.
- Dashboards: lock waits (`pg_locks` granted=false), blocking chains, lock_timeout error rates.

## 9. Coding Questions

1. Write the worker dequeue transaction (below) and explain crash-recovery (row lock released on crash → job re-claimable; add attempts counter).
2. Write the blocking-chain query using `pg_blocking_pids()`.

### 10. SQL Examples

```
-- Concurrent-safe dequeue
BEGIN;
SELECT id, payload FROM jobs
WHERE status = 'pending' AND run_at <= now()
ORDER BY run_at
FOR UPDATE SKIP LOCKED
LIMIT 10;
UPDATE jobs SET status='running', started_at=now() WHERE id = ANY($claimed);
COMMIT;   -- process, then mark done/failed in a new txn

-- Who blocks whom
SELECT pid, pg_blocking_pids(pid) AS blocked_by, state, wait_event_type, query
FROM pg_stat_activity
WHERE cardinality(pg_blocking_pids(pid)) > 0;

-- Safe DDL pattern
SET lock_timeout = '2s';
ALTER TABLE orders ADD COLUMN note text;   -- retried by the migration tool on timeout

-- Advisory lock: single-flight cron
SELECT pg_try_advisory_lock(hashtext('daily-billing'));  -- false → someone else runs it
```

### 11. Optimization Techniques

- Shard hot rows (counters, single-row config) to spread lock queues.
- Replace lock-wait polling with LISTEN/NOTIFY for queue wakeups.
- Use `FOR NO KEY UPDATE` instead of `FOR UPDATE` when you won't touch key columns — doesn't block FK inserts referencing the row.

### 12. Follow-up Questions

- "SKIP LOCKED worker dies mid-job after COMMIT of the claim — how does the job recover?" (visibility timeout / heartbeat + reaper; idempotent processing)
- "Compare with SQL Server lock escalation — why doesn't Postgres escalate?" (locks on tuples, not in a lock table)
- "When is an advisory lock better than a row lock?" (no row to lock yet — e.g., 'create if absent per key' without unique-violation churn; cross-table critical sections)

---

## Chapter 6.4 — Deadlocks

### 1. Why Interviewers Ask This

"Explain a deadlock you debugged" is a standard behavioral-technical hybrid. They test: the cycle definition, how DBs handle it (detection, not prevention), and *code patterns* that prevent it.

### 2. Core Concept

Deadlock = a cycle in the waits-for graph: T1 holds A waits for B; T2 holds B waits for A. Databases **detect and kill** (Postgres: after `deadlock_timeout` (1s), check the graph, abort one victim with error 40P01) — they don't prevent. Prevention is *your* job:

1. **Consistent lock ordering** — all code paths touch shared resources in the same global order (sort ids before multi-row updates!).
2. **Lock everything at once** — one `SELECT ... FOR UPDATE` with all target rows sorted, instead of incremental acquisition.
3. **Short transactions** — narrow the collision window.
4. **Retry on 40P01** — deadlocks may still happen; treat like serialization failures.

Subtle sources interviewers probe: FK locks (parent `FOR KEY SHARE` vs parent UPDATE), unique index insert waits, two multi-row UPDATEs with different scan orders, lock upgrades (FOR SHARE → UPDATE).

## 3. Internal Working

Postgres's detector runs in the *waiting* backend after deadlock_timeout: build local waits-for graph from the lock table, find cycles, abort the waiter that detected it (roughly — the victim choice is implementation detail; MySQL InnoDB picks the smaller-undo victim). The killed txn gets `deadlock detected` with DETAIL naming both queries — the log line you must know how to read. Multi-row UPDATE deadlocks happen because row lock acquisition follows *scan order*, which can differ between plans.

## 4. Visualization (ASCII)

```
T1: UPDATE accounts SET .. WHERE id=1;   -- holds row1
T2: UPDATE accounts SET .. WHERE id=2;   -- holds row2
T1: UPDATE accounts SET .. WHERE id=2;   -- waits on T2
T2: UPDATE accounts SET .. WHERE id=1;   -- waits on T1 → CYCLE

waits-for:  T1 ──▶ T2
            ▲      │
            └──────┘   detector: abort one (40P01), other proceeds

Fix (ordering): both transfer functions lock LEAST(id) then GREATEST(id):
T1: lock 1, lock 2.  T2: lock 1 (waits behind T1) — serialized, no cycle ✔
```

## 5. Real Production Example

Payments transfer service: `transfer(from, to)` locked `from` then `to`. Concurrent A→B and B→A = textbook cycle; at scale, hundreds of 40P01/hour. Fix: lock accounts in `ORDER BY id` inside one `SELECT ... FOR UPDATE`, plus retries. Second favorite: batch jobs updating rows in index order while OLTP updates them in a different order — nightly deadlock spikes; fix by chunking the batch in PK order with small transactions.

## 6. Common Interview Questions

- "What's a deadlock and how does the DB resolve it?"
- "Write transfer(from,to) so it can't deadlock." (ordering)
- "Deadlock between an INSERT and an UPDATE — how?" (FK/unique-index waits)
- "How do you investigate recurring deadlocks in prod?" (log lines with both queries → reconstruct interleaving → find inconsistent ordering)
- "Difference between deadlock and lock wait/timeout?"

## 7. Common Mistakes

- "Fixing" by lengthening deadlock_timeout (just detects later; waits longer).
- Retrying non-idempotent logic blindly.
- Ordering by business fields ("from before to") instead of a global total order (ids).
- Missing that a *single* multi-row UPDATE can deadlock against another due to scan order — "one statement" ≠ "atomic lock acquisition."

## 8. Best Practices

- Codify lock-ordering (sort ids) in shared helpers; review multi-entity transactions for it.
- Log and alert on deadlock rate; each recurring pair of queries in DETAIL is a bug ticket.
- Prefer designs that lock one row (single-owner mutations via queues) over multi-row locking.
- Keep `log_lock_waits = on` to see near-deadlocks (long waits) too.

## 9. Coding Questions

1. Write deadlock-free `transfer(a, b, amount)` (lock both rows sorted, verify balances, update both) with the retry wrapper.
2. Given the log: `Process 111 waits for ShareLock on transaction 222; blocked by process 333... DETAIL: Process 111: UPDATE orders SET ...; Process 333: UPDATE inventory SET ...` — reconstruct the interleaving and propose the ordering fix.

### 10. SQL Examples

```
-- Deadlock-free transfer core
BEGIN;
SELECT id, balance FROM accounts
WHERE id IN (:a, :b)
ORDER BY id                 -- global total order
FOR UPDATE;                 -- both locks acquired here, atomically from our view
UPDATE accounts SET balance = balance - :amt WHERE id = :a AND balance >= :amt;
-- check rowcount; then:
UPDATE accounts SET balance = balance + :amt WHERE id = :b;
COMMIT;

-- Chunked batch update in stable order (plays nice with OLTP)
UPDATE items SET price = price * 1.02
WHERE id IN (SELECT id FROM items WHERE category=7 ORDER BY id LIMIT 1000 OFFSET :i);
```

### 11. Optimization Techniques

- Reduce multi-row transactions to single-row via redesign (event per entity, saga).
- `SELECT FOR UPDATE ... ORDER BY` before UPDATE guarantees acquisition order regardless of update plan.
- In InnoDB, smaller transactions + same-index access paths reduce gap-lock deadlocks (know that MySQL RR gap/next-key locks create deadlocks Postgres doesn't have).

### 12. Follow-up Questions

- "Can SELECTs deadlock in Postgres?" (plain MVCC reads: no; FOR SHARE/UPDATE: yes; and DDL vs DML can)
- "Deadlocks across two *databases*/services — who detects?" (nobody — distributed deadlock needs timeouts; another reason to avoid distributed transactions)
- "Why do gap locks make MySQL deadlock-prone on inserts?" (adjacent-range locking under RR)

---

## Chapter 6.5 — Optimistic vs Pessimistic Locking

### 1. Why Interviewers Ask This

Application-level concurrency design — the question is "how would you prevent two users overwriting each other?" and they want you to *choose* based on contention math, not recite definitions.

### 2. Core Concept

- **Pessimistic**: assume conflict; take the lock before working (`SELECT ... FOR UPDATE`). Nobody else can even start. Costs: held locks (throughput ceiling), needs an open transaction spanning the operation → unusable across user think-time/HTTP requests.
- **Optimistic (OCC)**: assume no conflict; read a **version** with the data; write with `UPDATE ... WHERE id=? AND version=?` (and version=version+1). Rowcount 0 = someone else won → reload/merge/retry or surface a conflict. No locks held while thinking; cost = wasted work + retries under contention.

Decision rule: **contention low / operation long / spans requests → optimistic. Contention high / operation short / in-transaction → pessimistic (or better: atomic single statements).** Related: ETags/If-Match are OCC over HTTP; CAS in Redis (`WATCH/MULTI`) and DynamoDB (ConditionExpression) are the same idea.

### 3. Internal Working

OCC's guarded UPDATE is atomic because the row lock is taken *during* the update and the predicate re-checked against the current version (READ COMMITTED requery semantics working *for* you). Version column can be an integer, `xmin` (Postgres system column — neat trick), or a timestamp (beware clock resolution). Pessimistic `FOR UPDATE` parks competitors on the row's lock queue; with `NOWAIT`/`SKIP LOCKED` you convert waiting into control flow.

## 4. Visualization (ASCII)

```
OPTIMISTIC (version column):
T1: read (doc, v=5) ── user edits 3 min ──▶ UPDATE ... WHERE v=5 → v=6 ✔
T2: read (doc, v=5) ── user edits 4 min ──▶ UPDATE ... WHERE v=5 → 0 rows ✖
                                              → "document changed" → merge/retry
no locks held during the 4 minutes ✔

PESSIMISTIC:
T1: BEGIN; SELECT ... FOR UPDATE ──── work ────▶ COMMIT
T2:        SELECT ... FOR UPDATE ──── waits ──▶ proceeds after T1
throughput = serialized per row; safe but queued
```

## 5. Real Production Example

- **Optimistic**: CMS/document editing (two editors, minutes-long edits) — version column + conflict UI. Also every JPA `@Version`, Elasticsearch `_seq_no`, DynamoDB conditional writes.
- **Pessimistic**: warehouse inventory allocation during checkout — short transaction, high contention on hot SKUs; `FOR UPDATE` (or single-statement guarded decrement) is correct; optimistic here would retry-storm on popular items during a drop.

## 6. Common Interview Questions

- "Two admins edit the same record — design the protection." (OCC + conflict UX)
- "Implement optimistic locking in SQL." (guarded UPDATE, rowcount)
- "When does optimistic locking perform worse?" (high contention: retries × wasted work exceed lock waits)
- "How do you do pessimistic locking across two HTTP requests?" (you don't — that's a *lease*: locked_by/ locked_until columns, expiry, still OCC underneath)

## 7. Common Mistakes

- OCC without handling rowcount=0 (silently dropping the loser's write = the lost update you were preventing).
- Version check on read but not in the UPDATE's WHERE (TOCTOU).
- Holding `FOR UPDATE` transactions across user interaction / external APIs.
- Timestamp versions with second resolution (two updates same second).
- Using OCC for aggregate invariants across rows (that's write skew territory — 6.6).

## 8. Best Practices

- Default OCC for user-facing edits; version column + clear conflict responses (HTTP 409/412).
- Pessimistic (or atomic statements) for hot, short, machine-driven mutations.
- Leases (not held locks) for long exclusive claims: `claimed_by, claimed_until` + guarded claim UPDATE + expiry reaper.
- Bound retries with jitter; surface repeated conflicts, don't loop forever.

## 9. Coding Questions

1. Full OCC flow for a profile edit: read (id, version, fields) → guarded UPDATE → rowcount 0 path returning the fresh row for merge.
2. Lease-based claim: `UPDATE tasks SET claimed_by=$w, claimed_until=now()+'5 min' WHERE id=$t AND (claimed_by IS NULL OR claimed_until < now()) RETURNING *;` — explain every predicate.

### 10. SQL Examples

```
-- Optimistic concurrency
SELECT id, version, title, body FROM docs WHERE id = 42;      -- returns version=5
UPDATE docs
SET title=$1, body=$2, version = version + 1, updated_at = now()
WHERE id = 42 AND version = 5;                                 -- rowcount 0 ⇒ conflict

-- xmin trick (no schema change)
SELECT id, xmin AS version, ... FROM docs WHERE id=42;
UPDATE docs SET ... WHERE id=42 AND xmin = $version::text::xid;

-- Pessimistic with control flow instead of blocking
SELECT * FROM inventory WHERE sku=$1 FOR UPDATE NOWAIT;  -- 55P03 ⇒ tell user "busy, retry"
```

### 11. Optimization Techniques

- Reduce contention before choosing a scheme: shard hot rows, queue mutations to a single writer, batch increments.
- Combine: optimistic fast path, fall back to pessimistic after k conflicts.
- Keep versioned payloads small (don't re-write megabyte JSON per version bump — split hot fields).

### 12. Follow-up Questions

- "How does this map to DynamoDB / Mongo?" (ConditionExpression / findAndModify with query on version — same guarded-write shape)
- "OCC retry rate is 30% on one row — now what?" (it's a hot row: redesign — queue, shard, CRDT-style merge, or pessimistic)
- "What HTTP machinery expresses OCC to clients?" (ETag + If-Match → 412)

---

## Chapter 6.6 — SERIALIZABLE & Write Skew

### 1. Why Interviewers Ask This

Write skew is the staff-level discriminator: an anomaly that survives snapshot isolation and breaks real invariants (on-call rosters, budgets, double-booking). Explaining SSI marks you as genuinely deep in Postgres.

### 2. Core Concept

**Write skew**: T1 and T2 each read a shared predicate, then write **different** rows, each preserving the invariant *given what they read* — but together violating it. Canonical: "≥1 doctor must remain on call." Both on-call doctors check `count(on_call)=2 ≥ 2` and each takes *themselves* off call → 0 on call. No row was written by both → no write-write conflict → **snapshot isolation (PG REPEATABLE READ) permits it**.

**SERIALIZABLE in Postgres = SSI** (Serializable Snapshot Isolation, PG 9.1+): optimistic — runs like snapshot isolation while tracking read/write dependencies (predicate locks, "SIReadLocks"); when a dangerous dependency cycle appears, one transaction is aborted with 40001. No blocking, no extra locks held; cost = tracking overhead + retries + false positives (predicate locks can be page/relation-granular).

Alternatives when you can't/won't use SERIALIZABLE: - **Materialize the conflict**: make the invariant a *row* both transactions must touch — e.g., a `shift_slots` row per slot updated on every change (turns skew into a write-write conflict), or `SELECT ... FOR UPDATE` on a parent/guard row. - **Constraints**: unique / exclusion constraints check against *current* data, not snapshots — `EXCLUDE USING gist (room WITH =, during WITH &&)` kills double-booking outright.

### 3. Internal Working

SSI theory: snapshot-isolation anomalies always contain two consecutive **rw-antidependency** edges (T1 reads what T2 later writes) in the serialization graph. Postgres tracks rw-edges via SIRead locks taken by reads (kept until overlapping transactions finish); on detecting a T_a →rw→ T_b →rw→ T_c pattern with a committed pivot, abort one.

False positives arise from lock granularity promotion (row → page → relation under memory pressure: `max_pred_locks_per_transaction`).

## 4. Visualization (ASCII)

```
Write skew (both at REPEATABLE READ — permitted!):
invariant: at least 1 doctor on call.  state: alice=on, bob=on
T1: SELECT count(*) WHERE on_call → 2      T2: SELECT count(*) WHERE on_call → 2
T1: UPDATE alice SET on_call=false          T2: UPDATE bob SET on_call=false
T1: COMMIT ✔                                T2: COMMIT ✔      → 0 on call ✖✖

Under SERIALIZABLE (SSI):
T1 read the predicate T2 wrote to, and vice versa → rw-cycle detected
→ one gets ERROR 40001 → retries → sees count=1 → refuses. invariant holds ✔

Materialized conflict alternative:
both must  UPDATE oncall_guard SET version=version+1  → write-write conflict → serialized
```

## 5. Real Production Example

Real-world write skews interviewers recognize: double-booking a meeting room (both check "no overlap", insert different rows); spending a budget from two services (both read remaining, insert different expense rows); issuing the last two support slots twice. Stripe-class systems solve the money ones with **exclusion/unique constraints + single-row guards**, reserving SERIALIZABLE for genuinely predicate-shaped invariants. Booking systems (Module 9) use `EXCLUDE ... WITH &&` as the bedrock.

## 6. Common Interview Questions

- "Give an anomaly snapshot isolation does NOT prevent." (write skew, with the doctors example)
- "How does Postgres implement SERIALIZABLE without two-phase locking?" (SSI summary)
- "Prevent double-booking without SERIALIZABLE." (exclusion constraint / guard row FOR UPDATE)
- "What operational costs come with SERIALIZABLE?" (retries mandatory, predicate-lock memory, false positives, all-or-nothing per involved txns)
- "Why must the *whole set* of cooperating transactions be SERIALIZABLE?" (a READ COMMITTED bystander can still observe/create anomalies against them)

## 7. Common Mistakes

- Claiming REPEATABLE READ prevents "all anomalies except phantoms" — write skew survives.
- Using SERIALIZABLE without retry loops (40001 is expected behavior, not failure).
- Modeling invariants only in application checks when a constraint could enforce them against current data.
- Thinking SSI blocks like 2PL — it aborts instead; latency profile is different (fast until it isn't).

## 8. Best Practices

- Prefer declarative enforcement (unique/exclusion constraints) > materialized conflicts > SERIALIZABLE > hope.
- If SERIALIZABLE: keep transactions short, retry with backoff, watch `pg_stat_database.serialization_failures`-style metrics, size `max_pred_locks_per_transaction`.
- Document each invariant with *which mechanism* guards it — the artifact interviewers wish every team had.

## 9. Coding Questions

1. Doctors on call: write the schema, show the skew at RR, then fix three ways (SERIALIZABLE + retry; guard row; CHECK-via-trigger with FOR UPDATE) and rank them.
2. Meeting rooms: DDL with `EXCLUDE USING gist (room_id WITH =, during WITH &&)` and the insert that safely fails on overlap.

## 10. SQL Examples

```
-- Exclusion constraint: overlap-proof bookings (needs btree_gist)
CREATE EXTENSION IF NOT EXISTS btree_gist;
CREATE TABLE bookings (
  room_id int NOT NULL,
  during  tstzrange NOT NULL,
  EXCLUDE USING gist (room_id WITH =, during WITH &&)
);
-- concurrent overlapping inserts: exactly one succeeds, other gets 23P01 ✔

-- SERIALIZABLE with retry (shape)
BEGIN ISOLATION LEVEL SERIALIZABLE;
SELECT count(*) FROM doctors WHERE on_call;
UPDATE doctors SET on_call = false WHERE id = $me;  -- only if count would stay >= 1
COMMIT;  -- app retries on 40001

-- Materialized conflict (guard row)
BEGIN;
SELECT 1 FROM oncall_guard WHERE team=$t FOR UPDATE;   -- serialize the invariant check
-- re-check count, then update
COMMIT;
```

## 11. Optimization Techniques

- Scope SERIALIZABLE to the few endpoints owning the invariant; leave the rest at READ COMMITTED.
- `SET default_transaction_isolation` per service/role rather than sprinkling BEGIN options.
- Reduce false positives: smaller transactions, avoid seq scans inside serializable txns (relation-level SIRead locks), enough predicate-lock memory.

## 12. Follow-up Questions

- "How would MySQL handle the doctors case at SERIALIZABLE?" (2PL-style: reads take shared locks → blocks instead of aborting — different failure mode: deadlocks/waits)
- "Can you get write skew across microservices with per-service DBs?" (yes — and no DB fixes it; needs a coordinator/saga/single-owner design)
- "What's `SERIALIZABLE READ ONLY DEFERRABLE` for?" (waits for a known-safe snapshot: heavy reports with zero abort risk and zero tracking cost)

---

# Module 6 — Practice Problems

## Easy (5)

1. Match each anomaly (dirty read, non-repeatable read, phantom, lost update, write skew) to the weakest PG isolation level that prevents it.
2. Two concurrent `UPDATE counters SET n = n + 1 WHERE id=1` at READ COMMITTED — is an increment ever lost? Why (single-statement atomicity + row lock queue)?
3. What error codes do you retry, and what's the retry recipe? (40001, 40P01; bounded jittered backoff; idempotent bodies.)
4. Why does a plain SELECT never block an UPDATE in Postgres, and what statement *does* a plain SELECT block?
5. Write the SKIP LOCKED dequeue for a `jobs` table, claiming up to 10 jobs.

## Medium (5)

1. Reproduce a lost update with two psql sessions at READ COMMITTED (exact command sequence), then show the same sequence failing at REPEATABLE READ.
2. An ALTER TABLE on a hot table caused a 90-second full outage though it only needed 200ms of work. Reconstruct the queue mechanics and write the safe-DDL runbook.
3. Inventory oversell: `SELECT stock; if stock>0: UPDATE stock=stock-1` under 500 concurrent buyers. Show the race, then fix with (a) one guarded statement, (b) FOR UPDATE, (c) SERIALIZABLE — and compare throughput on ONE hot SKU.
4. Nightly job deadlocks with OLTP ~50x/night; logs show the job updating in category order, OLTP updating in id order. Design the fix (chunked PK-ordered batches + retries) and explain why ordering works.
5. Choose optimistic or pessimistic (and justify): (a) wiki page edits, (b) seat selection at checkout, (c) cron job leader election, (d) bank ledger postings, (e) shopping cart updates.

## Hard (5)

1. Design the on-call invariant ("≥1 doctor per team on call") for a team-scale product: schema, the write-skew proof at RR, chosen mechanism (guard row vs SSI vs trigger), retry policy, and monitoring for violation attempts.
2. A REPEATABLE READ report transaction runs 40 minutes on the primary. Enumerate every harm (vacuum horizon, bloat, wraparound clock, lock exposure on any writes, replica conflict if moved) and produce the architecture that removes it (replica + snapshot exports, or SERIALIZABLE DEFERRABLE on standby).
3. Build a double-booking-proof reservation system for 10k hotels without SERIALIZABLE: exclusion constraints per resource, hot-resource sharding, waitlist path on conflict, and the idempotency layer for client retries. Prove each concurrent scenario.
4. SSI false-positive storm: serialization failures spike 100x during a batch import touching the same pages as OLTP serializable txns. Explain predicate-lock granularity promotion, confirm via pg_locks (SIReadLock modes), and fix (batch at READ COMMITTED + constraints, memory sizing, schedule isolation).
5. Compare Postgres SSI vs MySQL InnoDB SERIALIZABLE (2PL, gap locks) for a ticket-sales workload: failure modes (aborts vs deadlocks/waits), throughput under hot rows, developer ergonomics, and which you'd pick with justification.

---

*Next: Module 7 — Database Scaling*

# MODULE 7 — Database Scaling

> The system-design-round module. Every "design X at scale" interview walks this ladder: optimize → cache → replicate reads → partition → shard. Know the ladder, the mechanics of each rung, and what each rung breaks.

Chapters: 7.1 Replication & Read Replicas 7.2 Partitioning 7.3 Sharding 7.4 Connection Pooling 7.5 Caching with Redis: Patterns, Hot Keys, Invalidation

---

## Chapter 7.1 — Replication & Read Replicas

### 1. Why Interviewers Ask This

First scaling move everyone makes, with two traps built in: **replication lag** (stale reads) and **failover data loss** (async replication). Senior candidates must articulate sync vs async trade-offs and read-your-writes strategies.

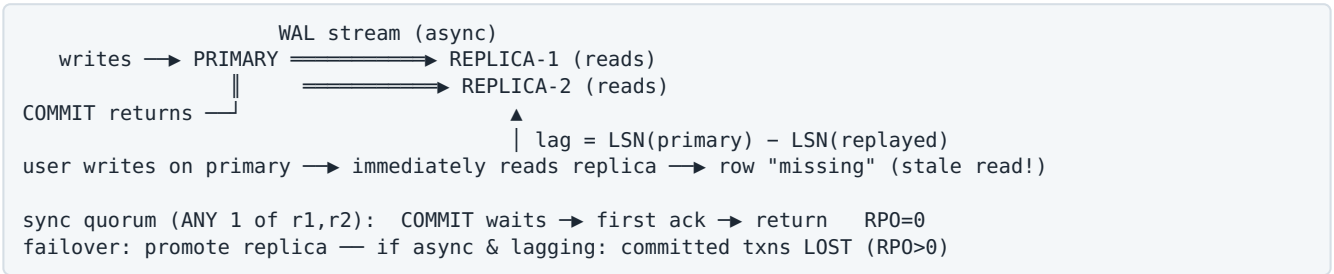
### 2. Core Concept

- **Physical/streaming replication** (Postgres default): ship the WAL byte-stream; replicas replay it → byte-identical, read-only standbys. Whole-cluster granularity.
- **Logical replication**: decode WAL into row changes, publish/subscribe per table → cross- version migrations, selective tables, fan-in — but no DDL replication, more care required.
- **Async** (default): primary commits without waiting → zero write latency cost; replica lag = potential data loss window on failover (RPO > 0).
- **Sync**: commit waits for replica(s) ack (`synchronous_standby_names`, quorum `ANY 1 (...)`) → RPO=0 at latency cost; a dead sync standby can stall writes (why you use quorum).
- **Read replicas** scale reads only — writes still hit one primary. Lag is normal (ms–s; unbounded during load/vacuum storms). **Failover**: promote a replica; clients re-point; split-brain protection (fencing, one-writer guarantee) is the hard part — usually delegated to RDS/Patroni/etc.
- MySQL parallel: binlog replication (row-based), semi-sync ≈ quorum-light, GTIDs for failover tracking.

### 3. Internal Working

Primary walsender → replica walreceiver → WAL written → startup process replays into pages. Replica queries run against MVCC snapshots as usual; **replication conflicts** occur when replay needs to remove row versions a replica query still uses (vacuum cleanup records) → replica either delays replay (`max_standby_streaming_delay`) or cancels queries; `hot_standby_feedback` exports the replica's xmin to the primary (no cancels, but primary bloat). Lag metrics: LSN delta (`pg_current_wal_lsn()` vs replay LSN) and time (`pg_last_xact_replay_timestamp`).

### 4. Visualization (ASCII)




### 5. Real Production Example

The classic: user updates their profile, next request lands on a lagged replica, sees the old name, files a bug — "the save button doesn't work." Standard fixes deployed at Meta/Uber-style stacks: **session pinning** (after a write, that session reads the primary for N seconds) or **LSN tokens** (client carries the write's LSN; replica serves only if replayed

past it). Second classic: failover during an incident loses 4 seconds of async-replicated payments → post-mortem moves the payments cluster to quorum-sync.

## 6. Common Interview Questions

- "Sync vs async replication — trade-offs and when each?" (latency/availability vs RPO)
- "How do you handle read-your-own-writes with replicas?" (pinning / LSN wait / stick writes- readers to primary)
- "What causes replication lag and how do you monitor it?" (write bursts, vacuum, big txns, replica I/O, long replica queries; LSN + time lag)
- "What happens to un-replicated commits on failover?"
- "Physical vs logical replication — when logical?" (upgrades, selective, cross-region fan-in, CDC)

## 7. Common Mistakes

- Routing all reads to replicas blindly (auth checks, post-write reads break).
- Treating replicas as backups (they replicate your DELETE too; PITR/base backups are backups).
- Single sync standby without quorum → standby death blocks all writes.
- Ignoring that replicas must replay serially-ish: a write-hot primary can outrun replica replay forever.
- Scaling *writes* with read replicas (they don't).

## 8. Best Practices

- Classify queries: primary-required (post-write, transactional reads) vs lag-tolerant (browse, analytics) — encode in the data-access layer.
- Alert on lag (bytes and seconds); cap acceptable staleness per endpoint.
- Quorum sync for money; async for the rest; measure the actual commit-latency cost.
- Use managed failover (RDS/Aurora/Patroni) and *test* failovers regularly.

## 9. Coding Questions

1. Write the lag queries: on primary (`pg_stat_replication`: sent vs replay LSN per replica, `write_lag/flush_lag/replay_lag`), on replica (replay timestamp delta).
2. Design the LSN-token read-your-writes flow: capture `pg_current_wal_lsn()` after commit, send as cookie, replica middleware compares `pg_last_wal_replay_lsn()`, waits ≤50ms, else forwards to primary.

## 10. SQL Examples

```
-- Primary: per-replica lag
SELECT application_name, state,
       pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn) AS lag_bytes,
       replay_lag
FROM pg_stat_replication;

-- Replica: staleness
SELECT now() - pg_last_xact_replay_timestamp() AS lag;

-- Quorum-sync durability for the cluster (postgresql.conf)
-- synchronous_standby_names = 'ANY 1 (replica1, replica2)'
-- Per-transaction opt-out for cheap writes:
SET LOCAL synchronous_commit = local;
```

## 11. Optimization Techniques

- Dedicated replicas per workload (OLTP reads vs analytics vs backups) — isolate cache and conflict profiles.
- `hot_standby_feedback` on the analytics replica only (accept primary bloat there), short `max_standby_streaming_delay` on OLTP replicas.
- Batch/segment huge writes so replay keeps pace; avoid single 100M-row transactions.

### 12. Follow-up Questions

- "Design multi-region: writes in one region, reads everywhere — what's user-visible?" (cross-region lag, read-your-writes across regions, failover RPO; leads to per-region write partitioning)
- "Aurora claims ~10ms replica lag and fast failover — what's architecturally different?" (shared distributed storage; replicas read the same storage rather than replaying full WAL locally)
- "When do you reach for logical replication into Kafka?" (CDC — cache invalidation, search indexing, warehouse feeds; Modules 5/8 tie-in)

---

## Chapter 7.2 — Partitioning

### 1. Why Interviewers Ask This

"The table hit a billion rows" is the prompt. Partitioning is the *single-node* answer (before sharding), and interviewers test whether you know what it does and does NOT solve, plus the partition-key discipline.

### 2. Core Concept

Split one logical table into physical child tables by a **partition key**: - **Range** (time, ids): time-series default; enables partition *lifecycle* (drop old months instantly). - **List** (region, tenant-group): data-residency, per-class management. - **Hash** (uniform spread): no natural range; spreads hot writes; no pruning for ranges.

What it buys: **pruning** (queries touching one partition scan 1/N of the data), **instant bulk deletes** (`DROP/DETACH PARTITION` vs DELETE of 500M rows), smaller per-partition indexes (fit in cache), parallelism per partition, targeted VACUUM. What it doesn't: more write throughput on one box (same disk/CPU), cross-partition queries (hit all partitions), and it *complicates* uniqueness: **every unique constraint / PK must include the partition key** (no global unique indexes in Postgres).

### 3. Internal Working

Declarative partitioning (PG10+): parent is a routing shell; tuple routing at insert; planner prunes at plan time (literal predicates) and **execution time** (parameters, join-driven — `Subplans Removed` in EXPLAIN). Each partition = full table (own indexes, stats, vacuum). Partition count sweet spot: dozens–low hundreds per query path (thousands inflate planning time/locks). `DETACH PARTITION CONCURRENTLY` for non-blocking removal; attach with a pre-added CHECK constraint to skip the validation scan.

### 4. Visualization (ASCII)

```
            events (parent — no data)
   ┌─────────────┬─────────────┬─────────────┬─────────────┐
 ev_2026_04  ev_2026_05  ev_2026_06  ev_2026_07   (range on created_at)
WHERE created_at >= '2026-06-10' AND < '2026-06-20'
        → planner prunes to ev_2026_06 only  (1/N of I/O, small hot index)
retention: DROP TABLE ev_2025_07;   -- 500M rows gone in milliseconds, no vacuum debt

anti-pattern: WHERE user_id = 42 (no partition-key predicate)
        → scans ALL partitions (N index probes) — worse than unpartitioned!
```

### 5. Real Production Example

Uber-style `trips/events` tables: monthly range partitions; every query carries a time predicate by convention (enforced in the query layer); retention = cron dropping month N−13. Before partitioning, deleting old data was a week-long DELETE causing bloat+lag; after, it's a metadata operation. Interview probe: "you partitioned by month but the app queries by user_id — what happens?" (all-partition scans; you chose the wrong key or need a composite approach).

### 6. Common Interview Questions

- "When do you partition and by what key?" (size/retention/pruning-driven; the key = the predicate every hot query carries)

- "Partitioning vs sharding?" (one node, transparent vs many nodes, app-visible)
- "Why must the PK include the partition key in Postgres?" (uniqueness enforced per-partition; no global index)
- "How does partitioning make deletes cheap?" (drop = unlink files, no row-by-row, no bloat)
- "What breaks if a query lacks the partition key?"

## 7. Common Mistakes

- Partitioning small tables (<~100GB) for fashion — pure overhead.
- Key mismatch with the workload (time-partitioned, id-queried).
- Thousands of tiny partitions (planning time, catalog bloat, per-partition overhead).
- Expecting global uniqueness on a non-key column (`email` unique across hash partitions — can't; needs a separate lookup table or app enforcement).
- Forgetting a default partition → inserts outside ranges fail (or conversely, default partition silently swallowing everything and killing pruning).

## 8. Best Practices

- Range-by-time for event-ish data; automate partition creation (pg_partman) and retention.
- Make the partition key mandatory in hot query paths (lint it in the data layer).
- Index per-partition to match queries; keep per-partition indexes cache-resident.
- Roll up before dropping: aggregate old partitions into summary tables, then drop.

## 9. Coding Questions

1. Write the DDL: `events` partitioned by month with per-partition `(tenant_id, created_at)` index, PK `(id, created_at)`, plus next-month creation and 12-month retention statements.
2. Show EXPLAIN evidence of pruning working and broken (with/without the time predicate) and fix the broken query.

## 10. SQL Examples

```
CREATE TABLE events (
  id bigint GENERATED ALWAYS AS IDENTITY,
  tenant_id bigint NOT NULL,
  created_at timestamptz NOT NULL,
  payload jsonb,
  PRIMARY KEY (id, created_at)                -- must include partition key
) PARTITION BY RANGE (created_at);

CREATE TABLE events_2026_07 PARTITION OF events
  FOR VALUES FROM ('2026-07-01') TO ('2026-08-01');
CREATE INDEX ON events_2026_07 (tenant_id, created_at DESC);

-- Retention: instant, no bloat
ALTER TABLE events DETACH PARTITION events_2025_07 CONCURRENTLY;
DROP TABLE events_2025_07;

-- Verify pruning
EXPLAIN SELECT * FROM events
WHERE tenant_id=42 AND created_at >= '2026-07-01' AND created_at < '2026-07-08';
-- plan lists ONLY events_2026_07 ✔
```

## 11. Optimization Techniques

- Sub-partitioning (range → hash) only with strong evidence; complexity grows fast.
- Partition-wise joins/aggregates (`enable_partitionwise_join/aggregate`) for co-partitioned big tables.
- Keep recent partitions on fast storage, old on cheap (tablespaces) — hot/cold tiering.

## 12. Follow-up Questions

- "How do you migrate a live 2TB unpartitioned table to partitions with minimal downtime?" (create partitioned shadow, backfill in chunks, dual-write or logical replication, cutover)

- "Global secondary index over partitions — options?" (separate lookup table maintained transactionally; or accept per-partition + app fan-out)
- "How does this compare to DynamoDB's partitioning?" (transparent hash by partition key — the *mandatory-key* discipline is the same lesson — Module 8)

---

# Chapter 7.3 — Sharding

## 1. Why Interviewers Ask This

The end-game scaling question: writes exceed one machine. They test key choice, resharding, and honest accounting of what you lose (cross-shard everything). Great candidates spend the first minute arguing whether sharding is needed at all.

## 2. Core Concept

Sharding = horizontal partitioning **across machines**; each shard is an independent DB holding a subset keyed by the **shard key**.

Strategies: - **Hash(key) → shard**: uniform spread; loses range queries across keys; the default for user-keyed data. Use **consistent hashing / many virtual buckets** (e.g., 4096 slots mapped to shards) so resharding moves buckets, not "all keys mod N". - **Range**: preserves key-locality/scans; hotspot risk (newest range gets all writes) — needs split/rebalance machinery (HBase/CockroachDB style). - **Directory/lookup**: mapping table key→shard; maximal flexibility (per-tenant placement, gradual migration) at the cost of running that mapping service (often combined: hash for users, directory for whale tenants).

What sharding breaks — say all of these: - **Cross-shard joins** (app-side joins, denormalize, duplicate reference tables to every shard) - **Cross-shard transactions** (avoid by design/colocate; else sagas/outbox; 2PC rarely) - **Global unique IDs** (Snowflake IDs, UUIDv7, per-shard sequences with shard bits) - **Cross-shard queries/analytics** (scatter-gather with merge, or CDC → warehouse) - **Rebalancing/resharding** (bucket moves with dual-read/write cutover) - **Hot keys** (one celebrity tenant > one shard's capacity — sub-shard or special-case them)

Shard key rule: the key that makes your **dominant access pattern single-shard** (usually user_id/tenant_id), immutable, high-cardinality, uniform-ish.

## 3. Internal Working

Routing layer options: client library computes bucket (Uber/Pinterest style), middleware/proxy (Vitess for MySQL, Citus for Postgres), or coordinator nodes (Mongo mongos). Scatter-gather reads: fan out, merge (k-way merge for sorted+limit — cursor carries per-shard positions). Resharding playbook: double buckets → copy bucket data (snapshot + CDC tail) → dual-write or pause-writes-per-bucket cutover → verify checksums → flip routing → clean up.

## 4. Visualization (ASCII)

```
                   router: bucket = hash(user_id) % 4096
                   bucket_map: [0..1023]→S1 [1024..2047]→S2 [2048..3071]→S3 [3072..4095]→S4
 user 42 ──► bucket 1732 ──► S2 (all of user 42's rows live together = single-shard ops ✔)

get user feed (single key)  → 1 shard ✔
"top sellers this week"     → scatter to 4 shards, gather+merge ✖ (or CDC→warehouse)
add shard S5: move buckets [3277..4095] S4→S5 (copy + tail + flip) — keys don't rehash ✔

hot key: tenant WHALE = 30% of writes → dedicated shard via directory override
```

## 5. Real Production Example

The canonical narratives interviewers expect you to know: **Instagram** sharded Postgres by user with thousands of logical shards (schemas) mapped to few machines — logical>physical from day one makes rebalancing "move a schema." **Vitess at YouTube/Slack**: proxy-based MySQL sharding with resharding workflows. **Stripe/Uber**: directory-based placement for whale accounts. Common thread: shard *late*, shard by the entity you always query by, and keep bucket count ≫ machine count.

## 6. Common Interview Questions

- "When do you shard, and what do you try first?" (vertical, replicas, cache, partitioning, workload split — shard last)
- "Choose the shard key for [marketplace/chat/payments] and defend it."
- "How do you add a shard without downtime?" (bucket migration playbook)
- "How do you handle a query that needs data from all shards?"
- "What about transactions across two users on different shards?" (avoid; saga; or route both entities into an owning aggregate)

## 7. Common Mistakes

- Sharding by a mutable or low-cardinality key (country → 5 giant shards, US hotspot).
- `mod N` directly on servers → resharding = rehash everything.
- Ignoring the reference-data problem (joins to `products` from every shard — replicate it).
- Promising global uniqueness/constraints the design can no longer enforce (Module 1.7 tie-in).
- Under-specifying the migration: "we'll just move the data" — the dual-write/tail/verify part is the actual work.

## 8. Best Practices

- Many logical buckets, few physical shards; carry `bucket_id` in every row.
- Colocate all tables sharing the shard key ("shard groups") so entity-local transactions stay ACID on one shard.
- CDC every shard into one warehouse for analytics; never scatter-gather for BI.
- Idempotency keys everywhere (retries across routing flips).
- Instrument per-shard load; hot-shard alerts drive rebalancing.

## 9. Coding Questions

1. Design the bucket map schema (`buckets(bucket_id PK, shard_id, state)` with states active/migrating/moved) and write the router pseudocode handling in-flight migration (read-both, write-new or write-both policy).
2. Global-ish unique order numbers on 16 shards: Snowflake layout (41 bits ms timestamp, 10 bits shard/worker, 12 bits sequence) — write the generator and collision argument.

## 10. SQL Examples

```
-- Per-shard DDL is normal Postgres; the sharding lives in routing metadata:
CREATE TABLE buckets (
  bucket_id int PRIMARY KEY,             -- 0..4095
  shard_id  int NOT NULL,
  state     text NOT NULL DEFAULT 'active'  -- active | migrating | moved
);

-- Scatter-gather top-N merge (app coordinates; per shard:)
SELECT * FROM orders WHERE created_at >= $1
ORDER BY created_at DESC, id DESC LIMIT 50;   -- run on each shard, k-way merge, take 50

-- Citus flavor (Postgres extension), for contrast:
SELECT create_distributed_table('orders', 'user_id');  -- co-locates by user_id
```

## 11. Optimization Techniques

- Route read-only single-key queries to shard replicas (sharding × replication compose).
- Cache the bucket map client-side with versioned invalidation (it changes rarely).
- Sub-shard hot tenants by a secondary key (tenant_id + hash(entity_id)) — accept fan-out for that tenant only.

## 12. Follow-up Questions

- "Why did Vitess/Citus-style middleware win over app-level sharding at many companies?" (centralizes routing/resharding/DDL fan-out; app stays SQL-ish)
- "How do NewSQL systems (Spanner, CockroachDB) change this conversation?" (automatic range-sharding + distributed transactions with consensus — you pay latency, drop the app-level machinery)

- "Design the un-shard: merging two shards after load dropped." (same bucket playbook, reversed)

---

# Chapter 7.4 — Connection Pooling

## 1. Why Interviewers Ask This

Connection exhaustion is a top-3 real Postgres incident, and serverless made it worse. It tests whether you know Postgres's process-per-connection cost and the pooler modes.

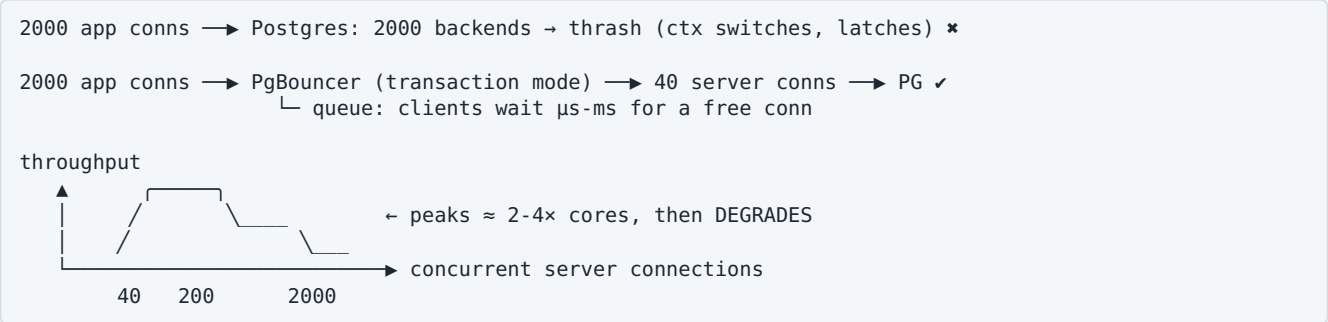
## 2. Core Concept

- Postgres connection = **forked OS process** (~5–10MB + scheduler load + lock-table entries). A few hundred active connections is healthy; thousands degrade *everything* (context switches, contention on shared structures) — even idle ones cost.
- **App-side pool** (HikariCP etc.): reuse per instance. Insufficient alone: 200 instances × 20 conns = 4000.
- **Server-side pooler** (PgBouncer, RDS Proxy): multiplexes thousands of client connections onto tens of server connections. Modes:
- **session**: 1 client ↔ 1 server conn for the session — safe, weak multiplexing.
- **transaction** (the production standard): server conn borrowed per transaction — breaks session state (SET, prepared statements pre-1.21, advisory locks, LISTEN, temp tables).
- **statement**: per statement; forbids multi-statement transactions.
- Sizing rule of thumb: active server connections ≈ cores × (2–4) for OLTP; queue the rest in the pooler. More is *slower*.

## 3. Internal Working

Why few connections outperform many: each backend competes for CPU, buffer-pool latches, lock manager, and snapshot computation (ProcArray) — throughput peaks near core count and *falls* beyond it; latency-vs-throughput curve is a classic interview sketch. PgBouncer is a single-threaded event loop (run several instances/so_reuseport at scale). Transaction mode + `DISCARD ALL`-style reset between borrows is how state leakage is prevented.

## 4. Visualization (ASCII)




## 5. Real Production Example

Incident every team has: deploy doubles pods; each pod opens 20 conns; Postgres hits `max_connections`, new conns refused, health checks fail, cascade. Or the serverless version: Lambda burst opens 3000 connections. Fix: PgBouncer/RDS Proxy in transaction mode, app pools sized down, `max_connections` modest, alerting on saturation. Interviewers often ask the Lambda variant explicitly.

## 6. Common Interview Questions

- "Why are Postgres connections expensive?" (process model + shared-structure contention)
- "PgBouncer transaction vs session mode — what breaks in transaction mode?"
- "How do you size a pool?" (cores-based, Little's law: conns ≈ QPS × avg query time)
- "Serverless + Postgres — what's the problem and fix?"

- "Symptoms of pool exhaustion vs DB slowness — how do you tell apart?" (client wait time vs server query time; pooler queue metrics — Module 11)

## 7. Common Mistakes

- Raising `max_connections` to 5000 as the "fix" (institutionalizes the thrash).
- Transaction mode with session-state reliance (SET search_path, advisory locks held across transactions, LISTEN) — subtle breakage.
- Pools sized "generously" per service, summing past DB capacity fleet-wide.
- Long transactions/idle-in-transaction hogging pooled server conns (pool multiplexing dies — ties to Module 6 timeouts).

## 8. Best Practices

- PgBouncer/RDS Proxy transaction mode as default architecture; session mode only for the exceptional session-stateful consumers.
- Budget connections fleet-wide: Σ(app pools) < pooler client limit; pooler server pool ≈ cores×3.
- `idle_in_transaction_session_timeout`, `statement_timeout` to protect the pool.
- Monitor: pooler queue wait, server conn saturation, PG `pg_stat_activity` state counts.

## 9. Coding Questions

1. Little's-law sizing: 8k QPS, mean query 5ms → ~40 busy conns; add p99 headroom → pool ~60. Show the math and what happens at mean 50ms (400 needed → redesign, don't just grow).
2. Write the monitoring query grouping `pg_stat_activity` by state (active / idle / idle in transaction) and flagging waits.

## 10. SQL Examples

```
-- Connection census
SELECT state, count(*) FROM pg_stat_activity GROUP BY state;

-- Who's hogging: idle-in-transaction sessions
SELECT pid, usename, now()-xact_start AS txn_age, left(query,60)
FROM pg_stat_activity
WHERE state = 'idle in transaction' ORDER BY xact_start;

-- Guardrails
ALTER SYSTEM SET idle_in_transaction_session_timeout = '60s';
ALTER SYSTEM SET statement_timeout = '30s';   -- per-role/app overrides as needed
```

## 11. Optimization Techniques

- Prepared statements via pooler (PgBouncer ≥1.21 supports protocol-level named statements in transaction mode) to reclaim parse/plan savings.
- Separate pools per workload (web / worker / cron) so batch jobs can't starve user traffic.
- Keep transactions short (Module 6) — the pool multiplexing ratio *is* your transaction duration profile.

## 12. Follow-up Questions

- "Why does MySQL tolerate more connections?" (thread-per-connection, lighter than processes — still not free)
- "Pooler is now the SPOF — how do you deploy it?" (pairs behind a TCP LB / per-node sidecar / managed proxy; connection draining on deploys)
- "How does Aurora/Neon 'serverless Postgres' attack this differently?" (separated proxy/ compute layers, connection multiplexing built-in)

---

# Chapter 7.5 — Caching with Redis: Patterns, Hot Keys, Invalidation

## 1. Why Interviewers Ask This

"Add a cache" is everyone's answer; the interview is about what follows: consistency, invalidation, stampedes, hot keys. ("There are only two hard things…" — they will quote it.)

## 2. Core Concept

Why Redis: in-memory (µs latency), rich structures (strings, hashes, sets, sorted sets, lists, streams), single-threaded per shard (atomic ops, no locks), Redis Cluster for sharding.

Patterns: - **Cache-aside** (lazy; the default): read → miss → load DB → SET with TTL; write → update DB → **invalidate (DEL)** the key. App owns the logic. - **Read-through / write-through**: cache layer loads/stores synchronously (consistency ↑, write latency ↑). - **Write-behind**: buffer writes in cache, flush async (fast, risky — loss window). - TTL always, even with explicit invalidation (TTL = corruption backstop).

Failure modes (the real interview content): - **Stale reads**: DB updated, cache not yet invalidated; or race: reader loads old value and SETs it *after* the invalidation (fix: short TTLs, versioned keys, or CAS/Lua). - **Stampede / dogpile**: hot key expires → thousands of concurrent DB loads. Fixes: per-key mutex (SET NX lock, one loader), probabilistic early refresh, background refresh, serve-stale-while-revalidate. - **Hot keys**: one key (celebrity profile) saturates one Redis shard/node. Fixes: local in-process cache layer, key replication (`key#1..N` random read), split the value. - **Big keys**: 10MB values / million-member sets block the single thread (`SCAN`, not `KEYS`; chunk values). - **Penetration**: misses for nonexistent ids hammer the DB — cache negative results (short TTL) or bloom filter. - **Eviction**: memory full → `maxmemory-policy` (allkeys-lru / volatile-ttl / lfu) — know that eviction ≠ invalidation.

Invalidation strategies ranked: TTL-only (simplest, staleness window) → explicit DEL on write (precise, needs discipline at every write path) → **CDC-driven** (Debezium tails WAL → consumer DELs keys: catches *every* write path incl. backfills; eventual by ms) → versioned keys (never invalidate; bump version pointer).

## 3. Internal Working

Redis single-threaded event loop: every command atomic; long commands block everything (hence big-key danger). Persistence: RDB snapshots / AOF appendfsync — cache use typically tolerates loss; "Redis as a database" needs AOF everysec + replicas and still isn't Postgres. Redis Cluster: 16384 hash slots over nodes; multi-key ops require same-slot (hash tags `{user:42}:...`). Replication is async → failover can lose recent writes (don't keep the only copy of anything precious).

## 4. Visualization (ASCII)

```
cache-aside read:                    write + invalidate race:
app ─GET k─▶ redis ─hit─▶ return     T1: read DB (v1) …slow…
   └─miss─▶ DB ─▶ SET k TTL ─▶ return T2: write DB (v2); DEL k
                                     T1: SET k = v1  ← STALE value cached!
stampede on expiry:                   fix: TTL backstop / version in key / compare-and-set
    k expires
1000 reqs ──▶ all miss ──▶ 1000 DB queries ✖
fix: first loader takes SET k:lock NX EX 5 → loads → SET k → DEL lock
     others: brief wait/serve stale

hot key: GET celeb:99 = 500k/s → one shard pinned
fix: app-local LRU (1s TTL) in front  → shard sees 1/1000th
```

## 5. Real Production Example

Twitter/Meta-scale celebrity problem: one profile key takes 500k reads/s — a single Redis node caps out; the answer stack: in-process cache with ~1s TTL (absorbs 99.9%), replicated keys, and precomputed fan-out. Stampede classic: a deploy flushes cache; DB gets the entire read load at once and falls over (thundering herd on cold cache) — mitigations: warm-up, staggered TTLs with jitter, request coalescing. These two stories cover most cache interview follow-ups.

## 6. Common Interview Questions

- "Walk through cache-aside and its consistency guarantees." (and the SET-after-DEL race)
- "Cache invalidation strategies — compare." (TTL / write-path DEL / CDC / versioned keys)
- "What's a cache stampede and three mitigations?"
- "How do you handle a hot key?" (local cache, replication, splitting)
- "Should you cache before or after optimizing the query?" (after — cache hides, doesn't fix; and cold-cache = raw query cost)
- "Delete or update the cache on write?" (DEL is safer: update races produce interleaved stale values; recompute on next read)

## 7. Common Mistakes

- No TTL "because we invalidate correctly" (until the one write path that doesn't).
- `KEYS pattern*` in prod (O(N), blocks the loop) — `SCAN`.
- Caching before fixing an unindexed query — cold cache melts the DB.
- One Redis for cache + queues + locks + rate limits → eviction policy conflicts (evicting "cache" evicts your locks).
- Treating Redis replication as durable (async; failover loses tail writes).

## 8. Best Practices

- Every key: TTL + jitter; every value: version/schema tag; every miss path: stampede-guarded.
- Separate Redis instances by role (cache vs data structures vs queues) with role-appropriate eviction/persistence.
- Cache at the right layer: precomputed view models (per-page payloads) beat caching raw rows.
- Measure hit ratio per key-class; below ~80% question the cache's existence.
- CDC-driven invalidation for multi-writer systems.

## 9. Coding Questions

1. Implement stampede-safe cache-aside pseudocode: GET → miss → `SET lock NX PX 3000` → winner loads DB and SETs (TTL+jitter) → losers retry/serve stale. Handle loader crash (lock PX).
2. Design keys/TTLs/invalidation for a product page: product core (1h + CDC DEL), price+stock (30s TTL only), reviews first page (5m, DEL on new review), rendered payload version-keyed.

## 10. SQL Examples

```
-- The DB side of cache-aside: the query worth caching should still be indexed
SELECT p.*, i.quantity FROM products p
JOIN inventory i USING (product_id) WHERE p.id = $1;

-- CDC invalidation source (logical replication publication)
CREATE PUBLICATION cache_inval FOR TABLE products, inventory, reviews;
-- Debezium/consumer: on change → DEL product:{id}, DEL product_page:{id}
```

```
# Redis command shapes to know cold:
SET product:42 "{...}" EX 3600
GET product:42 / DEL product:42
SET lock:product:42 token NX PX 3000          # stampede/mutex
INCRBY views:42 1 ; EXPIRE views:42 86400     # counters
ZADD leaderboard 9812 user:42                  # sorted set = ranking
MGET product:1 product:2 product:3             # batch (avoid N+1 to the cache!)
SCAN 0 MATCH product:* COUNT 1000              # never KEYS
```

## 11. Optimization Techniques

- Two-tier caching: in-process LRU (ms TTL, absorbs hot keys) → Redis → DB.
- Pipeline/MGET batches; hash tags to co-locate related keys in Cluster.
- Negative caching with short TTL for penetration; bloom filter for hard cases.
- TTL jitter (`ttl ± rand(10%)`) to de-synchronize mass expiry.

## 12. Follow-up Questions

- "Redis dies — what happens to your system, and what's your cold-start plan?" (degradation math: DB at full read load? load-shed? warm-up script?)
- "When is Redis the primary store legitimately?" (ephemeral-by-nature data: sessions, rate limits, presence — where loss is acceptable; Module 8)
- "How would you build rate limiting on Redis?" (INCR+EXPIRE fixed window / sorted-set sliding window / token bucket in Lua — classic mini-design)

---

# Module 7 — Practice Problems

## Easy (5)

1. Reads are 90% of load and the primary is CPU-bound. Ladder the next three moves in order and the risk each introduces.
2. Write the two lag queries (primary side and replica side) and define an alert threshold for a feed product vs a payments product.
3. Your `events` table is 2TB, queries always include `created_at` ranges, retention is 12 months. Design the partitioning (key, granularity, retention op).
4. 500 Lambda instances each open a DB connection on cold start. Explain the failure and the fix architecture.
5. A product page's cache key expires and 2,000 requests hit the DB simultaneously. Name the phenomenon and implement one mitigation in pseudocode.

## Medium (5)

1. Design read-your-own-writes for: web app, primary + 4 async replicas behind a router. Provide the LSN-token mechanism end-to-end and the fallback policy.
2. You must delete 800M rows older than 18 months from an unpartitioned table without downtime. Give the chunked-delete plan (with vacuum pacing) AND the partition-migration plan; argue which to choose at what table size.
3. Choose shard keys and justify: (a) B2B SaaS (tenants of wildly different sizes), (b) consumer chat app, (c) payments ledger, (d) IoT telemetry. For each, name the query that becomes expensive and the mitigation.
4. PgBouncer transaction mode broke: advisory-lock-based cron dedup and `SET search_path` multi-tenancy. Explain both breakages and redesign each feature pool-compatibly.
5. Design the caching layer for a news homepage (1M RPS peak, editors publish continuously): layers, TTLs, invalidation path from CMS publish to edge, stampede control, cold-start plan.

## Hard (5)

1. Full resharding design: 4 shards → 8, zero downtime, 4096 buckets. Specify: bucket map states and transitions, copy + CDC-tail mechanics, write policy during migration (dual-write vs brief per-bucket pause — argue one), verification (checksums/row counts), rollback points, and cutover.
2. Multi-region active-passive Postgres: writes in us-east, read replicas in eu-west (80ms). Design: replication topology, EU read staleness handling, failover runbook with RPO/RTO numbers for async vs quorum-sync, and how EU users' read-your-writes survives failover.
3. The celebrity problem end-to-end: one user with 80M followers posts; design the fan-out (push vs pull threshold), the hot-key strategy for their profile/post counters, and the cache consistency for edit/delete of a viral post.
4. A replica serves analytics and keeps cancelling long queries (vacuum conflicts). Compare the four fixes (`max_standby_streaming_delay`, `hot_standby_feedback`, dedicated delayed replica, CDC→warehouse) with their exact costs, pick per company stage.
5. Your cache hit ratio is 97%, but DB load doubles every deploy and quarterly cache flushes cause brownouts. Design deploy-safe caching: versioned keys with gradual rollover, dual-read during version transition, warm-up pipeline driven by top-K key logs, and load-shedding when hit ratio drops below a floor.

---

*Next: Module 8 — NoSQL*

# MODULE 8 — NoSQL

> System-design rounds require fluency in four systems — MongoDB, Redis, Cassandra, DynamoDB — plus the two mechanisms underneath them all: **LSM trees** and **consistent-hash replication**. The winning skill: choosing by access pattern and naming the sacrifice.

Chapters: 8.1 When to Choose NoSQL (Decision Framework) 8.2 Data Models: Document vs Key-Value vs Wide-Column 8.3 LSM Trees — The Write-Optimized Engine 8.4 MongoDB 8.5 Redis (as a Data Store) 8.6 Cassandra 8.7 DynamoDB 8.8 Replication, Sharding & CAP Across NoSQL (Comparison)

---

## Chapter 8.1 — When to Choose NoSQL

### 1. Why Interviewers Ask This

The opening move of most system-design interviews. They're screening for engineering judgment: candidates who choose by workload numbers and access patterns vs candidates who choose by hype.

### 2. Core Concept

Choose NoSQL when you can name **which of these you're buying**: 1. **Write scale beyond one primary** (Cassandra/DynamoDB: horizontal, masterless or managed). 2. **Access-pattern simplicity at huge data volume** (get/put by key; no ad-hoc queries needed). 3. **Availability over consistency** (must accept writes during partitions). 4. **Latency floors** (Redis in-memory; DynamoDB single-digit ms at any scale). 5. **Schema volatility on cheap terms** (documents; though Postgres jsonb covers much of this).

And you must name **what you pay**: joins, ad-hoc queries, multi-row ACID (limited), constraints, mature migrations — moved into application code. The senior framing: *"Postgres until a specific workload breaks it, then move that workload"* — and be ready to defend the reverse when the interviewer plays devil's advocate (known access patterns + huge scale from day one, e.g. IoT firehose → start with Cassandra/DynamoDB for that table).

### 3. Internal Working

The reason NoSQL scales writes is architectural, not magic: **no cross-node coordination on the write path** (per-key ownership via hashing, quorum-local decisions) + **write-optimized storage** (LSM — 8.3) + **no global constraints/ transactions to enforce**. Every capability they dropped is a coordination cost they don't pay. That's also the precise list of what you lose.

### 4. Visualization (ASCII)

```
Decision flow:
  need ad-hoc queries / joins / multi-row ACID?  ──yes──▶ RDBMS (+ cache/replicas)
        │ no
  access pattern = known key-based lookups?      ──no──▶ rethink; probably RDBMS/warehouse
        │ yes
  scale > single-primary write ceiling? latency floor? AP required?
        │ yes                                    │ no
        ▼                                        ▼
  KV/wide-column (Dynamo/Cassandra)         Postgres (jsonb if flexible shape)
  ephemeral/µs? → Redis    nested docs, mid-scale, rich secondary queries? → MongoDB
```

### 5. Real Production Example

Netflix: viewing history = per-member key, append-heavy, planet-scale, staleness-tolerant → Cassandra. Billing → MySQL. Amazon: cart availability during partitions → Dynamo lineage; orders → relational + strong consistency. Discord (famous case study): messages MongoDB → Cassandra → ScyllaDB as scale grew, keyed by channel — the access pattern (recent messages per channel) never changed, the engine did.

### 6. Common Interview Questions

- "SQL or NoSQL for [design prompt]?" (answer per-table/workload, not per-system)
- "What specifically breaks in Postgres at scale that Cassandra fixes?" (single-primary write ceiling, B+Tree random-write amplification, manual sharding)
- "What do you lose leaving Postgres?" (recite the tax list)
- "Polyglot persistence — how do you keep stores consistent?" (single source of truth + CDC; idempotent consumers)

### 7. Common Mistakes

- "NoSQL because scale" with no QPS/data-size numbers.
- Choosing a document DB because "schema flexibility" then hand-building joins/transactions.
- Ignoring Postgres jsonb / partitioning / replicas as the boring adequate answer.
- One database for everything (either direction) at a company past a certain size.

### 8. Best Practices

- Lead with access patterns + numbers, choose engine per workload, name the sacrifice, add the consistency-repair plan (CDC, reconciliation).
- Keep the system of record transactional wherever money/identity is involved.

### 9. Coding Questions

1. For a ride-hailing app, assign stores to: trips ledger, driver live locations, surge-pricing zones, chat, receipts search — with one-line justifications.
2. Estimate: 200k writes/s, 2KB each, key-value reads by id only, 99.9% availability across regions — argue Cassandra/DynamoDB over Postgres with arithmetic (per-node write ceilings, shard count).

### 10. SQL Examples

```
-- The honest middle ground first: Postgres as document store
CREATE TABLE profiles (
  user_id bigint PRIMARY KEY,
  doc jsonb NOT NULL
);
CREATE INDEX ON profiles USING gin (doc jsonb_path_ops);
SELECT doc->'settings'->>'theme' FROM profiles WHERE user_id = 42;
UPDATE profiles SET doc = jsonb_set(doc, '{settings,theme}', '"dark"') WHERE user_id = 42;
-- If this covers your "NoSQL need", you kept transactions and joins for free.
```

### 11. Optimization Techniques

- Hybrid: RDBMS system-of-record + NoSQL projections (feeds, caches, search) built via CDC — the architecture most FAANG answers converge to.
- Prove the bottleneck before migrating: pg_stat_statements + load tests beat vibes.

### 12. Follow-up Questions

- "Your PM wants 'flexible schema' — cheapest way to give it?" (jsonb column, not a new database)
- "When would you *start* on NoSQL day one?" (known KV pattern + guaranteed scale, e.g. telemetry; or team is DynamoDB-native and access patterns are stable)

---

## Chapter 8.2 — Data Models: Document, Key-Value, Wide-Column

### 1. Why Interviewers Ask This

Modeling is where NoSQL projects die. The interview form: "model X in DynamoDB/Mongo" — testing whether you design **from queries backwards** and denormalize on purpose.

## 2. Core Concept

- **Key-Value** (Redis, DynamoDB at its simplest): opaque value by key. One access pattern: get/put by key. Fastest, dumbest, scale-perfect.
- **Document** (MongoDB, DynamoDB items): JSON-ish tree per key; secondary indexes on fields; query within documents. Model: **embed what you read together** (1:few, bounded, co-accessed); **reference what grows unboundedly or is shared** (1:many-unbounded, many:many).
- **Wide-column** (Cassandra): rows grouped into **partitions** by partition key; within a partition, rows ordered by **clustering keys**. Think "one B-tree-ish sorted structure per partition key." Model: **one table per query**; partition key = the WHERE equality; clustering keys = the ORDER BY / range.

Universal NoSQL modeling law: **you denormalize; duplication is a feature; the application maintains consistency between copies.**

## 3. Internal Working

Embedding wins because one key fetch = one disk/network op returning everything (data locality); referencing forces N+1 fetches (no joins server-side — Mongo `$lookup` exists but is a single-node aggregation tool, not a scale primitive). Document size limits (Mongo 16MB; DynamoDB item 400KB) are why unbounded arrays (comments on a viral post) must be referenced or bucketed. Wide-column partitions map to a contiguous on-disk (LSM) run per node — in-partition range reads are sequential and cheap; cross-partition scans are cluster-wide and forbidden by culture.

## 4. Visualization (ASCII)

```
Document (embed vs reference):
order doc: { _id, user_id,                 user doc: { _id, name, ... }
  items: [ {sku, name, price, qty}, ... ]   ← embed: bounded, read together ✔
  status, total }
post doc:  { _id, author_id,
  comments: [ ...unbounded!... ] }          ← WRONG: reference/bucket comments ✖

Wide-column (Cassandra):
PARTITION KEY user_id │ CLUSTERING key created_at DESC
user:42 ──► [ (2026-07-01, order 9) (2026-06-28, order 8) (2026-06-01, order 7) ... ]
             └── sorted within partition: "last 10 orders of user 42" = one seek + scan ✔
"orders over $100 across all users" = full cluster scan ✖ (make another table for it)
```

## 5. Real Production Example

DynamoDB **single-table design** (AWS-canon): `PK=USER#42 / SK=PROFILE`, `PK=USER#42 / SK=ORDER#2026-07-01#9`, `PK=ORDER#9 / SK=ITEM#1` — one Query by PK returns a user's profile + recent orders in one round trip (an "item collection" replacing a join). Discord messages: partition = `(channel_id, bucket)` (time-bucketed to cap partition size), clustering = message_id desc — "recent messages in channel" is the one query that matters.

## 6. Common Interview Questions

- "Embed or reference for [user/orders, post/comments, product/reviews]?" (bounded? co-read? shared? growth?)
- "Model an e-commerce order system in DynamoDB." (single-table, item collections, GSIs)
- "Why does Cassandra require query-first modeling?" (partition-key-only access; no joins; duplication tables)
- "What's an unbounded-array bug in Mongo and its fix?" (16MB doc limit; bucketing pattern)

## 7. Common Mistakes

- Porting 3NF into documents (a document per SQL table + app joins = worst of both worlds).
- Embedding unbounded growth (comments, events) → document rewrite cost + size limits.
- Cassandra partition keys with unbounded partitions (partition per *user* for an IoT firehose → multi-GB partitions; add time buckets).
- Forgetting update fan-out: duplicated author name across 1M posts needs a rename strategy (async backfill + tolerate staleness — say it explicitly).

### 8. Best Practices

- Write the access-pattern table FIRST (operation, key, frequency); every table/index maps to rows of it.
- Bound everything: array sizes, partition sizes (Cassandra target <100MB, <~100k rows), item sizes.
- Duplicate immutable/slow-changing data freely; for mutable duplicates, define the sync mechanism and staleness budget at design time.

### 9. Coding Questions

1. Model Twitter-lite in Cassandra: tables for user timeline (fan-out-on-write), user's own tweets, tweet by id, followers — give full PRIMARY KEY (partition, clustering) for each.
2. Single-table DynamoDB for a hotel: entities Hotel, Room, Booking, Guest; list the access patterns and the PK/SK (+GSI) design that serves each.

### 10. SQL Examples

```
-- Cassandra CQL (query-first tables)
CREATE TABLE orders_by_user (
  user_id bigint, created_at timestamp, order_id bigint,
  total decimal, status text,
  PRIMARY KEY ((user_id), created_at, order_id)
) WITH CLUSTERING ORDER BY (created_at DESC);
SELECT * FROM orders_by_user WHERE user_id = 42 LIMIT 10;   -- the one true query

-- MongoDB
db.orders.insertOne({ user_id: 42, status: "paid",
  items: [{ sku: "A1", name: "Widget", price_cents: 4999, qty: 2 }] })
db.orders.find({ user_id: 42 }).sort({ created_at: -1 }).limit(10)
db.orders.createIndex({ user_id: 1, created_at: -1 })
```

### 11. Optimization Techniques

- Bucketing pattern for unbounded lists (comments page-1 doc, page-2 doc…).
- Computed/duplicated aggregates on the parent (counts, latest) maintained on write.
- Sparse GSIs (DynamoDB) / partial indexes (Mongo) for rare-state queries.

### 12. Follow-up Questions

- "A new access pattern arrives post-launch — compare the cost in Postgres vs DynamoDB." (add an index vs redesign keys/backfill a GSI — the flexibility tax made concrete)
- "How do you paginate within a Cassandra partition and across partitions?" (clustering-key keyset within; you don't across — design a table for it)

---

## Chapter 8.3 — LSM Trees

### 1. Why Interviewers Ask This

The storage-engine question that explains *why* Cassandra/RocksDB/DynamoDB-class systems write fast. Pairs with B+Tree (Module 4) as the fundamental engines duality; staff-level candidates are expected to compare them fluently.

### 2. Core Concept

**Log-Structured Merge tree**: never update in place — buffer writes in memory, flush sorted files, merge in background.

Write path: append to **WAL/commit log** (durability) → insert into **memtable** (sorted in-memory structure) → when full, flush as an immutable sorted file (**SSTable**) → background **compaction** merges SSTables (dedup keys, drop tombstones, maintain sorted runs).

Read path: memtable → recent SSTables → older ones; accelerated by **bloom filters** (skip files that definitely lack the key) and per-file index/summary blocks. Deletes = write a **tombstone** (the delete is data until compaction purges it).

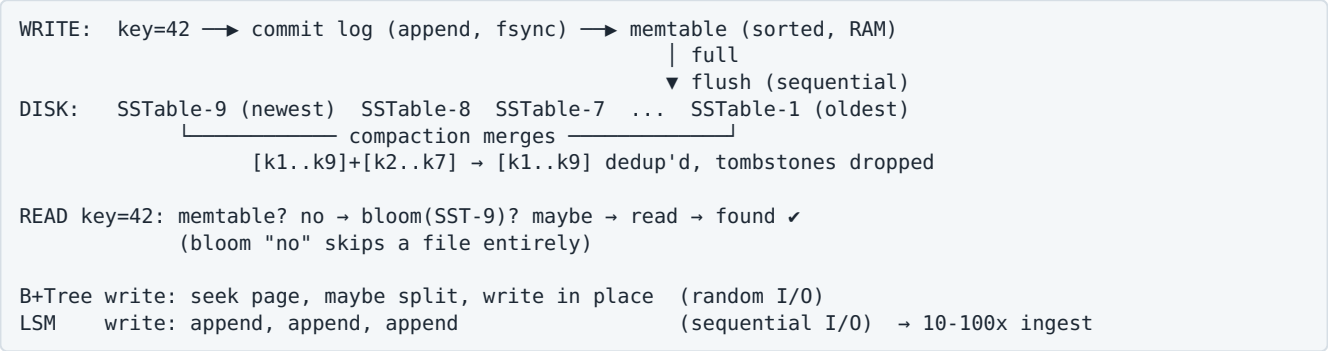
Trade profile vs B+Tree: - Writes: sequential-only I/O, no page splits → very high throughput ✔ - Reads: potentially multi-file (read amplification) ✖ (bloom filters mitigate point reads; range reads touch every overlapping file) - Space: duplicates+tombstones until compaction (space amplification) ✖ - Compaction: background I/O/CPU tax that *must* keep up (write stalls when it doesn't) ✖

Compaction strategies: **size-tiered** (merge similar-sized files; write-cheap, read/space- worse) vs **leveled** (RocksDB/LevelDB: non-overlapping levels; read/space-better, more write amplification) — choose by read/write ratio.

## 3. Internal Working

Everything sequential: commit-log append, SSTable flush, compaction streams — the disk never seeks on the write path (this single sentence is the "why fast" answer). SSTables immutable → no locks for readers, trivial caching, cheap replication (ship files). Read amplification math: point read worst case = memtable + one bloom-filter check per SSTable + 1–2 actual file reads; range scan = k-way merge across all overlapping files. Tombstone hazards: reading a key with millions of tombstones (queue-like workloads) scans them all — the famous "queues on Cassandra" anti-pattern.

## 4. Visualization (ASCII)




## 5. Real Production Example

RocksDB is the LSM everyone actually runs: it backs Kafka Streams state, CockroachDB (originally), TiKV, MyRocks (Meta's MySQL storage engine — Meta moved user DBs to MyRocks largely for **space amplification wins on SSDs**, ~50% storage savings), and every Cassandra-class store. Interview staple: "your ingest is 1M events/sec — B+Tree or LSM and why?" → LSM, sequential-write argument, then discuss the read/compaction bill.

## 6. Common Interview Questions

- "Explain LSM trees end to end." (write path, read path, compaction, tombstones)
- "LSM vs B+Tree — when each?" (write-heavy/append vs read-heavy/point-update)
- "What are bloom filters for here?" (skip SSTables on point reads; no false negatives)
- "Why are deletes expensive in LSM systems?" (tombstones live until compaction; range-delete pileups)
- "What are write/read/space amplification?" (define all three; compaction strategy trade)

## 7. Common Mistakes

- "LSM is faster" unqualified — faster *writes*; reads pay.
- Forgetting compaction is a first-class operational concern (falling behind = stalls, bloat).
- Queue workloads (write-then-delete) on LSM stores — tombstone hell.
- Missing that Postgres/InnoDB *also* sequence writes via WAL — the difference is the *data structure* maintenance (in-place B+Tree pages vs immutable merged files).

## 8. Best Practices

- Match compaction strategy to workload (size-tiered for ingest-heavy, leveled for read-heavy).
- Monitor compaction debt (pending compactions, SSTables-per-read) like you monitor vacuum in PG.
- Design keys so hot ranges don't collide with compaction (time-series: time-bucketed keys, TTL-based expiry drops whole SSTables free).

### 9. Coding Questions

1. Trace: write k=1..5, flush; write k=3(update),6, flush; delete k=2, flush. Show the three SSTables' contents, the answer to `read k=2` and `read k=3`, then the post-compaction file.
2. Estimate write amplification for leveled compaction with 10x level fanout and 5 levels (each byte rewritten ~once per level → ~5–10x) and compare with B+Tree page-write cost for random small updates.

### 10. SQL Examples

```
-- Cassandra: observe the LSM surface
nodetool tablestats keyspace.orders_by_user   -- SSTable count, bloom fp rate
nodetool compactionstats                      -- compaction backlog (debt!)

-- TTL as free deletion (expired data drops with whole SSTables)
INSERT INTO events (...) VALUES (...) USING TTL 2592000;  -- 30 days
```

### 11. Optimization Techniques

- Rate-limit/burst-schedule compaction off-peak; provision I/O headroom for it.
- Larger memtables → fewer, bigger SSTables → less read amplification (RAM trade).
- Partition data so time-expired data dies by file drop (TTL + time-windowed compaction strategy), never by tombstone flood.

### 12. Follow-up Questions

- "How does a bloom filter work and what's its guarantee?" (bit array + k hashes; false positives possible, false negatives never — sizing math if they push)
- "Why did Meta build MyRocks instead of staying on InnoDB?" (space+write amplification on SSDs; compression friendliness of immutable sorted files)
- "Where does Postgres feel LSM pressure?" (it doesn't have one natively; heavy-ingest use cases reach for timescale/partitioning or external stores — discuss honestly)

---

## Chapter 8.4 — MongoDB

### 1. Why Interviewers Ask This

The default document DB; interviewers test whether you know what it's actually good at (rich documents, developer velocity, built-in sharding) and its sharp edges (transactions, joins, the modeling discipline it silently demands).

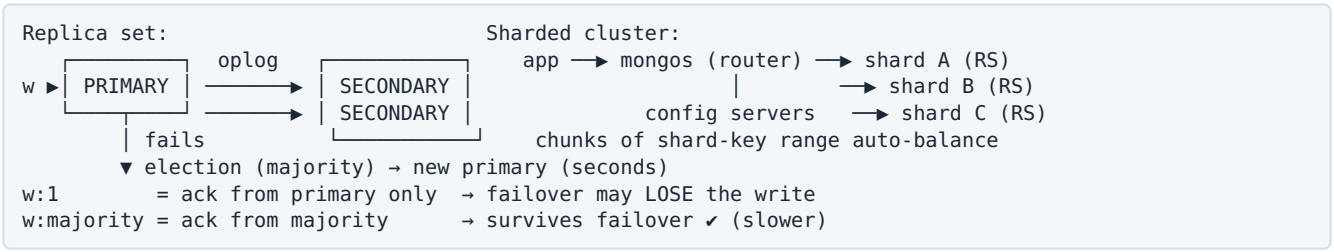
### 2. Core Concept

- Documents (BSON) in collections; `_id` primary key; secondary indexes (B+Tree, on any field, compound, partial, TTL, text, geo); rich query + aggregation pipeline.
- **Replica sets**: 1 primary + N secondaries, automatic elections (Raft-like). Write concern `w:1` (fast, loss window) → `w:"majority"` (durable); read concern/preference tune staleness (`readPreference: secondary` = replica-lag reads).
- **Sharding** built in: shard key (hashed or ranged), mongos routers, config servers; chunks auto-split/balance.
- **Transactions**: single-document atomicity always (the design center); multi-document ACID since 4.0 (replica set) / 4.2 (sharded) — works, but costs latency and has limits; heavy use signals wrong data model.
- Storage: WiredTiger — B+Tree, document-level concurrency (MVCC-ish), compression.

### 3. Internal Working

Elections: majority voting, ~seconds of write unavailability on primary failure (retryable writes paper over it). `w:majority` + `readConcern:majority` gives read-your-majority-writes; weaker settings can *lose acknowledged writes on failover* (the historical Jepsen findings — know they existed and that defaults improved: MongoDB 5+ defaults `w:majority`). Aggregation pipeline executes stages server-side ($match/$group/$lookup); $lookup is a left outer join — single-shard-friendly, cross-shard expensive.

## 4. Visualization (ASCII)




## 5. Real Production Example

Typical fit: product catalogs (deep varied attributes per category), CMS/content, user profiles, IoT device state — read-mostly rich documents where embedding matches the UI. The cautionary tale (interview-famous): startups modeling relational domains (orders ↔ inventory ↔ payments) in Mongo, discovering they need multi-document transactions everywhere, migrating to Postgres — the lesson is model-fit, not engine quality.

## 6. Common Interview Questions

- "When MongoDB over Postgres?" (document-shaped, embed-friendly data; horizontal write scale with built-in sharding; velocity on evolving schemas — vs jsonb rebuttal ready)
- "Explain write concern / read concern and the durability spectrum."
- "How does a Mongo failover work and what happens to in-flight writes?"
- "Is MongoDB ACID?" (nuanced: per-document yes; multi-doc since 4.0 with costs)
- "How do you choose a shard key?" (same rules as Module 7.3: cardinality, uniformity, monotonic-key hotspot → hashed)

## 7. Common Mistakes

- `w:1` for data you can't lose.
- Monotonic shard key (ObjectId/timestamp, ranged) → all inserts hit one chunk/shard.
- Unbounded embedded arrays; documents rewritten wholesale per append.
- Using $lookup as a general join at scale.
- Ignoring that indexes here cost like anywhere (write amplification, RAM).

## 8. Best Practices

- `w:majority` + retryable writes for anything durable; explicit staleness budget for secondary reads.
- Schema-on-write validation (JSON Schema in collMod) — "schemaless" ≠ ungoverned.
- Shard keys: hashed on high-cardinality id, or compound (tenant, id); test with realistic distribution before committing (shard key is ~forever; 5.0+ resharding exists but is heavy).

## 9. Coding Questions

1. Design the order document + indexes for: orders by user (recent first), by status (rare states), item-sku lookup — write `createIndex` calls (compound, partial).
2. Write an aggregation: daily revenue by country last 30 days ($match → $group on $dateTrunc → $sort), and state where it runs in a sharded cluster.

### 10. SQL Examples

```
// Durability spectrum
db.orders.insertOne(doc, { writeConcern: { w: "majority" } })

// Query + covering-ish index
db.orders.createIndex({ user_id: 1, created_at: -1 })
db.orders.find({ user_id: 42 }).sort({ created_at: -1 }).limit(10)

// Partial index for rare state
db.jobs.createIndex({ created_at: 1 }, { partialFilterExpression: { status: "pending" } })

// Multi-document transaction (use sparingly)
session.withTransaction(async () => {
  await orders.insertOne(order, { session });
  await inventory.updateOne({ sku, qty: { $gte: n } }, { $inc: { qty: -n } }, { session });
});
```

### 11. Optimization Techniques

- Keep the working set (indexes + hot docs) in RAM — WiredTiger cache sizing is the #1 knob.
- Project only needed fields (documents are fat; network+cache savings).
- Pre-aggregate with $merge into summary collections for dashboards.

### 12. Follow-up Questions

- "Postgres jsonb vs MongoDB — give the honest comparison." (PG: transactions/joins/one system, GIN indexes; Mongo: built-in sharding, richer doc-native ops, replica-set ergonomics)
- "What did Jepsen historically find and what changed?" (stale/lost writes under weak concerns; defaults hardened — shows you track correctness, not marketing)

---

## Chapter 8.5 — Redis (as a Data Store)

### 1. Why Interviewers Ask This

Beyond caching (Module 7.5), interviews use Redis to test data-structure-driven design: leaderboards, rate limiters, queues, presence — "design X" where X is a Redis structure away.

### 2. Core Concept

Data structures and their interview jobs: - **String** + INCR: counters, locks (SET NX PX), idempotency flags. - **Hash**: object fields (HSET user:42 name ...), partial updates without whole-value rewrite. - **List**: simple queues (LPUSH/BRPOP), capped feeds (LTRIM). - **Set**: uniqueness, tags, membership (SISMEMBER), intersections (mutual friends). - **Sorted Set (ZSET)**: leaderboards (ZADD/ZRANGE/ZRANK), sliding-window rate limits, delayed queues (score = run_at), top-K. - **Streams**: append-only log with consumer groups (Kafka-lite for modest scale). - HyperLogLog (approx distinct), bitmaps (daily-active flags), geo (nearby drivers), pub/sub (fire-and-forget fan-out).

Persistence reality: **RDB** snapshots (lose minutes) / **AOF** everysec (lose ~1s) / replication is async → Redis can be *a* database only for loss-tolerant, ephemeral-by-nature data (sessions, presence, rate limits, matchmaking). Single-threaded per instance → atomic commands + Lua/MULTI for compound atomicity; Cluster = 16384 slots (Module 7.5).

### 3. Internal Working

Event loop processes one command at a time → every command is a critical section (atomic INCR without locks; also why O(N) commands on big keys stall everyone). ZSET = skiplist + hash (O(log N) rank ops). Keyspace expiry: lazy + active sampling. Eviction under maxmemory by policy (LRU/LFU approximations). Fork-based RDB/AOF-rewrite uses copy-on-write — memory spikes on write-heavy instances (the classic "why did Redis OOM during BGSAVE" question).

## 4. Visualization (ASCII)

```
Leaderboard (ZSET):                    Sliding-window rate limit (ZSET):
ZADD lb 9812 user:42                   key = rl:user:42, member = req-id, score = now-ms
ZREVRANGE lb 0 9 WITHSCORES → top 10   ZADD rl now req; ZREMRANGEBYSCORE rl 0 now-60000
ZREVRANK lb user:42 → my rank          ZCARD rl → count in window → allow if < limit
                                       (wrap in Lua for atomicity)
Delayed job queue (ZSET):
ZADD delayed <run_at_ms> job:77
poller: ZRANGEBYSCORE delayed 0 <now> LIMIT 1 → ZREM (atomic via Lua) → run
```

## 5. Real Production Example

Uber-style driver presence/location: `GEOADD drivers lon lat driver:9`, nearby search `GEOSEARCH` — ephemeral by nature (refreshed every few seconds), perfect Redis fit; losing it on crash costs seconds of staleness, nothing more. Gaming leaderboards at scale are ZSETs verbatim. Session stores everywhere. Interviewers expect you to *reject* Redis for the ledger and *choose* it for these.

## 6. Common Interview Questions

- "Design a rate limiter." (fixed window INCR+EXPIRE → sliding-window ZSET → token bucket in Lua; discuss race-freedom and cost per request)
- "Design a leaderboard for 100M players." (ZSET + sharding by league/bucket; top-K global via merged bucket tops)
- "Can Redis be the primary database?" (loss-tolerance argument; AOF/replication limits)
- "How do distributed locks on Redis fail?" (expiry vs GC pauses, failover duplication — Redlock controversy; use fencing tokens / prefer DB advisory locks for correctness-critical)
- "Why is Redis single-threaded and why is that OK?" (RAM-bound ops; event loop; I/O threads in 6+ for network)

## 7. Common Mistakes

- Precious data in Redis-as-only-copy.
- O(N) commands on huge structures in the hot path (SMEMBERS a 10M set, KEYS, LRANGE 0 -1).
- Rate limiter with separate GET/INCR (racy) instead of atomic INCR or Lua.
- Redis pub/sub where delivery guarantees matter (fire-and-forget; use Streams/Kafka).
- One giant instance for cache + queues + locks (eviction policy conflicts — Module 7.5).

## 8. Best Practices

- Ephemeral-by-design data only, or accept AOF-everysec loss window explicitly.
- Lua (or MULTI/WATCH) for compound invariants; keep scripts O(1)-ish.
- Cap structure sizes (LTRIM, ZREMRANGEBYRANK); TTL everything; monitor big keys (`redis-cli --bigkeys`).
- Shard by hash tag when multi-key atomicity is needed in Cluster.

## 9. Coding Questions

1. Implement token-bucket rate limiting as an atomic Lua script (refill by elapsed time, consume, return allowed/deny + retry-after).
2. Build "trending hashtags last hour": per-minute ZSET buckets + ZUNIONSTORE with decay weights; state the memory bound and the top-K query.

### 10. SQL Examples

```
# Idempotency guard
SET idem:req-abc "1" NX EX 86400        → nil = duplicate request

# Leaderboard ops
ZADD lb:global 9812 user:42
ZREVRANGE lb:global 0 9 WITHSCORES
ZINCRBY lb:global 50 user:42

# Distributed lock (with the caveats!)
SET lock:invoice:77 <token> NX PX 10000
# release: Lua compare-token-then-DEL (never blind DEL)

# Streams consumer group (durable-ish queue)
XADD events * type checkout user 42
XREADGROUP GROUP g1 c1 COUNT 10 BLOCK 5000 STREAMS events >
XACK events g1 <id>
```

### 11. Optimization Techniques

- Pipelining/MGET to kill round trips (cache-side N+1).
- Hashes for small objects (ziplist/listpack encoding = memory-dense).
- Split monster keys; bucket time-series structures per window so old buckets expire whole.

### 12. Follow-up Questions

- "Your Lua script takes 80ms — what happens cluster-wide?" (everything on that shard queues; rewrite or move to app-side with CAS)
- "Redis Streams vs Kafka — when is each right?" (scale, retention, ecosystem, exactly-once tooling vs operational simplicity)
- "How do you migrate a hot Redis to Cluster with no downtime?" (dual-write or proxy with slot migration; hash-tag audit first)

---

## Chapter 8.6 — Cassandra

### 1. Why Interviewers Ask This

The reference masterless AP system: consistent hashing + quorums + LSM in one package. Even if you never run it, its concepts are the vocabulary of half of system design.

### 2. Core Concept

- **Masterless ring**: every node equal; data placed by consistent hashing of the partition key (vnodes); **RF** copies on distinct nodes. Any node coordinates any request.
- **Tunable consistency per query**: ONE / QUORUM / LOCAL_QUORUM / ALL for both reads and writes; `R + W > RF` ⇒ strong reads (Module 1.4). Multi-DC: LOCAL_QUORUM keeps latency in-region.
- **Write path**: LSM (commit log → memtable → SSTables) — writes are cheap and always accepted (even for updates: last-write-wins by timestamp, no read-before-write).
- **Repair machinery** (AP hygiene): hinted handoff (replay to recovered nodes), read repair (fix divergence seen during reads), anti-entropy repair (Merkle trees, scheduled).
- Query model: partition key equality (+ clustering ranges) ONLY. Secondary indexes exist but are scatter-gather traps; materialized views have a troubled history — duplicate tables by hand.
- **Counters** are special-cased; **lightweight transactions** (Paxos-based IF NOT EXISTS) exist but cost 4x round trips — avoid in hot paths.

## 3. Internal Working

Coordinator hashes the key → sends to RF replicas → acks per consistency level → returns. Conflicts resolved cell-wise by timestamp (LWW — clock skew can silently drop writes; the famous caveat). Reads at QUORUM compare digests across replicas, resolve newest, optionally repair. Tombstones (8.3) + LWW make delete-heavy and read-modify-write workloads hostile. Gossip protocol spreads membership/health; no config server, no election — availability through symmetry.

## 4. Visualization (ASCII)

```
        ring (RF=3)
      N1 ─── N2                write user:42 (CL=QUORUM):
     /          \              coordinator ──▶ N2 ✔, N3 ✔ (2/3 acks) ──▶ OK
    N6           N3                          └─▶ N4 down → hint stored on coordinator
     \          /              read (CL=QUORUM): ask N2,N3 → newest timestamp wins
      N5 ─── N4                     divergence found → read repair N3

R+W>RF:  W=QUORUM(2) + R=QUORUM(2) > RF(3) → overlap guaranteed → strong read ✔
W=ONE + R=ONE → fastest, may read stale ✖ (eventual)
```

## 5. Real Production Example

Netflix: ~everything user-state at planetary scale (viewing history, bookmarks) — multi-region active-active with LOCAL_QUORUM; Apple runs some of the largest Cassandra fleets (iCloud metadata); Discord's message store (then ScyllaDB — same model, C++ engine). Common shape: write-heavy, key-addressed, region-resilient, staleness-tolerant.

## 6. Common Interview Questions

- "How does a Cassandra write work end-to-end?" (coordinator → replicas → CL acks → hints)
- "Explain R+W>RF with numbers."
- "Why are writes so fast?" (LSM + no read-before-write + no master coordination)
- "What's LWW and its failure mode?" (clock skew drops concurrent writes silently)
- "Why is 'queue on Cassandra' an anti-pattern?" (tombstone scans)
- "Design the partition key for [time-series/chat/orders]." (bounded partitions — bucket!)

## 7. Common Mistakes

- Unbounded partitions (all events for one device forever) — bucket by time.
- Treating secondary indexes like RDBMS indexes (they're per-node; queries scatter).
- CL=ALL "for safety" (one dead node = unavailability; that's just worse CP).
- Read-modify-write patterns (racy under LWW; needs LWT and its cost).
- Ignoring repair operations until entropy bites (deleted data resurrecting via unrepaired replicas past gc_grace).

## 8. Best Practices

- Model one-table-per-query with bounded partitions (<100MB); duplicate writes to N tables in a logged batch when they share the partition key (else app-side).
- LOCAL_QUORUM writes+reads as the sane default; drop to ONE only for telemetry-grade data.
- Schedule repairs within gc_grace_seconds; monitor tombstones-per-read and SSTables-per-read.
- NTP discipline (LWW!) — or client-side timestamps from one source.

## 9. Coding Questions

1. Chat messages: design `messages_by_channel ((channel_id, bucket), message_id DESC)` with time-bucketing; write the CQL and the "load older messages" pagination query.
2. Given RF=3 across 2 DCs (3+3), compare CL choices (QUORUM=4 cross-DC vs LOCAL_QUORUM=2) for latency, consistency, and DC-failure behavior.

### 10. SQL Examples

```
CREATE KEYSPACE app WITH replication =
  {'class':'NetworkTopologyStrategy','dc1':3,'dc2':3};

CREATE TABLE messages_by_channel (
  channel_id bigint, bucket int,            -- bucket = day number: bounds the partition
  message_id timeuuid, author_id bigint, body text,
  PRIMARY KEY ((channel_id, bucket), message_id)
) WITH CLUSTERING ORDER BY (message_id DESC);

-- newest page
SELECT * FROM messages_by_channel
WHERE channel_id=42 AND bucket=20260702 LIMIT 50;

-- consistency is per-query
CONSISTENCY LOCAL_QUORUM;
INSERT INTO messages_by_channel (...) VALUES (...);

-- LWT (sparingly)
INSERT INTO usernames (name, user_id) VALUES ('neo', 42) IF NOT EXISTS;
```

### 11. Optimization Techniques

- Time-window compaction (TWCS) for time-series — expired buckets drop whole SSTables.
- Keep clustering rows narrow; move blobs elsewhere (object storage + reference).
- Token-aware client drivers (skip the coordinator hop).

### 12. Follow-up Questions

- "Cassandra vs DynamoDB — you're choosing for a new team." (ops burden vs managed; cost model; multi-cloud; feature deltas — next chapter)
- "How does ScyllaDB claim 10x?" (same model, shard-per-core C++ engine, no JVM/GC)
- "Explain a scenario where a committed write disappears." (LWW clock skew, or CL=ONE write to a node that dies before handoff/repair)

---

## Chapter 8.7 — DynamoDB

### 1. Why Interviewers Ask This

Amazon interviews assume it; everyone else uses it as "the managed NoSQL baseline." Tests: key design under rigid constraints, capacity/hot-partition thinking, and the single-table mindset.

### 2. Core Concept

- Fully managed KV/document store: tables → items (≤400KB) → attributes. **PK = partition key (+ optional sort key)**. Data distributed by hash(partition key); items sharing a partition key are sorted by sort key (an "item collection").
- Query patterns: `GetItem` (point), `Query` (one partition key + sort-key conditions — cheap), `Scan` (full table — forbidden in hot paths).
- **GSI** (global secondary index): alternate PK/SK — an automatically-maintained, **eventually consistent** copy (own capacity, async from base table). LSI: same partition key, alternate sort key, created at table birth.
- Consistency: eventually consistent reads by default; **strongly consistent reads** on the base table only (not GSIs); **transactions** (`TransactWriteItems`, ≤100 items, 2x cost).
- Capacity: on-demand or provisioned RCU/WCU; **adaptive capacity** helps but a single hot key still caps at ~1000 WCU / 3000 RCU per partition — hot-key design still matters.
- **DynamoDB Streams**: CDC feed → Lambda (projections, invalidation, fan-out).

## 3. Internal Working

Descended from the Dynamo paper but *not* the same: DynamoDB is multi-tenant, replicated across 3 AZs with a leader-ish per-partition consensus (strong reads = read from leader), auto-split partitions by size/throughput. Single-table design exists because (a) Query only works within a partition key, (b) you pre-join by co-locating heterogeneous items in one item collection, (c) fewer tables = shared capacity + streams. Global Tables = multi-region active-active with LWW conflict resolution.

## 4. Visualization (ASCII)

```
Single-table design ("pre-joined" item collection):
PK            SK                   attributes
USER#42       PROFILE              {name, email}
USER#42       ORDER#2026-07-01#9   {total, status}     ── Query PK=USER#42:
USER#42       ORDER#2026-06-28#8   {total, status}        profile + orders,
ORDER#9       ITEM#1               {sku, qty}             ONE round trip ✔
ORDER#9       ITEM#2               {sku, qty}

GSI: PK=STATUS#pending, SK=created_at   → "all pending orders" (eventually consistent)

hot partition: PK=DATE#2026-07-02 (all today's writes → one key) ✖
fix: write sharding  PK=DATE#2026-07-02#<rand 0-9>  → read = query 10 shards, merge
```

## 5. Real Production Example

Amazon retail runs cart/order-workflow tiers on DynamoDB (the 2019+ "we moved off Oracle" story); Lyft, Snap, and half of serverless-land use it as default OLTP. The canonical incident: a time-keyed partition (daily leaderboard, `PK=today`) throttles at peak despite low table-wide usage — per-partition limits + hot key; fixed by write sharding. Interviewers love this exact scenario.

## 6. Common Interview Questions

- "Design a DynamoDB table for [entity graph] — list access patterns first." (the ritual)
- "GSI vs LSI? What consistency do GSIs give?" (eventual only; capacity separate; GSI throttling back-pressures base-table writes!)
- "How do you handle a hot partition?" (key sharding, caching/DAX, redesign key)
- "Strongly consistent read — when available, when not?" (base table yes, GSI no, 2x RCU)
- "Model many-to-many." (adjacency list: both directions as items; or GSI-swapped keys)

## 7. Common Mistakes

- Designing entities-first, discovering an unsupported access pattern → Scan (game over).
- Low-cardinality or time-monotonic partition keys.
- Treating GSIs as free indexes (cost, lag, throttle coupling).
- Ignoring the 400KB item limit (embed → bucket/reference, same as Mongo).
- Missing idempotency on retried writes (SDK retries + non-idempotent updates).

## 8. Best Practices

- Access-pattern table first; single-table design where patterns are stable; generic key names (PK/SK) + entity-type attribute.
- Version attributes + conditional writes for OCC (Module 6.5 mapped here).
- Streams → Lambda for projections instead of dual-writing.
- TTL attribute for expiry; on-demand mode until traffic is predictable.

## 9. Coding Questions

1. Design the full key schema for a URL shortener with per-user link lists, click counts, and "top links today" (PK/SK, one GSI, sharded counter for hot links).

2. Write the OCC update: `UpdateItem` with `ConditionExpression: version = :v`, `UpdateExpression: SET ... , version = version + 1` and the retry policy on `ConditionalCheckFailedException`.

### 10. SQL Examples

```
// Query an item collection (profile + orders)
Query: KeyConditionExpression: "PK = :u AND begins_with(SK, :o)"
       :u = "USER#42", :o = "ORDER#"   ScanIndexForward: false, Limit: 10

// Conditional (optimistic) write
UpdateItem:
  Key: {PK:"DOC#7", SK:"META"}
  UpdateExpression: "SET body=:b, version = version + :one"
  ConditionExpression: "version = :expected"

// Transaction (bounded, 2x cost)
TransactWriteItems:
  - Put    order   (ConditionExpression: attribute_not_exists(PK))   // idempotency
  - Update inventory (ConditionExpression: qty >= :n)
```

### 11. Optimization Techniques

- DAX (managed read-through cache) for read-hot keys — mind its eventual consistency.
- Sparse GSIs (attribute present only on rare items) = cheap partial indexes.
- BatchGetItem/BatchWriteItem to collapse round trips (N+1 again).
- Compress large attributes; split hot/cold attributes across items sharing a PK.

### 12. Follow-up Questions

- "New access pattern post-launch: 'orders by SKU'. Walk the options." (new GSI + backfill implications; vs stream-projected table; vs the honest 'this is the flexibility tax')
- "DynamoDB vs Cassandra vs Postgres for a startup's core OLTP — decide with reasons." (managed+serverless vs ops control vs relational flexibility; lock-in; cost curves)
- "How do Global Tables resolve concurrent multi-region writes?" (LWW — design so conflicting concurrent writes don't happen or don't matter)

---

## Chapter 8.8 — Replication, Sharding & CAP Across NoSQL (Comparison)

### 1. Why Interviewers Ask This

The synthesis question: "compare these systems" — testing whether the mechanisms (leader vs masterless, B+Tree vs LSM, sync vs quorum) transfer across brand names.

## 2. Core Concept — The Comparison Table

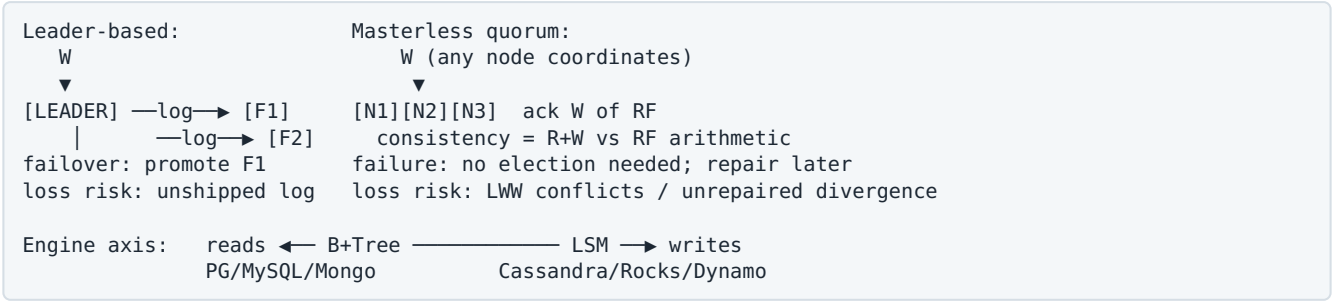
| | PostgreSQL | MySQL | MongoDB | Redis | Cassandra | DynamoDB |
|---|---|---|---|---|---|---|
| Model | Relational | Relational | Document | KV + structures | Wide-column | KV/Document |
| Storage engine | Heap + B+Tree, MVCC | InnoDB B+Tree (clustered), MVCC/ undo | WiredTiger B+Tree | RAM (+AOF/ RDB) | LSM | LSM-class (managed) |
| Replication | WAL streaming, leader | binlog, leader (+group repl.) | Replica set, elections | async, leader | masterless, RF quorums | 3-AZ managed, per-partition leader |
| Sharding | manual / Citus | manual / Vitess | built-in (mongos) | Cluster (16384 slots) | native (ring) | native (managed) |
| Consistency default | strong (single node) | strong (single node) | tunable (w/rc) | weak across replicas | tunable per query | eventual; strong opt-in |
| CAP posture (partition) | CP-ish w/ sync repl | CP-ish | CP-leaning (majority) | AP-ish (async) | AP, tunable to CP-per-op | AP-ish; strong reads in-region |
| Transactions | full ACID | full ACID | multi-doc (costly) | MULTI/Lua (single shard) | LWT (Paxos, slow) | TransactItems (≤100) |
| Best at | integrity + ad-hoc | same + read-scale ecosystems | rich documents | latency + structures | write scale, multi-DC | serverless scale, zero ops |

Mechanism clusters to reason with: - **Leader-based replication** (PG/MySQL/Mongo/Redis): simple mental model; failover = election /promotion; lag = staleness; writes bottleneck on the leader. - **Masterless quorum** (Cassandra/ Dynamo-lineage): availability through symmetry; consistency is arithmetic (R+W vs RF); conflicts need resolution (LWW/vector clocks). - **B+Tree engines** read-optimized; **LSM engines** write-optimized (Modules 4/8.3).

## 3. Internal Working

The deep commonality: everything is a **replicated log** — Postgres WAL, MySQL binlog, Mongo oplog, Cassandra commit log + mutation streams, Dynamo streams. Ordering + durability of that log determines consistency; who may append (one leader vs any replica) determines availability and conflict complexity. Say this in a synthesis question and you sound like the interviewer.

## 4. Visualization (ASCII)




## 5. Real Production Example

A realistic FAANG-adjacent stack, one line each: Postgres (orders, money — CP, ACID), Redis (sessions, rate limits — ephemeral), Cassandra/DynamoDB (events, feeds — AP write-scale), Elasticsearch (search projection), Kafka as the spine, CDC keeping projections honest. Interviews reward naming *why each* in one clause.

## 6. Common Interview Questions

- "Compare MongoDB and Cassandra replication." (elections+oplog vs masterless quorums)
- "Which of these lose acknowledged writes, and under what settings?" (Redis async failover; Mongo w:1; Cassandra CL=ONE + node loss; PG async replica promotion)

- "Map each store onto CAP/PACELC."
- "Same data, both engines: why is the LSM store faster to write and slower to read?"

## 7. Common Mistakes

- Comparing brands by adjectives instead of mechanisms.
- Assuming "NoSQL = eventual consistency" (Mongo majority, Dynamo strong reads, Cassandra R+W>RF all disprove it).
- Forgetting the ops axis (masterless repair, compaction, resharding are labor) when comparing self-hosted vs managed.

## 8. Best Practices

- Answer comparison questions on three axes: **replication topology, storage engine, consistency knobs** — everything else is derived.
- Keep one system of record; make every other store a rebuildable projection.

## 9. Coding Questions

1. For each system, state the setting that makes a committed write survive any single node loss: PG (`synchronous_standby_names` quorum), MySQL (semi-sync/group replication), Mongo (`w:majority`), Redis (WAIT — and its limits), Cassandra (CL≥QUORUM), DynamoDB (default).
2. Draft the polyglot data flow for an e-commerce checkout (order in PG → outbox → Kafka → projections in Redis/Elastic/warehouse) with the consistency contract at each hop.

## 10. SQL Examples

```
-- One durability knob per system (know these cold):
PG:        synchronous_standby_names = 'ANY 1 (r1,r2)'
MySQL:     rpl_semi_sync_source_enabled = ON
MongoDB:   { writeConcern: { w: "majority" } }
Redis:     WAIT 1 100        -- best-effort; not a real sync guarantee
Cassandra: CONSISTENCY QUORUM
DynamoDB:  (managed: 3-AZ by default; strong read = ConsistentRead: true)
```

## 11. Optimization Techniques

- Choose the engine axis by workload write/read ratio before choosing the brand.
- Use managed offerings' guarantees (Dynamo 3-AZ, Aurora storage) to delete whole problem classes from your design — and say you're doing it.

## 12. Follow-up Questions

- "Where do NewSQL systems (Spanner, CockroachDB, TiDB) land in this table?" (relational + Raft-per-range + distributed txns — CP with latency cost; the 'have both, pay latency' cell)
- "If you could only keep two stores company-wide, which and why?" (forcing-function question: usually Postgres + one of {Redis, DynamoDB/Cassandra} + defend the losses)

---

# Module 8 — Practice Problems

## Easy (5)

1. Classify by data model and CAP posture: Redis, MongoDB, Cassandra, DynamoDB, PostgreSQL.
2. Cassandra RF=3: for (W=ONE,R=ONE), (W=QUORUM,R=QUORUM), (W=ALL,R=ONE) state consistency and what happens when one replica is down.
3. Embed or reference in MongoDB (one line each): order items; post comments; user → profile-settings; product → category.
4. Why are LSM writes fast? Answer in exactly two sentences using "sequential" and "compaction".
5. Pick the store (one line why): session tokens; hotel bookings; IoT telemetry (1M/s); product catalog with faceted search; global leaderboard.

## Medium (5)

1. Design the Cassandra schema for a Twitter-like home timeline with fan-out-on-write: tables, keys, bucketing, and the celebrity exception.
2. A DynamoDB table throttles at 8% of provisioned capacity. Diagnose (hot partition), show how to confirm (CloudWatch per-key / contributor insights), and fix with write-sharded keys including the read-side merge.
3. Your Mongo replica set lost acknowledged writes during a failover. Reconstruct the exact sequence (w:1 ack → primary crash → election → rollback of un-majority-replicated ops) and the two settings that prevent it.
4. Compare running a queue on: Redis Streams, Cassandra, Postgres (SKIP LOCKED), Kafka — for ordering, delivery guarantees, tombstone/bloat behavior, and ops. Rank for a 10k msg/s payments pipeline.
5. Design DynamoDB single-table keys for a food-delivery app: restaurants, menus, orders by customer, orders by restaurant+status, courier active order. List each access pattern → key.

## Hard (5)

1. Netflix-style viewing history: 200M users, 5B events/day, reads = "continue watching" per user + per-title analytics. Design storage end-to-end (Cassandra keys+bucketing+TTL+TWCS, CDC to warehouse for analytics), including multi-region consistency choices.
2. Prove the LWW data-loss scenario: two Cassandra clients with 200ms clock skew write the same cell; show the timeline where the *earlier* real-world write wins, then design two mitigations (client-side monotonic timestamps from one tier; model as immutable events instead of updates).
3. Migrate a 5TB MongoDB cluster to DynamoDB with <1min write freeze: schema/key mapping decisions, backfill (parallel export respecting item limits), change-stream tailing, dual-read verification, cutover, and rollback plan.
4. Design a rate limiter service for 5M RPS across 3 regions on Redis: local vs global limits, Cluster key design, Lua atomicity, cross-region sync (CRDT-ish counters vs per-region budgets), degraded mode when Redis is down. Justify every consistency sacrifice.
5. Your company runs Postgres + Redis + Cassandra + Elasticsearch, and on-call is drowning. Consolidation review: for each store list the workloads on it, which could move to Postgres at current scale (with the math), which cannot and why — produce the target architecture and migration order.

---

*Next: Module 9 — Database Design*

# MODULE 9 — Database Design

> Schema-design rounds ask you to design real systems in 30–45 minutes. This module gives you the method plus complete reference schemas for the ten designs that actually get asked: users, orders, payments, products, inventory, hotel booking, flight booking, social media, chat, travel booking.

Chapters: 9.1 The Design Method (how to run the round) 9.2 Users & Identity 9.3 E-Commerce: Products, Inventory, Orders, Payments 9.4 Booking Systems: Hotel, Flight, Travel (the concurrency designs) 9.5 Social Media (feeds, follows, likes) 9.6 Chat & Messaging

---

## Chapter 9.1 — The Design Method

### 1. Why Interviewers Ask This

Schema design reveals more senior signal per minute than any other question: modeling judgment, integrity instincts, concurrency awareness, and scale honesty — all at once.

### 2. Core Concept — The 7-Step Script

1. **Clarify entities + access patterns first** ("what are the top 5 queries and the write rates?") — never start drawing tables silently.
2. **Model the write side in ~3NF**: entities, relationships (1:1, 1:N, M:N via junction), surrogate `bigint` PKs, natural keys as unique constraints.
3. **Encode invariants in DDL**: NOT NULL, CHECK, UNIQUE (incl. partial), FK with explicit ON DELETE, EXCLUDE for overlaps. Every constraint is a class of bugs deleted.
4. **Snapshot history** where facts must not drift (prices on orders, addresses on shipments).
5. **Design state machines explicitly**: status enums + allowed transitions (+ audit/event table for money).
6. **Index for the access patterns** (Module 4.7 method), including pagination keys.
7. **Name the scale plan**: what partitions, what caches, what denormalizes, what shards — and what breaks then (Modules 5/7).

Cross-cutting patterns you'll reuse in every design below: **idempotency keys** on any client-retriable write; **soft delete** (`deleted_at`) where audit matters; **ledger (append- only) over mutable balance** for money; **counter caches** for hot aggregates; **outbox** for DB+event atomicity.

### 3–4. Internal Working & Visualization

The artifact interviewers want on the whiteboard:

```
[entities] ──1:N──▶ [child tables]      per table:
   │                                     - PK (bigint identity)
   M:N via junction table                - natural keys → UNIQUE
   │                                     - FKs + ON DELETE policy
[state machines]: status + transitions   - status CHECK / enum
[money]: append-only ledger, never UPDATE amounts
[hot reads]: counter caches / denormalized read models (rebuildable)
```

### 5–12. (Method-level notes)

- **Interview questions**: "Why bigint surrogate PK?", "Where would you denormalize?", "How does this schema break at 100x?" — always coming; pre-plan answers per design.
- **Mistakes**: diving into DDL before access patterns; FLOAT for money (integer cents/numeric); EAV "flexible" schemas when jsonb suffices; storing mutable snapshots by joining live tables.
- **Best practice**: narrate trade-offs while drawing; you're graded on reasoning, not recall.
- **Follow-ups**: every design below ends with its own scale escalation.

---

# Chapter 9.2 — Users & Identity

## 1. Why Interviewers Ask This

Every system has users; the design has hidden depth: credentials vs profile separation, multiple auth providers, sessions, soft deletion vs GDPR erasure, email uniqueness semantics.

## 2. Core Concept

Split by lifecycle and sensitivity: `users` (identity core) / `user_auth_providers` (N ways to log in) / `user_profiles` (mutable, fat) / `sessions` (ephemeral, Redis-able). Never store plaintext passwords (bcrypt/argon2 hash only); email uniqueness must be case-insensitive and survive soft deletion.

## 3. SQL — Reference Schema

```
CREATE TABLE users (
  id            bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  public_id     uuid NOT NULL DEFAULT gen_random_uuid() UNIQUE,   -- external identifier
  email         citext NOT NULL,
  email_verified_at timestamptz,
  password_hash text,                          -- NULL when social-login only
  status        text NOT NULL DEFAULT 'active'
                CHECK (status IN ('active','suspended','deleted')),
  created_at    timestamptz NOT NULL DEFAULT now(),
  deleted_at    timestamptz
);
-- one LIVE account per email; freed on delete, kept for audit
CREATE UNIQUE INDEX users_email_live ON users (email) WHERE deleted_at IS NULL;

CREATE TABLE user_auth_providers (
  user_id     bigint NOT NULL REFERENCES users(id),
  provider    text  NOT NULL CHECK (provider IN ('google','github','apple')),
  provider_uid text NOT NULL,
  PRIMARY KEY (user_id, provider),
  UNIQUE (provider, provider_uid)              -- an external identity maps to ONE user
);

CREATE TABLE user_profiles (
  user_id     bigint PRIMARY KEY REFERENCES users(id),
  display_name text NOT NULL,
  avatar_url  text,
  bio         text,
  prefs       jsonb NOT NULL DEFAULT '{}'      -- genuinely flexible fragment
);

CREATE TABLE sessions (
  token_hash  bytea PRIMARY KEY,               -- store hash, never the token
  user_id     bigint NOT NULL REFERENCES users(id),
  created_at  timestamptz NOT NULL DEFAULT now(),
  expires_at  timestamptz NOT NULL,
  ip          inet, user_agent text
);
CREATE INDEX ON sessions (user_id);
CREATE INDEX ON sessions (expires_at);          -- reaper
```

## 4. Visualization

```
users 1──1 user_profiles          login flows:
  │ 1──N user_auth_providers      email+pw → users.password_hash
  │ 1──N sessions                 google   → auth_providers(provider,uid) → user
soft delete: deleted_at set, email freed via partial unique, PII scrubbed async (GDPR)
```

## 5–12. Interview Notes

- **Production example**: GDPR erasure = keep the `users` row (FK integrity for orders!) but null/scramble PII and set deleted; orders keep pointing at a tombstoned user.

- **Questions**: "same email re-signup after deletion?" (partial unique); "merge two accounts?" (auth_providers repoint + data migration job — hard, say it's hard); "sessions in PG or Redis?" (Redis for scale/TTL, PG acceptable early; hybrid: refresh tokens in PG, access in Redis).
- **Mistakes**: unique on raw `email` (case), profile columns bloating the identity row, deleting user rows (orphaned FKs everywhere).
- **Scale**: profiles cacheable; sessions → Redis; users table itself rarely the bottleneck — auth QPS is (cache the session check).

---

## Chapter 9.3 — E-Commerce: Products, Inventory, Orders, Payments

### 1. Why Interviewers Ask This

The most-asked design; it packs catalog modeling (variants), concurrency (inventory), state machines (orders), and financial integrity (payments/ledger) into one prompt.

### 2. Core Concept

- **Products** = product (marketing shell) + **SKU/variant** (the sellable, stockable unit) + flexible attributes (jsonb) + categories (M:N or tree).
- **Inventory** belongs to SKU × warehouse; overselling prevented by *atomic guarded decrements*, not read-check-write. Reservation pattern for checkout holds.
- **Orders** snapshot everything (name, unit price, tax) at purchase; status is a state machine; totals stored (not derived at read time) and verifiable.
- **Payments** are append-only **attempts/transactions**; never UPDATE an amount; refunds are new rows; idempotency keys on every external call; a `ledger` if you must track balances.

### 3. SQL — Reference Schema

```
CREATE TABLE products (
  id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name text NOT NULL, brand_id bigint REFERENCES brands(id),
  description text, attrs jsonb NOT NULL DEFAULT '{}',
  status text NOT NULL DEFAULT 'draft' CHECK (status IN ('draft','live','retired')),
  created_at timestamptz NOT NULL DEFAULT now()
);
CREATE TABLE skus (
  id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  product_id bigint NOT NULL REFERENCES products(id),
  sku_code text NOT NULL UNIQUE,
  variant jsonb NOT NULL DEFAULT '{}',          -- {"size":"L","color":"red"}
  price_cents int NOT NULL CHECK (price_cents >= 0),
  currency char(3) NOT NULL DEFAULT 'USD'
);
CREATE INDEX ON skus (product_id);

CREATE TABLE inventory (
  sku_id bigint NOT NULL REFERENCES skus(id),
  warehouse_id bigint NOT NULL REFERENCES warehouses(id),
  on_hand  int NOT NULL DEFAULT 0 CHECK (on_hand >= 0),
  reserved int NOT NULL DEFAULT 0 CHECK (reserved >= 0 AND reserved <= on_hand),
  PRIMARY KEY (sku_id, warehouse_id)
);

CREATE TABLE orders (
  id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  order_number text NOT NULL UNIQUE,            -- human/external id
  user_id bigint NOT NULL REFERENCES users(id),
  status text NOT NULL DEFAULT 'pending' CHECK (status IN
    ('pending','paid','fulfilled','shipped','delivered','cancelled','refunded')),
  subtotal_cents int NOT NULL, tax_cents int NOT NULL, total_cents int NOT NULL,
  shipping_address jsonb NOT NULL,              -- snapshot!
  created_at timestamptz NOT NULL DEFAULT now(), updated_at timestamptz
);
CREATE INDEX ON orders (user_id, created_at DESC);
CREATE INDEX ON orders (created_at) WHERE status = 'pending';  -- ops: stuck orders

CREATE TABLE order_items (
  order_id bigint NOT NULL REFERENCES orders(id),
  sku_id bigint NOT NULL REFERENCES skus(id),
  product_name text NOT NULL,                   -- snapshot
  unit_price_cents int NOT NULL,                -- snapshot
  quantity int NOT NULL CHECK (quantity > 0),
  PRIMARY KEY (order_id, sku_id)
);

CREATE TABLE payments (
  id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  order_id bigint NOT NULL REFERENCES orders(id),
  idempotency_key text NOT NULL UNIQUE,
  kind text NOT NULL CHECK (kind IN ('charge','refund')),
  amount_cents int NOT NULL CHECK (amount_cents > 0),
  status text NOT NULL DEFAULT 'pending'
         CHECK (status IN ('pending','succeeded','failed')),
  provider text NOT NULL, provider_ref text,    -- e.g. Stripe charge id
  created_at timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX ON payments (order_id);
```

Checkout core (concurrency-safe):

```
BEGIN;
-- reserve stock atomically; rowcount 0 = out of stock
UPDATE inventory SET reserved = reserved + :qty
WHERE sku_id = :sku AND warehouse_id = :wh
  AND on_hand - reserved >= :qty;
INSERT INTO orders (...) VALUES (...);
INSERT INTO order_items (...) VALUES (...);
INSERT INTO outbox (topic, payload) VALUES ('order.created', ...);  -- events, atomically
COMMIT;
-- then: charge card OUTSIDE the txn (idempotency_key!), then finalize:
--   success: on_hand -= qty, reserved -= qty, order → 'paid'
--   failure/timeout: reserved -= qty, order → 'cancelled' (reaper for expired holds)
```

### 4. Visualization

```
products 1─N skus 1─N inventory(×warehouse)
users 1─N orders 1─N order_items ─► skus (id ref + SNAPSHOT columns)
orders 1─N payments (append-only attempts; refunds = new rows)
checkout: reserve(guarded UPDATE) → charge(external, idempotent) → commit/release
          └──────────── reservation expiry reaper for abandoned checkouts ────────────┘
```

### 5–12. Interview Notes

- **Traps they'll pull**: "price changed after purchase — what shows on the old order?" (snapshots); "two buyers, one item left" (guarded UPDATE; show the SQL); "payment webhook arrives twice" (idempotency key + status transition guards); "totals don't match items" (store + verify with a CHECK or reconciliation job).
- **Ledger follow-up** (Stripe-flavored): double-entry — every movement is two rows (debit/credit) in an append-only `ledger_entries`; balances = SUM or maintained snapshot with reconciliation. Never UPDATE money rows.
- **Mistakes**: float money; deriving order totals by joining live SKU prices; one `inventory` count with no reservation concept; deleting order rows ever.
- **Scale plan**: products/skus cached + search in Elastic (CDC); orders partitioned by month (old ones cold); inventory hot rows — per-warehouse rows already shard the contention, hot SKUs may need queue-serialized decrements; payments append-only = easy to partition.

---

## Chapter 9.4 — Booking Systems: Hotel, Flight, Travel

### 1. Why Interviewers Ask This

Booking = the concurrency masterclass: prevent double-booking under contention, hold-then- confirm flows, and inventory that is *time-shaped* (nights, seats, legs). Hotel booking is the single most popular senior schema question.

### 2. Core Concept

Three equivalent strategies for "no double-booking" — know all three: 1. **Exclusion constraint on a range** (Postgres superpower): room × daterange must not overlap. Database-enforced, race-proof. 2. **Inventory-count model**: a row per (room_type, date) with `available` counter, decremented with a guarded UPDATE — books *types* not specific rooms (how real hotels/airlines sell), scales better, allows overbooking policy (`available` can start > physical). 3. **Unit-per-slot rows**: a row per (seat, flight) with a unique claim — flights' seat maps.

Holds: bookings start as `pending` with `expires_at`; payment confirms; reaper releases. Travel booking (multi-item trip) = saga across hotel+flight+car with compensations — the distributed-transaction discussion.

### 3. SQL — Reference Schemas

Hotel (both strategies):

```
-- Strategy 1: specific-room assignment, overlap-proof
CREATE EXTENSION IF NOT EXISTS btree_gist;
CREATE TABLE room_bookings (
  id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  room_id bigint NOT NULL REFERENCES rooms(id),
  guest_id bigint NOT NULL REFERENCES users(id),
  stay daterange NOT NULL,                          -- [check_in, check_out)
  status text NOT NULL DEFAULT 'pending'
       CHECK (status IN ('pending','confirmed','cancelled')),
  expires_at timestamptz,                            -- pending hold TTL
  EXCLUDE USING gist (room_id WITH =, stay WITH &&) WHERE (status <> 'cancelled')
);
-- concurrent overlapping inserts: exactly one wins (23P01), no app locking needed ✔

-- Strategy 2: sell room-types by night (scales; standard industry model)
CREATE TABLE room_type_inventory (
  hotel_id bigint NOT NULL, room_type_id bigint NOT NULL,
  night date NOT NULL,
  total int NOT NULL, available int NOT NULL CHECK (available >= 0),
  PRIMARY KEY (hotel_id, room_type_id, night)
);
-- book nights [d1, d2): all-or-nothing guarded decrement
UPDATE room_type_inventory
SET available = available - 1
WHERE hotel_id=:h AND room_type_id=:rt AND night >= :d1 AND night < :d2
  AND available > 0;
-- rowcount must equal (d2 - d1) nights; else ROLLBACK (partial nights unavailable)
```

Flight:

```
CREATE TABLE flights (
  id bigint PRIMARY KEY, flight_no text NOT NULL,
  departs_at timestamptz NOT NULL, origin char(3) NOT NULL, dest char(3) NOT NULL,
  UNIQUE (flight_no, departs_at)
);
CREATE TABLE flight_seats (
  flight_id bigint NOT NULL REFERENCES flights(id),
  seat_no text NOT NULL,
  booking_id bigint REFERENCES bookings(id),      -- NULL = free
  PRIMARY KEY (flight_id, seat_no)
);
-- claim a seat atomically:
UPDATE flight_seats SET booking_id = :b
WHERE flight_id=:f AND seat_no=:s AND booking_id IS NULL;   -- rowcount 0 = taken

CREATE TABLE bookings (               -- PNR-level record
  id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  pnr text NOT NULL UNIQUE,
  user_id bigint NOT NULL REFERENCES users(id),
  status text NOT NULL DEFAULT 'pending'
       CHECK (status IN ('pending','ticketed','cancelled')),
  expires_at timestamptz,
  total_cents int NOT NULL
);
CREATE TABLE booking_segments (       -- multi-leg itineraries
  booking_id bigint REFERENCES bookings(id),
  flight_id bigint REFERENCES flights(id),
  fare_class char(1) NOT NULL, price_cents int NOT NULL,   -- snapshot
  PRIMARY KEY (booking_id, flight_id)
);
```

Travel booking (trip saga):

```
CREATE TABLE trips (
  id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  user_id bigint NOT NULL,
  status text NOT NULL DEFAULT 'building' CHECK (status IN
    ('building','booking','confirmed','partially_failed','cancelled'))
);
CREATE TABLE trip_items (
  trip_id bigint REFERENCES trips(id),
  kind text NOT NULL CHECK (kind IN ('flight','hotel','car')),
  provider_ref text,                        -- external booking id
  status text NOT NULL DEFAULT 'pending' CHECK (status IN
    ('pending','reserved','confirmed','failed','compensated')),
  payload jsonb NOT NULL,
  PRIMARY KEY (trip_id, kind, provider_ref)
);
-- saga: reserve each item (idempotent) → confirm all → on any failure,
-- compensate (cancel reserved items) and mark partially_failed
```

### 4. Visualization

```
Hotel:  room_type_inventory[night]: |Jul1:3|Jul2:3|Jul3:0|Jul4:2|
        book Jul1-Jul3 → decrement Jul1,Jul2 rows atomically; Jul3 sold out? whole txn rolls back
Hold:   pending(expires 10min) ──pay──▶ confirmed
                        └─expired──▶ reaper releases (available++ / row freed)
Flight seat claim: UPDATE ... WHERE booking_id IS NULL  → exactly one winner per seat
Trip saga: [flight ✔ reserved][hotel ✔][car ✖ failed] → compensate flight+hotel → notify
```

### 5–12. Interview Notes

- **Guaranteed follow-ups**: "two users book the last room simultaneously — walk the exact mechanism" (constraint violation or rowcount=0 — no sleep-and-retry hand-waving); "user's payment takes 3 minutes — do you hold the room?" (pending + expires_at, reaper — never a held DB lock); "search available rooms for a date range" (inventory model: `GROUP BY room_type HAVING count(*) FILTER (WHERE available>0) = :nights` over the range — writable and indexable); "overbooking?" (inventory model supports it as policy: total > physical; exclusion model doesn't).
- **Mistakes**: `SELECT ... FOR UPDATE` on a *search* result set (locks the world); checking availability then inserting without a constraint (race); storing check_in/check_out without half-open semantics (adjacent bookings "overlap"); forgetting cancelled bookings must not block (the WHERE on the exclusion constraint).
- **Scale plan**: inventory model partitions by hotel_id naturally and shards the hot-row problem per (type, night); search moves to a read model (cache/Elastic) rebuilt from inventory; flights: seat maps per flight are small — the hot spot is fare/inventory buckets, same guarded-counter pattern; saga state must be crash-recoverable (the tables above are the saga log).

---

## Chapter 9.5 — Social Media (Feeds, Follows, Likes)

### 1. Why Interviewers Ask This

The read-amplification design: tiny writes, enormous fan-out reads, celebrity skew. Tests denormalization judgment and the push/pull feed decision.

### 2. Core Concept

Write model (normalized, Postgres): users, posts, follows (junction), likes (junction), comments. Read model (denormalized): **feeds**. The core decision: - **Fan-out on write (push)**: on post, insert into every follower's feed list (Redis/Cassandra). Feed read = one key fetch (fast). Celebrity posts = millions of writes. - **Fan-out on read (pull)**: feed read = merge recent posts of all followees. Cheap writes, expensive reads for users following thousands. - **Hybrid (the answer)**: push for normal authors; celebrities are pulled at read time and merged. Likes/comment *counts* are counter caches (sharded when hot).

## 3. SQL — Reference Schema

```
CREATE TABLE posts (
  id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  author_id bigint NOT NULL REFERENCES users(id),
  body text NOT NULL,
  media jsonb,
  like_count int NOT NULL DEFAULT 0,          -- counter cache (rebuildable)
  comment_count int NOT NULL DEFAULT 0,
  created_at timestamptz NOT NULL DEFAULT now(),
  deleted_at timestamptz
);
CREATE INDEX ON posts (author_id, created_at DESC) WHERE deleted_at IS NULL;

CREATE TABLE follows (
  follower_id bigint NOT NULL REFERENCES users(id),
  followee_id bigint NOT NULL REFERENCES users(id),
  created_at timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (follower_id, followee_id),
  CHECK (follower_id <> followee_id)
);
CREATE INDEX ON follows (followee_id);         -- "who follows X" (fan-out source)

CREATE TABLE likes (
  user_id bigint NOT NULL REFERENCES users(id),
  post_id bigint NOT NULL REFERENCES posts(id),
  created_at timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (user_id, post_id)               -- like-once enforced by PK
);
CREATE INDEX ON likes (post_id);

-- Pull-model feed query (works to ~thousands of followees):
SELECT p.* FROM posts p
JOIN follows f ON f.followee_id = p.author_id
WHERE f.follower_id = :me AND p.deleted_at IS NULL
  AND (p.created_at, p.id) < (:cursor_ts, :cursor_id)
ORDER BY p.created_at DESC, p.id DESC LIMIT 20;
```

Feed read model (push side, conceptually in Redis/Cassandra):

```
Redis:      LPUSH feed:{follower} post_id      (LTRIM feed:{follower} 0 799)
Cassandra:  feed_by_user ((user_id), created_at DESC, post_id)
read:       feed page = LRANGE / partition scan → hydrate posts by id (batched!)
```

## 4. Visualization

```
post by normal user (2k followers):        post by celebrity (80M followers):
write ─► posts ─► fan-out worker           write ─► posts (marked celebrity)
          └─► 2k feed-list inserts ✔                    └─► NO fan-out
read: feed:{me} → ids → MGET posts ✔       read: merge(feed:{me}, recent celebrity posts) ✔
like: PK(user,post) dedups; count = sharded counter, reconciled from likes table
```

## 5–12. Interview Notes

- **Follow-ups**: "unfollow — what happens to already-fanned-out posts?" (lazy filter at read, or async cleanup); "delete a viral post" (source-of-truth delete + read-time filter + async feed scrub + cache invalidation); "like count exact?" (no — counter cache + reconciliation; the *likes* table is exact); "feed consistency?" (eventual, seconds — say the product accepts it).
- **Mistakes**: COUNT(*) likes at render time; fan-out synchronously in the request; unbounded feed lists (trim!); follows table without the reverse index.
- **Scale plan**: posts partitioned by time; follows/likes shard by user; feed lists in Redis/Cassandra keyed by user (Module 8.2's exact pattern); counters sharded (Module 5.6); the celebrity threshold (~10k–100k followers) is a tunable — quote it.

---

# Chapter 9.6 — Chat & Messaging

## 1. Why Interviewers Ask This

Ordering, delivery states, unread counts, and huge append-only volume — plus the classic "Postgres first, Cassandra when" migration narrative (Discord).

## 2. Core Concept

Entities: conversations (direct or group), members (junction with per-member state), messages (append-only, keyed by conversation + time-ordered id). Critical decisions: - **Message ID = time-ordered** (snowflake/UUIDv7): sort key, pagination cursor, and dedup all in one. - **Per-member state on the junction** (`last_read_message_id`, muted, role) — unread count = messages after last_read (bounded query), or maintained counters for O(1). - **Direct conversations deduped** by a canonical pair key. - Delivery/read receipts: per-member watermark (scales), not per-message-per-user rows (explodes — only for small groups if product demands per-message ticks).

## 3. SQL — Reference Schema

```
CREATE TABLE conversations (
  id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  kind text NOT NULL CHECK (kind IN ('direct','group')),
  title text,                                    -- groups only
  -- dedup direct convos: canonical "smaller_id:larger_id"
  direct_key text UNIQUE,
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE conversation_members (
  conversation_id bigint NOT NULL REFERENCES conversations(id),
  user_id bigint NOT NULL REFERENCES users(id),
  role text NOT NULL DEFAULT 'member' CHECK (role IN ('member','admin')),
  last_read_message_id bigint NOT NULL DEFAULT 0,   -- read watermark
  muted_until timestamptz,
  PRIMARY KEY (conversation_id, user_id)
);
CREATE INDEX ON conversation_members (user_id);      -- "my conversations"

CREATE TABLE messages (
  id bigint PRIMARY KEY,                         -- snowflake: time-ordered, globally unique
  conversation_id bigint NOT NULL REFERENCES conversations(id),
  sender_id bigint NOT NULL REFERENCES users(id),
  body text,
  attachments jsonb,
  created_at timestamptz NOT NULL DEFAULT now(),
  deleted_at timestamptz                          -- soft delete ("message removed")
);
CREATE INDEX ON messages (conversation_id, id DESC);  -- the one hot index

-- history page (keyset):
SELECT * FROM messages
WHERE conversation_id = :c AND id < :cursor
ORDER BY id DESC LIMIT 50;

-- unread count per conversation (bounded by watermark):
SELECT count(*) FROM messages m
JOIN conversation_members cm
  ON cm.conversation_id = m.conversation_id AND cm.user_id = :me
WHERE m.conversation_id = :c AND m.id > cm.last_read_message_id;

-- conversation list with previews: denormalize last_message onto conversations
-- (updated on send) — avoids N top-1 subqueries on the inbox screen
```

## 4. Visualization

```
conversations 1—N messages (append-only, id = time-ordered)
      | 1—N conversation_members (watermarks: last_read_message_id)
inbox: my memberships → conversations (denormalized last_message preview) → unread badges
history: (conversation_id, id DESC) keyset pages —— infinite scroll
read:  advance watermark (1 UPDATE) — not N receipt rows
Cassandra migration (Discord path): messages_by_channel ((channel_id, bucket), id DESC)
```

## 5–12. Interview Notes

- **Follow-ups**: "message ordering across devices?" (server-assigned ids are the order; client timestamps are display-only); "exactly-once send?" (client-generated message id + PK dedup = idempotent retry); "typing indicators/ presence?" (Redis TTL keys, never the DB); "group of 100k members' unread counts?" (watermarks make it per-member O(1) state + lazy count; push badge counts via async).
- **Mistakes**: per-message-per-recipient delivery rows for large groups; OFFSET pagination on history; storing presence in Postgres; global autoincrement exposing message volume (fine internally, mask externally if it matters).
- **Scale plan**: messages = the biggest table you'll ever own — partition by month early (or conversation-bucket), archive cold partitions to object storage; when write volume outgrows one primary, this table is the textbook Cassandra/Scylla migration (channel-bucketed partitions — Module 8.6); everything else (users, members) stays relational far longer.

---

# Module 9 — Practice Problems

## Easy (5)

1. Add "wishlist" to the e-commerce schema: tables, keys, constraints, and the query "is this SKU in my wishlist" — in DDL.
2. Enforce: a user can review a product only once, and only after a delivered order containing it. (Unique constraint + the INSERT ... WHERE EXISTS shape.)
3. Why does `order_items` copy `unit_price_cents` when `skus.price_cents` exists? Name the principle and one bug the copy prevents.
4. Design the `coupons` + `coupon_redemptions` tables with per-user and global usage limits enforced in DDL as far as possible.
5. Write the inbox query: my 20 most recent conversations with unread counts, using the watermark design.

## Medium (5)

1. Extend hotel booking with rate plans (refundable/non-refundable, seasonal pricing): where do prices live, what gets snapshotted onto the booking, and how does availability interact with plan?
2. Add multi-warehouse fulfillment to e-commerce: allocation strategy tables, partial shipments, and the state machine from 'paid' to 'delivered' with split packages.
3. Design likes for 1M likes/minute peak: exact likes table + sharded counters + reconciliation + read path. Show the DDL and the flush job.
4. Flight search "NYC→LON, any airline, next Friday, 2 seats in economy": what schema/read model serves this in <100ms, given the booking tables? (Availability projection per (route, date, class) maintained by CDC.)
5. Convert the direct-message model to support "edit history + delete for me / delete for everyone": schema changes and the read query.

## Hard (5)

1. Full trip-booking saga: design the saga-state tables, idempotent reserve/confirm/compensate steps for flight+hotel+car with per-provider idempotency keys, crash recovery (resume from saga log), and the user-visible states. Walk a failure at each step.
2. Your hotel inventory model must now support: overbooking by policy (105%), day-of walk-ins, room-type upgrades at check-in, and channel managers (Expedia) with allotments. Extend the schema and identify each new race condition and its guard.
3. Design GDPR erasure across the whole e-commerce schema: what's deleted, scrambled, retained (legal), the FK strategy for tombstoned users, the async pipeline, and proof-of-erasure audit.
4. The messages table hits 10B rows on Postgres. Produce the full migration plan to Cassandra: target schema (bucketing math for 99p channel activity), dual-write phase, backfill strategy, read-cutover per channel cohort, verification, and rollback. Include what you do about the relational features you lose (FKs, transactions on member state).
5. Unify Modules 7+9: shard the social-media write model (users, posts, follows) across 8 Postgres shards. Choose the shard key, place each table, redesign the follows fan-out (cross-shard!), and specify the feed pipeline end-to-end with failure semantics.

---

*Next: Module 10 — Interview SQL*

# MODULE 10 — Interview SQL: The Problems That Actually Get Asked

> ~90% of SQL screen questions are instances of the 10 patterns below. Learn the *pattern*, and the "top 100 problems" become variations you improvise. Each pattern: why it's asked, the template, worked solutions, traps, and follow-ups.
>
> Standard tables used throughout: `employees(id, name, dept_id, salary, manager_id)` `orders(id, user_id, total_cents, status, created_at)` `events(user_id, event_type, ts)` `logins(user_id, login_date)`

## Pattern 1 — Ranking & Nth Highest (ROW_NUMBER / RANK / DENSE_RANK)

**Why asked**: the fastest window-function fluency check; "second highest salary" is the most famous SQL interview question in existence.

**Template**: rank in a subquery/CTE, filter outside (windows can't go in WHERE).

```
-- Second highest salary (value): DENSE_RANK handles ties correctly
SELECT DISTINCT salary
FROM (SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
      FROM employees) t
WHERE rnk = 2;

-- Nth highest as a parameter
SELECT salary FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk FROM employees
) t WHERE rnk = :n LIMIT 1;

-- Classic no-window alternatives (know them — they still get asked):
SELECT max(salary) FROM employees
WHERE salary < (SELECT max(salary) FROM employees);          -- 2nd highest

SELECT DISTINCT salary FROM employees e1
WHERE :n - 1 = (SELECT count(DISTINCT salary) FROM employees e2
                WHERE e2.salary > e1.salary);                -- Nth, correlated

-- Top 3 per department, ties included
SELECT * FROM (
  SELECT e.*, DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) rnk
  FROM employees e
) t WHERE rnk <= 3;
```

**Traps**: RANK skips positions after ties (1,1,3) — "3rd highest *value*" needs DENSE_RANK; "3rd highest *row*" needs ROW_NUMBER with a declared tiebreaker; Nth highest must return NULL (not empty tantrum) when fewer than N distinct values — `LIMIT 1` + scalar subquery shape, or LeetCode's `SELECT (SELECT ...)` wrapper. **Follow-ups**: "which of the three functions and why"; "do it without window functions"; "what index helps?" (`(dept_id, salary DESC)` feeds the partition sort).

## Pattern 2 — Duplicate Records (find, count, delete)

**Why asked**: real data-cleanup skill + window-function DELETE synthesis.

```
-- Find duplicated business keys
SELECT email, count(*) FROM users GROUP BY email HAVING count(*) > 1;

-- Full duplicate rows with ids (window version — shows all copies)
SELECT * FROM (
  SELECT u.*, count(*) OVER (PARTITION BY email) AS copies
  FROM users u
) t WHERE copies > 1;

-- Delete duplicates, keep lowest id (the canonical answer)
DELETE FROM users u
USING users u2
WHERE u.email = u2.email AND u.id > u2.id;

-- Delete keeping the most recent, arbitrary tie policy declared
DELETE FROM users
WHERE id IN (
  SELECT id FROM (
    SELECT id, ROW_NUMBER() OVER (PARTITION BY email
                                  ORDER BY last_seen DESC, id DESC) rn
    FROM users) t
  WHERE rn > 1
);
```

**Traps**: defining "duplicate" (whole row vs business key) — ask; keeping which copy — declare the ORDER BY; on huge tables, batch the delete and add the unique index *after* cleanup (`CREATE UNIQUE INDEX CONCURRENTLY`) so duplicates can't return. **Follow-ups**: "prevent them forever" (unique index — the real answer); "the table is 2B rows" (batched deletes by id range; or CTAS the survivors + swap).

---

## Pattern 3 — Running Totals & Moving Averages

**Why asked**: frames (ROWS/RANGE) separate people who *use* windows from people who understand them.

```
-- Running total per user
SELECT user_id, created_at, total_cents,
       SUM(total_cents) OVER (PARTITION BY user_id ORDER BY created_at, id) AS running
FROM orders;

-- 7-row moving average (physical rows)
SELECT day, revenue,
       AVG(revenue) OVER (ORDER BY day ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS ma7
FROM daily_revenue;

-- 7-DAY moving average (time-correct even with missing days)
SELECT day, revenue,
       AVG(revenue) OVER (ORDER BY day
                          RANGE BETWEEN interval '6 days' PRECEDING AND CURRENT ROW) AS ma7
FROM daily_revenue;

-- Month-over-month growth (LAG)
SELECT month, revenue,
       round(100.0 * (revenue - LAG(revenue) OVER (ORDER BY month))
             / NULLIF(LAG(revenue) OVER (ORDER BY month), 0), 1) AS growth_pct
FROM monthly_revenue;

-- Cumulative % of total (running / grand total)
SELECT day, revenue,
       SUM(revenue) OVER (ORDER BY day)::numeric
         / SUM(revenue) OVER () AS cum_share
FROM daily_revenue;
```

**Traps**: `ROWS` vs `RANGE` when days are missing or duplicated (ROWS counts rows; RANGE counts value-distance — the 7-day question is a RANGE or calendar-spine question); default frame with ORDER BY is RANGE-to-current-row *including peers*; always add a unique tiebreaker to ORDER BY for deterministic running sums. **Follow-ups**: "running total without window functions" (self-join `b.day <= a.day` — O(N²), say why it's worse); "reset the running total every month" (add month to PARTITION BY).

---

## Pattern 4 — Top-K per Group

**Why asked**: the workhorse pattern (top 3 products per category, latest order per user); tests both correctness and the performance alternatives.

```
-- Window version (general)
SELECT * FROM (
  SELECT o.*, ROW_NUMBER() OVER (PARTITION BY user_id
                                 ORDER BY created_at DESC, id DESC) rn
  FROM orders o
) t WHERE rn <= 3;

-- Latest-one-per-group: DISTINCT ON (Postgres idiom, terse)
SELECT DISTINCT ON (user_id) *
FROM orders ORDER BY user_id, created_at DESC, id DESC;

-- LATERAL version (index-driven; wins when groups ≪ rows)
SELECT o.* FROM users u
JOIN LATERAL (
  SELECT * FROM orders o WHERE o.user_id = u.id
  ORDER BY created_at DESC, id DESC LIMIT 3
) o ON true;      -- index: orders(user_id, created_at DESC, id DESC)
```

**Traps**: ROW_NUMBER vs RANK for "top 3 with ties" (RANK/DENSE_RANK ≤ 3 may return >3 rows — clarify the requirement); nondeterminism without a tiebreaker; the window version sorts *everything* — mention the LATERAL alternative for perf and you jump a level. **Follow-ups**: "which is fastest on 1B orders / 10M users?" (LATERAL probes 10M × index-top-3 vs global sort — depends on group count; discuss); "top 3 by SUM per group" (aggregate first, then rank the aggregates).

---

## Pattern 5 — Gaps & Islands

**Why asked**: the hardest common pattern; tests the row_number-difference trick. Shows up as consecutive login streaks, continuous sensor coverage, seat-range grouping.

**The trick**: for consecutive integers/dates, `value - ROW_NUMBER()` is constant within a consecutive run ("island") — group by that constant.

```
-- Longest consecutive-day login streak per user
WITH days AS (          -- dedup multiple logins per day
  SELECT DISTINCT user_id, login_date FROM logins
), grp AS (
  SELECT user_id, login_date,
         login_date - (ROW_NUMBER() OVER (PARTITION BY user_id
                                          ORDER BY login_date))::int AS island
  FROM days
)
SELECT user_id,
       count(*)          AS streak_len,
       min(login_date)   AS streak_start,
       max(login_date)   AS streak_end
FROM grp
GROUP BY user_id, island
ORDER BY streak_len DESC;

-- Gaps: missing id ranges in a sequence
SELECT prev_id + 1 AS gap_start, id - 1 AS gap_end
FROM (SELECT id, LAG(id) OVER (ORDER BY id) AS prev_id FROM tickets) t
WHERE id - prev_id > 1;

-- Islands with a tolerance (new island when gap > 30 min) — general LAG method
WITH flagged AS (
  SELECT *, CASE WHEN ts - LAG(ts) OVER (PARTITION BY user_id ORDER BY ts)
                      > interval '30 minutes'
                 THEN 1 ELSE 0 END AS new_island
  FROM events
), numbered AS (
  SELECT *, SUM(new_island) OVER (PARTITION BY user_id ORDER BY ts) AS island_id
  FROM flagged
)
SELECT user_id, island_id, min(ts), max(ts), count(*) FROM numbered
GROUP BY user_id, island_id;
```

**Traps**: duplicates break the row_number-difference trick (dedup first!); the difference trick only works for evenly-spaced values — irregular spacing needs the LAG+flag+running-sum method (which is also the sessionization solution); date arithmetic types (`date - int` works in PG; timestamps need care). **Follow-ups**: "streak *ending today*" (filter islands where max = current_date); "merge overlapping intervals" (same flag method with `start <= max(prev_end)` running max).

---

## Pattern 6 — Sessionization

**Why asked**: the analytics-engineering classic (Meta/Amazon product analytics): split an event stream into sessions with a 30-minute inactivity rule. It's Gaps & Islands in disguise — say that out loud.

```
WITH flagged AS (
  SELECT user_id, ts,
         CASE WHEN ts - LAG(ts) OVER (PARTITION BY user_id ORDER BY ts)
                   > interval '30 minutes'
              OR LAG(ts) OVER (PARTITION BY user_id ORDER BY ts) IS NULL
         THEN 1 ELSE 0 END AS session_start
  FROM events
), sessions AS (
  SELECT *, SUM(session_start) OVER (PARTITION BY user_id ORDER BY ts) AS session_no
  FROM flagged
)
SELECT user_id, session_no,
       min(ts) AS session_start,
       max(ts) AS session_end,
       max(ts) - min(ts) AS duration,
       count(*) AS events
FROM sessions
GROUP BY user_id, session_no;

-- Follow-on metrics interviewers ask next:
-- avg sessions/user/day, avg session duration, bounce sessions (events = 1)
```

**Traps**: forgetting the first event (LAG NULL) must open a session; timezone bucketing when reporting per-day; global vs per-user ordering (always PARTITION BY user). **Follow-ups**: "do it for 10B events" (this exact shape in Spark/warehouse; or incremental sessionization with state); "session ids stable across reruns?" (derive from user_id + session_start).

---

## Pattern 7 — Median & Percentiles

**Why asked**: tests knowledge of ordered-set aggregates (and the honest "SQL median is not AVG" check).

```
-- Median (interpolated) and p95
SELECT percentile_cont(0.5)  WITHIN GROUP (ORDER BY total_cents) AS median,
       percentile_cont(0.95) WITHIN GROUP (ORDER BY total_cents) AS p95,
       percentile_disc(0.5)  WITHIN GROUP (ORDER BY total_cents) AS median_actual_value
FROM orders;

-- Per group
SELECT dept_id,
       percentile_cont(0.5) WITHIN GROUP (ORDER BY salary) AS median_salary
FROM employees GROUP BY dept_id;

-- Median as a window (per-row context): PERCENTILE_CONT isn't a window fn in PG →
-- use two ROW_NUMBERs (the classic manual median):
WITH r AS (
  SELECT salary,
         ROW_NUMBER() OVER (ORDER BY salary)  rn,
         COUNT(*)     OVER ()                 n
  FROM employees
)
SELECT avg(salary) AS median FROM r WHERE rn IN ((n+1)/2, (n+2)/2);

-- Decile bucketing
SELECT id, salary, NTILE(10) OVER (ORDER BY salary) AS decile FROM employees;
```

**Traps**: `percentile_cont` interpolates (may return a value not in the data) vs `percentile_disc`; even-count median = average of middle two (the manual version must handle it — the `(n+1)/2, (n+2)/2` trick does); MySQL <8 has none of this → manual method is the portable answer. **Follow-ups**: "p99 latency over a rolling window" (percentile per time bucket; exact percentiles don't decompose — mention t-digest/HLL-style sketches for streaming).

---## Pattern 8 — Hierarchies & Recursive Queries

**Why asked**: org charts and category trees (Module 2.3's recursive CTE, in problem form).

```
-- All reports (direct+indirect) of manager 42, with depth and path
WITH RECURSIVE sub AS (
  SELECT id, name, manager_id, 1 AS depth, ARRAY[id] AS path
  FROM employees WHERE manager_id = 42
  UNION ALL
  SELECT e.id, e.name, e.manager_id, s.depth+1, s.path || e.id
  FROM employees e JOIN sub s ON e.manager_id = s.id
  WHERE NOT e.id = ANY(s.path)
)
SELECT * FROM sub ORDER BY path;

-- Chain UP: an employee's management chain to the CEO
WITH RECURSIVE chain AS (
  SELECT id, name, manager_id, 1 AS lvl FROM employees WHERE id = :emp
  UNION ALL
  SELECT e.id, e.name, e.manager_id, c.lvl+1
  FROM employees e JOIN chain c ON e.id = c.manager_id
)
SELECT * FROM chain;

-- Aggregate over subtree: headcount + total salary under each manager
WITH RECURSIVE sub AS (
  SELECT id AS root, id FROM employees
  UNION ALL
  SELECT s.root, e.id FROM employees e JOIN sub s ON e.manager_id = s.id
)
SELECT root, count(*) - 1 AS reports FROM sub GROUP BY root;

-- Employees earning more than their manager (the self-join classic)
SELECT e.name FROM employees e JOIN employees m ON m.id = e.manager_id
WHERE e.salary > m.salary;
```

**Traps**: infinite loops on cyclic data (path guard / PG14 `CYCLE` clause); UNION vs UNION ALL in the recursion (dedup per iteration vs not); the subtree-aggregate version fans out — fine for org charts, quadratic-ish for deep wide trees (closure table alternative). **Follow-ups**: "same in MySQL?" (8.0 WITH RECURSIVE — yes); "10M-node tree read constantly" (closure table / ltree / materialized path — Module 2.3).

---

## Pattern 9 — Pivots, Conditional Aggregation & Attribution

**Why asked**: reshaping rows→columns and funnel/attribution logic — the analytics screen staples.

```
-- Pivot: orders by status per month (rows → columns)
SELECT date_trunc('month', created_at) AS month,
       count(*) FILTER (WHERE status = 'paid')      AS paid,
       count(*) FILTER (WHERE status = 'cancelled') AS cancelled,
       count(*) FILTER (WHERE status = 'refunded')  AS refunded
FROM orders GROUP BY 1 ORDER BY 1;

-- Funnel: view → cart → purchase conversion within 7 days
WITH v AS (SELECT user_id, min(ts) AS t FROM events WHERE event_type='view'     GROUP BY 1),
     c AS (SELECT user_id, min(ts) AS t FROM events WHERE event_type='cart'     GROUP BY 1),
     p AS (SELECT user_id, min(ts) AS t FROM events WHERE event_type='purchase' GROUP BY 1)
SELECT count(v.user_id)                                AS viewed,
       count(c.user_id)                                AS carted,
       count(p.user_id) FILTER (WHERE p.t <= v.t + interval '7 days') AS purchased
FROM v LEFT JOIN c USING (user_id) LEFT JOIN p USING (user_id);

-- First-touch attribution: the channel of each user's first visit
SELECT DISTINCT ON (user_id) user_id, channel AS first_touch
FROM visits ORDER BY user_id, ts;

-- Retention: % of January signups active in each subsequent month (cohort)
SELECT date_trunc('month', a.ts) AS month,
       count(DISTINCT a.user_id)::numeric
         / (SELECT count(*) FROM users
            WHERE created_at >= '2026-01-01' AND created_at < '2026-02-01') AS retention
FROM events a
JOIN users u ON u.id = a.user_id
WHERE u.created_at >= '2026-01-01' AND u.created_at < '2026-02-01'
GROUP BY 1 ORDER BY 1;
```

**Traps**: funnel steps must be *ordered in time* (cart after view — join conditions on timestamps, not just existence); COUNT(DISTINCT) semantics in cohorts; pivot with dynamic columns isn't SQL's job (app/BI layer, or crosstab extension — say so). **Follow-ups**: "windowed funnel (step within 1h of previous step)" (conditional joins on time deltas or window LEAD over typed events).

---

## Pattern 10 — Set Operations, Anti-Joins & Data Diff

**Why asked**: "who churned / what's missing / reconcile these tables" — anti-join fluency plus the NOT IN trap (Module 2.2) in problem form.

```
-- Customers who never ordered
SELECT u.* FROM users u
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.user_id = u.id);

-- Churn: active last month, absent this month
SELECT user_id FROM activity WHERE month = '2026-05-01'
EXCEPT
SELECT user_id FROM activity WHERE month = '2026-06-01';

-- Table diff (schema-identical tables a, b): rows that differ in either direction
(TABLE a EXCEPT TABLE b) UNION ALL (TABLE b EXCEPT TABLE a);

-- Users who bought product X AND product Y (relational division, the sneaky one)
SELECT user_id FROM orders o JOIN order_items i ON i.order_id = o.id
WHERE i.product_id IN (10, 20)
GROUP BY user_id
HAVING count(DISTINCT i.product_id) = 2;

-- Users who bought EVERY product in a category (full relational division)
SELECT o.user_id
FROM orders o JOIN order_items i ON i.order_id = o.id
JOIN products p ON p.id = i.product_id AND p.category_id = 7
GROUP BY o.user_id
HAVING count(DISTINCT p.id) = (SELECT count(*) FROM products WHERE category_id = 7);
```

**Traps**: `NOT IN` + NULL (the module-2 landmine — reflex answer: NOT EXISTS); EXCEPT dedups (EXCEPT ALL if multiplicity matters); relational division via `HAVING count(DISTINCT) = total` is the expected idiom. **Follow-ups**:

"make the never-ordered query fast on 100M users" (anti hash join + index on orders(user_id); or maintained `last_order_at` column).

---

# The Top-100 Checklist (grouped by pattern)

Work through these; every one maps to a pattern above.  = solved cold, in one pass.

**Ranking / Nth (1–12)**: second highest salary · Nth highest salary · top 3 per department · rank scores (with each of the 3 functions) · department top earner with ties · first purchase per user · latest login per user · best-selling product per category · rank with gaps explanation · median rank · dense rank change after delete · top earner per dept per month.

**Duplicates (13–20)**: find dup emails · delete dups keep lowest id · delete dups keep newest · count fully-duplicate rows · dedupe with a business-key priority · find near-duplicates (same name, different email) · prevent duplicates (DDL) · dedupe a billion-row table (strategy).

**Aggregation & CASE (21–32)**: paid vs unpaid counts one scan · monthly revenue + growth % · AOV excluding refunds · revenue share per category · conditional sums by status · histogram buckets · percent of total per row · weighted average · min/max with their dates · count distinct buyers per month · first/last value per group · HAVING vs WHERE cases.

**Joins & anti-joins (33–44)**: customers with no orders · products never sold · users with orders in Jan but not Feb · employees vs managers salary · same-manager pairs · orders with all items in stock · reconcile ledger vs settlement · users with ≥2 distinct categories bought · self-join sequential events · full-join diff two snapshots · every-product buyers (division) · X-and-Y buyers.

**Windows: running/offset (45–58)**: running total · running total reset monthly · 7-row MA · 7-day MA with gaps · MoM growth · YoY same-month growth · days between consecutive orders · LAG price change detection · cumulative distinct users · first-order flag per row · percent-of-running-total · rolling 30-day distinct actives · rank of each day within its month · biggest single-day jump.

**Gaps & Islands / Sessions (59–70)**: longest login streak · current streak ending today · missing ids · missing dates · consecutive seats free · merge overlapping intervals · sessionize with 30-min gap · avg session duration · bounce rate · sessions per user per day · longest session · concurrent sessions max (interval overlap counting).

**Hierarchy / recursion (71–78)**: all reports of X · management chain up · tree depth · subtree headcount · cycle detection · category tree with product counts · BOM cost rollup · generate date series report.

**Median / percentiles (79–84)**: median salary · median per dept · p95 latency per endpoint · deciles · manual median (no percentile_cont) · trimmed mean.

**Design-flavored SQL (85–92)**: keyset pagination page-N · has_more trick · counter update race-free · upsert with conflict · idempotent insert · soft-delete-aware unique · queue claim SKIP LOCKED · optimistic-lock update.

**Classic LeetCode-style named problems (93–100)**: Employees earning more than managers · Duplicate emails · Customers who never order · Department highest salary · Department top three salaries · Rank scores · Consecutive numbers (3+ in a row — islands) · Trips and users (cancellation rate with filters).

# Module 10 — Practice Problems

## Easy (5)

1. Second-highest salary — three ways (window, max-below-max, OFFSET) — and state what each returns when the table has one distinct salary.
2. Find and delete duplicate `(user_id, product_id)` wishlist rows keeping the earliest; add the constraint preventing recurrence.
3. Monthly order counts pivoted into columns for statuses paid/cancelled/refunded, one scan.
4. Customers who never ordered — and explain why you didn't use NOT IN.
5. Rank scores per game with DENSE_RANK; return only players ranked ≤ 3, ties included.

## Medium (5)

1. Longest daily-login streak per user (full solution), then modify to "streak still alive today."
2. 7-day time-correct moving average of revenue with missing days, two ways (RANGE frame; calendar spine) — and when the answers differ.
3. Sessionize clickstream (30-min rule), then compute per user: sessions, avg duration, bounce rate — one query chain.
4. For each user: first order channel, last order channel, lifetime orders, days active span — one scan with windows/aggregates (no self-joins).
5. Consecutive-numbers problem: find numbers appearing ≥3 times consecutively (by id) — LAG/LEAD version and islands version.

## Hard (5)

1. Max concurrent sessions: given `(start_ts, end_ts)` rows, find the peak concurrency and when it occurred (event decomposition: +1/-1 rows, running sum, argmax).
2. Cohort retention triangle: for each signup month, retention % in months 0..5 — produce the month × offset grid in one query.
3. Median per department *as of each month end* over a year (percentiles over a moving population — window population construction + lateral percentile).
4. "Top 3 products per category by trailing-28-day revenue, refreshed daily, on a 2B-row order_items table" — write the query AND the incremental materialization design that makes it servable.
5. Gap-tolerant device uptime: sensors ping ~every 60s; compute per-device daily uptime % counting gaps >5 min as downtime (islands with tolerance + per-day slicing of intervals — handle intervals crossing midnight).

---

*Next: Module 11 — Production Debugging*

# MODULE 11 — Production Debugging

> The on-call module. Senior interviews end with "the database is slow / down — go." This module is organized as incident runbooks: symptom → triage → root causes → fixes → prevention. Most mechanisms were built in Modules 4–7; here they become diagnosis flows.

Chapters: 11.1 The Universal Triage Method 11.2 Slow Queries & Missing Indexes 11.3 Connection Pool Exhaustion 11.4 Deadlocks & Lock Contention 11.5 Replication Lag 11.6 Memory Issues 11.7 Large Tables & Bloat 11.8 Pagination Problems (deep-page incidents)

---

## Chapter 11.1 — The Universal Triage Method

### 1. Why Interviewers Ask This

Incident questions grade your *process* under ambiguity. A senior answer runs a fixed loop: scope → recent change → observe → hypothesize → verify → mitigate → root-cause → prevent.

### 2. Core Concept — The First 5 Minutes

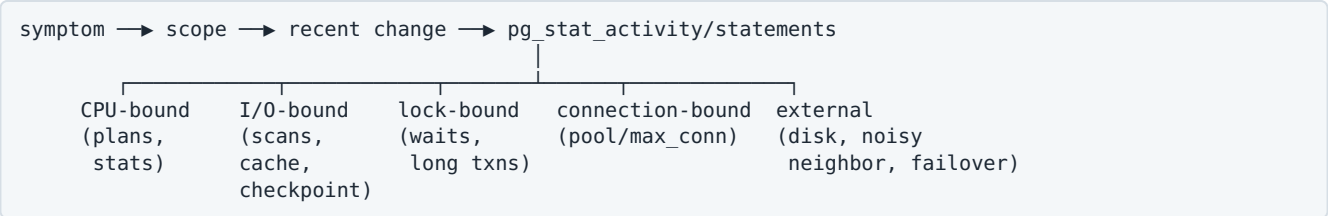
1. **Scope**: everything slow, or one query/endpoint/tenant? Reads, writes, or both? Primary or replicas? Since when?
2. **What changed**: deploys, migrations, traffic spikes, data growth crossing a threshold, config, infra events (failover, vacuum, backup).
3. **Look at the database's own account of itself** (the queries below — memorize them):

```
-- Who is doing what right now
SELECT pid, state, wait_event_type, wait_event,
       now() - query_start AS runtime, left(query, 80)
FROM pg_stat_activity
WHERE state <> 'idle' ORDER BY query_start;

-- Blocked ⇄ blocker chains
SELECT pid, pg_blocking_pids(pid) AS blocked_by, left(query, 60)
FROM pg_stat_activity WHERE cardinality(pg_blocking_pids(pid)) > 0;

-- Cumulative top offenders
SELECT calls, round(mean_exec_time,1) AS mean_ms,
       round(total_exec_time)::bigint AS total_ms, left(query, 80)
FROM pg_stat_statements ORDER BY total_exec_time DESC LIMIT 10;

-- Long transactions (the silent killer behind half of incidents)
SELECT pid, now() - xact_start AS age, state, left(query,60)
FROM pg_stat_activity WHERE xact_start IS NOT NULL ORDER BY xact_start LIMIT 5;
```

1. **Classify**: CPU-bound (plans/misestimates), I/O-bound (cache misses, scans, checkpoints), lock-bound (waits, not resource usage), or connection-bound (saturation ahead of the DB). `wait_event_type` tells you: `Lock` = lock-bound, `IO` = I/O, CPU pegged + no waits = plans.
2. **Mitigate before root-causing** when bleeding: kill the offending query (`pg_terminate_backend`), roll back the deploy, shed load, add the emergency index CONCURRENTLY. Say the mitigation/root-cause distinction out loud — it's senior signal.

### 3–12. (Method notes)

- **Visualization**:




- **Interview traps**: jumping to "add an index" before scoping; restarting the DB as step 1 (destroys evidence, drops cache — often makes it worse); not asking "what changed."
- **Best practice**: keep dashboards for the four classes (CPU/IO/locks/connections) + the four queries above as saved runbook snippets.

---

## Chapter 11.2 — Slow Queries & Missing Indexes

### 1. Why Interviewers Ask This

The most common incident. The interview version hands you symptoms ("this endpoint went from 50ms to 8s over a month") and expects a clean diagnosis chain.

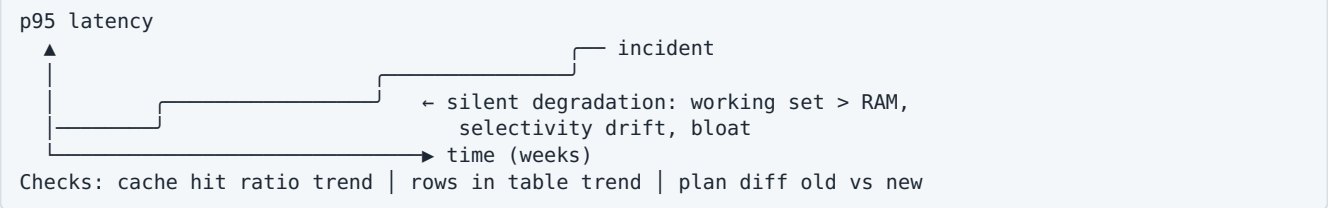
### 2. Core Concept — Diagnosis Chain

1. Identify the query: APM trace / `pg_stat_statements` (sort by `total_exec_time` for aggregate pain, `mean_exec_time` for user pain) / `auto_explain` logs.
2. `EXPLAIN (ANALYZE, BUFFERS)` with production parameters (Module 5.1).
3. Classify the cause — the frequency-ordered list: - **Missing/wrong index** (Seq Scan or Filter-heavy index scan; Rows Removed by Filter huge) - **Stale stats / misestimates** (est vs actual >10x; after bulk loads) - **Non-sargable predicate** (function on column, type cast, leading wildcard — Module 4.6) - **Plan flip** (prepared-statement generic plan; stats change; parameter skew) - **Data growth crossing thresholds** (the slow-over-a-month case: working set left RAM, or crossover from index to seq scan) - **Lock waits masquerading as slowness** (check wait_event first!) - **N+1 at the app layer** (fast queries, slow requests — Module 5.7)
4. Fix (index CONCURRENTLY / ANALYZE / rewrite / plan_cache_mode), verify with before/after plans, then prevent (slow-query alerting, index review in migrations, load tests with production-scale data).

### 3. Internal Working — "Slow over a month" mechanics

Nothing changed except data: (a) the hot working set outgrew `shared_buffers`+page cache → cache hit ratio slid → I/O-bound; (b) planner crossover — at 2% selectivity index scan won, at 9% it flips to seq scan (or should, but stats lag); (c) index bloat grew descent depth/cache footprint. This gradual class is *the* senior probe — name all three.

### 4. Visualization




### 5. Real Production Example

`WHERE status='pending'` job poller: brilliant at launch (1k rows), death at 400M rows with 99.9% terminal states — seq scan every 5 seconds. Fix: partial index (Module 4.4), instant. Second: nightly ETL bulk-loads a table; every morning dashboards are slow until autoanalyze catches up → add explicit `ANALYZE` to the ETL. Both are interview scripts, verbatim.

### 6–12. (Compressed)

- **Questions**: "walk me through debugging a slow query" (recite the chain); "it's slow only in production" (data volume, parameters, cache, stats — never 'works on my machine'); "it's slow only sometimes" (plan flips, lock waits, checkpoints, noisy batch jobs — correlate timestamps).
- **Mistakes**: indexing without reading the plan; testing on empty staging; measuring the second (cached) run only; "fixing" with `enable_seqscan=off`.
- **Best practices**: `pg_stat_statements` + `auto_explain` always on; every schema migration PR includes EXPLAIN of affected hot queries; production-scale fixture data in perf CI.
- **Follow-ups**: "index exists but unused" (Module 4.6 catalog — run it); "how do you find *missing* indexes proactively?" (top statements with high `shared_blks_read`/call + seq_scan counters in `pg_stat_user_tables`).

---

## Chapter 11.3 — Connection Pool Exhaustion

### 1. Why Interviewers Ask This

Top-3 real outage class ("FATAL: remaining connection slots are reserved" or app-side pool timeouts) with a counterintuitive fix (fewer connections, not more) — perfect senior filter.

### 2. Core Concept — Triage

Symptoms: app errors acquiring connections; DB *itself* often healthy and idle-ish. First split: **pool exhausted because queries got slow** (root cause = Ch 11.2/11.4: same throughput × longer hold time = more concurrent conns, Little's law) vs **pool exhausted because demand rose** (deploy doubled pods, traffic spike, connection leak).

```
SELECT count(*), state FROM pg_stat_activity GROUP BY state;
-- many 'idle in transaction'  → app leaks transactions (missing commit/close in a code path)
-- many 'active' same query    → that query got slow → fix the query, pool recovers
-- many 'idle'                 → fleet over-provisioned connections → pooler/sizing
```

Fix ladder: kill leakers (`pg_terminate_backend`) → `idle_in_transaction_session_timeout` → transaction-mode PgBouncer (Module 7.4) → right-size app pools fleet-wide → fix the slow query or leak that started it.

### 3. Internal Working

The death spiral interviewers want articulated: one slow query → requests stack → each holds a connection → pool saturates → *healthy* queries queue for connections → timeouts → retries → more load → total outage from one bad query. Connection pressure is a **symptom amplifier**; always ask what consumed the time-slices first.

### 4. Visualization

```
slow query (500ms→8s)                          Little's law:
   │ same 1000 RPS                             busy_conns = QPS × latency
   ▼                                           1000 × 0.05s = 50   (fine)
conns held 16x longer ──▶ pool full ──▶        1000 × 0.8s  = 800  (dead: pool=100)
timeouts ──▶ retries ──▶ MORE load ──▶ ✖
mitigation order: kill slow query → shed load → THEN tune pools
```

### 5–12. (Compressed)

- **Production example**: deploy adds a missing `await`/`close()` — every request leaks one `idle in transaction` conn; 20 minutes later, full outage. The `state` census finds it in one query.
- **Questions**: "app says pool timeout, DB looks idle — go"; "should we raise max_connections?" (almost never the fix — Module 7.4 throughput curve); "how do serverless functions make this worse?"
- **Mistakes**: raising limits everywhere (moves the cliff, adds thrash); restarting the app (clears symptom, loses the leak evidence); no per-service connection budget.
- **Prevention**: pooler in transaction mode, statement/idle-txn timeouts, per-service budgets summing under capacity, leak detection in the driver (max lifetime, abandoned tracking), pool-wait metrics + alerts.

- **Follow-ups**: "PgBouncer added — what breaks?" (session state — Module 7.4); "connection storms after failover?" (thundering reconnect — jittered backoff, connection warm-up rate limits).

---

## Chapter 11.4 — Deadlocks & Lock Contention

### 1. Why Interviewers Ask This

Distinguishes "knows locks exist" from "can read a lock graph at 3am." Also covers the sneaky class: everything slow, CPU idle — pure waiting.

### 2. Core Concept — Triage

Symptoms split: **deadlocks** (errors 40P01 in logs, DETAIL shows both queries) vs **lock contention** (no errors, just latency; `wait_event_type='Lock'` in pg_stat_activity).

Contention flow: find blocked pids → `pg_blocking_pids()` → walk to the **root blocker** → what is it? (long transaction? idle in transaction? DDL waiting? batch job?) → kill or wait → then design fix (shorter txns, lock ordering, SKIP LOCKED, hot-row sharding — Module 6).

Deadlock flow: collect log DETAILs → reconstruct the two access orders → impose one global order (or single-statement atomicity) → add retries.

```
-- The lock-chain one-liner during an incident
SELECT a.pid, a.state, now()-a.xact_start AS txn_age,
       pg_blocking_pids(a.pid) AS blocked_by, left(a.query, 70)
FROM pg_stat_activity a
WHERE a.pid IN (SELECT unnest(pg_blocking_pids(pid)) FROM pg_stat_activity)
   OR cardinality(pg_blocking_pids(a.pid)) > 0;
```

### 3. Internal Working

The pileup shapes (from Module 6.3, now as diagnosis): hot-row convoy (all waits on one tuple — counters, singleton config rows), DDL barrier (ACCESS EXCLUSIVE queued behind a long read, everything queues behind it), FK contention (child inserts vs parent updates), batch-vs-OLTP ordering deadlocks (nightly spikes). Each has a signature in the chain query: one root pid with dozens of waiters (convoy), an `ALTER TABLE` mid-chain (DDL), etc.

### 4. Visualization

```
lock chain:  [root blocker: idle in txn, 42min]
                  ▲              ▲
         [UPDATE waits]   [ALTER TABLE waits]
                                 ▲
                      [200 SELECTs wait behind DDL]  ← "the site is down"
read the tree root-first: kill/finish the ROOT, the tree drains itself
```

### 5–12. (Compressed)

- **Production example**: migrations at peak: `ALTER TABLE` queued behind an analytics query → full brownout (Module 6.3's story, now from the debugging side). Detection time is the metric: with `log_lock_waits=on` and the chain query saved, minutes; without, an hour of confusion.
- **Questions**: "DB slow, CPU 10%, disks idle — first check?" (wait events / lock chains); "recurring 40P01 every night at 2am" (batch ordering — reconstruct from log DETAIL); "how do you kill safely?" (`pg_cancel_backend` first (query), `pg_terminate_backend` if needed (connection); know rollback cost of killing a huge write txn).
- **Mistakes**: killing waiters instead of the root; DDL retried in a tight loop re-poisoning the queue; treating deadlock errors as data corruption (they're not — one txn rolled back cleanly).
- **Prevention**: `log_lock_waits=on`, `lock_timeout` for DDL, `deadlock` alert on log parsing, lock-ordering conventions, idle-in-transaction timeout (again — it prevents half this chapter).
- **Follow-ups**: "lock contention with no long transactions at all?" (pure hot-row throughput ceiling — serialize through a queue or shard the row); "how does this look in MySQL?" (`SHOW ENGINE INNODB STATUS`, `data_locks`; gap-lock deadlocks class Postgres lacks).

# Chapter 11.5 — Replication Lag

## 1. Why Interviewers Ask This

Lag incidents blend infrastructure and application: stale reads, backlogged failover risk, and cascading cache bugs. Tests whether you know causes *and* the consumer-side contract.

## 2. Core Concept — Triage

Measure first (Module 7.1 queries): is lag growing linearly (replica can't keep up), spiky (bursts, vacuum storms, big transactions), or plateaued (replay blocked)?

Cause catalog: - **Write burst / bulk load** on primary (replay is more serial than primary's parallel apply). - **One giant transaction** (100M-row UPDATE arrives as a wall of WAL; replicas stall then jump). - **Replica query conflicts**: long replica reads force replay to wait (`max_standby_streaming_delay`) — lag *caused by the readers themselves*. - **Replica hardware/IO inferior** to primary; network throughput ceiling. - **Vacuum/checkpoint storms** generating WAL floods (full-page writes after checkpoint).

Fixes map 1:1: chunk big writes; move long reads to a dedicated replica (or `hot_standby_feedback` with its bloat tax); scale replica IO; increase WAL bandwidth; and on the app side — **staleness contracts** (Module 7.1: pin-after-write, LSN tokens) so lag degrades gracefully instead of correctness-breaking.

## 3–4. Internal Working & Visualization

```
lag_bytes = pg_current_wal_lsn(primary) − replay_lsn(replica)
growth patterns:
  /````/````/  spiky: batch jobs → chunk them, schedule off-peak
  ————————/    step: one giant txn → break it up
  ↗↗↗↗↗↗↗↗     linear: replica underpowered / replay conflict-throttled → fix capacity or
                         reader placement
replay blocked check: SELECT * FROM pg_stat_activity ON REPLICA where backend blocks recovery
                      (conflict with recovery messages in logs)
```

## 5–12. (Compressed)

- **Production example**: nightly ETL UPDATE of 80M rows → 20 min replica lag → users see yesterday's balances → support flood. Fixes: chunked ETL + LSN-gated reads for balance-bearing endpoints + lag-based replica ejection from the read LB (serve primary when lag > threshold).
- **Questions**: "users intermittently see old data" (lag + read routing — trace one request); "replica lag alarms during vacuums — why?" (WAL volume from cleanup + full-page writes); "failover with 30s lag — what's lost?" (async: those 30s of acked writes — RPO conversation, Module 7.1).
- **Mistakes**: alerting only on seconds-lag (idle systems show huge time-lag with zero byte-lag — alert on both); hot_standby_feedback everywhere (primary bloat); assuming logical replication has the same profile (it's row-based, single-apply-worker bottlenecks differ).
- **Prevention**: chunked batch writes as policy; lag SLO per replica class; read-router with staleness budget; capacity-match replicas to primary.
- **Follow-ups**: "design the read layer so lag can never cause a correctness bug" (LSN tokens end-to-end — be able to draw it); "what's different in Aurora?" (shared storage: ~ms lag, different failure modes).

# Chapter 11.6 — Memory Issues

## 1. Why Interviewers Ask This

OOM kills and memory pressure are opaque until you know the Postgres memory model — a great test of systems depth.

### 2. Core Concept — The Memory Map

- **shared_buffers** (global, fixed): page cache (~25% RAM).
- **work_mem** (per sort/hash **node**, per connection!): the multiplication bomb — 200 connections × 4 nodes × 64MB = 51GB *potential*.
- **maintenance_work_mem** (vacuum, index builds), **wal_buffers**, per-backend overhead (~5–10MB × connections), OS page cache (the other half of your RAM — `effective_cache_size` tells the planner about it).

Incident classes: - **OOM-killed backend / postmaster** (Linux OOM killer → crash-restart of the whole cluster): usually work_mem × concurrency, a runaway hash aggregate misestimate (planner thought 1k groups, got 100M), or too many connections. - **Slow due to cache misses** (not OOM): working set outgrew RAM — hit ratio slide (11.2). - **Temp-file explosions** (disk, but memory-caused): spills from undersized work_mem — the *safe* failure mode; don't "fix" it into the unsafe one by raising work_mem globally.

```
-- Cache effectiveness
SELECT sum(blks_hit)*100.0/nullif(sum(blks_hit)+sum(blks_read),0) AS hit_pct
FROM pg_stat_database;
-- Spill volume per query (find the work_mem candidates, fix locally)
SELECT left(query,60), temp_blks_written FROM pg_stat_statements
ORDER BY temp_blks_written DESC LIMIT 5;
```

### 3–4. Internal Working & Visualization

```
RAM (64GB):
[ shared_buffers 16GB ][ OS page cache ~30GB ][ backends: 200 × 10MB = 2GB ]
[ work_mem exposure: 200 conns × N nodes × work_mem  ← the UNBOUNDED slice ]
OOM story: misestimate → HashAgg plans "1k groups" → builds 100M-group table in RAM
           → grows past work_mem *estimate-based* decisions → OOM killer → cluster restart
(PG13+ hash aggs can spill; misestimates still hurt)
```

### 5–12. (Compressed)

- **Production example**: analytics query with a misestimated GROUP BY OOM-kills the primary at month-end, every month. Chain: auto_explain catches it → estimate fix (extended stats) + move to replica + session-local work_mem cap. The "monthly recurring OOM" framing is a common interview scenario.
- **Questions**: "walk the Postgres memory model"; "OOM killer took the DB — how do you find the culprit?" (kernel log timestamp → pg_stat_statements/auto_explain around it → plan); "why not raise work_mem globally?" (the multiplication argument — say the arithmetic).
- **Mistakes**: shared_buffers at 80% of RAM (starves OS cache + work areas); overcommit-enabled kernels letting the OOM killer choose Postgres (set `vm.overcommit_memory=2`, adjust oom_score for postmaster); confusing spill (safe, slow) with OOM (fatal).
- **Prevention**: connection caps via pooler (bounds the multiplier), per-role/session work_mem for heavy jobs, memory dashboards split by class, statement_timeout as a runaway bound.
- **Follow-ups**: "same question for MySQL" (innodb_buffer_pool + per-thread buffers — same multiplication trap); "how does work_mem interact with parallel query?" (per worker! another multiplier).

---

## Chapter 11.7 — Large Tables & Bloat

### 1. Why Interviewers Ask This

"The table is 3TB and everything about it is slow" — tests VACUUM understanding (Module 6.2), partitioning judgment (7.2), and safe-operations discipline.

### 2. Core Concept — The Big-Table Playbook

Diagnose first: is it *bloat* (dead space) or *legitimate size*?

```
SELECT relname, n_live_tup, n_dead_tup,
       round(n_dead_tup*100.0/nullif(n_live_tup+n_dead_tup,0),1) AS dead_pct,
       last_autovacuum
FROM pg_stat_user_tables ORDER BY n_dead_tup DESC LIMIT 10;
-- pgstattuple for precise dead-space %, pg_relation_size for the raw truth
```

- **Bloat** (dead_pct high, autovacuum behind): find the blocker (long transactions! replication slots! prepared transactions!) → unblock → tune autovacuum (scale_factor per table, cost limits) → reclaim with `pg_repack` (online) — never casual `VACUUM FULL` (exclusive lock).
- **Legitimately huge**: partition (7.2) for pruning + retention; archive cold data; split hot/cold columns; move blobs to object storage.
- **Operations on big tables have their own physics**: every ALTER/backfill/index-build must be chunked/CONCURRENTLY/NOT VALID+VALIDATE; a naive `UPDATE SET flag=true` on 2B rows = 2B new row versions = self-inflicted bloat bomb + replication flood.

### 3–4. Internal Working & Visualization

```
why the 2B-row UPDATE is a disaster:
UPDATE all rows → 2B new versions (MVCC) → table doubles on disk
→ WAL flood → replicas lag hours → autovacuum weeks of debt → indexes bloat too
correct: batch 10k-100k rows per txn, sleep between, vacuum checkpoints, or
         CTAS-rebuild + swap when touching >30-50% of the table
autovacuum starvation loop: big table × default scale_factor(0.2) = vacuums only
every 400M dead rows → each run takes days → always behind → lower per-table thresholds
```

### 5–12. (Compressed)

- **Production example**: "add a column with default + backfill" on a 1TB table done naively → weekend outage; done right → invisible (PG11+ fast default, then chunked backfill of derived values, then NOT NULL via CHECK NOT VALID → VALIDATE). This migration question is asked at every senior loop in some form.
- **Questions**: "count of dead tuples keeps growing despite autovacuum running — why?" (long txn / stale replication slot pinning the horizon — find with the Module 6.2 queries); "delete 90% of a huge table" (don't DELETE: partition-drop, or CTAS survivors + swap); "why is VACUUM FULL dangerous and what replaces it?" (pg_repack).
- **Mistakes**: DELETE for retention on unpartitioned tables (bloat instead of space); ignoring index bloat (REINDEX CONCURRENTLY exists); disabling autovacuum on the busiest table.
- **Prevention**: partition event-like tables at design time; per-table autovacuum tuning as tables cross ~50GB; replication-slot and long-txn monitoring; migration playbooks with chunking as default.
- **Follow-ups**: "TOAST — what is it and when does it bite?" (out-of-line storage for big values; update patterns rewrite whole toasted values); "wraparound emergency — walk the runbook" (age monitoring → aggressive vacuum → single-user mode worst case).

---

## Chapter 11.8 — Pagination Problems (deep-page incidents)

### 1. Why Interviewers Ask This

A compact, real incident class that ties together Modules 5.5, 4.3, and API design — often used as a closing practical question.

### 2. Core Concept — The Incident Catalog

- **Deep OFFSET scans**: crawler/export hits `?page=50000` → each request scans-and-discards 1M rows → DB saturated by a single client. Detect: pg_stat_statements shows the same query shape with huge `rows` scanned vs tiny returned; fix: keyset pagination + hard offset caps + rate-limit/redirect exports to snapshots (Module 5.5).
- **Unstable pages under writes**: users see duplicated/skipped items during infinite scroll (rows shifting across page boundaries). Fix: keyset with unique tiebreaker (stable anchors).
- **COUNT-per-page**: every page render runs `count(*)` over the filtered set → half the endpoint's cost is the total nobody reads. Fix: has_more (k+1 fetch), estimates, or cached counts (Module 5.6).

- **Missing composite index for the sort**: pagination "works" but each page sorts the world — `(filter_cols, sort_col, id)` index makes pages O(page size).
- **Cursor drift on filtered lists**: cursor encodes position but filters changed between requests → wrong slices. Fix: encode filters (or their hash) in the cursor; reject on mismatch.

### 3–4. Internal Working & Visualization

```
OFFSET-depth cost:  page1 │ page100 ▌ page1000 ▉ page10000 ████████  (O(offset))
keyset:             page1 │ page10000 │                               (O(k))
incident signature in pg_stat_statements:
  query LIKE '%OFFSET%'  calls=40k/hr  mean=900ms  rows=20  shared_blks_read=huge
  → one API client paging to infinity
```

### 5–12. (Compressed)

- **Production example**: partner integration syncs the catalog nightly via `?page=N` to page 80,000; DB CPU pegs for 3 hours nightly. Fixes shipped: cursor API + `updated_since` incremental sync + nightly export file — and a max-offset guard (HTTP 400 past page 500) to stop the bleeding immediately. The "stop the bleeding vs fix the API" sequencing is what interviewers grade.
- **Questions**: "the DB dies every night at 2am; here's pg_stat_statements — find it" (the OFFSET signature); "user reports seeing the same item on two consecutive pages" (instability under writes — keyset + tiebreaker); "how would you paginate a 100M-row export API?" (cursor / snapshot-to-object-storage / server-side cursor streaming).
- **Mistakes**: caching page results as a fix (position-keyed cache invalidates on every write); adding replicas to absorb OFFSET abuse (scales the waste); switching to keyset without backfilling the supporting index.
- **Prevention**: keyset-by-default in the API framework; offset caps + pagination-abuse rate limits; `updated_since` incremental endpoints for sync use-cases; page-depth metrics.
- **Follow-ups**: "cursor pagination across shards?" (per-shard cursors merged — Module 5.5/7.3); "how does Stripe's API do it?" (`starting_after`/`ending_before` object-id cursors — cite it).

---

# Module 11 — Practice Problems

## Easy (5)

1. Write the four triage queries (activity, blockers, top statements, long transactions) from memory and state what each rules in or out.
2. `FATAL: remaining connection slots are reserved` — list the diagnosis steps in order and the fix ladder.
3. pg_stat_user_tables shows n_dead_tup=800M and last_autovacuum=3 days ago on a hot table. Give the three most likely blockers and the query to find each.
4. An endpoint is slow only for one huge tenant. Name three mechanisms that make identical queries tenant-dependent and the fix for each (skew/plans, missing composite index, hot partition).
5. Replica time-lag alarm fires at 4am daily for 20 minutes. What do you correlate it with, and what are the two standard fixes?

## Medium (5)

1. Given pg_stat_statements output (queries with calls, mean, total, temp_blks, blks_read), triage which of five listed statements you'd attack first and why — construct the reasoning with total-time × user-impact.
2. The site is down; pg_stat_activity shows 300 sessions waiting, all blocked (transitively) on one `idle in transaction` pid from the admin VPN. Write the exact commands to confirm, kill, and the two config changes that prevent recurrence.
3. Design the "safe migration" checklist for a 2TB orders table: add column with default, backfill computed values, add NOT NULL, add an index, add an FK — each step with its lock profile and chunking strategy.
4. Every deploy causes 5 minutes of DB latency. List four mechanisms (cold app pools × connection storm, prepared-statement re-plan, cache eviction from restarted services, ORM migrations taking locks) and how to confirm each from metrics.
5. Nightly OOM kill during a reporting query. Reconstruct the full chain from kernel log to query plan to root cause (HashAgg misestimate), and write the three-layer fix (stats, session work_mem, workload placement).

## Hard (5)

1. Compose the full incident: at 09:00 a deploy ships a new filter that makes a hot query non-sargable; by 09:20 the pool is exhausted; by 09:25 retries have tripled load and the replica lags 10 minutes. Write the minute-by-minute response: detection signals, mitigation order (and why that order), root-cause confirmation, and the three prevention items.
2. You inherit a 6TB single-table Postgres with no partitioning, 40% bloat, autovacuum always behind, quarterly wraparound scares, and 5s p99. Produce the six-month stabilization plan with dependency ordering: vacuum unblocking, repack sequencing, partition migration (dual-write design), index audit, and the risk register for each step.
3. Build the observability stack for databases from scratch: exact metrics per incident class (11.2–11.8), log lines to parse (lock waits, deadlocks, autovacuum, checkpoints, temp files), auto_explain config, alert thresholds with rationale, and the runbook index. Argue what pages a human at 3am vs what waits for morning.
4. A multi-tenant SaaS suffers noisy-neighbor incidents: one tenant's analytics queries starve everyone. Design the isolation ladder — statement_timeout per role, per-tenant connection budgets via pooler, workload routing to replicas, tenant-partitioned tables, and finally tenant sharding — with the trigger criteria for escalating each rung.
5. Post-mortem synthesis: pick any three incidents from this module and write the shared root cause taxonomy (long transactions, unbounded work, missing backpressure) — then propose the three platform-level invariants (timeouts everywhere, chunk-by-default batch framework, staleness-budgeted read routing) and prove each incident is prevented or bounded by them.

## You're Done — The Final Checklist

Before your interview loop, verify you can do these cold:

- [ ] Explain WAL → commit → crash recovery in 60 seconds (M1)
- [ ] Recite the clause evaluation order and the NOT IN + NULL trap (M2)
- [ ] Whiteboard all three join algorithms with when-each-wins (M3)
- [ ] Design a composite index from a query using ESR and defend column order (M4)
- [ ] Read an EXPLAIN ANALYZE bottom-up: estimates, loops, spills (M5)
- [ ] Produce lost-update and write-skew interleavings + all fixes (M6)
- [ ] Walk the scaling ladder: cache → replicas → partition → shard, with what breaks (M7)
- [ ] Compare B+Tree vs LSM and place all six databases on the comparison table (M8)
- [ ] Design hotel booking with race-proof no-double-booking, three ways (M9)
- [ ] Solve: Nth salary, dedup delete, streaks, sessionization, top-K per group (M10)
- [ ] Run the universal triage loop on any "DB is slow" prompt (M11)